Mastering JavaScript and TypeScript

A Compressed Book for Experienced Programmers - Complete Edition

Patrick Lin

July 22, 2026

Contents

Introduction: Static Guarantees and Runtime Behavior

JavaScript determines what executes. TypeScript analyzes the program before execution and reports whether its expressions, assignments, calls, and declarations satisfy inferred or declared type constraints. Type annotations and inferred types generally do not survive into the emitted JavaScript; classes and enums are important cases where a source name can also correspond to a runtime value. Production behavior also depends on the runtime host, module loader, package metadata, compiler configuration, external data, storage systems, and process lifetime. Mastery requires identifying which component supplies each guarantee, which component can enforce it, and which failures remain possible elsewhere.

This book is compressed by design. It presents a connected technical explanation rather than a sequence of gentle tutorials. You will make its concepts usable and retrievable by predicting program behavior, reconstructing explanations from memory, implementing examples and abstractions independently, presenting them aloud, and repairing only gaps demonstrated by your own work.

The chapters move through five connected regions: runtime state; asynchronous execution and resource lifetime; TypeScript's static analysis; runtime truth and the source-to-execution pipeline; and production correctness. The regions are contiguous and ordered: Chapters 1–5, 6–9, 10–15, 16–20, and 21–26, respectively. Integration chapters treat important relationships as subjects in their own right. The capstone requires all five regions to operate together in one program.

Pedagogical diagnosis and design decision

This book was designed for a particular learning problem. An experienced programmer can often recognize JavaScript and TypeScript syntax while still making incorrect predictions when several components interact. The difficult questions are usually not "What does this keyword mean?" They are "What executes?", "Which component checked or enforced this fact?", "When was that fact established?", and "What can still fail outside that component's authority?" A programmer may understand promises and types separately but still expect a promise return type to guarantee completion. The same programmer may understand imports and compilation separately but still treat editor success as evidence that Node can load the deployed file. The book therefore concentrates on relationships, execution boundaries, and the allocation of guarantees.

Other book structures solve different problems. A beginner sequence can introduce syntax gently, but it would spend much of this reader's attention on already familiar programming constructs and postpone the cross-component failures that motivated the book. An encyclopedic reference makes isolated facts easy to look up, but it does not necessarily produce a connected explanation from which behavior can be predicted. A framework cookbook can produce an application quickly, but recipes can hide which guarantees come from JavaScript, TypeScript, Node, a library, a database, or deployment configuration, and the recipes age with the tools. A type-system-centered treatment can develop considerable static sophistication while leaving runtime data, module loading, asynchronous lifetime, storage, and process failure underexplained. A capstone-first approach exposes the whole system immediately, but it makes a wrong result difficult to localize because too many prerequisites can fail at once.

The five regions are ordered to reduce that ambiguity. Runtime values and state come first because every later TypeScript description refers, directly or indirectly, to JavaScript values and operations. Asynchronous

execution follows because time, interleaving, cancellation, and cleanup alter what state can be trusted. The TypeScript static model comes next, after the runtime behavior it describes is visible. Erasure, validation, modules, and compilation then reconnect source descriptions to the artifact and data that actually execute. Production chapters finally distribute guarantees among application code, tests, parsers, databases, hosts, and lifecycle owners. Integration chapters exist because learning the endpoints does not automatically teach the relationship between them.

The resulting pedagogy is differential rather than uniformly expansive. The canonical chapters provide one compact technical spine for every reader. When a particular passage causes trouble, the reader pastes the passage, current understanding, prediction or attempt, and observed result into Codex. The cause might be an unfamiliar term, a missing relationship, a false imported assumption, an implementation step that is not yet usable, or knowledge that was recognized but not retrievable. The reader does not have to choose among those possibilities. Codex uses the evidence in the report to add a patient, example-led section to the personal appendix that repairs only the demonstrated gap. The section links back to the exact passage so the reader can resume without searching. It may include a focused prompt when a prompt materially helps the explanation, but it does not inherit a standard question template. One reader's difficulty remains personal; the same failure recurring across readers is evidence that the canonical passage may itself need revision.

The book is trying to convey more than a collection of JavaScript and TypeScript features. It presents a way to account for a production program: name the operation, the component that performs it, the evidence or guarantee that operation supplies, the time and scope in which the guarantee holds, and the failures that remain possible elsewhere. Prediction, independent implementation, diagnosis, presentation, and delayed reconstruction are used because a connected explanation is useful only when the reader can retrieve and apply it without the page in view.

When a passage produces friction, paste the relevant text or code into Codex and explain in ordinary language what you currently understand, what you expected, what happened, and what remains confusing. You do not need to identify or classify the gap yourself. Codex will determine what explanation or practice is missing and record the resulting repair in your personal appendix rather than expanding the canonical chapter. Do not confuse density with a requirement to struggle alone, but do not turn every incomplete recognition into a long prerequisite detour. Repair the smallest demonstrated gap, then return to the original passage.

Chapter 1

Values, References, Identity, and Mutation

Chapter type: Acquisition, with early JavaScript/TypeScript integration
Prerequisites: Variables, assignment, function calls, ordinary objects and collections
Capabilities introduced: Predict rebinding, mutation, identity, aliasing, function-argument effects, shallow copying, and compile-time versus runtime immutability controls
Earlier chapters integrated: None
Later chapters enabled: 2–5 directly; asynchronous state, narrowing, domain modeling, and runtime truth indirectly
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch01/src/`, `examples/ch01/test/`, and `examples/ch01/lab/`

Compressed thesis

JavaScript variables are **bindings** to values. Primitive values can be copied from one binding to another without creating shared mutable state. Object creation produces a distinct identity. Copying the reference value that gives access to an object creates another access path to that same identity; such access paths are **aliases**. Rebinding changes the value held by one binding, whereas mutation changes an existing object that every alias can observe.

JavaScript passes argument values. When an argument value gives access to an object, the parameter initially becomes another alias; property writes through that parameter mutate the shared object, but assignment to the parameter rebinds only the parameter. Object and array spread create a new outer container while copying its property or element values, so nested objects may remain shared. A `const` declaration prevents rebinding one binding, TypeScript `readonly` rejects selected writes during typechecking, and `Object.freeze` prevents selected property changes on one object at runtime. None of them introduces ownership or prevents every alias from mutating reachable state.

This chapter uses **reference value** as an operational term for the copied value that preserves access to one object identity. The ECMAScript specification uses *Reference Record* for an internal result of evaluating locations such as identifiers and properties. These terms name different concepts: the former explains observable aliasing, while the latter describes specification evaluation steps.

1. Bindings and values

A declaration introduces a binding in an environment. Evaluation supplies values, and assignment changes which value a writable binding contains. In `let score = 41`, `score` names a binding and `41` is the value initially held there. The binding and its current value remain distinct: the binding may later hold `42`, while the number `41` has not itself changed.

Primitive values—`undefined`, `null`, booleans, numbers, bigints, strings, and symbols—have no mutable properties of their own. Some primitives expose property-like operations, such as `"dense".toUpperCase()`, but that does not turn the string into a mutable object. Do not invent a persistent wrapper object to explain ordinary primitive use.

Predict before running: What are `copiedScore` and `score` after the final assignment? Did either number mutate?

```
let score = 41;
const copiedScore = score;
score = 42;

console.log(copiedScore, score);
```

Output:

```
41 42
```

The second declaration copied the current number value into a different binding. The final assignment rebounded `score`; it did not reach backward and modify `copiedScore`, nor did it mutate the number `41` into `42`.

`const` changes which assignments are permitted for a binding. A `const` binding must be initialized and cannot later be rebound. It says nothing yet about whether a value reached through that binding contains mutable state:

```
const session = { attempts: 1 };
session.attempts += 1;       // allowed: mutate the object
// session = { attempts: 0 }; // error: rebind the const binding
```

The false model "`const` makes an object immutable" collapses binding and object into one entity. The declaration constrains the binding named `session`; the property assignment operates on an object.

Destructuring also becomes predictable when treated as binding initialization rather than a permanent connection to a property. Primitive property values are copied into new bindings; object property values can create new aliases:

```
const state = {
  count: 1,
  options: { trace: false },
};
const { count, options } = state;

state.count = 2;
options.trace = true;

console.log(count, state.options.trace);
```

The output is `1 true`. `count` received the number value that `state.count` held during destructuring. `options` received a value that reaches the same nested object as `state.options`. Destructuring did not create live bindings back to ordinary object properties. Later module chapters will show that imported names are live bindings established by module semantics; ordinary object destructuring does not establish that relationship.

2. Object identity and reference values

An object has identity in addition to its current properties. Each evaluation of an object, array, function, class, map, set, or other object-creating expression can produce a distinct identity. Two objects can carry equal-looking data and remain different objects.

Predict before running: Which comparisons are true? Separate the comparison of property values from the comparison of objects.

```
const first = { language: "TypeScript" };
const second = { language: "TypeScript" };
const alias = first;

console.log(first === second);
console.log(first === alias);
console.log(first.language === second.language);
```

Output:

```
false
true
true
```

Strict equality compares these object operands by identity. It does not recursively compare their properties. The strings in the third comparison have equal primitive values, so that comparison is true. If an application needs structural equality, it must define the relevant structure and comparison rules; `===` does not infer them.

The statement "an object is a reference" is too compressed for the reasoning required here. Keep three roles visible: an object has identity and state; a binding contains a value; copying the value that provides access to the object can make another alias without constructing another object. These distinctions let you ask separately whether an assignment changed a binding, mutated an existing object, or created a new identity.

Identity is not a database identifier. Two runtime objects can both contain `userId: "17"` and still have different runtime identities. Conversely, one runtime object can be reached through many aliases while containing no domain ID at all.

Arrays and functions follow the same identity rules because both are objects. Assigning an array to another binding does not copy its elements, and assigning a function does not copy its code into a new callable identity. A class instance likewise retains its identity when assigned to a base-shaped or `readonly` TypeScript variable. The annotation may cause TypeScript to accept or reject an operation during checking; it does not replace the runtime object.

Representing a running heap fragment as a graph makes aliases visible. Objects are nodes with identities; properties, collection entries, and bindings can carry edges that reach those nodes. Primitive values can appear directly at an edge or property without introducing a mutable node. Object equality asks whether two evaluated operands designate the same node, not whether the reachable subgraphs look alike. This graph predicts identity and mutation; it does not describe an engine's memory layout.

3. Aliasing and mutation

Aliasing exists whenever multiple access paths reach the same object. The paths may be variables, parameters, object properties, closure-captured bindings, array elements, map values, or results returned from APIs. Mutation through one path changes the object, so later reads through every other path can observe the new state.

Predict before running: Does `primary.attempts` remain `1` because only `alias` appears on the left side of the property assignment?

```
const primary = { attempts: 1 };
const alias = primary;

alias.attempts += 1;

console.log(primary === alias, primary.attempts);
```

Output:

```
true 2
```

The assignment to `alias` copied an access value; it did not duplicate the object. The property update therefore acts on the identity also reached through `primary`.

Aliasing is not automatically bad. A cache deliberately gives many callers access to shared entries. A DOM node is expected to preserve identity as code updates it. A connection or transaction object often represents one resource that collaborators must share. The cost is a larger reasoning boundary: to know whether an object can change, you may need to find every alias and every operation available through them. Hidden aliases make a local expression depend on nonlocal behavior.

Rebinding and mutation can both be described as changing one part of the graph, but they change different parts. `primary = replacement` redirects one binding edge and leaves the old object untouched. `primary.attempts = 3` changes a property of the node reached through `primary`; all edges to that node still reach it and can observe the new property value. `primary.child = replacementChild` mutates the primary object by redirecting one of its property edges. The last case is both a property assignment and a change in alias topology, which is why the slogan "assignment means rebinding" is too broad: the left-hand location determines what is updated.

An alias may also be read-only by convention or by a TypeScript view and still be an alias. Aliasing describes shared identity, not permission. Permission determines which operations one access path exposes; another path may expose more. Keeping those questions separate will later prevent structural types from being mistaken for ownership capabilities.

Because hidden aliases expand the code that can affect an object, mutation policy belongs in an API contract. An API may mutate a supplied object, return a new object, expose a readonly view, transfer practical control by convention, or encapsulate mutation behind methods. None of these policies follows merely from an object-shaped type.

4. Function arguments: values are passed

At a call, JavaScript evaluates each argument and initializes the corresponding parameter with the resulting value. For an object argument, the value copied into the parameter preserves access to the same object identity. The parameter is therefore initially an alias, but its binding belongs to the function call.

Predict before running: What are the caller's points, and does `callerAccount` acquire the replacement object's identity?

```
interface Account {
  owner: string;
  points: number;
}

function mutateThenRebind(account: Account): Account {
  account.points += 5;
  account = { owner: "replacement", points: 0 };
  account.points += 1;
  return account;
}

const callerAccount = { owner: "caller", points: 10 };
const returnedAccount = mutateThenRebind(callerAccount);

console.log(callerAccount);
console.log(returnedAccount);
console.log(callerAccount === returnedAccount);
```

Output:

```
{ owner: 'caller', points: 15 }
{ owner: 'replacement', points: 1 }
false
```

The first property update uses the parameter's access path to mutate the caller's object. The later assignment changes only the parameter binding, so subsequent work uses a new object. The caller's binding is not available as an assignable location inside the function.

Both `account.points += 5` and `account = replacement` use assignment syntax, but their left-hand sides evaluate differently. The first identifies a property on the shared object. The second identifies the call-local parameter binding. That difference determines the result; vague phrases such as "the function changes the object" are insufficient because one assignment changes it and the other does not.

A function can make the caller choose a new identity by returning one. If the caller writes `callerAccount = mutateThenRebind(callerAccount)` and its binding is writable, the caller performs the final rebinding after the function returns. The callee still did not rebind the caller's variable directly. Separating return from caller assignment matters for APIs that offer immutable updates: creation of a new value does not automatically replace any caller-held binding.

"Objects are passed by reference" predicts the mutation but often predicts parameter rebinding incorrectly. Under a calling convention that passed the caller's variable itself by reference, assigning the parameter could assign that caller variable. JavaScript instead passes the argument value. "Pass by value, where the value may provide access to an object" predicts both observations without a special exception.

Type annotations in the example constrain which calls and property operations TypeScript accepts; they do not change how JavaScript passes the argument value. Removing the annotations yields the same emitted runtime relationships.

5. Copying and nested sharing

Object spread creates a new ordinary object and copies selected own enumerable property values into it. Array spread creates a new array and copies the source element values. Neither operation recursively recreates objects reached through those copied values. If one copied value gives access to a nested object, the new container and old container remain aliases for that nested identity.

Predict before running: Which identity comparisons are true, and what theme is visible through `original` after the update through `copy`?

```
const original = {
  id: "profile-1",
  preferences: { theme: "dark", notifications: true },
};

const copy = { ...original };
copy.preferences.theme = "light";

console.log(original === copy);
console.log(original.preferences === copy.preferences);
console.log(original.preferences.theme);
```

Output:

```
false
true
light
```

The outer identity is new, but the value copied for `preferences` still reaches the original nested object. "Spread makes a deep copy" fails because it does not identify which objects are recreated. Draw each created object, then draw an edge for each property whose value reaches another object. Copying the outer node does not recursively copy every node reachable from it.

An immutable update creates a new object for each object on the property path being changed:

```
const updated = {
  ...original,
  preferences: {
    ...original.preferences,
    theme: "light",
  },
};
```

Now `updated !== original` and `updated.preferences !== original.preferences`. Other unchanged subobjects may remain shared when the program's mutation policy makes that safe. This selective copying preserves useful identity and avoids an indiscriminate traversal of the entire graph.

Selective copying is path-sensitive. To change `profile.preferences.theme` without modifying the old graph, create a new object for each mutable node on the path from the returned root to `theme`: the outer profile and the preferences object. A sibling such as `profile.identity` is not on that path and may be reused if its contract prevents unsafe mutation. Copying too few path nodes leaks the update into the old graph; copying every reachable node discards identities and performs work the transition did not require.

Arrays of objects make the same rule visible in a familiar shape. `[...orders]` creates a new array node, but each element value still reaches its previous order object. Replacing `copy[0]` changes only the copied array's element edge. Mutating `copy[0].status` changes the shared order node and is visible through the original array. "The array was copied" therefore answers only the outermost identity question.

JSON serialization is not a general deep-copy operation. It accepts only a restricted data model, invokes serialization behavior, rejects some graphs such as cycles, and loses or transforms values and prototypes. Use it for JSON interchange when JSON is the intended representation, not as folklore for cloning arbitrary JavaScript state.

6. Compile-time and runtime immutability

Four different questions are often compressed into "is it immutable?"

1. Can this binding be rebound?

2. Will the typechecker permit a write through this access path?
3. Will the runtime accept a property change on this object?
4. Can another alias still reach mutable nested state?

`const` prevents later assignment to one binding. TypeScript `readonly` rejects selected writes through a typed access path during checking; the `readonly` modifier is absent from emitted JavaScript and adds no runtime enforcement. The official TypeScript object-types documentation also warns that readonly and writable views can coexist through aliasing.

```
interface Profile {
  name: string;
  preferences: { theme: string };
}

const mutable: Profile = {
  name: "Ada",
  preferences: { theme: "dark" },
};
const readonlyView: Readonly<Profile> = mutable;

// readonlyView.name = "Grace"; // rejected by TypeScript
mutable.name = "Grace";         // accepted; same object
mutable.preferences.theme = "light";

console.log(readonlyView.name, readonlyView.preferences.theme);
```

The output is `Grace light`. The readonly view did not acquire ownership or a snapshot. Moreover, `Readonly<Profile>` is shallow: it prevents assignment to `readonlyView.preferences`, but the nested object type still has a writable `theme` property.

`Object.freeze` acts at runtime on an object. It prevents changes to that object's own property set and descriptors, so assigning a frozen top-level data property in strict-mode code throws a `TypeError`. Freeze is shallow: it does not recursively freeze a nested object merely because a property reaches it.

The depth distinction has two forms. `Readonly<Profile>` transforms the declared top-level properties of `Profile`; it does not recursively transform the type of the object stored in `preferences`. `Object.freeze(profile)` freezes the top-level runtime object; it does not recursively invoke freeze on the preferences object. The static and runtime operations are shallow for different reasons, but they produce a similar boundary: the outer edge may be protected while the nested node remains writable.

Predict before running: Which write succeeds, and which runtime write throws in the ESM examples?

```
const frozen = Object.freeze({
  label: "fixed",
  nested: { count: 0 },
});

frozen.nested.count += 1; // nested object was not frozen
// frozen.label = "changed"; // TypeScript rejects; runtime also rejects the write
```

The nested count becomes `1`; the top-level label remains `"fixed"`. The runnable case deliberately creates a writable static view to prove that runtime freeze still rejects the top-level write. A type assertion can silence a checker complaint, but it cannot thaw an object.

TypeScript `readonly` and `Object.freeze` can reinforce one another, but they are not interchangeable. A readonly API communicates intent and causes the checker to reject selected writes. Freeze changes the runtime object's property descriptors so JavaScript rejects selected mutations. Neither proves that the full reachable object graph will never change, and TypeScript cannot generally prove that an object has one unique owner. JavaScript and TypeScript do not supply Rust-like borrowing or ownership checking.

Runtime enforcement also depends on the exact operation and execution mode. In strict code, an invalid

ordinary assignment to a frozen property throws; in non-strict legacy code, some failed assignments can be ignored. The canonical repository is ESM, and modules are strict code, so the caught `TypeError` is deterministic there. TypeScript's rejection occurs earlier and only for code it checks; untyped JavaScript, an assertion, or a writable alias can attempt the runtime operation, where the actual object decides the result.

7. Consequences for program design

An API's mutation policy changes what callers can safely infer. If `normalize(profile)` mutates its input, the name, documentation, types, and tests should make that effect visible. If `withNormalizedEmail(profile)` returns a new profile, callers need to know which nested identities remain shared. A readonly parameter can reject accidental writes in checked code, but it does not prove the caller lacks a mutable alias.

Shared mutable state increases reasoning cost because the observed values depend on both alias topology and operation order. Immutable updates can shrink the relevant reasoning boundary: old access paths retain old state while the returned graph represents the transition. The benefit is not free. Selective copying requires identifying every changed object on the update path, creates allocations, and can still preserve dangerous aliases when nested mutation remains possible.

Mutation is reasonable when identity is itself the abstraction or when change is tightly encapsulated: updating a private cache, advancing a local parser cursor, maintaining a connection state machine, or filling a newly created object before publication. The governing question is not "is there mutation?" but "who can observe it, through which aliases, during what lifetime, and what invariant controls it?"

Identity preservation can itself be part of a contract. Replacing a cached object may leave existing consumers attached to a stale identity; mutating it may be the intended notification mechanism. In another API, preserving the old value is essential for comparison, rollback, or concurrent reads, so a new graph is the correct transition result. JavaScript's identity and aliasing rules do not select one universal style; they let you predict the consequences of either style.

Later chapters increase the stakes. A closure can retain an alias after the apparent local scope has ended. A collection can hide aliases among elements. An asynchronous continuation can mutate an object after other code has observed or modified it. State-machine types can restrict supported transitions but do not change runtime identity. Each consequence follows from the binding, identity, and aliasing rules established here.

Garbage collection supplies a final boundary: object lifetime depends on reachability as determined by the runtime, not on the continued existence of one particular variable name. Rebinding one alias does not make the object collectible if another reachable alias remains. Detailed collector algorithms are outside this chapter.

False models, repaired

- **"Objects are passed by reference."** Argument values are passed. A copied object-access value creates a parameter alias; parameter rebinding remains local.
- **"`const` makes an object immutable."** `const` prevents rebinding one binding.
- **"Spread makes a deep copy."** Spread copies one layer of values; nested object identities may remain shared.
- **"Two objects with the same properties are equal."** `===` compares object identity, not recursive structure.
- **"TypeScript `readonly` means the object cannot change at runtime."** It restricts checked writes through a typed access path and is erased.
- **"Mutation is always bad."** Mutation has costs and valid uses; alias visibility, lifetime, and invariants determine the risk.
- **"Reassigning a parameter reassigns the caller's variable."** The parameter is a separate binding initialized with the argument value.

Mastery questions

Answer without running prediction cases until you have committed to outputs and causal explanations.

1. A number is copied to a second variable and the first is incremented. Predict both values and distinguish value copying from rebinding.
2. Two variables alias an array of objects. One pushes a new object and mutates an existing element. What changes are visible through the other variable?
3. Construct two separate objects whose properties are equal. Which comparisons establish property equality, and which establish identity?
4. A function mutates `parameter.count` and then assigns a new object to `parameter`. Predict the caller's object and derive the result from parameter-binding initialization.
5. Explain one concrete wrong prediction produced by "pass by reference."
6. Draw or describe the alias graph before and after `{ ...profile }` when `profile.preferences` is an object.
7. Draw the graph after selectively copying both `profile` and `profile.preferences`. Which edges may still be shared?
8. Diagnose a bug where `[...orders]` is treated as protection against mutations to an order object.
9. Compare `const`, a readonly property, `Readonly<T>`, and `Object.freeze` by restricted operation, depth, enforcement phase, and responsible component.
10. Show how a readonly view and mutable alias can coexist, then state what TypeScript did and did not guarantee.
11. Design a function signature and naming policy for an API that intentionally sorts an array in place. What must callers know?
12. Apply the identity and copying rules to a `Map<string, User>` copied with `new Map(original)`: which identities are new and which `User` objects remain shared?
13. Give a situation where mutation improves the design and identify the boundary that contains its reasoning cost.
14. Explain why rebinding the last visible local variable is insufficient evidence that an object is unreachable.

The full answer workspace and lab specification are in `exercises/ch01.md`.

Implementation lab — Immutable Preferences Update

Implement `updatePreferences` in `examples/ch01/lab/starter.ts`. It receives a nested `UserProfile` and a partial preferences update. Return a new outer profile without mutating the input. Preserve the identity and audit subobjects because their properties do not change; replace the preferences object because the update changes its properties. Test both values and identities.

Before coding, state which of these must be equal by identity: original and result; their identity objects; their preferences objects; their audit objects. Then inspect the deliberately incorrect behavior only after implementing your own version. The evaluator's faulty version copies the outer profile and mutates the still-shared preferences object.

Presentation prompt

Give a 10–15 minute explanation in your own words covering binding, value, identity, reference value, aliasing, mutation versus rebinding, argument passing, nested sharing after spread, the distinct guarantees of `const`, `readonly`, and freeze, one reasonable mutation, one API-design consequence, and one point that remains fragile. Include one predicted code case and one derivation rather than only definitions. The recording checklist is in `presentations/prompts/ch01.md`.

Ready-when test

Proceed only when, without notes, you can predict all six canonical examples; explain why object arguments are not truly passed by reference; separate mutation from rebinding in a new case; identify nested aliases after a shallow copy; distinguish `const`, `readonly`, and `Object.freeze`; implement and test `updatePreferences`; and deliver the presentation without a major mechanistic error. Imperfect wording is acceptable. An explanation that predicts the wrong identity, enforcing component, or enforcement phase is not.

Chapter 2

Functions, Scope, Closures, and this

Chapter type: Acquisition and integration
Prerequisites: Chapter 1
Capabilities introduced: Predict lexical name resolution, closure-retained state, receiver selection, method extraction, arrow-function capture, and TypeScript `this` checking
Earlier chapters integrated: Bindings, identity, aliasing, mutation, and lifetime from Chapter 1
Later chapters enabled: Prototypes and classes, collection callbacks, asynchronous state, modules, and resource lifetime
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch02/src/`, `examples/ch02/test/`, and `examples/ch02/lab/`

Compressed thesis

A JavaScript function is an object with identity that can be stored, passed, returned, and invoked. Creating a function associates the function object with the lexical environment in which its body was defined. A later call resolves free identifiers through that retained environment, so a closure preserves access to bindings rather than copying their current values. Each invocation of an enclosing function creates new parameter and local bindings; several functions created during one invocation can share those bindings, while functions created during separate invocations retain separate bindings.

For an ordinary function, the invocation operation supplies `this`; the function's lexical location and the object property that once stored it do not permanently fix a receiver. `account.describe()` supplies `account`, while extracting the same function and calling `describe()` supplies `undefined` in this book's strict ECMAScript modules. An arrow function has no receiver binding of its own and reads `this` from the enclosing function or module context. Lexical capture and receiver selection are therefore separate operations even when both make external state visible inside a function.

TypeScript can reject calls whose receiver conflicts with an explicit `this` parameter, and `noImplicitThis` can report receiver uses whose type would otherwise be implicit `any`. Neither checker operation supplies a runtime receiver. A method call, `call`, `apply`, `bind`, constructor call, or enclosing arrow context supplies or fixes the runtime receiver; a TypeScript annotation only changes which checked source operations are accepted. This canonical chapter retains the shared technical spine. Any explanation required only because of an observed reader gap belongs in a source-linked section of the personal appendix.

1. Function values, identities, and calls

Function declarations, function expressions, arrow expressions, methods, and class methods all produce or expose callable objects. A binding can hold a function value exactly as it can hold an array or map value. Assigning the function to another binding preserves its identity; evaluating a new function expression creates a new function object even when the source text is identical.

Predict before running: Which comparisons are true, and what does the final call return?

```
function increment(value: number): number {
  return value + 1;
}

const alias = increment;
const another = (value: number) => value + 1;

console.log(alias === increment);
console.log(another === increment);
console.log(alias(4));
```

Output:

```
true
false
5
```

The first assignment copies the value that designates the existing function object. The arrow expression constructs a different callable object. Equal results do not imply equal function identity, just as equal-looking object properties did not imply equal object identity in Chapter 1.

A call evaluates the callee expression, evaluates each argument, creates the function call's parameter bindings, and begins executing the body. A function can mutate an object reached through an argument or a closure, but the call does not grant access to rebind the caller's local variables. Chapter 1's argument-evaluation and parameter-binding rules remain in force.

2. Lexical environments and name resolution

JavaScript resolves an identifier according to the nesting of source scopes. A block containing `let`, `const`, or `class` declarations creates bindings for that block. A function call initializes parameter and local bindings. A module supplies module-scoped bindings. When an expression evaluates `rate`, JavaScript first seeks a matching binding in the current lexical environment, then follows the environment's outer link until it finds the binding or reaches the end of the chain.

Lexical scope is determined by where code is defined, not by which function later calls it. Passing a callback into another module does not transplant the callback's free names into the caller's scope.

Predict before running: Does the caller's `label` binding affect the callback?

```
const label = "module";

function makeReporter() {
  const label = "factory";
  return function report() {
    return label;
  };
}

function invoke(callback: () => string) {
  const label = "invoker";
  return callback();
}

console.log(invoke(makeReporter()));
```

Output:

```
factory
```

`report` was defined inside `makeReporter`, so its free use of `label` resolves through that invocation's environment. The binding named `label` inside `invoke` is not an enclosing binding for `report`, despite being active when `report` runs.

This rule differs from dynamic scoping, where callers would affect free-name resolution. It also differs from receiver lookup: `this.label` asks for a property on a receiver object, not for a lexical binding named `label`.

3. Closures retain bindings, not snapshots

A closure consists operationally of a function object and retained access to the lexical bindings its execution may use. Saying that a closure "captures a value" predicts incorrectly when the binding is later assigned a different value. The function reads the binding when it executes.

Predict before running: Does `readStatus` preserve `"queued"`, or observe the later assignment?

```
let status = "queued";
const readStatus = () => status;

status = "running";
console.log(readStatus());
```

Output:

```
running
```

The closure retains access to the `status` binding. Assignment changes that binding's current value, so the later read produces `"running"`. If a snapshot is required, code must create a different binding initialized with the current value:

```
const snapshot = status;
const readSnapshot = () => snapshot;
```

Object-valued bindings add Chapter 1 aliasing. If a closure reaches an object and another alias mutates it, the closure can observe the mutation. If the captured binding is rebound to a different object, the closure subsequently reaches the replacement. Predicting closure behavior therefore requires two questions: which binding is resolved, and which value or object does that binding currently provide?

A reachable closure provides a path to the environment and captured values that its body may observe. Returning or registering that closure can therefore extend the practical lifetime of state beyond the return of the function call that created it. This does not freeze the state, expose a particular memory layout, or promise when collection will occur; ECMAScript does not guarantee that any particular unreachable object will be collected on a schedule.

4. Separate environments and shared closure state

Each ordinary call creates a new set of parameter and local bindings. Two calls to a factory can return closures with independent state. Multiple closures created during one call can instead share the same bindings.

Predict before running: Which increments affect which reads?

```
function createCounter() {
  let value = 0;
  return {
    increment: () => {
      value += 1;
    },
    read: () => value,
  };
}

const first = createCounter();
const second = createCounter();

first.increment();
first.increment();
second.increment();

console.log(first.read(), second.read());
```

Output:

```
2 1
```

`first.increment` and `first.read` share the `value` binding from the first call. The corresponding functions in `second` share a different binding created by the second call. The function bodies may originate from the same source, but their retained environments differ.

Loop closures require the same binding analysis. A `for (let index = ...)` loop creates a per-iteration binding for `index`, so callbacks created on different iterations can read different values. A function-scoped `var index` supplies one shared binding, so callbacks executed after the loop read the final value. The important distinction is not old syntax versus new syntax; it is one shared binding versus distinct per-iteration bindings.

Closures are a form of encapsulation, not automatic isolation. If returned operations expose a mutable object, callers can still create aliases. If several closures share a binding, mutations by one operation are intentionally visible to the others. The API must specify that shared state transition.

5. Ordinary `this` comes from invocation

For an ordinary function, `this` is selected by the invocation operation. In `receiver.method()`, evaluation retains the base object and the call supplies it as `this`. Parentheses do not remove that relationship, but copying the function into an independent binding does.

Predict before running: Why does the extracted call fail in this ESM program?

```
const account = {
  owner: "Ada",
  describe() {
    return `owner:${this.owner}`;
  },
};

console.log(account.describe());

const describe = account.describe;
try {
  console.log(describe());
} catch (error) {
  console.log(error instanceof TypeError);
}
```

Output:

```
owner:Ada
true
```

The property access in `account.describe()` contributes both the function and the receiver. The binding call `describe()` supplies `undefined` as `this` in strict code; ECMAScript modules are strict. Reading `this.owner` therefore throws. The function did not permanently belong to `account` merely because it was stored there.

`call` invokes a function with an explicit receiver followed by separate arguments. `apply` supplies the receiver plus an array-like argument collection. `bind` does not invoke immediately; it creates a new callable object that stores a receiver and optionally leading arguments:

```
function describe(this: { owner: string }, prefix: string, suffix: string) {
  return `${prefix}${this.owner}${suffix}`;
}

const bound = describe.bind({ owner: "Grace" }, "owner:");
console.log(describe.call({ owner: "Ada" }, "owner:", "!"));
console.log(describe.apply({ owner: "Lin" }, ["owner:", "?"]));
console.log(bound("."));
```

Output:

```
owner:Ada!
owner:Lin?
owner:Grace.
```

The TypeScript `this` parameter is not a JavaScript parameter and is absent from emitted code. `call`, `apply`, and the bound wrapper supply runtime receivers; the annotation lets the checker reject incompatible receivers in checked source.

Constructor calls form another call category. `new Constructor()` creates a candidate instance, associates its prototype, calls the constructor with the candidate as `this`, and normally returns that object. Chapter 3 develops that process. Receiver selection here is runtime call semantics, not lexical name capture.

6. Arrow functions read lexical `this`

An arrow function does not create an ordinary `this` binding. A use of `this` in its body obtains the surrounding context's `this`, analogous to an identifier search continuing into an enclosing lexical environment. Consequently, `call`, `apply`, and `bind` cannot replace an arrow's `this` value.

Predict before running: Does calling the returned arrow as a standalone function lose the queue instance?

```
class Queue {
  label = "jobs";

  formatter() {
    return (count: number) => `${this.label}:${count}`;
  }
}

const format = new Queue().formatter();
console.log(format(3));
```

Output:

```
jobs:3
```

The arrow is created while `formatter` executes with the queue instance as `this`, so its later read uses that receiver. Extraction does not break it.

The same rule makes an arrow a poor choice for an object-literal method that expects the object at the call site to become the receiver:

```
function makeFormatter(this: { label: string }) {
  return {
    label: "object",
    format: () => this.label,
  };
}

const formatter = makeFormatter.call({ label: "enclosing" });
console.log(formatter.format()); // enclosing, not object
```

The property call does not make `formatter` the arrow's receiver. In a module, a top-level arrow would similarly retain top-level `this`, whose value is `undefined`; an object literal around the arrow does not supply a new one. Arrow functions also cannot be used as constructors and have no own `arguments` binding. These results follow from the absence of an ordinary receiver binding, not from arrows merely being abbreviated methods.

Class field arrows trade receiver stability for per-instance allocation and identity. A prototype method is normally one function shared through the prototype. An arrow field is initialized for each instance and closes over that instance's `this`. Choose according to extraction requirements and allocation policy rather than adopting one form universally.

7. Callbacks, APIs, and TypeScript checks

Passing a method as a callback is method extraction. An API that later calls `callback(value)` does not reconstruct the original receiver. Repair choices include binding the method, wrapping it in an arrow that performs a method call, defining a receiver-stable arrow field, or designing the callback as a receiver-free function.

TypeScript can express a callback's receiver expectations:

```
interface Row {
  value: number;
}

function inspect(this: Row) {
  return this.value;
}

inspect.call({ value: 4 });
// inspect(); // rejected: the required receiver is absent
```

The synthetic `this` parameter participates in checking but is erased from JavaScript. It neither binds nor validates the runtime receiver. Untyped JavaScript, an assertion, or another unchecked boundary can still call the emitted function incorrectly. Conversely, a callback type can declare `this: void` so the checker rejects a receiver-dependent function supplied as that callback.

Avoid hiding mutable closure state without a contract. A long-lived registry that retains a callback creates a reachability path to that callback's captured values. An event listener may therefore extend state and resource lifetime until application code unregisters it or removes another retaining path. Do not use garbage-collection timing as cleanup logic: ECMAScript deliberately leaves collection timing largely unguaranteed. Later chapters connect explicit unregistration to cancellation and cleanup.

8. Consequences for program design

Use lexical state when an operation should retain definition-time bindings. Use an ordinary method when an operation should act on a call-supplied receiver. Use an arrow when preserving an enclosing `this` is part of the function's contract. These choices select different state sources and should not be treated as stylistic synonyms.

Receiver-dependent APIs are fragile under extraction. Receiver-free functions are easier to pass through transformation pipelines. Closure-based APIs can prevent callers from naming private bindings but may conceal shared state and lifetime. Classes provide explicit instance organization, yet their prototype methods still depend on call form. The design questions are therefore concrete: which bindings or objects supply state, who may mutate them, which callbacks retain them, and what invocation operation supplies each receiver?

False models, repaired

- **"A closure stores the value as it was when the function was created."** A closure retains access to bindings; later reads can observe rebinding or object mutation.
- **"The caller's locals determine a callback's free variables."** Lexical source nesting determines the enclosing environments.
- **"A method permanently belongs to its object."** An ordinary function receives a receiver from the call form; extraction can remove it.
- **"Arrow functions are shorter methods."** An arrow has lexical `this`, cannot construct, and has no own `arguments`.
- **"TypeScript's `this` parameter binds the receiver."** It affects checking and is absent from emitted JavaScript.
- **"Closures make state private and therefore safe."** They can encapsulate names while still exposing aliases, races, unbounded retention, or invalid transitions.

Mastery questions

1. Predict the identity comparisons among a declared function, its alias, and two separately evaluated but textually identical arrows.
2. Construct a callback whose free name resolves differently under lexical and hypothetical dynamic scoping; derive JavaScript's result.

3. Show a closure observing both a rebinding and a mutation of a captured object-valued binding.
4. Explain why two functions returned by one factory call may share state while functions returned by two calls do not.
5. Predict callbacks created by `for (var index...)` and `for (let index...)`; name the exact binding difference.
6. Diagnose why `const send = client.send; send(message)` fails when `send` reads `this.transport`.
7. Repair the extracted method with `bind` and with an arrow wrapper; compare function identity and allocation.
8. Give a case where an ordinary method is preferable to an arrow field and a case where the arrow field's receiver stability is worth its per-instance function.
9. Explain what an explicit TypeScript `this` parameter proves and how emitted JavaScript can still be called incorrectly.
10. Design a closure-based API that does not leak its mutable state object through any returned value.
11. Identify the reachability path by which a registered callback can retain a resource wrapper after its original owner releases its reference.
12. Apply the chapter's lexical-capture and receiver-selection rules to a framework callback whose documentation supplies a receiver: what must be checked before replacing its ordinary function with an arrow?

The full answer workspace and lab specification are in `exercises/ch02.md`.

Implementation lab — Bounded Account Ledger

Implement `createLedger` in `examples/ch02/lab/starter.ts`. Each call must create independent closure state. Return receiver-free operations for credit, debit, balance lookup, and an immutable transaction snapshot. Do not expose the mutable transaction array. Extracting any returned operation into a standalone binding must preserve its behavior.

The lab must reject non-positive amounts and overdrafts without partially mutating state. Tests must establish independence between ledgers, shared state among one ledger's operations, snapshot isolation, and correct behavior after method extraction.

Presentation prompt

Give a 10–15 minute explanation covering function identity, lexical name resolution, binding capture, shared and separate closure environments, ordinary receiver selection, method extraction, arrow `this`, and the exact role of a TypeScript `this` parameter. Include one output prediction, one derivation from Chapter 1 aliasing, one false model, one lifetime boundary, and one unresolved question. The recording checklist is in `presentations/prompts/ch02.md`.

Ready-when test

Proceed only when, without notes, you can resolve every free name in a nested callback; predict rebinding and mutation visible through a closure; distinguish state shared within one factory call from state separated across calls; derive `this` from ordinary, method, bound, constructor, and arrow invocation; repair a receiver-loss bug in two ways; state exactly what TypeScript checked; implement and test `createLedger`; and present the chapter without treating lexical capture and receiver selection as one operation.

Chapter 3

Prototypes, Classes, and Object Organization

Chapter type: Acquisition and integration
Prerequisites: Chapters 1–2
Capabilities introduced: Predict prototype lookup, shadowing, construction, class-member placement, private-field access, `instanceof`, and composition/inheritance tradeoffs
Earlier chapters integrated: Identity, mutation, function objects, call forms, receivers, and closure-based encapsulation
Later chapters enabled: Collections, structural typing, runtime truth, domain modeling, and public API design
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch03/src/`, `examples/ch03/test/`, and `examples/ch03/lab/`

Compressed thesis

Ordinary JavaScript property access begins at the receiver object. If the receiver lacks the requested own property, the runtime follows the receiver's `[[Prototype]]` link and repeats the lookup until it finds the property or reaches `null`. Constructor functions and classes arrange prototype links so that many instances can find shared methods while retaining their own state. Prototype lookup finds a property value; the subsequent call form independently determines `this`.

`new Constructor(...)` invokes the constructor's `[[Construct]]` operation: for an ordinary base constructor, JavaScript creates a candidate object whose `[[Prototype]]` normally refers to `Constructor.prototype`, calls the constructor with that candidate as `this`, and returns either the constructor's explicit object result or the candidate. Class syntax uses these prototype relationships, but also supplies real semantics: class bodies are strict, class constructors reject calls without `new`, fields follow defined initialization order, private elements use runtime brands, and `extends` connects both instance and constructor inheritance.

TypeScript usually compares public class instance types by structure. Its `private` modifier restricts checked source access but normally emits an ordinary property; JavaScript `#private` elements are runtime-enforced branded slots and are not string-keyed properties. The canonical chapter preserves these shared technical relationships. Explanations required only after an observed reader failure belong in source-linked sections of the personal appendix.

1. Own properties, prototypes, and lookup

Every ordinary object has an internal `[[Prototype]]` slot containing another object or `null`. `Object.getPrototypeOf(value)` exposes that link. A property read such as `receiver.role` first checks `receiver` for an

own property named `role`. If none exists, the runtime repeats the search on `receiver.[[Prototype]]`; inherited lookup continues until a match is found or the next prototype is `null`.

An assignment such as `receiver.role = "admin"` normally creates or updates an own property on `receiver`; it does not overwrite a same-named data property on the prototype. The new own property **shadows** the inherited property. Deleting the own property can reveal the inherited value again. Accessors and proxies add boundaries because their setters or traps can intercept operations; the ordinary data-property case is the semantic spine here.

Predict before running: Which values are own, which are inherited, and what changes when the shared prototype is mutated?

```
const memberPrototype = {
  role: "reader",
  describe(this: { name: string; role: string }) {
    return `${this.name}:${this.role}`;
  },
};

const ada = Object.assign(Object.create(memberPrototype), { name: "Ada" });
const lin = Object.assign(Object.create(memberPrototype), { name: "Lin" });

ada.role = "admin";
memberPrototype.role = "member";

console.log(ada.describe());
console.log(lin.describe());
console.log(Object.hasOwn(ada, "role"), Object.hasOwn(lin, "role"));
```

Output:

```
Ada:admin
Lin:member
true false
```

Both objects inherit `describe`. Lookup finds the same function on `memberPrototype`, but `ada.describe()` and `lin.describe()` supply different receivers. Inside the function, `this.name` starts lookup from the receiver, and `this.role` finds either `ada`'s shadowing own property or the prototype's mutated property. Chapter 2's receiver rule therefore remains separate from prototype lookup: lookup selects a function value; method-call syntax supplies `this`.

Mutating a shared prototype can affect every object that still inherits the changed property. Existing shadowing properties remain decisive because lookup stops at the first match. Replacing an object's `[[Prototype]]` with `Object.setPrototypeOf` is possible but can invalidate performance assumptions and makes relationships harder to inspect; establish prototype links during object construction when practical.

2. Constructor objects, `.prototype`, and `new`

`Object.getPrototypeOf(instance)` and `Constructor.prototype` answer different questions. The first reads an internal link from a particular object. The second reads an ordinary property on a constructor object. Construction commonly makes them refer to the same prototype object:

```
function Ticket(this: { id: string }, id: string) {
  this.id = id;
}

const ticket = new (Ticket as unknown as new (id: string) => { id: string })("T1");
console.log(Object.getPrototypeOf(ticket) === Ticket.prototype);
```

The output is `true`. `Ticket.prototype` is not the prototype of the function object `Ticket`; `Object.getPrototypeOf(Ticket)` normally reaches `Function.prototype`. The constructor's `.prototype` property is instead the object selected as the new instance's `[[Prototype]]` by ordinary construction.

For an ordinary base constructor, derive `new C(args)` in four steps: evaluate `C` and the arguments; create a candidate object using the object in `C.prototype` when that value is suitable; call `C` with the candidate as `this`; return an explicit object result if the constructor supplies one, otherwise return the candidate. An explicit primitive return does not replace the candidate. Derived class construction has additional `super()` rules, so this four-step account is not a substitute for every inheritance detail.

Predict before running: Which result keeps `Candidate.prototype`, and which constructor assignment remains visible?

```
function Candidate(this: { source: string }, source: string) {
  this.source = `candidate:${source}`;
  if (source === "override") return { source: "replacement" };
  return undefined;
}

const Constructable = Candidate as unknown as new (source: string) => {
  source: string;
};
const ordinary = new Constructable("ordinary");
const replaced = new Constructable("override");

console.log(ordinary.source, ordinary instanceof Constructable);
console.log(replaced.source, replaced instanceof Constructable);
```

Output:

```
candidate:ordinary true
replacement false
```

The explicit object return replaces the candidate, including its prototype relationship. This boundary explains why "`new` always returns the object it first creates" is inaccurate, without making constructor return overriding a normal design recommendation.

3. What class syntax places where

A class declaration evaluates to a constructor object. Public instance fields become own properties of each instance. Ordinary instance methods are installed on the object at `Constructor.prototype`, so instances normally share one function identity. Static fields and methods are properties of the constructor object itself, not properties of instances or of `Constructor.prototype`.

Predict before running: Which properties are own, which method identities are shared, and where does the static member exist?

```
class Counter {
  static category = "counter";
  value = 0;

  increment() {
    this.value += 1;
  }
}

const first = new Counter();
const second = new Counter();

console.log(Object.hasOwn(first, "value"), Object.hasOwn(first, "increment"));
console.log(first.increment === second.increment);
console.log(Object.hasOwn(Counter, "category"), "category" in first);
```

Output:

```
true false
true
true false
```

An arrow-valued instance field would instead create an own function for each instance. That can preserve an enclosing instance receiver under extraction, as Chapter 2 explained, but it changes function identity and allocation. Ordinary prototype methods remain receiver-dependent: `const increment = first.increment; increment()` loses the instance receiver even though lookup originally found the method through `Counter.prototype`.

Classes are therefore more than cosmetic constructor notation. Their bodies run in strict mode; invoking a class constructor as an ordinary function throws; method definitions have class-specific installation semantics; `extends` establishes prototype relationships; and public and private fields follow specified initialization operations. A class name also exists as a runtime value, unlike an interface or type alias that disappears from emitted JavaScript.

4. Initialization order is observable

For a base class, instance fields initialize before the constructor body. During derived construction, the base fields initialize and the base constructor runs as part of `super()`; derived fields then initialize before the remaining derived constructor statements. A base constructor that calls an overridable method or reads a field that a derived class will later initialize can therefore observe incomplete derived state.

Predict before running: What does the base constructor observe, and can the class be called without `new`?

```
const log: string[] = [];

class BaseLabel {
  label = "base";
  constructor() {
    log.push(this.label);
  }
}

class DerivedLabel extends BaseLabel {
  label = "derived";
}

const instance = new DerivedLabel();
let plainCallThrew = false;
try {
  (DerivedLabel as unknown as () => unknown)();
} catch (error: unknown) {
  plainCallThrew = error instanceof TypeError;
}

console.log(log.join(","), instance.label, plainCallThrew);
```

Output:

```
base derived true
```

The base constructor runs before the derived field initializer replaces `label`. This is a reason to avoid invoking overridable instance behavior from constructors when that behavior may require derived fields. The thrown ordinary call is runtime enforcement by the class constructor, not a TypeScript diagnostic.

5. Runtime private brands and TypeScript access control

A declaration such as `#secret` creates a private name scoped to one class definition. Instances initialized by that class carry the corresponding private element. The private element is not an ordinary property keyed by the string `"#secret"` or `"secret"`: property enumeration, bracket access, proxies, and `Object` reflection do not expose it. Access through `value.#secret` checks the class's runtime brand and throws `TypeError` when the receiver lacks it. Inside the declaring class, `#secret in value` performs the brand check without reading the value.

TypeScript `private secret`, in contrast, normally emits a property named `secret`. The checker rejects selected source accesses and uses the declaration origin when comparing class types containing private or protected members, but ordinary JavaScript reflection can still observe the emitted property. TypeScript `private` is static access control; `#secret` is JavaScript runtime privacy.

Predict before running: Why can one hidden value be read through reflection while the other cannot, and why does the forged object pass only one check?

```
class Vault {
  #secret = "sealed";
  reveal() {
    return this.#secret;
  }
  static hasBrand(value: object) {
    return #secret in value;
  }
}

class SoftVault {
  private secret = "visible-at-runtime";
}

const vault = new Vault();
const forged = Object.create(Vault.prototype) as Vault;
const soft = new SoftVault();

console.log(Vault.hasBrand(vault), Vault.hasBrand(forged));
console.log(Reflect.get(soft, "secret"));
console.log(forged instanceof Vault);
```

Output:

```
true false
visible-at-runtime
true
```

The forged object has the expected prototype chain but lacks the private brand. Calling `forged.reveal()` would throw. A prototype relationship and a private-field brand are therefore distinct runtime facts.

For public members, TypeScript generally compares class instance types structurally: an independently declared object or class with compatible public members can be assignable without sharing a constructor or prototype. TypeScript private and protected members introduce an origin requirement into compatibility, but that checker rule still does not perform `instanceof` or validate runtime provenance.

6. What `instanceof` establishes

In its ordinary function case, `value instanceof Constructor` searches `value`'s prototype chain for the current object stored in `Constructor.prototype`. It does not compare TypeScript types, compare public shape, or prove that `Constructor` originally allocated the object. Reassigning `Constructor.prototype` can make old and new instances produce different results, and `Object.create(Constructor.prototype)` can create a match without running the constructor.

The operation also has two important boundaries. A constructor can customize the result through `Symbol.hasInstance`. Separate realms have separate built-in constructor and prototype identities, so a value from another realm may fail an `instanceof` check against the local built-in; `Array.isArray` is the cross-realm test for arrays. Use `instanceof` when the prototype-chain or customized relationship is the intended evidence, not as a universal runtime type test.

7. Inheritance and composition organize different relationships

Inheritance arranges delegated property lookup and supports substitution when the derived object genuinely satisfies the base contract. It is useful when shared operations act through a stable receiver interface and derived instances should participate in the base prototype relationship. Its costs include coupling to initialization order, overridable behavior, shared mutable prototypes, and base-class assumptions that subclasses can violate.

Composition stores or closes over collaborators instead. A `QuotaCounter` can contain an audit sink rather than inherit from one; replacement then changes one dependency without asserting that the counter *is* a sink. Composition does not eliminate identity, mutation, or receiver issues. It makes the collaboration an explicit property, constructor argument, or captured binding, whose lifetime and alias policy still require design.

Choose by relationship, not slogan. Prefer inheritance when prototype participation and substitutability are actual requirements. Prefer composition when one object uses a replaceable capability or when several independent axes of behavior would produce a fragile hierarchy. In either case, specify who owns mutable state, which methods depend on receivers, and which invariants constructors establish.

Consequences for program design

Prototype methods make shared behavior cheap in identity terms, but a shared function can still mutate each receiver's own state because the call supplies that receiver. Public fields expose ordinary properties and therefore expose aliasing and mutation policies from Chapter 1. Private fields prevent external property access at runtime, but they do not make referenced objects deeply immutable or automatically protect invariants across asynchronous operations.

Avoid treating a class annotation as proof of construction history. If an API needs runtime provenance, use evidence supplied at runtime: a reliable prototype check, a private brand check exposed by the class, or explicit parsing and validation. If an API only needs a capability, a structural interface can accept more implementations and avoid unnecessary coupling to one constructor identity.

False models, repaired

- **"The prototype is the constructor's `.prototype` property."** Every object has an internal `[[Prototype]]`; a constructor separately has an ordinary `.prototype` property used during construction.
- **"Inherited methods run with the prototype as `this`."** Lookup may find the function on a prototype, while method-call syntax supplies the original receiver.
- **"Classes are only syntax sugar."** They use prototypes and also impose strict bodies, construction-only calls, field initialization, private names, and inheritance semantics.
- **"Class fields and methods live in the same place."** Public instance fields are own properties; ordinary instance methods are normally shared through the prototype.
- **"TypeScript `private` is runtime privacy."** The checker restricts source access; JavaScript `#private` elements enforce branded access while executing.
- **"`instanceof` proves which constructor ran."** It normally proves a current prototype-chain relationship and can be customized.
- **"Composition is always better than inheritance."** Each represents a different relationship and carries different coupling, substitution, and state-lifetime costs.

Mastery questions

1. Predict a read when an object, its prototype, and its prototype's prototype all define the same property; then predict the result after deleting the object's own property.
2. Two instances inherit one method. Prove which function identities are equal, then explain why the method can mutate different own fields on each receiver.
3. Derive the difference between `Object.getPrototypeOf(instance)`, `Constructor.prototype`, and `Object.getPrototypeOf(Constructor)`.
4. Predict the result of a constructor that mutates its candidate receiver and then returns a different object. Which identity and prototype survive?
5. Transform a per-instance arrow field into a prototype method. State the receiver and function-identity consequences.
6. Diagnose a base constructor that calls an overridden method before derived fields initialize. Give two repairs with different API shapes.

7. Construct an object that passes ordinary `instanceof C` without running `C`, then explain why a `#private` access can still fail.
8. Compare TypeScript `private` and JavaScript `#private` by syntax, checker restrictions, emitted/runtime representation, reflection, and subclass access.
9. Show two unrelated classes whose public instance types are structurally assignable. Add a private member and explain the changed checker result without claiming runtime validation.
10. Explain one cross-realm failure of `instanceof` and select an appropriate check for an array crossing that boundary.
11. Design an inheritance relationship where substitutability and prototype participation are useful; identify the constructor-order risk.
12. Redesign the same capability using composition; identify the collaborator's identity, lifetime, and mutation policy.
13. A library mutates `Service.prototype.run` after instances already exist. Predict which instances observe the change and which own-property shadow would prevent it.
14. Transfer the chapter rules to a built-in such as `Map`: distinguish the constructor object, its `.prototype`, an instance's own state, inherited methods, and runtime brand requirements.

The full answer workspace and lab specification are in `exercises/ch03.md`.

Implementation lab — Branded Quota Counter

Implement `QuotaCounter` in `examples/ch03/lab/starter.ts`. Each instance owns an ID and private usage state, shares ordinary methods through the class prototype, and delegates audit recording to a composed sink. Reject invalid consumption without partial mutation. Expose a static private-brand check without exposing the stored count. Tests must establish values, own/prototype placement, method identity, brand behavior, and collaborator calls.

The evaluator includes a deliberately faulty counter whose public `used` property exposes mutable state and whose arrow-field operations allocate new functions per instance. Inspect it only after completing your own implementation and predictions.

Presentation prompt

Give a 10–15 minute explanation in your own words covering ordinary property lookup, shadowing, the distinction between `[[Prototype]]` and constructor `.prototype`, derivation of `new`, class member placement, initialization order, runtime private brands, TypeScript structural class types, and `instanceof` boundaries. Include one output prediction, Chapter 2's lookup/receiver connection, one composition/inheritance tradeoff, one false model, and one unresolved point. The recording checklist is in `presentations/prompts/ch03.md`.

Ready-when test

Proceed only when, without notes, you can trace property lookup through an unfamiliar chain; separate lookup from receiver selection; derive ordinary `new`, including object-return overriding; locate public fields, prototype methods, and static members; predict base/derived initialization; distinguish TypeScript `private` from JavaScript private brands; state what ordinary `instanceof` establishes and where it fails; defend an inheritance or composition choice; implement and test `QuotaCounter`; and present the chapter without confusing static assignability with runtime provenance.

Chapter 4

Collections, Iteration, and Transformation

Chapter type: Acquisition
Prerequisites: Chapters 1–3
Capabilities introduced: Select collection equality and access semantics; predict iterator state, generator demand, callback order, copy identity, and mutation effects; choose explicit loops or transformation pipelines from operational requirements
Earlier chapters integrated: Identity and shallow copying from Chapter 1; callbacks, closures, and receivers from Chapter 2; prototypes, classes, and method lookup from Chapter 3
Later chapters enabled: Aliasing and state-transition analysis, bounded asynchronous workflows, generic container APIs, and trusted data pipelines
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch04/src/`, `examples/ch04/test/`, and `examples/ch04/lab/`

Compressed thesis

An `Array` is an ordered, integer-indexed sequence whose elements may contain aliases. A `Map` associates arbitrary key values with values, and a `Set` retains at most one occurrence of each value; `Map` and `Set` use SameValueZero to distinguish entries, so object keys and object members are distinguished by identity while `NaN` equals `NaN` for this purpose. Copying an array, map, or set creates a new outer collection but normally preserves the identities of stored objects.

An **iterable** supplies iterator objects through `[Symbol.iterator]()`. An **iterator** owns traversal state and advances when `next()` is called. `for...of`, array spread, array destructuring, `Array.from`, and collection constructors are consumers of iterable values. A generator-function call returns a suspended iterator object; the JavaScript runtime executes the generator body only as a consumer requests values. TypeScript's `Iterable<T>` and `Iterator<T>` describe operations accepted by the checker, but those types neither install `[Symbol.iterator]` nor prove that an untrusted runtime value follows the protocol.

Array `map`, `filter`, and `reduce` execute their callbacks eagerly and in index order over the source indices selected by each method's algorithm. Their callbacks may mutate reachable state, throw, or depend on closure state; pipeline syntax therefore proves neither purity nor speed. Explicit loops make state, early termination, and allocation policy visible. Pipelines can make independent transformations easier to recognize. Choose from those concrete requirements. This canonical chapter preserves the shared technical spine; explanations needed only after an observed reader failure belong in source-linked sections of the personal appendix.

1. Collection choice fixes observable semantics

Collection choice determines how code addresses entries and decides whether a requested entry is already present. Arrays expose an ordered sequence through nonnegative integer indices and a `length`; an array can contain the same object in several positions. `Map` accepts keys of any JavaScript language type. `Set` stores values without a separate key. Both keyed collections preserve insertion order during their standard iteration.

`Map` key comparison and `Set` membership use SameValueZero. For ordinary primitive values, this mostly agrees with `===`; the important numerical differences are that `NaN` matches itself and negative zero matches positive zero. Object values match only the same identity. Two separately created `{ id: "A" }` objects are therefore distinct map keys and distinct set members even when their properties are equal. TypeScript generic arguments such as `Map<UserId, User>` constrain checked calls; they do not replace runtime key comparison or validate values arriving from unchecked code.

Predict before running: How many map and set entries remain, and can an equal-looking object retrieve the stored value?

```
const storedKey = { id: "A" };
const lookalikeKey = { id: "A" };

const byObject = new Map<object, string>();
byObject.set(storedKey, "stored");
byObject.set(storedKey, "replaced");

const unique = new Set([NaN, NaN, 0, -0, storedKey, lookalikeKey]);

console.log(byObject.size, byObject.get(storedKey));
console.log(byObject.has(lookalikeKey));
console.log(unique.size);
```

Output:

```
1 replaced
false
4
```

The second `set` finds the existing object identity and replaces its associated value. The lookalike is a different identity. The set contains one `NaN`, one zero, and two objects. If domain equality should depend on `id`, application code must instead use the string ID as the key or perform an explicit domain comparison; neither `Map` nor `Set` recursively compares object properties.

2. Collection copies preserve stored identities

Array spread consumes the source array's iterable and creates a new array containing the produced element values. `new Map(existingMap)` consumes key-value entries and creates new map storage. `new Set(existingSet)` consumes values and creates new set storage. These operations separate the outer containers; they do not recursively recreate object keys, elements, or values.

```
const key = { queue: "priority" };
const job = { id: "J1", status: "queued" };

const originalArray = [job];
const copiedArray = [...originalArray];
const originalMap = new Map([[key, job]]);
const copiedMap = new Map(originalMap);
const originalSet = new Set([job]);
const copiedSet = new Set(originalSet);

copiedArray[0]!.status = "running";

console.log(originalArray === copiedArray);
console.log(originalArray[0] === copiedArray[0]);
console.log(originalMap === copiedMap);
console.log(copiedMap.get(key) === job);
console.log(originalSet === copiedSet);
console.log(copiedSet.has(job));
console.log(originalMap.get(key)!.status);
```

Output:

```
false
true
false
true
false
true
running
```

Replacing `copiedArray[0]` would change one element in the copied array only. Mutating `copiedArray[0].status` changes the shared job object. A copied map similarly preserves both the object-key identity and the object-value identity; a copied set preserves the identity of each object member. Chapter 1's rule applies at every collection boundary: ask which container was created, then inspect each stored value for an access path to an existing identity.

3. Iterables create iterators; iterators advance

The iterable protocol requires a callable property keyed by `Symbol.iterator`. Calling that method must return an iterator. The iterator protocol requires `next()`, which returns an object describing either a produced value or completion. Traversal position belongs to the iterator object, not abstractly to the underlying array, map, set, or custom collection.

```
const values = [10, 20];
const cursor = values[Symbol.iterator]();

console.log(cursor.next());
console.log(cursor.next());
console.log(cursor.next());
console.log(cursor.next());

const fresh = values[Symbol.iterator]();
console.log(fresh.next());
```

Output:

```
{ value: 10, done: false }
{ value: 20, done: false }
{ value: undefined, done: true }
{ value: undefined, done: true }
{ value: 10, done: false }
```

The first iterator is exhausted and stays exhausted. The array remains reiterable because each `[Symbol.iterator]()` call creates a fresh array iterator. Some iterator objects are also iterable: their `[Symbol.iterator]()` method returns themselves. A generator object has this single-use shape. "Iterable" therefore does not imply repeatable traversal; repeatability depends on whether the iterable produces fresh independent iterators.

`for...of`, array spread, array destructuring, argument spread, `Array.from`, `new Map(iterable)`, and `new Set(iterable)` request and advance iterators. Object spread is different: it copies selected own enumerable properties and does not consume the iterable protocol. A `for...of` loop that exits abruptly with `break` asks a conforming iterator to close through its `return` method when one exists. Iterator cleanup becomes important when a generator holds a resource in a `try...finally` block.

`Iterable<T>` and `Iterator<T>` are TypeScript structural types. The checker uses their declared members to accept or reject source operations. The emitted JavaScript still needs a callable `[Symbol.iterator]` and a conforming `next()` at execution. A type assertion can suppress a diagnostic without adding either method; validation or trusted construction is required at an external boundary.

4. Generators suspend execution and expose demand

A generator declaration defines a function. Calling that function creates a generator object without executing the body. The first `next()` starts execution and runs until `yield`, `return`, or a thrown value. Each later `next()` resumes after the prior `yield`. The generator object carries both iterator state and the iterable method that returns itself.

Predict before running: When does each trace entry appear?

```
const trace: string[] = [];

function* produce(): Generator<number, void, unknown> {
  trace.push("start");
  yield 1;
  trace.push("middle");
  yield 2;
  trace.push("end");
}

const produced = produce();
console.log(trace);
console.log(produced.next(), trace);
console.log(produced.next(), trace);
console.log(produced.next(), trace);
```

Output:

```
[]
{ value: 1, done: false } [ 'start' ]
{ value: 2, done: false } [ 'start', 'middle' ]
{ value: undefined, done: true } [ 'start', 'middle', 'end' ]
```

The function call allocates suspended traversal state; consumer demand advances the body. This permits incremental transformations and finite consumption of an unbounded source. Laziness is not automatic merely because code uses functions: array `map` is eager, while a generator performing the analogous transformation can defer work until `next()`.

5. Array transformations are eager callback programs

`map`, `filter`, and `reduce` call ordinary JavaScript functions synchronously. For a normal dense array, each method visits indices in ascending order. `map` creates an output array and assigns one transformed result per visited source index. `filter` creates an output array containing values whose callback results are truthy. `reduce` carries an accumulator from one callback invocation to the next. The calls finish or throw before the method returns.

The methods determine the source length before callback iteration begins, and they test whether each scheduled index is present when its turn arrives. A callback that pushes beyond the original length does not extend the current traversal. A callback that changes a not-yet-visited element can change the value observed later. Deleting or shifting elements can make scheduled indices absent or change which value occupies them. Mutation during traversal is therefore governed by exact operation order, not by a snapshot abstraction.

Predict before running: Which values reach the callback, and is the pushed element transformed?

```
const source = [1, 2, 3];
const seen: number[] = [];

const doubled = source.map((value, index) => {
  seen.push(value);
  if (index === 0) {
    source[1] = 20;
    source.push(4);
  }
  return value * 2;
});

console.log(seen);
console.log(doubled);
console.log(source);
```

Output:

```
[ 1, 20, 3 ]
[ 2, 40, 6 ]
[ 1, 20, 3, 4 ]
```

`map` itself does not assign into `source`, but the callback does. The callback may also mutate captured objects, increment counters, perform I/O, or throw. Pipeline notation neither prevents these effects nor makes the callback pure. Sparse-array details are deliberately secondary here, but they explain why "exactly once for every integer below length" is too broad: iterative array methods generally skip absent properties.

6. Explicit loops and pipelines expose different facts

A `filter(...).map(...)` pipeline names two independent transformations and can make their composition immediately visible. It also traverses eagerly and ordinarily allocates the filtered intermediate array before `map` begins. A single explicit loop can combine selection and transformation, keep counters or budgets visible, push directly into one result, and `break` once enough output exists. A generator pipeline can defer work and avoid an intermediate array, but it introduces single-use traversal state and moves execution to consumer demand.

`reduce` is useful when an accumulator transition is the central operation. It is a poor compression when the callback hides several unrelated counters, early-stop rules, and mutation effects. Conversely, an explicit loop filled with independent element-wise expressions may obscure a simple filter-and-map relation. Neither syntax is automatically faster: allocation count, callback calls, source representation, early termination, runtime optimization, and workload determine cost. Measure a relevant workload after choosing code whose state transitions can be reviewed correctly.

When processing an iterable that may be large or unbounded, `[...]`, `Array.from`, and eager array methods require materialization before later array operations can run. An explicit loop or lazy iterator can bound consumption. If the source represents a file cursor, database cursor, or later asynchronous stream, early termination also controls resource lifetime. Chapter 8 will make cleanup and cancellation explicit; Chapter 9 will add concurrency limits.

7. Integration with functions and object organization

Collection copying extends Chapter 1's alias graph: a new container can still expose old element, key, and value identities. Callback-driven transformations extend Chapter 2: each callback is a function value, resolves free names through its retained lexical environment, and may mutate captured state. Passing an extracted receiver-dependent method as `array.map(object.method)` loses the original receiver unless the method is bound or wrapped; array iteration does not reconstruct it.

Array, map, set, and iterator operations are methods found through prototype lookup from Chapter 3. Built-in collection methods also require the receiver to carry the appropriate internal collection state, so borrowing `Map.`

`prototype.get` for a plain object fails. A custom class can implement `[Symbol.iterator]` as a prototype method and return a new iterator for each traversal. TypeScript can check that instances are assignable to `Iterable<T>`, but static assignability still does not prove that an external runtime object has a working method.

Consequences for program design

Choose `Map` when arbitrary keys and keyed lookup are part of the contract, `Set` when uniqueness under SameValueZero is the contract, and `Array` when positional order and sequence operations are central. If domain equality differs from object identity, normalize a stable domain key explicitly.

Document whether an API consumes an iterable once, may stop early, preserves element identities, or returns a fully materialized array. Those choices determine whether callers may reuse a cursor, whether generators run completely, which resources close, and whether mutations remain shared through returned collections.

False models, repaired

- **"A copied collection contains copied objects."** The outer collection is new; stored object identities normally remain shared.
- **"Equal-looking objects are the same map key."** Object keys match by identity under SameValueZero.
- **"Iterable and iterator are synonyms."** An iterable produces an iterator; an iterator owns `next()` state. One object may implement both roles.
- **"An iterable can always be traversed repeatedly."** A single-use iterable iterator can return itself and remain exhausted.
- **"Calling a generator runs its body."** Calling creates a suspended generator object; `next()` begins or resumes execution.
- **"`map` is lazy and pure."** Array `map` invokes callbacks eagerly, and callbacks can perform effects.
- **"Functional pipelines are automatically faster."** Pipelines may allocate intermediates and traverse more elements; actual cost depends on the operations and workload.
- **"A TypeScript `Iterable<T>` annotation makes a value iterable."** The checker accepts declared operations; the runtime still requires the protocol methods.

Mastery questions

1. Predict the size and lookups of a `Map` given repeated `NaN`, positive and negative zero, one object alias, and one equal-looking object.
2. Copy an array of map values, a map with object keys, and a set of objects. Draw the new outer identities and every preserved stored identity.
3. Given an iterable whose `[Symbol.iterator]()` returns a fresh cursor and a generator object whose method returns itself, predict the result of two traversals of each.
4. Manually call `next()` through completion, then call it again. State which object contains the traversal position and what remains reusable.
5. Explain how array destructuring, array spread, `Array.from`, and `for...of` depend on the same protocol while object spread does not.
6. Construct a generator trace proving that its body does not execute at function-call time and that later statements run only after additional demand.
7. Predict a `map` callback that mutates a later element and pushes one element. Derive which values are visited from saved length and per-index lookup.
8. Diagnose a pipeline whose callback closes over and mutates a shared counter. Which chapter rules disprove the assertion that the pipeline is pure?
9. Rewrite `filter(...).map(...)` as one explicit loop. Compare allocation, callback count, early termination, and visibility of state without asserting which is universally faster.
10. Design an API accepting `Iterable<Job>` that consumes at most five accepted jobs. State whether it permits a single-use iterator and what it promises about overconsumption.

11. Diagnose an extracted instance method passed to `map` when the method reads `this.rate`; give two repairs and compare their function identities.
12. An untrusted value is asserted as `Iterable<Order>`. Explain what TypeScript checked, what the assertion suppressed, and which runtime failures remain possible.

The full answer workspace and lab specification are in `exercises/ch04.md`.

Implementation lab — Bounded Selection from an Iterable

Implement `selectWithinBudget` in `examples/ch04/lab/starter.ts`. Consume an `Iterable<T>` in encounter order. Reject candidates that fail the supplied predicate. Accept qualified candidates while their nonnegative cost fits the remaining budget, and stop before consuming further input once the item limit is reached. If the next qualified candidate would exceed the budget, stop without accepting it. Return the accepted items, examined count, and accepted total cost.

Do not materialize the iterable. The source may be large, unbounded, or backed by a resource. Preserve accepted item identities; the result array is a new outer container. Tests must prove bounded demand, generator closing on early exit, encounter order, budget handling, and zero-limit nonconsumption. The evaluator's faulty implementation eagerly spreads the entire source before selecting, so it returns plausible values while violating the consumption contract.

Presentation prompt

Give a 10–15 minute explanation covering collection access and equality, outer-container copying, iterable versus iterator, exhaustion and repeatability, generator demand, eager array callbacks, mutation during traversal, and the loop-versus-pipeline decision. Include one output prediction, one derivation from Chapters 1 or 2, one false model, one external-boundary failure, and one unresolved question. The recording checklist is in `presentations/prompts/ch04.md`.

Ready-when test

Proceed only when, without notes, you can select an array, map, or set from its equality and access contract; identify preserved identities after a collection copy; distinguish an iterable from an iterator; predict exhaustion, fresh traversal, generator demand, and callback mutation order; compare a loop, eager pipeline, and generator by allocation and termination; state exactly what TypeScript checked; implement and test bounded selection without overconsumption; and present the chapter without treating pipeline syntax as proof of purity or performance.

Chapter 5

Aliasing, State Transitions, and Local Reasoning

Chapter type: Integration and emergent
Prerequisites: Chapters 1–4
Capabilities introduced: Trace hidden sharing across objects and collections; specify state ownership, mutation surfaces, snapshots, invariants, commit points, and observation timing; implement rejection-safe in-memory transitions
Earlier chapters integrated: Object identity and nested aliases; closure-retained bindings and receiver-dependent methods; prototype-shared behavior and private state; collection-held identities and eager or lazy transformations
Later chapters enabled: Event-loop interleavings, asynchronous state, state machines, domain modeling, transactions, and distributed correctness
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch05/src/`, `examples/ch05/test/`, and `examples/ch05/lab/`

Compressed thesis

Local reasoning is possible when a reader can determine an operation's result from the operation, its explicit inputs, and a bounded set of owned state. Hidden sharing breaks that boundary: a nested object, closure-retained binding, receiver field, prototype property, collection member, or callback can supply an unlisted path by which earlier or distant code changes a later observation. A new outer object, a `readonly` view, a private field, or a transformation pipeline restricts particular operations; none automatically proves that all reachable state is isolated.

A predictable state transition names its state owner, admits mutation through a narrow operation, validates every condition before commit, and specifies which identities a result preserves or replaces. Its snapshot contract says whether callers receive a live alias, a detached copy, or an immutable runtime value. Its observation contract says when committed state becomes visible. Mutation remains defensible when persistent identity is the abstraction and these boundaries are narrow; an immutable update remains unsafe when it preserves an undocumented mutable alias.

Within one ordinary synchronous JavaScript job, another queued job does not preempt a running function. That scheduling fact does not undo partial writes, prevent reentrant observation through a synchronously called callback, persist data across process failure, or create a database transaction. This canonical chapter retains the shared discipline. If a reader cannot reconstruct one term, relationship, prediction, procedure, or boundary, the smallest repair belongs in a personal appendix rather than an expansion of this chapter.

1. Local reasoning requires a bounded state dependency

Consider a function named `price(order)`. If its result depends only on the supplied order and named pricing rules, the call has a small reasoning boundary. If it also reads a closure-retained multiplier, a mutable map entry, the receiver's private counter, and a callback-updated cache, the signature no longer exposes the complete dependency set. The function can still be correct, but a reader must locate every retained binding and reachable object before predicting the result.

The relevant structure is a directed alias graph. Bindings, properties, closure environments, private fields, array elements, and map values can all provide paths to object identities. Local reasoning fails when an operation appears to own or copy a region but another undisclosed path reaches a mutable node in that region. The problem is not mutation alone. The problem is a mismatch between the boundary a reader is invited to inspect and the boundary that can actually change the result.

State ownership is therefore an API fact, not a TypeScript inference. An owner has the authority and responsibility to preserve an invariant. A class with a `#state` field can prevent outside syntax from naming that field, and a closure can prevent callers from naming its local binding. Either owner can still leak an alias by returning the stored map, an element object, or a shallow copy containing those elements.

2. New containers can preserve the dangerous identity

Suppose an update creates both a new root object and a new `Map`. Those two allocations do not recursively recreate orders stored as map values or line arrays stored inside each order.

Predict before running: Which identities remain shared, and what quantity is visible through `original`?

```
const original = {
  orders: new Map([
    ["order-1", { id: "order-1", lines: [{ sku: "book", quantity: 1 }] }],
  ]),
};

const updated = {
  ...original,
  orders: new Map(original.orders),
};

updated.orders.get("order-1")!.lines[0]!.quantity = 2;

const originalOrder = original.orders.get("order-1")!;
const updatedOrder = updated.orders.get("order-1")!;

console.log(original === updated);
console.log(original.orders === updated.orders);
console.log(originalOrder === updatedOrder);
console.log(originalOrder.lines === updatedOrder.lines);
console.log(originalOrder.lines[0]!.quantity);
```

Output:

```
false
false
true
true
2
```

Object spread replaced the root identity. The `Map` constructor replaced the collection identity while inserting the same key and value objects. The order and its line array therefore remain shared. Calling the operation an "immutable update" describes an intention but does not establish it. The copy policy must identify every node recreated on a changed path and every preserved node whose future mutation remains possible.

TypeScript `readonly` can reject `quantity = 2` through a readonly-typed path, but a writable alias can still perform the assignment. `Object.freeze` can reject selected runtime changes to one frozen object, but freezing the map wrapper or root does not recursively freeze stored orders. Copying and restrictions on one access path reduce available operations; alias analysis determines whether another path remains.

3. Retained state makes observation time part of the input

A closure reads the current value of its retained binding when invoked. A receiver-dependent method reads the state of the object supplied by the call. Both operations can therefore return different results for identical explicit arguments at different observation times.

Predict before running: Why does the second quote change, why do two instances share a method function but not a counter, and why does extraction fail?

```
function createPricer() {
  let factor = 1;
  return {
    quote: (base: number) => base * factor,
    setFactor: (next: number) => { factor = next; },
  };
}

class Sequence {
  #value = 0;
  next() {
    this.#value += 1;
    return this.#value;
  }
}

const pricer = createPricer();
console.log(pricer.quote(10));
pricer.setFactor(3);
console.log(pricer.quote(10));

const left = new Sequence();
const right = new Sequence();
console.log(left.next === right.next);
console.log(left.next(), left.next(), right.next());
```

Output:

```
10
30
true
1 2 1
```

The two pricer operations share one factory invocation's `factor` binding. The class installs one `next` function on `Sequence.prototype`, while each constructed instance carries its own private value. The call `left.next()` supplies `left` as receiver. Extracting `const next = left.next; next()` supplies no instance receiver in this strict ESM program, so private-field access throws `TypeError`.

Closures and private fields provide name-level encapsulation. They do not by themselves supply snapshot semantics, transition validity, or freedom from sharing. Several closures can intentionally mutate one binding. Several methods can intentionally mutate one receiver. A private field can contain a public mutable object that a caller also holds. The contract must state whether a later read is intended to observe the latest committed state or a value detached at an earlier time.

4. Collections and pipelines do not certify isolation

An array, map, or set is both an owner of collection structure and a holder of element, key, or value identities. Copying the outer collection separates additions, removals, and replacements in that container. It does not separate mutations to stored objects. A snapshot implemented as `[...records.values()]` is a new array of aliases, not a detached record snapshot.

A transformation pipeline similarly identifies evaluation order and result allocation, not purity. `map` and `filter` invoke callbacks synchronously and eagerly. Those callbacks may read closure state, mutate element objects, call receiver-dependent methods, or update an audit collection. A generator defers callback-like work until demand, which changes observation timing but does not remove effects. The word "pipeline" therefore supplies no ownership or mutation guarantee.

Snapshot policy should answer three concrete questions. Which outer containers are new? Which stored object identities are preserved? Which runtime operation prevents or permits later writes? A live view may be correct for an identity-preserving cache. A detached copy may be required for audit output or before/after comparison. A frozen detached copy may additionally reject accidental writes by unchecked JavaScript. Each choice has allocation and identity costs; none should be inferred from a return type alone.

5. A six-part discipline for state transitions

For a transition such as `reserve(request)`, make six decisions reviewable:

1. **State owner.** Name the closure invocation, class instance, module, or other component authorized to replace or mutate the state. Do not accept caller-owned mutable structures as private state without copying or an explicit ownership-transfer contract.
2. **Controlled mutation surface.** Expose a small set of operations such as `reserve`, `release`, or `advance`; do not export the writable map or record objects those operations govern.
3. **Transition inputs and results.** Accept all facts required to decide the transition and return an explicit accepted or rejected result. Hidden time, counters, receivers, and callbacks are dependencies even when absent from a parameter list.
4. **Snapshot and alias policy.** State which identities results reuse, which they replace, and whether returned observations remain live. Test identities as well as property values.
5. **Invariant enforcement and commit point.** Check the complete proposal before changing owned state. Build a candidate state without publishing it, then make one narrow replacement or perform a commit sequence that cannot reject for an already-known domain reason.
6. **Observation timing.** State when readers may observe the result and whether any callback can run during preparation or commit. Later asynchronous work introduces observations between separate jobs even when each individual job runs to completion.

This discipline does not require immutable data everywhere. It requires the code to reveal where state lives, how it may change, and what evidence establishes a valid transition.

6. Validate a complete proposal before commit

Mutating while validating makes a later rejection dangerous. A stock movement that decrements the source before checking the destination or the full requested quantity can expose a partial state. Prepare the transition from the old state instead.

Predict before running: Which identity is returned on rejection, and which quantities remain in the original after the accepted transition is constructed?

```
function moveStock(
  current: ReadonlyMap<string, number>,
  from: string,
  to: string,
  quantity: number,
) {
  const fromQuantity = current.get(from);
  const toQuantity = current.get(to);
  if (
    !Number.isInteger(quantity) || quantity <= 0 ||
    fromQuantity === undefined || toQuantity === undefined ||
    fromQuantity < quantity
  ) {
    return { accepted: false as const, stock: current };
  }

  const next = new Map(current);
  next.set(from, fromQuantity - quantity);
  next.set(to, toQuantity + quantity);
  return { accepted: true as const, stock: next };
}

const initial = new Map([["warehouse", 3], ["store", 0]]);
const rejected = moveStock(initial, "warehouse", "store", 4);
const accepted = moveStock(initial, "warehouse", "store", 2);

console.log(rejected.stock === initial, initial.get("warehouse"), initial.get("store"));
console.log(accepted.stock !== initial, accepted.stock.get("warehouse"), accepted.stock.get("store"))
    ;
```

Output:

```
true 3 0
true 1 2
```

The function performs every domain check before creating the next map. Rejection returns the unchanged identity. Acceptance constructs a candidate and exposes it only as the result. A class can use the same pattern with one private root state: build replacement maps and records locally, then assign the private root once.

“Atomic” requires qualification. Run-to-completion prevents another queued job on the same JavaScript agent from interleaving with this synchronous call. It does not roll back writes performed before a thrown exception, and synchronously invoked user code can observe state if the transition calls it midway. It also does not make an external file write, database update, or network request all-or-nothing, and a process can terminate before durable storage records anything. Database atomicity requires a database transaction; process recovery requires persistence and recovery design. This chapter establishes only rejection-safe in-memory transitions inside one synchronous call.

7. Controlled mutation can preserve the right identity

Replacing state is useful when old and new observations must coexist. In-place mutation is useful when a stable identity is the abstraction: a parser cursor advances, a connection changes phase, a cache entry accumulates observations, or a UI node is updated. The defensible design keeps the mutable identity behind a narrow operation and prevents callers from obtaining unrestricted aliases to invariant-bearing state.

A class with prototype methods and private fields is one way to express that boundary. A closure factory returning receiver-free operations is another. Select the form from call and identity requirements. If method extraction is expected, a closure or arrow wrapper can make the receiver unnecessary. If instances should share behavior and method extraction is not part of the contract, prototype methods avoid per-instance function allocation. Neither organization replaces validation, snapshot design, or tests for partial change.

Consequences for program design

Test transitions at their boundaries. For rejection, compare complete before and after snapshots and assert that no callback or audit event occurred. For acceptance, assert invariant equations, changed and preserved identities, commit count, and snapshot detachment. A value-only assertion can miss a dangerous alias; an identity-only assertion can miss an invalid quantity.

Future scheduling makes observation timing more demanding. Separate callbacks may read and update one owner across different jobs. Each callback still runs to completion, but the state observed by a later callback depends on which earlier transitions committed. Chapters 6–9 add scheduling, cancellation, and workflows; the owner, alias, invariant, and commit discipline established here remains the starting point.

False models, repaired

- **"A new root object means the update is immutable."** Nested objects or collection entries may preserve mutable identities.
- **"`readonly` proves the state cannot change."** The checker rejects selected writes through one typed path; writable aliases and unchecked JavaScript remain possible.
- **"A closure or private field makes state safe."** Name-level encapsulation can still expose aliases, permit invalid transitions, or hide observation timing.
- **"A transformation pipeline is pure."** Callbacks can mutate captured or element state, and lazy traversal moves effects to consumer demand.
- **"All mutation defeats local reasoning."** Mutation behind one owner and a narrow invariant-preserving operation can create a smaller boundary than widespread selective copies.
- **"One synchronous function is automatically transactional."** Run-to-completion prevents queued-job interleaving; it does not provide rollback, durability, database isolation, or protection from synchronous reentrancy.
- **"Validation can safely occur while applying each line."** A later rejection can leave earlier lines committed unless preparation and commit are separated.

Mastery questions

1. Draw the identities after copying a root object, a map, and an array stored in one map value. Which mutation leaks back to the original graph?
2. Transform the hidden-sharing prediction so the changed line object and every container on its path receive new identities while an unchanged order remains shared.
3. Predict a closure-based quote function before and after another returned operation rebinds its multiplier. Which unlisted input changed?
4. Two class instances share one prototype method. Explain why one method identity can mutate separate private state and why extraction changes the result.
5. Compare a live map view, an array of map-value aliases, a detached record array, and a frozen detached record array by allocation, identity, and allowed runtime writes.

6. Diagnose an eager pipeline whose callback increments a closure counter and mutates each source element. Which outputs and outside observations change?
7. Apply the six-part discipline to a shopping-cart quantity update. Name the owner, mutation surface, result, alias policy, invariant, commit point, and observation time.
8. Construct a validate-as-you-mutate failure with two request lines where the second line rejects. Prove the partial change with before and after observations.
9. Rewrite that transition so every domain rejection occurs before one root-state assignment. Which errors can still occur outside the promised boundary?
10. Defend in-place mutation for a parser cursor or cache entry. Which stable identity matters, which aliases exist, and which operation preserves the invariant?
11. Explain precisely what run-to-completion prevents. Give one synchronous reentrancy failure, one process-failure boundary, and one database operation it does not make atomic.
12. Transfer the discipline to two later callbacks that update the same store. Without describing promise mechanics, identify which state each callback must read and when it may commit.

The full closed-book workspace and lab specification are in `exercises/ch05.md`.

Implementation lab — In-Memory Inventory Reservation

Implement `InventoryReservationStore` in `examples/ch05/lab/starter.ts`. The class owns stock and reservation state. `reserve` accepts a multi-line request, aggregates duplicate SKUs, validates the complete proposal, and commits either every line or none. Use one private root-state replacement as the commit point. Expose prototype methods, not the internal maps. `snapshot` returns deterministic detached frozen observations whose mutation cannot reach the store.

The evaluator's deliberately faulty store validates and mutates one line at a time. A request whose first line fits and second line exceeds stock is rejected after the first reservation has already changed inventory. Inspect that implementation only after your own tests prove rejection safety, ownership, snapshot detachment, invariant preservation, and method placement.

Presentation prompt

Give a 10–15 minute explanation in your own words covering hidden aliases across nested objects and collections, retained closure state, receiver-dependent private state, snapshot policy, the six-part transition discipline, validate-before-commit, one defensible mutation, and the boundary between synchronous run-to-completion and transactional durability. Include one output prediction, one causal derivation from Chapters 1–4, one false model, one failure case, and one unresolved uncertainty. The recording checklist is in `presentations/prompts/ch05.md`.

Ready-when test

Proceed only when, without notes, you can trace hidden sharing through a nested collection update; explain why closures, classes, `readonly`, copies, and pipelines each restrict some operations without proving isolation; derive a later result from retained state and observation time; specify all six parts of a new transition; distinguish a live view from a detached snapshot; implement and test an inventory reservation that commits no partial change on rejection; defend one narrow mutation boundary; and distinguish synchronous run-to-completion from rollback, persistence, database atomicity, and later callback interleavings.

Chapter 6

Execution Contexts and the Event Loop

Chapter type: Acquisition
Prerequisites: Chapters 1–5
Capabilities introduced: Trace synchronous calls, distinguish language Jobs from host queues, predict microtask ordering, reason about timer eligibility, and diagnose state interleaving across callbacks
Earlier chapters integrated: Mutable identity, closure-retained bindings, callback invocation, collection state, and explicit state transitions
Later chapters enabled: Promises and `async`, cancellation, cleanup, bounded concurrency, and reliable asynchronous workflows
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch06/src/`, `examples/ch06/test/`, and `examples/ch06/lab/`

Compressed thesis

ECMAScript execution contexts track active code evaluation. Entering a function call pushes a context onto an agent's execution-context stack; returning or throwing removes that context. This synchronous call stack does not schedule timers, I/O, or promise continuations. ECMAScript defines Jobs and the promise algorithms that create promise reaction Jobs, while a host such as a browser or Node.js decides when eligible Jobs and callbacks run. HTML specifies task queues and a microtask queue. Node.js supplies an event loop with host phases plus a separate next-tick queue. These terms describe related but noninterchangeable structures.

Within one agent, one Job or host callback runs to completion before another begins. That guarantee protects one uninterrupted synchronous segment; it does not make a workflow spanning several callbacks atomic. A closure can retain mutable state between callbacks, and separately scheduled operations can read and later overwrite that state in an application-level interleaving even when no JavaScript statements execute in parallel.

Timer delays are minimum eligibility thresholds, not precise sleep durations. HTML and Node process promise reactions and `queueMicrotask` callbacks as microtasks, while Node timers and immediates enter different host phases. Predict only ordering guaranteed by the responsible specification and context. This canonical chapter preserves the shared technical spine; explanations required by an observed reader gap belong in a source-linked section of the personal appendix.

1. Execution contexts track synchronous evaluation

An ECMAScript execution context is a specification device that records the state required to evaluate code. Each agent has an execution-context stack and at most one running execution context. Calling an ordinary function creates and pushes a new context; the caller remains below it until the callee returns or throws. Lexical environments, the current function, realm information, and receiver-related state are associated with execution contexts as required by the relevant evaluation rules.

The phrase **call stack** is a useful operational description of this last-in, first-out nesting. It is not a promise about an engine's physical machine stack, nor is it a queue of future callbacks. A generator can suspend an execution context and later resume it, as Chapter 4 showed, so "popped means destroyed" is also too broad.

Predict before running: In what order do the trace entries appear?

```
const trace: string[] = [];

function inner() {
  trace.push("inner");
}

function outer() {
  trace.push("outer:start");
  inner();
  trace.push("outer:end");
}

trace.push("entry:start");
outer();
trace.push("entry:end");
console.log(trace);
```

Output:

```
[ 'entry:start', 'outer:start', 'inner', 'outer:end', 'entry:end' ]
```

`inner` must finish before evaluation resumes after `inner()` in `outer`; `outer` must finish before the entry code continues. No event-loop rule is needed for this prediction. Ordinary call evaluation and the execution-context stack determine it.

2. Language execution and host scheduling have separate authorities

JavaScript programs need a host to obtain timers, network operations, file I/O, UI events, and process lifetime. ECMAScript defines a **Job** as an abstract closure that begins an ECMAScript computation when no other ECMAScript computation is active in that agent. It also defines host hooks such as `HostEnqueuePromiseJob`. Promise settlement creates reaction Jobs and asks the host to enqueue them; the host must preserve the ordering constraints imposed by that hook.

ECMAScript does not define one universal event loop or a `setTimeout` API. HTML specifies event loops containing one or more task queues and a distinct microtask queue. Its processing algorithm selects a runnable task, performs it, and then performs a microtask checkpoint. Node.js instead documents phases including timers, poll, check, and close callbacks, together with microtask processing and a Node-specific next-tick queue. Calling every scheduled callback a "task" erases which specification supplies its order.

Use the terms with an owner:

Term	Responsible authority	What it tracks or schedules
execution context	ECMAScript	current code evaluation and associated state
Job / promise reaction Job	ECMAScript plus a host enqueue hook	a future ECMAScript computation
task / microtask	HTML	host work selected by an HTML event loop
timers, poll, check phases	Node.js/libuv host	categories of Node callbacks
next-tick queue	Node.js	callbacks requested with `process.nextTick`

The table is a vocabulary boundary, not an assertion that each row maps one-to-one onto an operating-system thread. Worker agents and shared memory introduce additional concurrency; this chapter reasons about one ordinary Node agent and defers worker threads.

3. Run-to-completion is local, not transactional

Once a Job starts in an agent, ECMAScript requires that Job to run to completion before another Job starts in that agent. A host callback likewise executes its synchronous calls without another event-loop callback being inserted midway through a statement sequence. Nested functions can call more functions, enqueue later work, mutate objects, and throw, but the next scheduled Job or callback waits until the current synchronous stack unwinds.

Run-to-completion does not mean that an entire asynchronous operation is atomic. If a request reads state, yields by scheduling a continuation, and later writes based on the old read, another logical operation may perform its own read and write between those steps. The individual callbacks remain uninterrupted; the multi-callback state transition does not.

The guarantee also explains blocking. A CPU-heavy loop, synchronous filesystem operation, or expensive callback prevents the host from selecting timer, I/O, or microtask work until the current synchronous work reaches the applicable scheduling checkpoint. A timer may already be eligible while its callback still cannot begin.

4. Promise reactions and microtasks drain before later host work

When a promise settles, ECMAScript's promise algorithms create reaction Jobs and call the host's promise-enqueue hook. In HTML and Node, promise handlers are processed through the microtask queue. `queueMicrotask` asks the host to append a callback to that same queue in Node. Queue insertion order therefore determines the following Node 24 trace.

Predict before running: Include the nested work; do not stop after the two initially queued callbacks.

```
const trace = ["sync:start"];

Promise.resolve().then(() => {
  trace.push("promise:first");
  queueMicrotask(() => trace.push("microtask:nested"));
});

queueMicrotask(() => {
  trace.push("microtask:first");
  Promise.resolve().then(() => trace.push("promise:nested"));
});

trace.push("sync:end");
```

After the current microtask checkpoint completes, the trace is:

```
sync:start
sync:end
promise:first
microtask:first
microtask:nested
promise:nested
```

The synchronous segment finishes first. The initial promise reaction precedes the initial `queueMicrotask` callback because it was enqueued first. Each callback appends another microtask. A microtask checkpoint continues until the queue is empty, so both nested entries run before the host selects a later timer or I/O callback.

This draining rule creates a starvation boundary. A microtask that indefinitely queues another microtask can

prevent the host from reaching tasks, rendering, timers, or I/O callbacks. "Runs soon" is not a license for unbounded recursive scheduling.

5. Timers request eligibility after a threshold

In Node, `setTimeout(callback, delay)` registers host work. The delay specifies a threshold after which the callback may be selected; it does not suspend the caller and does not guarantee an exact start time. Current synchronous work, microtasks, other callbacks, operating-system scheduling, and the event-loop phase can all move execution later.

Predict before running: Which relative order is guaranteed without measuring milliseconds?

```
const trace = ["sync:start"];

setTimeout(() => trace.push("timer"), 0);
queueMicrotask(() => trace.push("microtask"));
trace.push("sync:end");
```

The guaranteed Node trace begins `sync:start`, `sync:end`, `microtask`, then `timer`. A zero delay does not mean "invoke now." The timer becomes eligible only after its threshold and cannot preempt the current synchronous execution or the intervening microtask checkpoint.

Node's `setImmediate` schedules work for the check phase after poll. At top level, this pair has no portable fixed order:

```
setTimeout(() => console.log("timeout"), 0);
setImmediate(() => console.log("immediate"));
```

Node documents the top-level order as context-dependent and potentially nondeterministic. Inside an I/O callback, Node documents the immediate as preceding timers scheduled in that callback. Do not convert the top-level observation from one machine into a test or language rule. Node 20 and later also use changed libuv timer-phase placement, so phase-level predictions must retain a Node-version boundary.

6. `process.nextTick` is a narrow Node-only exception

`process.nextTick` adds a callback to Node's next-tick queue. Node drains that queue after the current JavaScript operation completes and before the event loop continues; recursive use can starve I/O. Node 24 labels the API Legacy and recommends `queueMicrotask` for most user code.

Ordering comparisons require context. After an ordinary host callback, Node drains next-tick callbacks and then the microtask queue. The runnable example schedules both from inside a `setImmediate` callback and produces:

```
immediate:callback
nextTick
microtask
```

Top-level ESM evaluation is already processed through microtask machinery. Node's documentation consequently shows a top-level ESM `queueMicrotask` callback running before a simultaneously requested `process.nextTick` callback, unlike the corresponding CommonJS example. The chapter does not promote that contextual reversal into a general rule. Prefer the portable microtask API unless a Node API contract specifically requires next-tick semantics.

7. Closures carry mutable state into later callbacks

Chapter 2 established that a closure retains access to bindings, and Chapter 1 established that copied object access creates aliases. Scheduling a closure changes when it uses those access paths; scheduling does not snapshot their values or isolate the object.

Predict before running: Why is the final balance `80`, although each callback runs to completion?

```
const account = { balance: 100 };

function scheduleAdjustment(delta: number) {
  const observed = account.balance;
  queueMicrotask(() => {
    account.balance = observed + delta;
  });
}

scheduleAdjustment(10);
scheduleAdjustment(-20);
```

Both calls read `100` before either microtask runs. The first callback writes `110`; the second then writes `80` from its retained snapshot. No statements execute in parallel. The application-level operations overlap because each operation spans a synchronous read and a later write, allowing their steps to interleave. Chapter 5's transition discipline exposes the missing facts: the account is the state owner, each adjustment lacks one invariant-checked commit point, and its observation time precedes the write. A correct repair might perform the read, invariant check, and write in one scheduled state transition, serialize operations, or use storage-level atomicity; run-to-completion alone cannot repair a transition split across callbacks.

This interaction is the bridge from Chapter 5 state-transition discipline to Chapters 7–9. Promises and `await` make boundaries easier to write, but they do not remove aliasing, stale reads, or missing atomicity.

8. Stack traces are evidence, not execution histories

An error stack usually records the active synchronous call chain at the point where the engine constructs or captures the error, subject to engine formatting and frame limits. A timer callback, event handler, or promise reaction begins after the earlier synchronous stack has unwound. V8 can enrich some stack traces with selected asynchronous frames, especially across supported `await` paths, but `Error.prototype.stack` is not standardized and no stack string is a complete log of every enqueue, dequeue, state read, or callback.

Use a stack trace to locate active calls and possible async ancestry. Use explicit trace events, correlation IDs, structured logs, or runtime tracing to reconstruct a workflow across scheduling boundaries. An absent frame does not prove that earlier code did not schedule or influence the callback.

Consequences for program design

Make callback boundaries visible where correctness depends on state order. Keep synchronous work bounded, because every extra millisecond postpones other eligible host work. Use microtasks for short ordering repairs or invariant completion, not for unbounded background processing. Treat timer delays as scheduling thresholds and encode deadlines against a clock or cancellation policy rather than assuming exact wakeup.

Tests should assert specified order only. For workflow logic, inject a controlled scheduler or record semantic trace events so the test chooses when queues advance. Never stabilize a test by sleeping for a guessed duration, and never assert top-level timer/immediate order that Node explicitly leaves context-dependent.

False models, repaired

- **"The call stack is the event loop."** The execution-context stack tracks active evaluation; the host event loop selects future work.
- **"Every queued callback is an ECMAScript Job."** ECMAScript defines Jobs and host enqueue hooks; HTML tasks and Node phase callbacks are host concepts with their own rules.
- **"Run-to-completion makes an async workflow atomic."** It protects one Job or callback, not a transition split across several callbacks.

- **"A zero-delay timer runs immediately."** The delay is a minimum eligibility threshold; the host selects the callback later.
- **"Microtasks run once per turn."** A checkpoint drains nested microtasks until the queue is empty.
- **"Single-threaded code cannot have races."** Logical operations can interleave across callbacks and overwrite closure-retained shared state without parallel statement execution.
- **"`process.nextTick` always precedes promise reactions."** Its order depends on Node execution context; top-level ESM is a documented counterexample.
- **"A stack trace is the program's execution history."** It is bounded, engine-dependent evidence centered on one capture point.

Mastery questions

1. Trace the execution-context stack for three nested calls where the innermost throws and the outermost catches. Which contexts are active at the capture point?
2. Classify each term by authority: execution context, promise reaction Job, HTML task, HTML microtask, Node poll phase, and next-tick queue.
3. Predict a trace containing two initial microtasks where each queues another microtask. Derive the result from FIFO insertion and checkpoint draining.
4. Explain why a promise handler cannot run between two adjacent synchronous statements in the current callback.
5. Construct a counterexample to "run-to-completion makes my request handler atomic" using a read before `await` and a write after it.
6. Diagnose why `setTimeout(callback, 50)` is not evidence that the callback began exactly 50 milliseconds later. Name at least three sources of additional delay.
7. A top-level Node test asserts that `setTimeout(..., 0)` always beats `setImmediate`. Explain why the test is invalid and identify one context where Node documents a stronger order.
8. Compare a promise reaction, `queueMicrotask`, `process.nextTick`, `setTimeout`, and `setImmediate` by responsible authority and portability.
9. Show how recursive microtasks or recursive next-tick callbacks can starve I/O even though each individual callback is short.
10. Transform the lost-balance example so each adjustment reads and writes in one scheduled callback. Which stale-read edge disappears?
11. Explain how a closure can extend the practical lifetime of mutable state until a later callback executes.
12. Diagnose a service whose CPU-heavy callback causes timer and socket latency. Why does adding more zero-delay timers not create preemption?
13. Given an async stack trace with an absent scheduling function, state what the trace proves, what it does not prove, and which trace events you would add.
14. Design a deterministic test for code that requires microtasks to finish before a timeout callback without sleeping or asserting elapsed milliseconds.

The full answer workspace and lab specification are in `exercises/ch06.md`.

Implementation lab — Controlled Publication Turn

Implement `schedulePublication` in `examples/ch06/lab/starter.ts`. It accepts mutable publication state, a controlled scheduler, and a trace recorder. Start validation synchronously; validate in one microtask; enqueue a nested microtask that commits and increments the revision; and register a timer that observes the committed state. The evaluator advances queues explicitly, so exact order is testable without wall-clock timing.

Before coding, predict the state and trace immediately after the function returns and after one controlled turn. Then inspect the deliberately incorrect timer-as-sleep implementation only after your solution passes.

Presentation prompt

Give a 10–15 minute explanation in your own words covering execution contexts, the call stack, ECMAScript Jobs, host scheduling, tasks and microtasks, run-to-completion, timers, one Node-specific boundary, closure-retained state, and stack-trace limitations. Include one output prediction, one causal derivation from Chapters 1–5, one false model, one unspecified order, and one unresolved uncertainty. The recording checklist is in `presentations/prompts/ch06.md`.

Ready-when test

Proceed only when, without notes, you can trace nested synchronous calls; distinguish an ECMAScript Job from HTML and Node scheduling terms; predict initial and nested microtasks; explain timer thresholds and the unspecified top-level timer/immediate order; bound a `process.nextTick` statement to Node 24 and its scheduling context; derive an application-level interleaving from closure-retained state; explain why long synchronous work blocks host progress; implement and test the controlled scheduler lab; and present the chapter without treating run-to-completion as workflow atomicity.

Chapter 7

Promises, async, and Control Flow

Chapter type: Acquisition and integration
Prerequisites: Chapter 6; function calls, closures, and iterable consumption
Capabilities introduced: Distinguish promise state, settlement, and resolution; derive chain outcomes; predict `async`/`await` ordering; preserve dependency and error propagation; select promise combinators by contract
Earlier chapters integrated: Function return and throw behavior from Chapter 2; iteration order from Chapter 4; ECMAScript Jobs and host scheduling from Chapter 6
Later chapters enabled: Errors, cancellation, cleanup, retries, bounded concurrency, and production workflows
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch07/src/`, `examples/ch07/test/`, and `examples/ch07/lab/`

Compressed thesis

A promise is an object whose state is **pending**, **fulfilled**, or **rejected**. Fulfillment and rejection are the two settled states. **Resolution** is a separate procedure: resolving a promise with an ordinary value fulfills it, but resolving it with a promise or thenable locks its eventual outcome to that adopted value. A resolved promise can therefore remain pending. The `Promise` constructor calls its executor synchronously; promise reactions run later as ECMAScript Jobs after the current synchronous computation has completed.

Every call to `then`, `catch`, or `finally` returns a new promise. The returned promise's outcome follows from the selected handler: an ordinary return value fulfills it, a thrown value rejects it, and a returned promise or thenable is assimilated so the chain waits for that value's eventual outcome. An `async` call likewise returns a new promise. `await` converts its operand through promise resolution, suspends only the enclosing async continuation, and resumes that continuation through promise reaction machinery; it does not block the operating-system thread or process.

Correct composition therefore requires explicit dependency structure. Start independent operations before awaiting them together; await dependent operations in their required order; and return or await every promise whose completion and rejection belong to the enclosing operation. This canonical chapter retains that shared technical spine. Explanations needed only after an observed reader failure belong in source-linked sections of the personal appendix.

1. State, settlement, resolution, and assimilation

Promise **state** answers one question: is the promise pending, fulfilled with a value, or rejected with a reason? A promise is **settled** exactly when it is fulfilled or rejected. Once settled, its state and result do not change.

Resolution answers a different question: has the promise's outcome been fixed directly or linked to another promise-like value? Calling a promise's resolving function with `17` normally fulfills it with `17`. Calling the same

function with a pending promise resolves the outer promise immediately in the sense that later calls to either resolving function have no effect, yet the outer promise remains pending until the adopted promise settles. If the adopted promise fulfills, the outer promise fulfills with the same eventual value; if it rejects, the outer promise rejects with the same reason.

A **thenable** is an object whose `then` property is callable. Promise resolution reads that property and, when callable, schedules a Promise Resolve Thenable Job to call it. This **thenable assimilation** lets native promises adopt foreign promise-like values while guarding against multiple attempts to settle. The call to the thenable's `then` method occurs after surrounding synchronous evaluation, although property access needed to obtain `then` can itself run synchronously.

This terminology prevents a common contradiction: "resolved" does not always mean "fulfilled." A promise resolved to a pending thenable is resolved and pending, but it is not settled.

Predict before running: Which entries exist before the first `console.log`, and when does the reaction append its entry?

```
const trace: string[] = [];

const operation = new Promise<string>((resolve) => {
  trace.push("executor");
  resolve("ready");
});

operation.then((value) => trace.push(`reaction:${value}`));
trace.push("caller");

console.log(trace);
await operation;
console.log(trace);
```

Output:

```
[ 'executor', 'caller' ]
[ 'executor', 'caller', 'reaction:ready' ]
```

The constructor invokes the executor during construction, so `"executor"` is recorded first. Fulfillment makes the reaction eligible, but JavaScript does not call the reaction inside `resolve`. The runtime asks the host to enqueue a Promise Reaction Job. The current module computation records `"caller"` and reaches its suspension point before that job calls the handler.

Promises represent eventual completion; they do not make arbitrary work asynchronous. CPU-intensive code placed directly in an executor still runs synchronously and delays the constructor call from returning.

2. A chain is derived from handler outcomes

`source.then(onFulfilled, onRejected)` creates and returns a new promise immediately. When `source` settles, exactly the applicable reaction handler runs as a Job. The handler's completion determines the derived promise:

- returning an ordinary value resolves the derived promise with that value, normally fulfilling it;
- throwing a value rejects the derived promise with that value;
- returning a promise or thenable resolves the derived promise to that value, so resolution waits for and adopts its eventual outcome;
- reaching the end of the handler returns `undefined`, so the derived promise fulfills with `undefined`.

The third rule is commonly called **flattening**. JavaScript does not fulfill a chain link with a still-wrapped promise and require the next handler to unwrap it manually. The resolution procedure assimilates the returned promise or thenable. TypeScript's `Awaited<T>` and standard `Promise` declarations describe the same recursive unwrapping for checked return types; the JavaScript runtime performs the actual resolution.

Predict before running: What final string is produced, and which handler changes rejection back to fulfillment?

```
const result = await Promise.resolve(2)
  .then((value) => value + 1)
  .then((value) => Promise.resolve(value * 10))
  .then((value) => {
    throw new Error(`E${value}`);
  })
  .catch((reason: unknown) =>
    reason instanceof Error ? `recovered:${reason.message}` : "recovered:unknown",
  );

console.log(result);
```

Output:

```
recovered:E30
```

The first handler returns `3`. The second returns a fulfilled promise, so the next link adopts it and supplies `30`, not `Promise<30>`, to the third handler. That handler throws, rejecting its derived promise. The `catch` handler returns an ordinary string, which recovers by fulfilling the final promise.

Missing `return` is therefore a control-flow bug, not cosmetic style. If a handler starts an operation but does not return its promise, the handler returns `undefined`; the outer chain can fulfill before the started operation finishes, and that operation's later rejection is no longer propagated through the outer chain.

```
source.then((record) => {
  saveRecord(record); // fault: the returned promise is not returned
}).then(() => {
  publishCompletion(); // can run before saveRecord settles
});
```

Repair the edge by returning `saveRecord(record)`, using the expression-bodied arrow `record => saveRecord(record)`, or awaiting it inside an async handler. Indentation cannot create the dependency. Only the returned or awaited promise connects the operations.

3. Rejection, `catch`, and `finally`

A thrown value and a promise rejection are related but not identical. A `throw` produces abrupt control flow while JavaScript is evaluating a function. A rejection is a settled state and reason stored by a promise. The Promise constructor converts an uncaught throw from its executor into rejection of the constructed promise. A Promise Reaction Job converts a handler's throw into rejection of the derived promise. An async function converts an uncaught throw in its body into rejection of the promise returned by that call. In contrast, an ordinary function that throws before returning any promise throws synchronously to its caller.

At `await`, the relationship runs in the other direction: rejection resumes the suspended async context with a throw completion. An enclosing `try/catch` in that async function can therefore handle the rejected reason. Promise rejection does not mean that a hidden exception is continuously bubbling while the promise is pending.

`promise.catch(onRejected)` performs rejection handling equivalent to `promise.then(undefined, onRejected)`. If `onRejected` returns normally, the derived promise is fulfilled and the chain has recovered. To preserve failure, the handler must throw or return a rejected promise. A terminal `catch` that logs and returns accidentally can therefore swallow a failure.

`finally(onFinally)` runs after either fulfillment or rejection and normally preserves the prior outcome. Its callback receives neither value nor reason. Returning an ordinary value—or a promise that fulfills—does not replace the prior fulfillment value or rejection reason. If the callback throws or returns a rejected promise, that new failure rejects the promise returned by `finally`.

Predict before running: What value survives cleanup, and in which order are the entries recorded?

```
const events: string[] = [];

const value = await Promise.reject(new Error("primary"))
  .catch((reason: unknown) => {
    events.push(reason instanceof Error ? reason.message : "unknown");
    return "fallback";
  })
  .finally(() => {
    events.push("cleanup");
    return "ignored";
  });

console.log(value, events);
```

Output:

```
fallback [ 'primary', 'cleanup' ]
```

The `catch` converts rejection to fulfillment with `"fallback"`. The ordinary return from `finally` is transparent, so the fulfillment value remains `"fallback"`. Chapter 8 develops cleanup failures and competing errors; this chapter establishes only the propagation contract.

4. `async` calls and `await` continuations

Calling an async function creates a new promise and starts evaluating the body. Evaluation before the first reached `await` runs synchronously as part of the call. A normal `return value` resolves the call's promise with `value`; returning a promise causes adoption rather than nested fulfillment. An uncaught throw rejects the call's promise. TypeScript represents `async function load(): Promise<Record>` as a promise-returning function, but that annotation neither schedules work nor handles rejection at runtime.

`await expression` first resolves the expression's value as a promise, including thenable assimilation. JavaScript then suspends the enclosing async execution context and returns control to its caller. Fulfillment later resumes that context with a normal value; rejection resumes it by throwing the reason at the `await` expression. Even an already fulfilled promise or a non-thenable value causes the continuation after `await` to run later through promise machinery.

Predict before running: Which code runs before `inspect()` returns its promise to the caller?

```
const trace: string[] = [];

async function inspect(): Promise<number> {
  trace.push("before");
  await 0;
  trace.push("after");
  return 7;
}

const resultPromise = inspect();
trace.push("caller");
console.log(trace);
console.log(await resultPromise, trace);
```

Output:

```
[ 'before', 'caller' ]
7 [ 'before', 'caller', 'after' ]
```

`await` suspends only `inspect`'s continuation. It does not sleep the process, block the operating-system thread, or stop the host from running other eligible work. Chapter 6's distinction remains essential: ECMAScript defines promise reaction Jobs and the `HostEnqueuePromiseJob` ordering constraints; Node services promise reactions through its microtask queue, whose checkpoint drains nested microtasks before later host work. Not every queued operation is an event-loop task, and timers are host operations rather than promise states.

5. Dependency structure and initiation time

Sequential syntax is correct when the second operation requires the first result:

```
const account = await loadAccount(userId);
const entries = await loadEntries(account.accountId);
```

`loadEntries` cannot be called correctly until `account.accountId` exists. By contrast, two independent calls should usually be initiated before either result is awaited:

```
const accountPromise = loadAccount(userId);
const policyPromise = loadPolicy(userId);
const [account, policy] = await Promise.all([accountPromise, policyPromise]);
```

The calls initiate the operations; `Promise.all` observes their returned promises. Passing the functions themselves would not call them. Writing `await loadAccount()` before calling `loadPolicy()` introduces a dependency edge even when the data flow does not require one.

`await` inside a loop is not automatically wrong. Sequential awaiting is required when iteration `n + 1` depends on iteration `n`, when order is part of the API contract, or when an external capacity rule permits only one operation at a time. Initiating every iteration at once is appropriate only when the operations are independent and the resulting concurrency is safe. Chapter 9 owns explicit concurrency limits.

6. Promise combinators are completion contracts

For a finite iterable that can be consumed normally, the four standard combinators differ by the outcome needed from their input promises:

Combinator	Fulfillment contract	Rejection contract	Empty iterable
`Promise.all`	All inputs fulfill; returns their values in input order	First input rejection observed by the aggregate	Fulfills with `[]`
`Promise.allSettled`	All inputs settle; returns fulfillment/rejection records in input order	An input rejection becomes a result record rather than rejecting the aggregate	Fulfills with `[]`
`Promise.race`	First input settlement is a fulfillment	First input settlement is a rejection	Remains pending
`Promise.any`	First input fulfillment	All inputs reject; rejects with `AggregateError`, whose reasons retain input order	Rejects with `AggregateError`

Non-promise values and thenables are processed through promise resolution. Abrupt failure while consuming the iterable can reject the returned promise; the table describes ordinary successful iteration. Result order for `all`, `allSettled`, and `AggregateError.errors` follows input order, not completion order.

Short-circuiting settles only the aggregate promise. If `Promise.race` obtains one result, `Promise.any` obtains one fulfillment, or `Promise.all` observes a rejection, the other operations continue unless their own APIs receive and honor an independent cancellation request. A promise has no general operation that reverses the work it represents. Chapter 8 introduces cancellation and cleanup without changing these combinator contracts.

7. Consequences for program design

An async API should return a promise representing all work the caller is entitled to observe. If cleanup, persistence, or publication occurs after the returned promise fulfills, the API has defined premature completion unless that detached lifetime is explicit. A precise TypeScript return type can expose some missing returns, but the checker cannot prove that every started operation belongs to the returned promise.

Choose error boundaries intentionally. Catch where recovery, translation, or added context is possible; otherwise preserve rejection for the caller. JavaScript permits any rejection reason, so code must not assume `Error` without checking. Host reporting of unhandled rejection is diagnostic evidence, not a substitute for connecting the promise chain.

False models, repaired

- **"Resolved means fulfilled."** Resolution to a pending promise locks the outcome while the resolving promise remains pending.
- **"The Promise constructor makes its executor asynchronous."** Construction invokes the executor synchronously; reactions and thenable calls run through later Jobs.
- **"`then` nests returned promises."** The derived promise assimilates a returned promise or thenable and exposes its eventual value or rejection.
- **"Indentation makes an inner operation part of the chain."** Only returning or awaiting its promise creates the completion and error-propagation edge.
- **"A rejection is a thrown exception waiting on the stack."** Rejection is promise state; `await` can convert it into a throw completion when the async context resumes.
- **"`await` blocks JavaScript."** It suspends one async continuation and returns control to the caller.
- **"Every `await` in a loop is accidental serialization."** Dependencies, ordering, and capacity constraints can require sequential initiation.
- **"A winning combinator cancels the losers."** The aggregate settles; independent operations continue unless separately cancelled.

Mastery questions

1. Construct a promise that is resolved but still pending, and explain why a later call to its reject function has no effect.
2. Predict the ordering among a Promise constructor executor, synchronous caller code, a `then` reaction, and an `await` continuation. Name each synchronous computation or Job.
3. Derive the final outcome of a four-link chain whose handlers return an ordinary value, return a thenable, throw, and recover.
4. Diagnose a missing-`return` handler that publishes completion before a write finishes. State both the premature value and the lost rejection path.
5. Explain why a handler returning `Promise<Promise<number>>`-shaped work does not deliver a promise object to the next fulfillment handler.
6. Give one synchronously thrown failure that `.catch()` attached to a later promise cannot intercept, and one thrown failure that becomes rejection of a derived promise.
7. Predict the outcomes of `finally` after fulfillment, after rejection, after an ordinary cleanup return, and after a cleanup rejection.
8. Rewrite a three-step chain as `async`/`await` without changing its dependency or recovery boundaries.
9. Two independent operations are currently awaited one after the other. Transform the code to initiate both first, and identify the exact statement that changes initiation time.

10. Defend a sequential `await` loop using a dependency, ordering, or capacity contract; then give a case where the same shape is accidental serialization.
11. Select among `all`, `allSettled`, `race`, and `any` for four production scenarios. For each, state fulfillment, rejection, and losing-operation behavior.
12. Design a deterministic test proving that an aggregate can settle while a losing operation continues.
13. Explain what `Promise<Report>` in a TypeScript signature permits the checker to verify and which runtime completion, validation, and cancellation facts remain unproved.
14. Transfer the chapter to an unfamiliar callback wrapper: identify where the executor runs, which callback calls resolve or reject, and how duplicate callback invocations are handled.

The full closed-book workspace and lab specification are in `exercises/ch07.md`.

Implementation lab — Controlled Account Summary

Implement `buildAccountSummary` in `examples/ch07/lab/starter.ts`. Load a user first because its result supplies account and policy identifiers. Then initiate balance and policy loads without waiting for one before starting the other. Fulfill with a summary only after both fulfill; propagate either rejection without translating it.

The evaluator uses controllable deferred promises rather than timers. Tests prove the dependency boundary, concurrent initiation of independent work, completion only after both results, and rejection identity. A deliberately faulty implementation omits the return from a nested `Promise.all` handler and therefore fulfills with `undefined` while child operations remain pending.

Presentation prompt

Give a 10–15 minute explanation in your own words covering state, settlement, resolution, thenable assimilation, executor and reaction timing, chain derivation, missing returns, rejection versus throw, `catch`, transparent `finally`, async call results, await suspension, dependency-aware initiation, and the four combinator contracts. Include one prediction, one causal derivation, one Chapter 6 connection, one false model, one failure boundary, and one unresolved uncertainty. The structured checklist is in `presentations/prompts/ch07.md`.

Ready-when test

Proceed only when, without notes, you can distinguish pending, fulfilled, rejected, settled, and resolved; derive every chain result from handler return or throw; repair a missing completion edge; predict executor, reaction, and await order using Chapter 6 terminology; preserve dependency and error propagation while changing sequential initiation to concurrent initiation; select all four combinators by exact outcome contract; implement and test the controlled workflow; and explain why aggregate settlement does not cancel losing operations. A wrong settlement state, Job ordering, or propagation edge is a mechanistic error.

Chapter 8

Errors, Cancellation, and Cleanup

Chapter type: Integration
Prerequisites: Chapters 6–7
Capabilities introduced: Classify failure channels; derive `try`/`catch`/`finally` and promise propagation; design cooperative cancellation and timeout policy; assign resource ownership; guarantee exactly-once cleanup; preserve competing failures
Earlier chapters integrated: Run-to-completion and host scheduling from Chapter 6; promise settlement, reaction Jobs, `await`, and transparent `finally` from Chapter 7; state ownership and commit boundaries from Chapter 5
Later chapters enabled: Reliable asynchronous workflows, transaction lifetime, public error contracts, graceful shutdown, and partial-failure diagnosis
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch08/src/`, `examples/ch08/test/`, and `examples/ch08/lab/`

Compressed thesis

Failure handling begins by naming the representation and the control-flow edge. A synchronous `throw` abruptly exits the current evaluation until a matching `catch` handles its value. A promise rejection is settled promise state; a reaction handler or `await` can turn that state into later abrupt evaluation. A domain failure value returns normally and must be inspected by application code. An abort signal records and broadcasts a cancellation request. A timeout is a policy decision made when a deadline wins. Cleanup can itself fail. Treating all six as an undifferentiated "error" hides who can observe, recover, cancel, or release.

Resource correctness requires an ownership path: acquire, use, release. The operation that successfully acquires a lease, stream, listener, or cancellation scope normally owns its release unless the API explicitly transfers that obligation. Every exit after acquisition—success, primary failure, observed cancellation, or timeout policy—must reach one exactly-once release path. If primary work and cleanup both fail, the boundary needs a stated precedence or aggregation policy that preserves evidence of both.

Cancellation is cooperative. `AbortController.abort(reason)` changes its signal's state and notifies observers; it does not stop arbitrary JavaScript, roll back completed effects, close unrelated resources, or settle an operation that never observes the signal. `Promise.race` likewise settles only its aggregate promise. This canonical chapter retains that shared technical spine; explanations required by an observed reader gap belong in source-linked sections of the personal appendix rather than expansions of the chapter.

1. Six outcomes that must remain distinct

A synchronous throw, promise rejection, domain failure, cancellation notification, timeout decision, and cleanup failure can eventually cross the same API boundary as a rejected promise, but they originate differently.

Outcome	Representation	Who produces or observes it
synchronous failure	throw completion carrying any JavaScript value	current JavaScript evaluation and enclosing `catch`
asynchronous failure	rejected promise with a reason	promise resolution/reaction algorithms; caller handlers or `await`
expected domain refusal	ordinary result such as `{ ok: false, reason }`	application branch that inspects the result
cancellation request	aborted signal state, reason, and abort notification	controller and operations that inspect or subscribe to the signal
timeout policy	application or host deadline chooses an outcome	deadline owner; cancellation requires a separate supported operation
cleanup failure	throw or rejection from release work	resource boundary executing `close`, `dispose`, or equivalent

Predict before running: Which failures enter which `catch`?

```
function reserve(quantity: number) {
  if (quantity <= 0) return { ok: false as const, reason: "quantity" };
  return { ok: true as const, reservationId: "R1" };
}

try {
  const result = reserve(0);
  console.log(result.ok ? "reserved" : `declined:${result.reason}`);
} catch {
  console.log("caught");
}

async function load() {
  throw new Error("offline");
}

const pending = load();
console.log("after-call");
try {
  await pending;
} catch {
  console.log("rejected");
}
```

Output:

```
declined:quantity
after-call
rejected
```

The domain refusal is a normal return, so `catch` is not selected. Calling `load` creates a promise; the uncaught throw in its async body rejects that promise. `await` later resumes the module evaluation with a throw completion, which the surrounding `catch` handles. A caller should not infer retry, cancellation, or HTTP status merely from the fact that a promise rejected; that translation belongs to a named boundary.

2. `finally` preserves a prior completion only when cleanup completes normally

ECMAScript represents normal return and abrupt exits—including `throw`, `return`, `break`, and `continue`—as completions. A `try` block produces a completion. A matching `catch` can replace a throw completion with the catch

block's completion. JavaScript then evaluates `finally`. If the `finally` block completes normally, the earlier completion continues. If `finally` returns, throws, breaks, or continues, its abrupt completion replaces the earlier one.

Predict before running: Which reason escapes?

```
function replaceFailure() {
  try {
    throw new Error("primary");
  } finally {
    throw new Error("cleanup");
  }
}

try {
  replaceFailure();
} catch (reason: unknown) {
  console.log(reason instanceof Error ? reason.message : "unknown");
}
```

Output:

```
cleanup
```

The `finally` block did run, but its throw discarded direct propagation of the primary failure. A `return` in `finally` can similarly replace an earlier return or throw. Avoid control-flow exits from cleanup unless replacement is the explicit policy.

`Promise.prototype.finally` has the corresponding asynchronous contract. It returns a new promise and invokes its callback after the source settles. An ordinary return or fulfilled promise from the callback preserves the source outcome. A throw or rejected promise from the callback rejects the returned promise with the cleanup reason. An `async finally` block behaves through ordinary `try` semantics plus `await`: if awaited cleanup rejects, that resumed throw can replace the pending return or failure.

Neither form promises execution after the process is forcibly terminated, the machine loses power, the runtime aborts, or code never reaches the construct. "`finally` always runs" is too broad. Its guarantee concerns control flowing through the JavaScript `try` statement or promise chain in the running agent.

3. Resource ownership defines the release obligation

Track a resource through three phases:

1. **Acquire.** Identify the operation after which a resource exists and who receives its handle.
2. **Use.** Identify every awaited step, partial external effect, and point where cancellation is observed.
3. **Release.** Identify the one control-flow path that closes the handle and whether release is synchronous or asynchronous.

Place `try` after successful acquisition. If acquisition itself fails before a handle exists, there is nothing to close. Once acquisition succeeds, the `finally` path must cover every later exit:

```
const lease = await pool.acquire();
try {
  return await useLease(lease);
} finally {
  await lease.close();
}
```

The function that acquired `lease` owns closure in this example. Returning the lease would require an explicit transfer contract: the caller becomes responsible for one close, and the callee must not also close it. Merely passing a handle to a helper does not transfer ownership. The same rule applies to an `AbortController`: the component that creates the cancellation scope normally decides when that scope ends; callees receive its signal and should not assume authority to abort unrelated sibling work.

Exactly-once cleanup needs an idempotence decision. The owner should invoke release once on its control-flow graph. A resource may additionally make `close()` idempotent because duplicate calls can arise during defensive shutdown, but callers must not assume idempotence without a contract. Abort listeners are resources too: remove a listener when work settles successfully or fails for another reason. `{ once: true }` removes it after an abort event, but it does not remove a never-fired listener from a long-lived signal when the operation finishes first.

4. Cancellation is a request that an operation must observe

`AbortController` and `AbortSignal` are Web-platform APIs also exposed by Node 24. Calling `abort(reason)` records the reason, changes `signal.aborted` to `true`, and dispatches the signal's one abort notification. An abort-aware operation checks an already-aborted signal before starting and observes a later notification while pending. The operation then performs its own stopping and cleanup steps and ordinarily rejects an unsettled promise with `signal.reason`.

Checking only once is insufficient for a multi-step operation. Subscribing without checking is also insufficient because a signal that was already aborted will not replay its abort event for a newly added listener. A robust operation checks before acquisition when possible, subscribes after it has the resource that must be stopped, checks again after subscription, checks at cooperative boundaries, and removes the listener when the operation settles.

The notification does not preempt a running synchronous loop. The loop must call `throwIfAborted()` or read `aborted` at selected points. Nor does cancellation undo a request already sent, bytes already written, or a state transition already committed. Node's file APIs explicitly describe some cancellation as best effort because internal buffering may stop while operating-system requests or written data remain. Rollback requires a separate transactional or compensating operation; database rollback belongs to Chapter 23.

The controller owner chooses cancellation scope. If one controller covers two sibling operations, abort notifies both. If only one child should stop, give that child a separate controller or a derived signal with an explicit lifetime. Reusing an already-aborted signal causes later operations to begin in the aborted state.

5. A timeout policy must connect settlement to stopping work

A timeout says what the caller will do after a deadline: reject, return a fallback, emit a diagnostic, request cancellation, or detach work. `Promise.race([work, deadline])` implements only first-settlement selection. When the deadline rejects first, the race promise rejects; the work promise and its underlying operation continue.

Predict before running: Does the work effect occur after the deadline wins?

```
const effects: string[] = [];
let finishWork!: () => void;
let expire!: () => void;

const work = new Promise<void>((resolve) => { finishWork = resolve; })
  .then(() => effects.push("work:finished"));
const deadline = new Promise<never>((_, reject) => {
  expire = () => reject(new Error("timeout"));
});

const raced = Promise.race([work, deadline]);
expire();
await raced.catch(() => effects.push("race:rejected"));
finishWork();
await work;
console.log(effects);
```

Output:

```
[ 'race:rejected', 'work:finished' ]
```

A bounded operation needs a cancellation-capable API and a connection from deadline to that API, commonly a timeout signal passed into the operation. Even then, "timed out" means cancellation was requested at the deadline; actual stopping, cleanup completion, and residual effects depend on the operation's contract. If the caller abandons the losing promise, its later rejection still needs an observer to avoid detached failure.

6. Preserve both primary and cleanup failures

Cleanup failure creates a precedence problem. If use succeeds and close fails, the operation should normally reject with the close failure because the resource contract was not completed. If use fails and close succeeds, preserve the primary failure. If both fail, silently replacing either destroys evidence.

One explicit policy is to reject with a boundary-specific error containing both identities:

```
class OperationCleanupFailure extends Error {
  constructor(
    readonly primaryFailure: unknown,
    readonly cleanupFailure: unknown,
  ) {
    super("Operation and cleanup both failed", { cause: primaryFailure });
  }
}
```

An `AggregateError` is another valid representation. The important property is not the class name but retention: logs, retry classification, tests, and callers must be able to inspect the primary and cleanup failures. A `finally` block that simply throws the cleanup reason follows JavaScript's replacement rule but loses the primary reason unless the cleanup code explicitly packages it.

Do not retry automatically at this boundary. The primary operation may have completed an external effect before failing, while cleanup failure may leave resource state uncertain. Chapter 9 develops retries and idempotency; Chapter 24 develops shutdown and partial failure.

Consequences for program design

Make lifetime visible in API shape. Accept an `AbortSignal`, not an `AbortController`, when a callee may observe cancellation but must not cancel the caller's scope. Return a promise that settles only after required cleanup

completes. Document whether a domain refusal is returned or thrown, which abort reason is propagated, which effects may survive cancellation, and whether ownership is retained or transferred.

Tests should control promise settlement and abort delivery rather than sleep. Assert acquisition count, completed steps, cancellation calls, listener removal, release count, result or failure identity, and the composite shape when both phases fail. A test that checks only the final rejection misses leaks and duplicate cleanup.

False models, repaired

- **"All failures are exceptions."** Domain refusals can be ordinary values; cancellation and timeout begin as separate signals or policies.
- **"`finally` preserves the original result."** Only normal cleanup preserves it; abrupt cleanup replaces it.
- **"`finally` runs under every termination."** It governs JavaScript control reaching the construct, not forced process loss.
- **"Calling `abort()` stops the operation."** It changes signal state and notifies observers; the operation must cooperate.
- **"Cancellation rolls back completed effects."** Stopping future work and reversing prior effects are different operations.
- **"`Promise.race` cancels losers."** It settles an aggregate promise and leaves inputs unchanged.
- **"`{ once: true }` removes every abort listener when work completes."** It removes a listener after the event fires; successful work must remove an unused listener.
- **"Cleanup failure should replace the primary failure."** Replacement is JavaScript's default abrupt-completion result, not an adequate evidence policy when both matter.

Mastery questions

1. Classify six outcomes as synchronous throw, rejection, domain failure value, cancellation notification, timeout decision, or cleanup failure; state which code observes each.
2. Predict a `try` that returns from its body and throws from `finally`. Which completion escapes, and why?
3. Derive the promise returned by `source.finally(cleanup)` for source fulfillment/rejection crossed with cleanup fulfillment/rejection.
4. Transform an async function whose resource is acquired inside `try` so acquisition failure does not call `close` on an uninitialized handle.
5. Draw acquire, two use steps, and release across success, primary failure, and cancellation paths. Prove release has one owner and one invocation.
6. Diagnose an abort-aware API that subscribes to `abort` but never checks `signal.aborted` before starting.
7. Diagnose the inverse API that checks once but never subscribes or checks again during a long operation.
8. A write completes its first external effect and then observes cancellation. What may stop, what remains true, and what separate guarantee would rollback require?
9. Predict a controlled `Promise.race` where a timeout rejects first and the work later mutates an array. Which promise settled and which effect still occurs?
10. Design a timeout policy that requests cancellation, waits for cleanup, and retains any late cleanup failure.
11. A successful operation's `close` rejects. A failed operation's `close` also rejects. Specify the returned failure for both cases without discarding evidence.
12. Explain why a function accepting only `AbortSignal` expresses less authority than one accepting `AbortController`.
13. Diagnose a successful operation that leaves an abort listener on a process-lifetime signal. What remains reachable, and which cleanup path is missing?
14. Transfer the acquire-use-release analysis to an event subscription, stream reader, lock lease, or temporary file without assuming database rollback.

The full closed-book workspace and lab specification are in `exercises/ch08.md`.

Implementation lab — Cancellable Two-Part Reader

Implement `readDocument` in `examples/ch08/lab/starter.ts`. It acquires one reader lease, reads a header and body in order, observes an `AbortSignal`, removes its listener, and closes the lease exactly once after acquisition. The evaluator controls both reads, abort delivery, primary failures, and close failures without timers.

Preserve the abort reason identity. If reading and closure both fail, reject with `OperationCleanupFailure` containing both values. The deliberately faulty implementation leaves its abort listener registered; a later abort therefore calls cancellation on an already closed lease.

Presentation prompt

Give a 10–15 minute explanation in your own words covering the six failure representations, completion replacement in both forms of `finally`, acquire-use-release ownership, cooperative cancellation, pre-abort handling, listener removal, timeout policy, completed effects, and competing failures. Include one prediction, one Chapters 6–7 derivation, one false model, one host boundary, and one unresolved uncertainty. The structured checklist is in `presentations/prompts/ch08.md`.

Ready-when test

Proceed only when, without notes, you can classify all six outcomes; derive synchronous and promise `finally` replacement; trace acquire, use, and release on every exit; implement pre-abort and mid-operation cancellation without treating notification as preemption; prove a raced loser continues; preserve completed effects and both competing failures; remove listeners and close resources exactly once; implement and test the controlled reader; and present the chapter without treating timeout, cancellation, rollback, or cleanup as synonyms.

Chapter 9

Asynchronous State and Reliable Workflows

Chapter type: Integration and emergent
Prerequisites: Chapters 1–8
Capabilities introduced: Diagnose stale asynchronous state; choose serial, unbounded, or bounded initiation; specify worker-pool invariants; propagate cancellation; classify retries; define idempotency boundaries; aggregate partial outcomes
Earlier chapters integrated: Identity and aliasing; closure-retained state; validate-before-commit ownership discipline; run-to-completion; promise initiation and settlement; cooperative cancellation and exactly-once cleanup
Later chapters enabled: Domain state machines, durable idempotency, transaction boundaries, background work, graceful shutdown, and production partial-failure handling
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch09/src/`, `examples/ch09/test/`, and `examples/ch09/lab/`

Compressed thesis

One JavaScript agent executes one Job or host callback at a time, yet application-level races still occur when one logical state transition spans several Jobs or callbacks. A logical operation can read owned state, suspend at `await`, allow a competing operation to commit, and later overwrite that commit using the stale read. Run-to-completion protects each synchronous segment; it does not combine separated segments into one transition.

A reliable asynchronous workflow therefore extends Chapter 5's discipline across time. It names the state owner, invariant, commit point, admission policy, cancellation scope, cleanup owner, retry policy, idempotency boundary, and result contract. Serial execution admits one operation after the preceding operation settles. Unbounded initiation calls every operation before observing their promises. Bounded execution admits work only while an active-count invariant permits another start. `Promise.all` observes input promises; it does not impose a concurrency limit on operations already initiated by `items.map(start)`.

Cancellation stops new admission and is propagated to active operations, but active operations must cooperate. Every acquired resource still follows one exactly-once cleanup path. Retrying is a policy over classified failures, attempt limits, delay or backoff, and cancellation. Idempotency is a different property: repeated processing of the same logical request produces no additional committed effect under a stated key, scope, and retention boundary. This chapter's deduplication store is process-local and non-durable; database transactions, locks, and durable idempotency records remain later enforcement boundaries.

1. A logical transition can race with itself across time

Chapter 6's run-to-completion rule says another queued Job does not interrupt the current Job. The rule does not protect a value read before `await` from changes made before the continuation resumes. Each asynchronous call is a logical operation containing several separately scheduled synchronous segments.

Predict before running: Both calls read before either can commit. Which read becomes stale, which operation commits between that read and its write, and which state is overwritten?

```
const releaseCommits = Promise.withResolvers<void>();
const account = { balance: 100 };
const trace: string[] = [];

async function adjust(label: string, delta: number) {
  const observed = account.balance;
  trace.push(`${label}:read:${observed}`);
  await releaseCommits.promise;
  account.balance = observed + delta;
  trace.push(`${label}:commit:${account.balance}`);
}

const credit = adjust("credit", 10);
const debit = adjust("debit", -20);
releaseCommits.resolve();
await Promise.all([credit, debit]);

console.log(account.balance, trace);
```

Output:

```
80 [
  'credit:read:100',
  'debit:read:100',
  'credit:commit:110',
  'debit:commit:80'
]
```

The debit operation's stale read is `100`. Its `await` is the suspension boundary. The credit continuation is the competing commit and publishes `110`. The debit continuation then assigns `100 - 20`, overwriting the committed `110`. No two JavaScript statements ran simultaneously. The race exists because two logical transitions interleaved across the suspension boundary.

Naming those four points—stale read, suspension boundary, competing commit, overwritten state—turns "async timing issue" into a diagnosis. The same pattern appears in caches, revision updates, token refresh, inventory, and read-modify-write API handlers.

2. Carry ownership, invariants, and commit points across `await`

An `await` does not invalidate Chapter 5's six-part transition discipline. It makes observation timing explicit. The owner still decides which operation may publish state; the invariant still defines valid next state; the commit point still identifies when readers can observe the replacement.

Three repairs address different contracts. **Serialize** transitions when they must observe and commit in admission order. **Re-read before commit** when a calculation can be repeated against the latest state. **Reject stale proposals** by capturing a revision and comparing it immediately before commit. Each repair must keep the check and in-memory commit in one synchronous segment so another Job cannot run between them on the same agent.

A revision check supplies qualified in-memory protection only. It does not exclude another process, make a network write atomic, survive process termination, or coordinate independent database clients. A database

transaction, conditional write, or lock can enforce a later storage boundary; this chapter does not simulate those guarantees with an in-memory promise queue.

Do not hold a stale domain proposal merely because an asynchronous dependency was necessary. Separate preparation from publication: obtain external data, then ask the state owner to validate the complete proposal against current state and perform one narrow commit. If external work itself commits partial effects, the workflow must expose those effects rather than pretending the final in-memory assignment made the whole operation atomic.

3. Serial, unbounded, and bounded initiation are different contracts

Serial execution starts the next operation only after the preceding promise settles:

```
const results: Receipt[] = [];
for (const item of items) {
  results.push(await process(item));
}
```

This shape is correct when order, dependency, or capacity requires one active operation. It may be unnecessary serialization when operations are independent and the resource can safely support more than one.

Unbounded concurrent initiation calls every operation while constructing the array:

```
const operations = items.map((item) => process(item));
const results = await Promise.all(operations);
```

`map` calls `process` once for every visited element before returning. If `process` initiates I/O before its first suspension, every call has already initiated work before `Promise.all` receives the promise array. `Promise.all` attaches fulfillment and rejection reactions and produces an aggregate promise. It does not reach backward into `process` to limit starts.

Bounded concurrency requires an admission algorithm: a semaphore, fixed worker pool, or queue. The lab uses a fixed number of worker loops. Each worker claims at most one logical item at a time, retains that slot through all attempts and cleanup for the item, and claims another item only after the preceding outcome is complete. If `active` counts claimed but incomplete logical items and `limit` is a positive integer, the invariant is `0 <= active <= limit`.

The lab admits a fixed input array in index order and preserves that order in the outcome array. It does not guarantee completion order. It also does not claim universal fairness: host scheduling, an operation that never settles, multiple runner instances, external service queues, and process starvation lie outside this worker pool's authority. "FIFO admission for this queue" is a narrower and testable statement than "the system is fair."

4. Cancellation closes admission and requests active work to stop

The runner checks its `AbortSignal` before claiming another input. Once the signal is aborted, unstarted items receive rejected outcomes with zero attempts. Every acquired or pending attempt receives the same signal through its context. An abort-aware operation checks pre-abort, subscribes while pending, observes later cancellation at cooperative boundaries, and removes its listener when it settles.

That contract separates two responsibilities. The runner guarantees that it will not admit another item after observing abort. The operation decides how and when its active work can stop. A synchronous loop that never checks the signal, a foreign API that ignores it, or an external request already committed can continue. Cancellation does not preempt JavaScript or roll back completed effects.

Cleanup remains mandatory after cancellation. In the lab, successful acquisition returns a lease. Exactly one runner path invokes `lease.close()` once for that acquired attempt, whether `run` fulfills, rejects, or observes abort. Acquisition failure produces no lease and therefore no close. If `run` and `close` both fail, `AttemptCleanupFailure` retains both failure identities instead of allowing cleanup to erase the primary failure.

The runner commits only a successfully prepared value after cleanup and a final abort check. That contract is deliberately narrow: the attempt's `run` method must not perform the committed effect represented by the commit store. If it sends email, charges a card, or writes another service before rejecting, the effect has crossed a different boundary and needs its own idempotency or compensation design.

5. Retry is an explicit policy, not a reaction to any rejection

A retry policy answers five questions:

1. Which failure representations are classified as transient, permanent, or uncertain?
2. How many total attempts may occur?
3. What delay or backoff follows each failed attempt?
4. Does cancellation prevent waiting and the next attempt?
5. Which effects may already have committed before the failure was observed?

The attempt number counts initial execution as attempt one. `maxAttempts: 3` therefore permits at most two retries. Delay and exponential backoff are application algorithms, not promise semantics; inject the wait operation so tests can complete it without wall-clock sleeps. The wait must accept the workflow's signal so cancellation stops the backoff and prevents the next acquisition.

Do not retry permanent validation, authentication, authorization, or unsupported-operation failures by folklore. Repeating identical invalid input does not supply new evidence. A transport reset or capacity response may be transient under a named service contract. An unknown failure after a partial external write is not automatically safe: retrying can duplicate the effect.

Each attempt owns its acquired resource and completes cleanup before the retry classifier admits another attempt. A retry loop that leaks one lease per failure satisfies an attempt count while violating resource lifetime. A cleanup failure also changes classification because the resource's final state may be uncertain.

6. Idempotency constrains committed effects

For this chapter, **idempotency** means that repeated processing of the same logical request produces no additional committed effect under a stated key, scope, and retention boundary. The key identifies sameness, the scope identifies which component can see prior processing, and retention states how long that evidence remains available.

The lab's `InMemoryCommitStore` uses a string key, one store instance as scope, and the lifetime of its map as retention. The first synchronous `commit(key, value)` stores a prepared result. Later commits with that key return the stored result and do not add another entry. Because the lookup-and-set occurs in one synchronous call on one agent, concurrently prepared duplicates still produce one in-memory committed entry.

This is not durability. Restarting the process, creating another store instance, running a second process, or deleting an entry removes the evidence. The store cannot prevent an effect performed inside `run` before its own commit point. Durable deduplication requires storage with an atomic key constraint or transaction at the appropriate service boundary, which later chapters supply.

Idempotency is not retry. Retry decides whether and when to attempt processing again. Idempotency constrains the committed effect when the same logical request is processed again. A workflow can retry a non-idempotent operation and duplicate effects; an idempotent operation can be called repeatedly without any retry policy at all.

7. Partial success requires per-item outcomes

Batch work can produce some fulfilled items, some permanent refusals, some exhausted transient failures, and some cancellations. Rejecting one aggregate promise at the first observed failure hides those sibling outcomes from the aggregate's rejection value even though the sibling operations continue. `Promise.all` does not undo their effects or cancel their promises.

`Promise.allSettled` observes every input and returns one settlement record per input position. A workflow runner can provide a domain-specific equivalent containing item identity, value or reason, attempt count, and

commit source. Preserving input association lets a caller retry selected failures, report successful commits, and distinguish an unstarted cancellation from a failed active attempt.

Outcome aggregation is an observation contract, not transaction atomicity. If a batch must commit all items or none, collecting records afterward is too late; the enforcing database or state owner must support a single transaction over the complete proposal. The Chapter 9 lab intentionally permits partial success and reports it accurately.

Consequences for program design

Make admission, completion, and commit separate events in traces and tests. Record item ID, idempotency key, attempt, start, cleanup, classification, and commit source. Assert maximum active count rather than elapsed time. Assert that cancellation leaves unstarted items at zero attempts, that every acquired lease closes once, and that a repeated key adds no committed entry.

Choose the concurrency limit from a named capacity boundary: connection pool size, service quota, memory budget, or downstream contract. A constant with no stated resource merely moves an unexplained assumption into code. Measure and revise the limit in production; JavaScript cannot infer external capacity from a promise type.

False models, repaired

- **"Single-threaded JavaScript cannot have races."** Logical operations can interleave across callbacks or `await` continuations and overwrite stale state on one agent.
- **"Run-to-completion makes my async transition atomic."** It protects one synchronous segment, not read and commit segments separated by suspension.
- **"`Promise.all(items.map(start))` limits concurrency."** `map` has already called `start` for each visited item; `Promise.all` observes the returned promises.
- **"A worker pool is fair."** This runner promises FIFO admission for one fixed queue and an active-count bound, not completion order or universal scheduling fairness.
- **"Abort stops active work."** Abort records and communicates a request; active operations must observe it and perform their own stopping and cleanup.
- **"Any rejection should be retried."** Classification, attempt limits, backoff, cancellation, and possible prior effects decide whether another attempt is permitted.
- **"Retry makes an operation idempotent."** Retry repeats processing; idempotency prevents another committed effect for the same logical request within a named boundary.
- **"An in-memory key protects every deployment."** Its evidence is process-local, non-durable, and lost outside the store's scope or retention lifetime.
- **"An aggregate rejection means no work succeeded."** Sibling operations may already have fulfilled or committed; preserve per-item outcomes.

Mastery questions

1. Identify the stale read, suspension boundary, competing commit, and overwritten state in two async balance adjustments.
2. Transform the race so the state owner validates a revision and commits in one synchronous segment. What cross-process failures remain possible?
3. Compare serial execution, `items.map(start)` followed by `Promise.all`, and a two-worker pool by initiation time, maximum active count, and result order.
4. Prove why adding `Promise.all` after four calls cannot reduce the number of operations those calls already initiated.
5. State the active-count invariant for a semaphore or worker pool and construct a deterministic test that would falsify it.
6. Give one fairness guarantee a local FIFO queue can supply and three scheduling or completion guarantees it cannot supply.

7. Trace cancellation through two active items and two queued items. Which receive the signal, which have cleanup obligations, and which report zero attempts?
8. Design a retry classifier for transient capacity failure, malformed input, invalid credentials, and uncertain failure after a partial write. Defend every decision.
9. Explain why cleanup belongs inside each attempt rather than outside the retry loop. How should competing operation and cleanup failures be represented?
10. Define an idempotency contract for a payment request by key, scope, retention, and committed effect. Then state what process restart invalidates in an in-memory version.
11. Construct one retry without idempotency and one idempotent operation without retry. What property differs?
12. A batch produces two successes, one validation failure, and one cancellation. Design the returned outcome records and explain why they do not create an all-or-nothing transaction.

The full closed-book workspace and lab specification are in `exercises/ch09.md`.

Implementation lab — Deterministic Bounded Workflow Runner

Implement `InMemoryCommitStore` and `runBoundedWorkflow` in `examples/ch09/lab/starter.ts`. Use a fixed number of worker loops, preserve input/result association, stop claiming inputs after abort, pass the signal to active attempts, close each acquired lease exactly once, apply retry classification and attempt limits, and commit through one process-local idempotency store. Return every item as a fulfilled or rejected outcome rather than rejecting the whole batch.

The evaluator uses controlled deferred operations. Its tests prove maximum active count, input/result association, cancellation admission, signal propagation, cleanup counts, retry classification, attempt limits, repeated-key behavior, competing failures, and the deliberate `Promise.all(items.map(start))` limit fault without wall-clock sleeps.

Presentation prompt

Give a 10–15 minute explanation in your own words covering logical interleaving, the four-point stale-state diagnosis, Chapter 5 ownership and commit discipline, all three initiation contracts, the active-count invariant, the fairness boundary, cooperative cancellation, exactly-once cleanup, retry policy, idempotency boundaries, and partial outcome aggregation. Include one code prediction, one derivation from Chapters 6–8, one false model, one database boundary, and one unresolved uncertainty. The structured checklist is in `presentations/prompts/ch09.md`.

Ready-when test

Proceed only when, without notes, you can diagnose all four points of a stale asynchronous write; preserve Chapter 5's owner, invariant, and commit point across suspension; distinguish serial, unbounded, and bounded initiation; prove an active-count bound without claiming universal fairness; propagate cancellation while preserving exactly-once cleanup; specify retry classification, attempts, delay, and cancellation; define idempotency by key, scope, retention, and effect; explain why the in-memory store is process-local and non-durable; aggregate partial outcomes; implement and test the controlled runner; and state where database transactions or locks must provide a later guarantee.

Chapter 10

Inference, Annotations, and Sets of Possible Values

Chapter type: Acquisition
Prerequisites: Chapters 1–9; JavaScript values, bindings, object identity, functions, collections, state transitions, asynchronous workflows, and strict TypeScript compilation
Capabilities introduced: Read types as compile-time descriptions of possible runtime values and checked operations; trace inference and widening; apply directional assignability; choose among annotations, assertions, `as const`, and `satisfies`; interpret object, function, union, and intersection types
Earlier chapters integrated: Runtime values and mutation from Chapters 1 and 5; function calls and callbacks from Chapters 2 and 4; workflow values and outcomes from Chapters 7–9
Later chapters enabled: Narrowing and exhaustiveness; structural typing; generics; type construction; domain modeling; erasure and runtime validation
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch10/src/`, `examples/ch10/test/`, and `examples/ch10/lab/`

Compressed thesis

TypeScript analyzes source expressions, assignments, calls, and declarations before JavaScript executes. For a binding, property, parameter, or return position, the checker either infers a type from available evidence or checks an explicit type. Read that type as a compile-time description of possible runtime values and the operations permitted without further evidence. The checker does not create a runtime container that admits or rejects values; ordinary JavaScript evaluation still creates, passes, mutates, and compares the values described.

The set-of-values interpretation makes simple relations usable: `"queued"` describes one string value, `string` describes many string values, and `"queued" | "running"` describes either alternative. Assignability is directional: the checker asks whether a source type may be used at a target position. A value described only as `string` is not assignable to a target that permits only two literals, even if its present runtime value happens to be `"queued"`. The set interpretation is a reasoning aid, not a statement about compiler storage or a complete formalization of TypeScript; contextual inference, structural relations, `any`, type parameters, and later type operators require additional rules.

Inference preserves information when the checker has evidence and widens information when a mutable location must admit future writes. An annotation constrains and checks a location. A type assertion asks the checker to use a different description without adding runtime evidence. A `const` assertion prevents selected literal widening and adds readonly descriptions to literal members; `satisfies` checks an expression against a target while retaining useful inferred information. None performs runtime validation. This canonical chapter retains that shared technical spine; explanations required by an observed reader gap belong in source-linked sections of the personal appendix rather than expansions of the chapter.

1. Types describe runtime possibilities; JavaScript supplies the values

Chapter 1 distinguished a binding from the value it contains. TypeScript adds a compile-time description associated with expressions and named locations. If the checker describes `state` as `"queued" | "running"`, a runtime read of `state` still produces one JavaScript string value, not a union object or a bag containing two candidates. The union records what the checker considers possible before that read executes.

A type also controls which source operations the checker accepts. For `string | number`, both alternatives support `toString`, so the checker accepts `value.toString()`. Only strings support `toUpperCase`, so the checker rejects `value.toUpperCase()` until later control-flow evidence excludes the number alternative. Chapter 11 develops that exclusion process. Chapter 10 stays with the description available before narrowing.

Chapter 9's bounded runner returns ordinary per-item outcome objects: each completed array element is one fulfilled or rejected record with an item identity, attempt count, and result-specific fields. TypeScript can describe those alternatives with a union and preserve a literal tag on each source object. The runtime array still contains individual objects scheduled and created by JavaScript. The union neither schedules the attempts nor aggregates the outcomes; it records the alternatives that checked consumers must be prepared to receive.

The set metaphor is strongest for ordinary literal unions: the possible values of `"queued"` are contained among the possible values of `string`. It becomes incomplete when treated as compiler implementation detail. The checker represents and relates types through algorithms, contextual information, object members, signatures, type parameters, and special types such as `any`; it does not enumerate every JavaScript value. Use "set" to reason about admitted values and safe operations, not to predict the checker's internal data structures.

2. Inference follows evidence from initializers, returns, calls, and context

Inference means that the checker derives a type without a written annotation at that location. A variable initializer supplies evidence for the variable. Return expressions supply evidence for a function's return type. Arguments and parameters can supply candidates for a generic call. A function expression can receive parameter types from the callback position where it appears.

Predict before checking: Which declarations retain one literal, which admit later string assignments, and what runtime array is returned?

```
function outcome(succeeded: boolean) {
  return succeeded ? "committed" as const : "retry" as const;
}

const initial = "queued";
let current = "queued";
current = "running";

console.log([initial, current, outcome(false)]);
```

Output:

```
[ 'queued', 'running', 'retry' ]
```

The runtime output follows ordinary binding assignment and function evaluation. During checking, `initial` has literal type `"queued"` because its `const` binding cannot be rebound. `current` is widened to `string` because its `let` binding admits later string values. The two return expressions cause `outcome` to have return type `"committed" | "retry"`. Those inferred descriptions do not alter the three runtime strings.

Generic inference expresses relationships among positions and belongs primarily to Chapter 13. One boundary is useful now:

```
function identity<Value>(value: Value): Value {
  return value;
}

const retained = identity("queued"); // "queued"
```

The call infers a type argument from `"queued"`, and the result retains that literal type because the declaration relates its input and output through `Value`. That is different from guessing a runtime value. The checker is solving a declared compile-time relationship using the call expression as evidence.

Contextual typing runs from a known use site into an expression. If `processSteps` accepts `(step: WorkflowStep ) => string`, then the unannotated parameter in `processSteps(step => step.name)` is checked as `WorkflowStep`. Moving the same arrow into a standalone declaration removes that call-site context; under this repository's `noImplicitAny` rule, an otherwise untyped parameter is rejected. Context changes checker evidence, not the argument JavaScript passes when the callback runs.

3. Assignability is a directional question

For an assignment, argument, return, or initializer with a declared target, TypeScript compares a **source type** with a **target type**. "Compatible" is too vague because reversing the direction can reverse the result.

```
type WorkflowState = "queued" | "running" | "committed";

const precise = "queued";
const general: string = precise;       // source "queued" to target string
const state: WorkflowState = precise;  // source "queued" to target union

declare const uncertain: string;
// const rejected: WorkflowState = uncertain;
```

The first two assignments are accepted because every value described by the source literal type is admitted by each target. The commented assignment is rejected because `string` includes values such as `"cancelled"` that the target union does not admit. A programmer's belief that `uncertain` currently contains `"queued"` is not checker evidence.

This direction also governs calls: the argument expression supplies the source type, and the parameter supplies the target. It governs returns: each returned expression supplies a source checked against the declared return type. Structural object assignability adds further rules and hazards in Chapter 12, so the subset intuition must not be mistaken for the entire assignability algorithm.

4. Annotations constrain; assertions replace evidence

An annotation declares the target description for a location and asks the checker to verify incoming values. In `let state: WorkflowState = "queued"`, the initializer is checked against `WorkflowState`, and later assignments must also be assignable to that target. A return annotation checks every reachable return expression. A parameter annotation checks argument expressions at calls visible to the checker.

A type assertion operates differently:

```
const text: string = "cancelled";
const asserted = text as WorkflowState;
```

The assertion tells the checker to treat the expression as `WorkflowState` within TypeScript's permitted assertion rules. It neither inspects the string nor changes it. At runtime, `asserted` contains `"cancelled"`. An assertion can therefore suppress or replace useful checker evidence. Chapter 16 treats assertion boundaries and dual type/value

constructs in depth; here the governing rule is simple: an annotation invites a check against a declared target, while an assertion supplies no runtime proof that the target description is true.

Prefer inference when the inferred type expresses the intended contract. Add an annotation when an API boundary, mutable location, or intended abstraction needs a stable target. Use an assertion only when evidence exists outside the checker's view and make that evidence reviewable. External JSON, environment variables, and database rows require runtime parsing or validation, not assertions.

5. Literal types, widening, and `as const`

Literal information is useful when a value selects behavior: workflow state, command kind, HTTP method, or result tag. TypeScript nevertheless widens a property when ordinary mutation is plausible:

```
const binding = "queued";                  // "queued"
const mutable = { state: "queued" };       // { state: string }
const described = { state: "queued" } as const;
// { readonly state: "queued" }
```

The `const` declaration prevents rebinding `mutable`; it does not prevent `mutable.state = "running"`, so the inferred property type admits other strings. The `as const` expression prevents literal widening within that literal and gives object properties readonly descriptions. It does not call `Object.freeze`, recursively secure aliases, or change emitted runtime behavior. `Object.isFrozen(described)` is `false` unless application code separately freezes the object.

Preserving every literal is not always desirable. A mutable status variable intended to move through many strings should not be trapped at its initializer's one literal. Conversely, widening a command's `kind` from `"query"` to `string` discards information later checks could use. The correct precision follows the location's intended writes and consumers, not a universal preference for narrower types.

6. Object types, function types, and optional presence

An object type describes required and optional properties and the types of their values. It does not allocate an object, inspect an object received from an untyped source, or establish runtime identity. A function type describes accepted arguments and returned values for checked calls; it does not wrap the function or monitor calls made by unchecked JavaScript.

```
type Step = { name: string; retryAfterMs?: number };
type DescribeStep = (step: Step) => string;

const describe: DescribeStep = step => step.name;
```

The function annotation contextually types `step` as `Step`. The checker verifies the arrow's return expression against `string`. When JavaScript calls `describe`, it passes one object value using the Chapter 1 argument rules; no type object accompanies that value.

With `exactOptionalPropertyTypes`, `retryAfterMs?: number` means the property may be absent, or may be present with a number. It does not permit an explicit `undefined` write unless the declared property type includes `undefined`. A read still produces `number | undefined`, because JavaScript returns `undefined` when the property is absent. Runtime presence remains observable: `"retryAfterMs" in {}` is false, while it is true for `{ retryAfterMs : undefined }`. The compiler option aligns writes with that distinction; it does not create the distinction.

7. Unions express alternatives; intersections require all constraints

A union `A | B` describes values admitted by either member. Before further evidence, the checker accepts only operations valid for every possible member. A union does not mean that a runtime value simultaneously has both forms, nor does it dispatch automatically.

An intersection `A & B` describes values that satisfy both member constraints. For object types, one runtime object can contain all required properties:

```
type Identified = { id: string };
type Timed = { createdAt: string };
type StoredRecord = Identified & Timed;

const record: StoredRecord = {
  id: "job-17",
  createdAt: "2026-07-21T00:00:00.000Z",
};
```

The `&` operator did not merge two runtime objects. The object literal evaluation created one object, and the checker verified that its type satisfied both constraints. Intersections can also conflict: `{ state: "queued" } & { state: "done" }` requires a property value satisfying both distinct literals. No ordinary string does, and later chapters name such impossible possibilities with `never`.

8. `satisfies` checks a target while retaining useful inference

A broad annotation can discard distinctions the initializer supplied. `satisfies` checks that an expression is assignable to a target but leaves the expression with useful inferred property information:

```
type CommandSpec = {
  kind: "query" | "mutation";
  run: (subject: string) => string;
};

const commands = {
  inspect: {
    kind: "query",
    run: subject => `inspect:${subject}`,
  },
  rebuild: {
    kind: "mutation",
    run: subject => `rebuild:${subject}`,
  },
} as const satisfies Record<"inspect" | "rebuild", CommandSpec>;
```

The target checks the required keys, each `kind`, and each callback. It also contextually types `subject` as `string`. The inferred type of `commands.inspect.kind` remains `"query"`, so consumers do not lose the per-entry distinction. By contrast, annotating the variable as a table of `CommandSpec` values describes each `kind` only as `"query" | "mutation"`.

`satisfies` is not runtime validation. If unchecked code, an assertion, or external data produces a malformed table, the emitted JavaScript contains no `satisfies` check. Its value is at the source boundary: verify a shape while preserving distinctions that downstream checked code can use.

Consequences for program design

Place annotations where they express a durable contract: exported function parameters and results, mutable state boundaries, callback protocols, and domain interfaces. Allow local inference to retain details that the implementation has already established. Excessive annotations can erase useful literals or duplicate facts; missing boundary annotations can let accidental implementation changes silently redefine an API.

Preserve discriminants at their source. A command table whose entries retain `"query"` and `"mutation"` can support later exhaustive reasoning, while a table widened to `string` cannot. That static precision does not validate configuration loaded at runtime. Parsing remains responsible for establishing that external values actually have

the required fields and literals.

Treat checker acceptance as one source of evidence. It proves that the checked source obeyed the configured assignability rules using the declarations available to TypeScript 6.0.3. It does not prove that declarations describe external libraries correctly, that assertions are honest, that untyped callers comply, that runtime values were validated, or that asynchronous workflows satisfy operational invariants.

False models, repaired

- **"A type is a container around the runtime value."** JavaScript holds the value; TypeScript associates compile-time descriptions with source expressions and locations.
- **"Inference means TypeScript inspects execution."** The checker derives descriptions from source evidence before execution.
- **"Assignable means the two types are interchangeable."** Assignability compares a source to a target; reversing them can change the result.
- **"An annotation converts the initializer."** It checks the initializer and constrains the location; JavaScript receives the same emitted value expression.
- **"A type assertion proves the value has that type."** It supplies no runtime evidence and can discard checker information.
- **"`const` preserves every literal inside an object."** It prevents binding rebinding; mutable properties normally widen.
- **"`as const` freezes the object."** It changes inferred literal and readonly descriptions, not runtime property descriptors.
- **"Optional means present with `undefined`."** Under `exactOptionalPropertyTypes`, absence and explicit `undefined` are distinct unless `undefined` is declared.
- **"A union value contains all members."** One runtime value is among the alternatives; only common operations are accepted before further evidence.
- **"An intersection merges objects at runtime."** It requires one value to satisfy all member constraints; runtime construction is separate.
- **"`satisfies` validates loaded configuration."** It checks a source expression during TypeScript analysis and emits no validator.

Mastery questions

1. Explain what exists at runtime when a binding has type `"queued" | "running"`. What does not exist?
2. Predict the inferred types of `const status = "queued"`, `let status = "queued"`, and `const record = { status: "queued" }`. Derive each result from permitted writes.
3. A function returns `"done" as const` on one path and `"retry" as const` on another. Derive its inferred return type without executing it.
4. Reverse the assignment `const broad: string = "queued" as const`. Why does `string` fail as a source for target `"queued"`, although the first direction succeeds?
5. Transform an over-annotated command table so every entry is checked while its literal `kind` remains available.
6. Diagnose a callback whose parameter becomes implicit `any` after the callback is moved out of a typed call expression.
7. Compare an annotation, ordinary type assertion, `as const`, and `satisfies` by checker operation, retained evidence, emitted JavaScript, and remaining runtime failures.
8. Explain why `{ retryAfterMs: undefined }` and `{}` differ at runtime. State the static write rule under `exactOptionalPropertyTypes`.
9. Design an object type for a retry step where omission and explicit reset-to-default must be different operations.
10. For `string | { id: string }`, list operations the checker can accept before narrowing. Do not perform the narrowing.
11. Construct one object satisfying `{ id: string } & { createdAt: string }`. Then construct an intersection whose property requirements cannot both be satisfied.

12. Explain why an annotation on a runtime-loaded JSON value is not validation. Name the component that must establish the facts.
13. Transfer the set interpretation to a union of three domain result literals, then identify one point where the metaphor does not specify TypeScript's algorithm.
14. An async workflow returns `{ outcome: "committed" }` and `{ outcome: "retry" }` from separate paths. Show two ways to preserve the literal alternatives without teaching control-flow narrowing.

The full closed-book workspace and lab specification are in `exercises/ch10.md`.

Implementation lab — Checked Command Table

Complete `examples/ch10/lab/starter.ts`. Preserve each command's literal `kind` while checking the entire object against `CommandTableShape`. Implement the two handlers, derive exported callable type aliases from the handler values with `typeof`, and keep optional timeout presence exact. Runtime tests verify command results and property presence; static assertions verify the retained discriminants and contextual parameter types.

After your implementation passes, inspect `lab/evaluator/incorrect-broad-annotation.ts`. Its broad variable annotation accepts the values but widens each entry's `kind` to the full union. Explain why this is a loss of static evidence rather than a runtime mutation.

Presentation prompt

Give a 10–15 minute explanation in your own words covering types as descriptions of possible values, the boundary of the set metaphor, four inference sources, directional assignability, annotations versus assertions, literal widening, `const` versus `as const`, exact optional properties, unions, intersections, contextual callbacks, and `satisfies`. Include one Chapters 1–9 runtime value, one prediction, one causal derivation, one false model, one runtime-validation boundary, and one unresolved uncertainty. The structured checklist is in `presentations/prompts/ch10.md`.

Ready-when test

Proceed only when, without notes, you can separate a runtime value from its inferred or declared type; derive initializer, return, generic-call, and contextual inference in unfamiliar code; evaluate assignability in the correct direction; choose among an annotation, assertion, `as const`, and `satisfies`; predict literal widening and optional-property presence; interpret unions and intersections without inventing runtime containers; implement and statically verify the command table; and present the chapter without treating checker acceptance as runtime validation.

Chapter 11

Narrowing, unknown, never, and Exhaustive Reasoning

Chapter type: Integration
Prerequisites: Chapter 10; JavaScript runtime control flow, equality, property lookup, and prototypes
Capabilities introduced: Establish runtime evidence for uncertain values; predict TypeScript control-flow narrowing; distinguish `unknown` from `any`; audit user-defined predicates and assertion functions; derive exhaustive handling from discriminated unions and `never`
Earlier chapters integrated: Runtime values and execution from Chapters 1–9; possible-value descriptions, inference, assignability, and erasure from Chapter 10
Later chapters enabled: Structural typing, trusted domain construction, impossible-state design, runtime truth, and external-data parsing
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch11/src/`, `examples/ch11/test/`, and `examples/ch11/lab/`

Compressed thesis

TypeScript's control-flow analysis narrows the possible values of a binding at a source position when guards, assignments, and reachability supply evidence. A JavaScript `typeof`, equality, property-presence, prototype-chain, or discriminant check executes at runtime; the checker recognizes selected expression forms and permits operations supported by the remaining alternatives. The checker does not execute the program, inspect future external values, or add runtime validation to emitted JavaScript.

`unknown` can describe any incoming value while requiring evidence before value-specific operations. `any` suppresses those constraints: property access, calls, assignments, and results can carry an unchecked assumption into otherwise checked code. A user-defined predicate or assertion function can package runtime evidence, but its signature transfers responsibility to the implementation; TypeScript trusts the declared refinement and does not prove the check sufficient.

When control flow eliminates every member of a union, the remaining static type is `never`. No runtime `never` value is created. That result supports exhaustive discriminated-union handling: a new variant fails checking where the supposedly unreachable value must be assignable to `never`. This canonical chapter preserves the shared technical spine; explanations required by an observed reader gap belong in source-linked sections of the personal appendix rather than expansions of the chapter.

1. Narrowing refines possible values along control-flow paths

Chapter 10 treated a type as a compile-time description of possible runtime values and permitted operations. Narrowing makes that description position-sensitive. Given `string | number`, the checker initially permits only

operations safe for both alternatives. Inside a branch guarded by `typeof value === "string"`, it describes `value` as `string`; on a path where that branch returned, the checker can exclude `string` from the code that follows.

The declared type remains the assignment target. Narrowing produces an **observed type** at one source position; it does not permanently rewrite the declaration. If `let result: string | number` currently holds a number, `result.toFixed()` is accepted there. A later string assignment remains legal because its source type is checked against the declared target `string | number`, after which the observed type becomes `string`.

Control-flow analysis follows source structure rather than executing the function. It tracks recognized guards, assignments, branch splits, returns, throws, and rejoined paths. Runtime execution supplies one actual value and takes one path. Type annotations and inferred refinements generally do not survive emission, so no runtime union object shrinks when a branch runs.

2. `unknown` demands evidence; `any` suppresses constraints

Both `unknown` and `any` can receive any JavaScript value. Their permitted operations differ. The checker rejects property access, calls, arithmetic, or assignment to a narrower target from `unknown` until source evidence supports the operation. This makes `unknown` appropriate at a boundary whose caller may supply arbitrary data: uncertainty remains visible in every consumer.

`any` disables ordinary checking for the affected expression. A property read from `any` is usually also `any`, an `any` value is assignable to most targets, and an unchecked result can therefore contaminate downstream inference. The annotation `const delayMs: number = unchecked.delayMs` does not convert or inspect the property; if the runtime property is a string, the binding still holds a string.

Predict before running: What are the runtime type and sum, despite the annotation?

```
const unchecked: any = { delayMs: "500" };
const delayMs: number = unchecked.delayMs;

console.log(typeof delayMs, delayMs + 1);
```

Output:

```
string 5001
```

By contrast, an `unknown` input requires the program to perform the relevant check:

```
function format(value: unknown): string {
  if (typeof value === "string") return value.toUpperCase();
  if (typeof value === "number") return value.toFixed(1);
  return "unsupported";
}
```

The two `typeof` expressions run in JavaScript and protect the corresponding operations. TypeScript recognizes their control-flow consequences. The return annotation checks the result but validates no input by itself.

3. A guard is only as precise as its runtime operation

Narrowing must preserve the exact JavaScript boundary of the recognized expression.

- `typeof` distinguishes primitive categories and functions, but `typeof null` is `"object"`. An object branch that must exclude `null` needs `value !== null`. Arrays, dates, and ordinary records all report `"object"`.
- Equality against a literal can retain union members compatible with that literal. `value !== null` removes only `null`; `value != null` deliberately removes both `null` and `undefined` according to JavaScript's loose-equality rules.
- A truthiness check excludes falsy values on the true path. It can therefore discard valid `0`, `""`, `false`, `0n`, and `NaN` along with `null` and `undefined`. Use an explicit nullish check when those values belong to the domain.

- `"field" in value` requires an object operand at runtime and reports properties found either on the object or its prototype chain. For union narrowing, a member with an optional property can remain possible on both branches. Use `Object.hasOwn` when own-property presence is the required runtime fact.
- `value instanceof Constructor` normally tests whether `Constructor.prototype` occurs in the value's prototype chain, and `Symbol.hasInstance` can customize the test. It does not structurally validate every property expected by a TypeScript class or interface.
- A comparison such as `command.kind === "rename"` can narrow a discriminated union when each member gives the common property a distinct literal type.

Predict before running: Which fallback preserves zero?

```
const value: number | null = 0;
console.log(value ? value : "fallback");
console.log(value ?? "fallback");
console.log(typeof null);
```

Output:

```
fallback
0
object
```

The first conditional tests truthiness. The nullish coalescing operator uses its right operand only for `null` or `undefined`; it does not itself validate an arbitrary `unknown` value as a number.

4. Assignment, reachability, aliases, and callbacks bound the evidence

Early exits often express a guard more clearly than a nested positive branch:

```
function render(value: string | number | null): string {
  if (value === null) return "missing";
  if (typeof value === "number") return value.toFixed(1);
  return value.toUpperCase();
}
```

At the final return, reaching that statement excludes `null` and `number`, so only `string` remains. A throwing branch supplies the same reachability fact because the call cannot return normally on that path.

Assignments change evidence. After `value = 3`, the observed type can be `number`; after `value = "done"`, it can be `string`. An assignment to a narrowed binding or property can invalidate the earlier refinement. A `const` boolean holding `typeof value === "string"` can carry the recognized condition into a later `if` when neither the boolean nor the guarded parameter is reassigned.

Callbacks require a lifetime question. A captured `let` binding may be reassigned before a queued or deferred callback executes, so the checker does not generally preserve an outer narrowing for that mutable binding inside the callback. Copy the proven value into a `const` binding when the callback needs a stable snapshot. That snapshot preserves the copied value, not deep immutability of an object reached through it.

Aliasing creates a second boundary. A function call can mutate an object through another access path even where TypeScript retains a property refinement across the call. Checker acceptance is not proof that an arbitrary callee is pure or that an object has unique ownership. When correctness depends on a mutable property, copy the validated primitive, prevent mutation by contract and runtime design, or recheck after the call.

5. Predicates and assertion functions transfer proof obligations

A function returning `value is TextEnvelope` is a user-defined type predicate. On its true branch, the checker uses `TextEnvelope` as though the implementation established every required fact. An assertion signature `asserts`

`value is TextEnvelope` says that normal return establishes the same description; the implementation should throw when the facts do not hold.

```
interface TextEnvelope {
  kind: "text";
  text: string;
}

function isTextEnvelope(value: unknown): value is TextEnvelope {
  return typeof value === "object" && value !== null
    && "kind" in value && value.kind === "text"
    && "text" in value && typeof value.text === "string";
}
```

This predicate checks object-ness, non-nullness, the discriminant, property presence, and the field value's runtime category. The returned object remains the original identity; narrowing does not construct a domain object or freeze its properties.

Predict before running: Will TypeScript reject the call, and what does JavaScript do?

```
function isTextEnvelopeLying(value: unknown): value is TextEnvelope {
  return typeof value === "object" && value !== null
    && "kind" in value && value.kind === "text";
}

const candidate: unknown = { kind: "text", text: 17 };
if (isTextEnvelopeLying(candidate)) {
  console.log(candidate.text.toUpperCase());
}
```

The checker accepts `toUpperCase` because it trusts the predicate signature. JavaScript throws `TypeError` because the predicate never checked `text`. TypeScript verifies that the predicate declaration is syntactically meaningful and that its implementation returns a boolean; it does not prove the boolean condition implies the declared target type. Predicate and assertion-function implementations therefore deserve boundary tests with malformed values.

6. `never` records elimination and enables exhaustiveness

`never` describes no possible values. It appears when control flow eliminates every member of a union and as the return type of a function that cannot return normally, such as one that always throws. JavaScript never creates a special `never` value. A throwing `assertNever` function receives an ordinary runtime argument only if a static assumption was bypassed or the program's declarations were false.

A discriminated union supports reliable elimination because each member carries one distinct literal and the fields required for that alternative:

```
type DeliveryEvent =
  | { kind: "accepted"; id: string }
  | { kind: "delayed"; id: string; delayMs: number }
  | { kind: "rejected"; id: string; reason: string };

function assertNever(value: never): never {
  throw new Error(`Unhandled event: ${JSON.stringify(value)}`);
}

function summarize(event: DeliveryEvent): string {
  switch (event.kind) {
    case "accepted": return `accepted:${event.id}`;
    case "delayed": return `delayed:${event.delayMs}`;
    case "rejected": return `rejected:${event.reason}`;
    default: return assertNever(event);
  }
}
```

After the three cases, the checker finds no remaining `DeliveryEvent` member, so `event` is assignable to the `never` parameter. If the union gains `{ kind: "redirected"; id: string }`, the default branch contains that member and checking fails at `assertNever(event)`. The diagnostic's exact wording is not the contract; the required fact is that a remaining source type is not assignable to target `never`.

An alternative places `event satisfies never` after a `switch` whose cases all return or throw. This checks exhaustiveness without calling a helper. `assertNever` additionally provides a runtime failure if unchecked input reaches the branch. A generic `default: return "unknown"` defeats the static alarm because it accepts every future variant.

Exhaustiveness is relative to the declared union. It does not prove that external JSON belongs to that union. A runtime parser must first establish the discriminant and member fields; Chapter 17 develops that boundary. Nor does a closed union by itself make every application state valid; Chapter 15 combines unions with domain invariants.

Consequences for program design

Keep uncertain values uncertain at their entry point. An `unknown` parameter forces each consumer to establish facts, while a small parser or trustworthy guard can centralize repeated checks and return a closed union. Do not publish a predicate stronger than its runtime implementation.

Choose guards by the fact the operation actually establishes. Truthiness is appropriate when falsy values are intentionally absent; it is not shorthand for nullishness. `in` is appropriate for prototype-inclusive presence; it is not an own-property validator. `instanceof` is appropriate for a trusted runtime constructor relationship; it is not an interface check.

Preserve discriminants during inference and construction, then require exhaustiveness at transition, rendering, dispatch, and serialization boundaries. The resulting checker failure makes a newly added alternative visible at every exhaustive consumer. Runtime validation, tests, authorization, and persistence constraints remain separate responsibilities.

False models, repaired

- **"Narrowing changes the runtime value."** Runtime checks select a branch; the checker refines its possible-value description for source positions on that path.
- **"`unknown` means the value is invalid."** Any value is assignable to `unknown`; operations require evidence because its useful properties are not yet established.
- **"`any` is a more convenient `unknown`."** `any` suppresses constraints and can contaminate downstream results with false descriptions.

- **"`typeof value === "object"` proves a non-null record."** `null`, arrays, dates, and many other objects satisfy that result.
- **"A truthiness guard removes only missing values."** It also excludes valid falsy primitives such as zero and the empty string.
- **"`in` proves an own property with the expected value type."** It includes the prototype chain and establishes presence, not the property's value category.
- **"TypeScript verifies a type predicate's logic."** The checker trusts the declared predicate or assertion signature.
- **"A narrowing remains true after arbitrary mutation."** Assignments, callback timing, and aliases can invalidate the runtime fact.
- **"`never` is a runtime sentinel."** It is a static description used when no values remain or normal return is impossible.
- **"A default branch makes a switch safely exhaustive."** A permissive default can hide a newly added union member; a `never` check exposes it.

Mastery questions

1. For `unknown`, `any`, and `string | number`, list which operations the checker accepts before additional evidence and identify the remaining runtime risks.
2. Predict `typeof null`, `typeof []`, and `typeof (() => {})`. Design the checks needed to accept only a non-null record with an own `kind` property.
3. Transform a truthiness check on `number | null | undefined` so that zero remains valid. Explain the runtime and static differences.
4. For `"name" in value`, derive what JavaScript checks and why neither own-property status nor a string-valued `name` follows automatically.
5. Compare `instanceof Date` with checking an object-shaped `{ toISOString(): string }` interface. Which runtime fact does each approach establish or fail to establish?
6. Trace the observed and declared types of a `let` binding through a number assignment, a string assignment, two branches, and a merged path.
7. Rewrite a nested nullable function with early returns, then derive the remaining type at the final statement from reachability.
8. Diagnose why a narrowed captured `let` is unsafe in a deferred callback. Repair it with a stable primitive snapshot and state what the snapshot does not protect.
9. Construct a lying predicate that causes checked code to throw. Identify the exact missing runtime check.
10. Compare a boolean-returning guard, a type predicate, an assertion function, and a type assertion by runtime execution, checker evidence, and failure behavior.
11. Add a fourth member to a discriminated union and predict the checker result for a complete `assertNever` switch, a `satisfies never` switch, and a permissive default.
12. Explain both meanings of `never` used here: elimination of union alternatives and inability to return normally. Why does neither introduce a runtime value?
13. Design a dispatcher that accepts `unknown`, establishes a closed command union, and rejects malformed values before exhaustive handling.
14. Transfer exhaustiveness to a production result union containing success, domain refusal, retryable failure, and cancelled. Identify one correctness property the union still cannot enforce.

The closed-book workspace and lab specification are in `exercises/ch11.md`.

Implementation lab — Unknown Command Dispatcher

Complete `examples/ch11/lab/starter.ts`. Parse an `unknown` input into the closed `CounterCommand` union using runtime checks for every consumed field. Then implement an exhaustive reducer for increment, rename, and reset commands, and a dispatcher that rejects malformed input without replacing the prior state. Zero increments and empty labels are valid; truthiness checks must not discard them.

After the runtime and static checks pass, inspect `lab/evaluator/incorrect-any.ts`. Its `any` boundary admits

`{ kind: "increment", amount: "2" }`; the exhaustive reducer then performs string concatenation while its return type describes a number. Explain why exhaustive handling can be correct relative to a false input guarantee.

Presentation prompt

Give a 10–15 minute explanation in your own words covering control-flow evidence, `unknown` versus `any`, precise guard boundaries, assignment and reachability, callback or alias invalidation, predicate responsibility, discriminated unions, both meanings of `never`, and exhaustive checks. Include one prediction, one causal derivation, one lying predicate, one connection to Chapter 10's possible-value and assignability terms, one runtime-validation boundary, and one unresolved uncertainty. The structured checklist is in `presentations/prompts/ch11.md`.

Ready-when test

Proceed only when, without notes, you can derive a binding's narrowed type from guards, assignments, and reachability; choose a runtime check whose boundary matches the required fact; preserve zero and empty-string values when excluding nullish input; explain why `unknown` contains uncertainty while `any` suppresses constraints; audit a predicate or assertion implementation against its signature; repair a callback whose captured binding can change; add a discriminated variant and predict every exhaustive checker failure; implement and test the unknown command dispatcher; and present the chapter without treating checker acceptance as runtime validation.

Chapter 12

Structural Typing and Semantic Identity

Chapter type: Acquisition and integration
Prerequisites: Chapters 1, 3, 10, and 11; object identity, public class instances, directional assignability, and control-flow evidence
Capabilities introduced: Evaluate structural assignability by source and target; diagnose excess-property checks; separate public class shape from provenance; reason about readonly aliases; prevent semantic category swaps with static brands
Earlier chapters integrated: Runtime and domain identity from Chapter 1; class construction and private brands from Chapter 3; directional assignability from Chapter 10; runtime evidence and narrowing from Chapter 11
Later chapters enabled: Generic API design, type construction, domain modeling, runtime parsing, and durable public boundaries
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch12/src/`, `examples/ch12/test/`, and `examples/ch12/lab/`

Compressed thesis

TypeScript assignability is directional and primarily structural. When an expression supplies a source type for a target position, the checker asks whether the source supplies every member required by the target under the applicable rules; additional source members are usually permitted. Compatible members establish that checked code may perform the target's operations. They do not establish that the runtime value has a particular object identity, prototype, constructor history, validator history, authorization decision, or domain meaning.

Fresh object literals receive contextual excess-property checks that can catch misspelled or unintended keys. A variable holding the same properties can remain assignable to a narrower target. The difference is a checking rule for particular source expressions, not a general exact-object guarantee, and neither accepted assignment removes runtime properties. Public class instance types are also normally structural. TypeScript `private` and `protected` members add a declaration-origin requirement during compatibility checking; JavaScript `#private` fields instead supply a distinct runtime brand.

Structural representation is therefore insufficient when two categories share one representation but must not be interchanged. A declaration-scoped `unique symbol` phantom property can distinguish `UserId` from `OrderId` during checking, while trusted constructors or parsers control where each branded value is created. The phantom property does not exist on the emitted string. Assertions, untyped JavaScript, or unvalidated external data can still forge the checker's belief. This canonical chapter preserves the shared technical spine; explanations needed only after an observed reader failure belong in a source-linked section of the personal appendix.

1. Structural assignability compares a source with a target

Chapter 10 established the direction of an assignability question: an initializer or argument expression supplies a **source type**, and the annotated variable or parameter supplies a **target type**. For object types, TypeScript normally examines the target's required members and asks whether the source has corresponding members whose types are compatible. The comparison continues through nested properties and call signatures.

```
interface DispatchTarget {
  readonly id: string;
  deliver(message: string): string;
}

const worker = {
  id: "worker-17",
  region: "us-east",
  deliver(message: string) {
    return `${this.id}:${message}`;
  },
};

const target: DispatchTarget = worker;
```

The assignment is accepted because the source supplies `id` and `deliver` with compatible member types. The target does not require `region`, so its presence on the source does not cause ordinary structural comparison to fail. Reversing the example changes the question: `{ id: string }` cannot be used as a `DispatchTarget` because the source lacks `deliver`.

Member compatibility matters, not member names alone. A source with `id: number` does not satisfy target member `id: string`; a method must also satisfy the target call signature. TypeScript has additional rules for function parameters, overloads, optional members, index signatures, and generic positions. Chapter 13 treats those relationships. The usable rule here is narrower: object width permits extra source members, but every required target member must still be present and compatible under the configured checker.

Predict before running: What does JavaScript print, and did the assignment discard `region` or construct a new object?

```
console.log(target.deliver("ready"));
console.log(target === worker);
console.log(Reflect.get(target, "region"));
```

Output:

```
worker-17:ready
true
us-east
```

The annotation changes which source operations the checker accepts through `target`. JavaScript receives the original object value. The assignment copies access to the same runtime identity, and the `region` property remains present.

2. Excess-property checks are contextual, not exact object types

TypeScript gives a fresh object literal additional scrutiny when the literal is contextually checked against an object target. A key that the target does not recognize commonly produces an excess-property diagnostic:

```
interface DeliveryOptions {
  readonly priority: "normal" | "urgent";
}

const direct: DeliveryOptions = {
  priority: "urgent",
  // diagnostic: traceId is not a known DeliveryOptions property
  traceId: "trace-12",
};
```

The diagnostic is useful for option bags because an unexpected key often represents a misspelling or a mistaken API assumption. It does not mean that `DeliveryOptions` describes an object with exactly one runtime property. The same values can be staged in a variable and then supplied as the source for ordinary structural comparison:

```
const staged = {
  priority: "urgent" as const,
  traceId: "trace-12",
};

const accepted: DeliveryOptions = staged;
```

Predict before running: If a function receives `accepted`, which keys does `Object.keys` observe?

```
console.log(Object.keys(accepted).sort());
```

Output:

```
[ 'priority', 'traceId' ]
```

No conversion, projection, or property deletion occurred. Freshness and contextual target information affect TypeScript's diagnostic decision at the source expression. JavaScript later executes with the object that was created. Assertions can also suppress this diagnostic, but Chapter 10's boundary still applies: an assertion supplies no runtime evidence. If an API must reject unknown input fields, application parsing or validation must inspect them at runtime.

Excess-property behavior contains compiler details and special cases. State the concrete TypeScript baseline when an edge matters instead of treating "freshness" as a universal language law. Under this repository's TypeScript 6.0.3 configuration, the static fixture verifies that the direct literal is rejected and the staged variable is accepted.

3. Compatible shape does not establish identity or provenance

Structural assignability answers whether checked code may use the target's declared operations. It does not answer any of these runtime questions:

- Is this the same object observed earlier?
- Does this object inherit from a particular prototype?
- Did a particular constructor, parser, or database query produce it?
- Was its content authorized for the current caller?
- Do equal-looking fields denote the same domain entity?

Chapter 1 separated object identity from equal representation and from domain identifiers. Structural checking adds no runtime identity test. In the first example, `target === worker` is true because the assignment preserved one object identity, not because the two static types were compatible. Another object with the same members could also satisfy `DispatchTarget` while comparing unequal to `worker`.

Public class instance types make this boundary conspicuous:

```
class QueueTicket {
  constructor(readonly id: string) {}
  describe() {
    return `ticket:${this.id}`;
  }
}

const independent = {
  id: "Q-17",
  describe() {
    return `independent:${this.id}`;
  },
};

const acceptedAsTicket: QueueTicket = independent;
console.log(acceptedAsTicket.describe());
console.log(acceptedAsTicket instanceof QueueTicket);
```

Output:

```
independent:Q-17
false
```

The checker finds the required public instance members. JavaScript does not run `QueueTicket`'s constructor or change the object's prototype. A class annotation is therefore not construction evidence.

TypeScript `private` and `protected` members form an important checker exception. If the target instance type contains one of those members, a compatible source must contain a corresponding member originating in the same class declaration. Two unrelated classes with textually identical private members are rejected as incompatible. The origin rule introduces nominal-like checking for those members; it still does not execute `instanceof` or inspect a runtime object.

JavaScript `#private` is a different boundary from Chapter 3. A class initializes a runtime private element, and private access checks whether the receiver carries that class's private brand. Matching public members or a forged prototype chain cannot supply the brand. Do not use the phrase "branded type" without specifying whether it means this runtime private-field evidence or the static phantom discipline introduced below.

4. `readonly` restricts a checked access path

A readonly property tells TypeScript to reject assignment through an access path whose type marks that property readonly. The modifier does not freeze the runtime object, claim ownership, copy a value, or prevent a writable alias from reaching the same identity.

Predict before running: What state is read through `view`, and is the object frozen?

```
interface MutableStatus {
  state: string;
}

interface ReadonlyStatus {
  readonly state: string;
}

const mutable: MutableStatus = { state: "queued" };
const view: ReadonlyStatus = mutable;

mutable.state = "running";
console.log(view.state, mutable === view, Object.isFrozen(view));
```

Output:

```
running true false
```

The checker would reject `view.state = "done"`, but it accepts the write through `mutable`. Both bindings still reach one object identity. In current TypeScript compatibility checking, readonly property modifiers are not generally used to distinguish otherwise compatible property shapes; the repository's static fixture even verifies assignment from the readonly view back to a writable interface. That permissive rule is version-sensitive checker behavior, not a statement that writes are safe through every alias.

Use readonly views to express which operations checked consumers should perform. Use encapsulation or explicit ownership conventions to control who can mutate. Use `Object.freeze` or another runtime operation when selected writes must fail during execution. No one keyword supplies all three guarantees.

5. Representation does not supply domain meaning

Consider two aliases:

```
type UserId = string;
type OrderId = string;

function ownershipLabel(userId: UserId, orderId: OrderId) {
  return `user ${userId} owns order ${orderId}`;
}

const userId: UserId = "user_17";
const orderId: OrderId = "order_90";
console.log(ownershipLabel(orderId, userId));
```

Output:

```
user order_90 owns order user_17
```

Both aliases name `string`, so the checker has no member or literal distinction with which to reject the swap. The JavaScript call succeeds because strings are valid runtime values in either position. The output is syntactically valid and semantically wrong.

The same hazard appears with `number` values such as meters and seconds, or with objects such as `{ value: number }` used for both currencies. Equal representation does not imply semantic interchangeability. Domain identity is also not object identity: two separately allocated records can denote one user, while a branded `UserId` can be a primitive string with no object identity at all.

6. Phantom brands add static distinctions

A branded or opaque type can add a property that ordinary values do not expose. A declaration-scoped `unique symbol` makes the property key specific to one declaration:

```
declare const userIdBrand: unique symbol;
declare const orderIdBrand: unique symbol;

type UserId = string & { readonly [userIdBrand]: "UserId" };
type OrderId = string & { readonly [orderIdBrand]: "OrderId" };
```

`UserId` and `OrderId` retain the runtime representation of a string but require different phantom members during checking. Code outside the trusted construction boundary cannot ordinarily supply those unexported computed properties. The API can therefore demand one category in each position:

```
function createOwnership(userId: UserId, orderId: OrderId) {
  return { userId, orderId };
}

declare const userId: UserId;
declare const orderId: OrderId;

createOwnership(userId, orderId); // accepted
// createOwnership(orderId, userId); // rejected
```

A trusted constructor establishes the minimal runtime fact required by the API and keeps the assertion local:

```
function parseUserId(text: string): UserId {
  if (!/^user_[1-9]\d*$/.test(text)) {
    throw new TypeError(`invalid UserId: ${text}`);
  }
  return text as UserId;
}
```

The assertion is justified only by the preceding check and the module's construction policy. A corresponding `parseOrderId` checks the order prefix and returns `OrderId`. This chapter does not design a general validation library or prove that an ID exists in storage; Chapter 17 handles richer external-data boundaries.

Predict before running: What is `typeof parseUserId("user_17")`, and how many brand-symbol properties exist on the returned value?

Output:

```
string
0
```

The `declare const` brand keys and intersection members generate no runtime properties here. The returned value is the original primitive string. Consequently, a type assertion such as `"order_90" as UserId`, an inaccurate declaration file, unchecked JavaScript, or external data asserted without parsing can forge the static belief. If runtime provenance must survive, represent it separately—for example with a validated tagged record or a class whose JavaScript private field supplies runtime brand evidence. Static branding is API discipline, not an unforgeable security boundary.

Consequences for program design

Accept structural inputs when an operation truly needs a capability rather than a construction pedigree. A logger requiring `write(message): void` can remain open to many implementations. Require runtime evidence

when construction history, validation, authorization, or private state is load-bearing. The TypeScript target type cannot retroactively supply those facts.

Use excess-property diagnostics as typo detection, not as a security boundary or serializer. If exact external keys matter, parse the input. Use readonly access paths to state consumer permissions, not to imply unique ownership. Use brands at stable domain boundaries where identical representations invite costly category errors, but keep brand creation narrow and auditable. Branding every incidental string adds friction without useful semantic separation.

Public APIs should state which guarantee each device supplies: structural types check available operations; readonly rejects selected checked writes; static brands reject accidental checked category swaps; parsers establish selected runtime facts; JavaScript private fields provide runtime class-specific access evidence. Failures remain possible whenever code bypasses the responsible component.

False models, repaired

- **"Compatible object types describe identical objects."** The source may have extra members, and assignment preserves its runtime identity and properties.
- **"Excess-property checks make TypeScript object types exact."** They apply contextually to selected fresh literals; staged variables can remain assignable.
- **"A class annotation proves that the constructor ran."** Public instance types are normally structural; runtime provenance requires runtime evidence.
- **"Private members make TypeScript nominal."** TypeScript `private` and `protected` add a declaration-origin rule only where those members participate.
- **"`readonly` means nobody can mutate the object."** It rejects writes through selected checked access paths; writable aliases can coexist.
- **"Two aliases with different names are different types."** `type UserId = string` and `type OrderId = string` describe the same representation.
- **"A phantom brand exists on the runtime string."** The declaration and phantom property are erased in this pattern.
- **"A brand validates external data."** Only the trusted constructor's runtime checks establish the facts it explicitly tests; assertions can forge the type.

Mastery questions

Answer static cases before invoking the checker and runtime cases before executing JavaScript.

1. Given source `{ id: string; region: string }` and target `{ id: string }`, evaluate assignability in both directions and name the missing target member in the rejected direction.
2. Add `deliver(message: string): string` to a target. Construct one wider accepted source and two rejected sources with different member incompatibilities.
3. Predict whether a direct object literal and a staged variable with the same extra key are accepted as `DeliveryOptions`. Then state which runtime keys remain.
4. Diagnose a security review that treats excess-property checks as rejection of unknown JSON keys.
5. Two separately created objects satisfy one interface. Explain why structural compatibility establishes neither `===` identity nor shared domain identity.
6. Construct an object assignable to a public class instance type that fails `instanceof` for that class. Explain every result.
7. Add separately declared TypeScript private members to two otherwise matching classes. Why does the checker reject assignment, and what runtime fact has not been tested?
8. Distinguish a TypeScript private-member origin rule, a JavaScript `#private` runtime brand, and a `unique symbol` phantom brand.
9. Show a readonly view and writable alias to one object. Which assignment is rejected, which mutation succeeds, and what does each access path later read?
10. Design a counterexample to the proposition that readonly compatibility implies ownership or freezing.

11. Demonstrate a semantic swap involving meters and seconds or two identifier categories that plain `number` or `string` aliases cannot prevent.
12. Define two `unique symbol`-branded string types and a function that rejects their interchange in checked code.
13. Explain why a trusted brand constructor may contain an assertion after validation while ordinary consumers should not assert the brand.
14. Transfer the chapter to an API accepting `{ write(message: string): void }`: decide when structural openness is valuable and when runtime provenance would be necessary.

The closed-book workspace and lab specification are in `exercises/ch12.md`.

Implementation lab — Trusted Semantic Identifiers

Implement the trusted constructors and ownership API in `examples/ch12/lab/starter.ts`. `UserId` and `OrderId` are declaration-scoped `unique symbol`-branded strings in `lab/types.ts`. Accept only `user_<positive integer>` and `order_<positive integer>`, then construct and format an ownership record without allowing checked category swaps. Keep each assertion inside its matching parser.

Before reading the evaluator, prove the successful runtime values and the intended checker rejections. Then inspect `lab/evaluator/incorrect-unbranded.ts`. Its two parameters are plain strings, so the reversed call compiles and JavaScript constructs `{ userId: "order_90", orderId: "user_17" }`.

Presentation prompt

Give a 10–15 minute explanation in your own words covering directional structural assignability, object width, member compatibility, fresh-literal excess checks, runtime identity, public class structure, private/protected origin, JavaScript private brands, readonly aliases, semantic category errors, and phantom brands. Include one prediction, one causal derivation, one false model, one assertion or untyped-code boundary, one connection to Chapters 1, 3, 10, or 11, and one unresolved uncertainty. The complete checklist is in `presentations/prompts/ch12.md`.

Ready-when test

Proceed only when, without notes, you can evaluate structural assignability in the correct direction; distinguish ordinary width from fresh-literal excess checking; state what compatible shape cannot prove; explain public class structure and private/protected origin without inventing runtime validation; predict mutation through readonly aliases; design two semantically distinct branded values; identify every way their static distinction can be forged; implement and test the trusted identifier lab; and present the chapter without confusing domain identity, object identity, static brands, or JavaScript private brands.

Chapter 13

Generics and Relationships Between Types

Chapter type: Acquisition
Prerequisites: Chapters 10–12; function calls, callbacks, arrays, and structural assignability
Capabilities introduced: Design and inspect generic declarations that preserve relationships among inputs, outputs, containers, keys, callbacks, constraints, defaults, and factories; predict type-argument inference; recognize variance and method-compatibility boundaries
Earlier chapters integrated: Inference and directional assignability from Chapter 10; evidence and `unknown` from Chapter 11; structural compatibility and semantic identity from Chapter 12
Later chapters enabled: Type construction, domain modeling, erased-type boundaries, and reusable public contracts
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch13/src/`, `examples/ch13/test/`, and `examples/ch13/lab/`

Compressed thesis

A generic declaration introduces one or more type parameters and uses them in checked positions. Its value is the relationship among those positions: an input and output have the same type, an array element determines a callback parameter, a callback return determines an output collection, or an object and key determine a property result. A type parameter used once often expresses no relationship and may be replaceable by a concrete type or `unknown`.

At a call, TypeScript infers type arguments from argument and contextual evidence or checks explicit type arguments. A constraint restricts admissible type arguments and permits operations known on the constraint; it does not authorize an implementation to fabricate an arbitrary subtype. A default supplies a type argument only when inference and explicit syntax supply none. Type parameters, constraints, defaults, and inferred arguments generally emit no JavaScript representation, constructor, validation, or runtime default.

Generic relationships operate through TypeScript's assignability rules, including structural compatibility and selected unsound allowances. They preserve only information that the declaration actually connects. This canonical chapter retains the shared technical spine; explanations required by an observed reader gap belong in source-linked sections of the personal appendix rather than expansions of the chapter.

1. A type parameter connects checked positions

Chapter 10 introduced a minimal identity function:

```
function retain<Value>(value: Value): Value {
  return value;
}
```

`Value` does not mean "any value" during execution. It states that the parameter and return positions use one type argument for each call. If the argument is inferred as `{ readonly kind: "archive"; readonly id: "job-7" }`, the return expression retains that description. JavaScript receives and returns the same object identity; TypeScript emits no `Value` object.

Contrast a parameter used once:

```
function describe<Value>(value: Value): string {
  return String(value);
}
```

Nothing else depends on `Value`. If the implementation needs no operations beyond those available to every input, `value: unknown` expresses the useful contract more directly. A generic parameter is not valuable merely because it replaces `any`; it should preserve a dependency a caller can use.

Common useful relationships include:

- input to output: `(value: T) => T`;
- element to container: `readonly T[]` to `T` or `T[]`;
- input element through callback to output element: `T[]` and `(T) => U` to `U[]`;
- object and key to property: `T`, `K extends keyof T`, and `T[K]`;
- factory to constructed value: `() => T` to `T`.

The repeated parameter positions are the contract. Renaming `T` to `Input` changes nothing; removing or broadening one occurrence can sever the relationship.

2. Inference solves declared relationships from call evidence

Consider a collection transformation:

```
function mapValues<Input, Output>(
  values: readonly Input[],
  transform: (value: Input, index: number) => Output,
): Output[] {
  return values.map(transform);
}

const labels = mapValues(
  [{ id: "A", attempts: 1 }, { id: "B", attempts: 3 }],
  (job, index) => `${index}:${job.id}/${job.attempts}`,
);
```

The array argument supplies evidence for `Input`. That inferred argument contextually types the callback parameter, and the callback return expressions supply evidence for `Output`. The result is `string[]`. The checker does not run the callback to discover its result; it analyzes the expressions and call relationships.

Predict before running: What does JavaScript return, and what types does the checker infer?

```
function pair<Value>(left: Value, right: Value): readonly [Value, Value] {
  return [left, right];
}

const same = pair(2, 5);
const mixed = pair<string | number>(2, "five");
```

Runtime values are `[2, 5]` and `[2, "five"]`. The first call infers `Value` as `number`. The second explicitly chooses `string | number`, then checks both arguments against it. Under this project's TypeScript 6.0.3 baseline, `pair(2, "five")` without the explicit union is rejected: inference does not promise to synthesize a convenient union from every conflict.

Explicit arguments are useful when evidence is absent, intentionally broader than one argument, or resolves poorly. They are not casts: `pair<boolean>(2, 5)` is rejected because the runtime arguments remain numbers and their source types are not assignable to target `boolean`.

Annotations can accidentally discard relationships. Replacing `mapValues` with `(values: readonly unknown[], transform: (value: any) => any): any[]` admits the call but loses both the input evidence and callback result. Likewise, annotating a precise generic result as `unknown[]` discards information at that receiving location even when the function preserved it.

3. Constraints permit operations; they do not manufacture subtypes

An unconstrained parameter could represent any admissible type, so its implementation may use only operations available for that possibility. A constraint narrows the admissible arguments:

```
interface HasLength {
  length: number;
}

function longer<Value extends HasLength>(left: Value, right: Value): Value {
  return left.length >= right.length ? left : right;
}
```

`extends HasLength` lets the implementation read `.length`, and the return type promises one value with the chosen `Value`, not merely any length-bearing object. Structural assignability from Chapter 12 determines which types satisfy the constraint.

That distinction rejects a tempting implementation:

```
function atLeast<Value extends HasLength>(value: Value, minimum: number): Value {
  if (value.length >= minimum) return value;
  // rejected: { length: number } need not contain every member of Value
  return { length: minimum };
}
```

A caller could supply `{ length: 2, id: "A" }`. Returning `{ length: 5 }` satisfies the constraint but violates the promised subtype by omitting `id`. Repair the contract in one of three ways: return `HasLength` and admit the information loss; accept a factory `(length: number) => Value`; or return a union/result that represents replacement explicitly. A constraint grants checked operations. It is not a constructor for `Value`.

The same rule exposes fake creation functions. `<Value>(): Value` cannot honestly produce every requested `Value` without input evidence, a constructor, a factory, validation of an external source, or another genuine source. An assertion can silence the diagnostic but cannot create the promised runtime value.

4. Object, key, and property form one indexed relationship

The smallest useful indexed API uses two type operators:

```
function getProperty<ObjectType, Key extends keyof ObjectType>(
  object: ObjectType,
  key: Key,
): ObjectType[Key] {
  return object[key];
}
```

For an object type, `keyof ObjectType` describes its permitted property-key types. Constraining `Key` to that union rejects known-absent keys. The indexed access `ObjectType[Key]` describes the selected property's value type. Given `{ id: string; attempts: number }`, key `"id"` produces `string`, while key `"attempts"` produces `number`.

JavaScript still performs ordinary bracket lookup. No runtime key list or property validator is emitted. If an untyped caller passes a missing key, runtime lookup can produce `undefined`; if an assertion falsifies the object type, the return description can be wrong. Index signatures also change the result of `keyof`, and symbols and numeric keys require care. Chapter 14 develops those operators and their composition; this chapter uses them only to expose the object/key/result relationship.

5. Defaults fill missing type evidence, not runtime values

A default makes a type argument optional:

```
function emptyBucket<Element = string>(): Element[] {
  return [];
}

const names = emptyBucket();          // string[]
const flags = emptyBucket<boolean>(); // boolean[]
```

The default selects `string` because the call supplies neither explicit syntax nor inference evidence for `Element`. It does not insert a string into the array, instantiate `String`, or validate later JavaScript writes. Both runtime arrays are empty.

When inference finds a candidate, the candidate takes precedence. A defaulted parameter must satisfy its constraint, and required type parameters cannot follow optional ones. Defaults improve APIs whose omitted evidence has one defensible checked interpretation. They conceal uncertainty when chosen only to make diagnostics disappear.

6. Parameter and return positions change safe substitution

Generic relationships inherit TypeScript's function assignability rules. Use concrete call authority rather than variance slogans. Let `Dog` structurally extend `Animal`.

```
type Producer<Value> = () => Value;
type Consumer<Value> = (value: Value) => string;
```

A `Producer<Dog>` may be used as `Producer<Animal>` because every produced dog supplies the members an animal consumer may read. The reverse substitution is unsafe because a general animal need not supply dog members.

A `Consumer<Animal>` may be used as `Consumer<Dog>` because code authorized to call the target supplies dogs, and the source function can consume any animal. A `Consumer<Dog>` may not be used as `Consumer<Animal>`: the target contract authorizes a caller to pass a non-dog animal. With `strictFunctionTypes`, TypeScript rejects that assignment for function-property syntax.

Methods retain a compatibility exception. A type written as `{ consume(value: Value): string }` can accept a narrower method in a wider target position. Calling the resulting wider view with a non-dog can throw when the implementation reads dog-only members. This bivariant method boundary supports common JavaScript and library patterns but is not a proof of runtime safety. TypeScript is deliberately not fully sound; API authors should prefer function properties when strict callback parameter checking is required.

Variance annotations are not the acquisition goal. TypeScript normally derives variance from structural use, and annotations do not rewrite a type's structural behavior. The practical question is: which values may target callers supply or receive, and can the source implementation satisfy every authorized call?

7. Erasure exposes fake generics

Predict before running: Does the explicit type argument convert the string?

```
function unsafePretend<Value>(value: any): Value {
  return value as Value;
}

const count = unsafePretend<number>("not-a-number");
console.log(typeof count, count);
```

Output:

```
string not-a-number
```

`any` suppresses the relationship between input and output, and the assertion supplies no evidence. `Value`, `<number>`, and `as Value` are absent from emitted JavaScript. This is a fake generic: the caller selects a return description that execution need not satisfy.

Use a relationship-backed declaration instead. `retain<T>(value: T): T` returns supplied evidence. `makeWith<T>(factory: () => T): T` invokes a source that produces `T`. A parser can return `T` only when runtime checks establish the relevant shape and meaning; Chapter 17 covers that boundary. A type parameter cannot make external data true.

Consequences for program design

Start an API design by naming dependencies callers need preserved. If the output is independent of the input, do not invent a shared type parameter. If the callback result determines the collection result, give it its own parameter. If a key selects a property, connect the object, key, and result. Fewer, better-used parameters usually improve inference and error messages.

Keep structural and semantic boundaries visible. A constraint accepts every structurally assignable input, not every value with the intended provenance. A branded `UserId` from Chapter 12 can flow through `retain<UserId>` without losing its distinction, but a generic cannot validate or create the brand. `unknown` from Chapter 11 remains the honest description for unproved input.

Test runtime behavior and static relationships separately. Runtime tests establish returned values, identities, calls, and failures. Static fixtures establish inferred results and expected rejections. Neither substitutes for the other.

Defer type computation until a simpler relationship fails. Mapped types, conditional types, advanced `infer`, and recursive construction belong to Chapter 14; state-machine domain design belongs to Chapter 15; assertion and erasure policy belongs to Chapter 16; runtime parsing belongs to Chapter 17.

False models, repaired

- **"A generic means the function accepts anything."** A generic relates declared positions; constraints can restrict admissible arguments.

- **"Using a type parameter once makes an API reusable."** One use often preserves no caller-visible dependency and may be clearer as `unknown` or a concrete type.
- **"A constraint is the concrete type inside the function."** It permits known operations, but the function must still honor the caller's specific type argument.
- **"`T extends HasLength` permits returning any `HasLength`."** A declared `T` result must preserve members of the selected subtype.
- **"Inference always chooses the union I intended."** Conflicting evidence can fail; explicit type arguments or a redesigned contract may be required.
- **"A default creates a default runtime value."** It supplies a missing type argument and emits no value.
- **"A generic return type validates external data."** Erased type parameters inspect nothing at runtime.
- **"Strict mode makes every callback substitution sound."** `strictFunctionTypes` tightens function syntax; method syntax retains a bivariant compatibility boundary.
- **"Variance annotations force safer structural behavior."** They normally document or optimize an already matching relationship; they do not redesign structural comparison.

Mastery questions

1. Compare `<T>(value: T) => T`, `<T>(value: T) => string`, and `(value: unknown) => string`. Which declarations preserve a caller-visible relationship, and when is the type parameter unnecessary?
2. Derive the inferred input, callback-parameter, callback-return, and result types for a `mapValues` call over branded identifiers.
3. Predict the checker result for `pair(1, "two")` and `pair<string | number>(1, "two")`; then predict both runtime arrays.
4. Diagnose an API that returns `any[]` after accepting a generic callback. Identify the exact positions whose relationship was severed and repair them.
5. Explain why `<T extends { length: number }>(value: T): T` may return `value` but may not fabricate `{ length: 3 }`.
6. Redesign the failed fabrication API once with a factory and once with a wider return type. State what each caller loses or supplies.
7. For `getProperty`, derive the accepted keys and result types for an unfamiliar object with string, numeric, and object-valued properties.
8. Give a runtime counterexample showing why `getProperty` cannot validate an object received through `any` or an assertion.
9. Design a defaulted generic whose no-evidence call has a defensible static result. State why the default does not produce runtime data.
10. Given `Animal` and `Dog`, derive producer and consumer assignment directions from what target callers may receive or pass.
11. Construct a method-syntax assignment accepted at the bivariant boundary that throws on a target-authorized call. Convert it to function-property syntax and predict the diagnostic.
12. Explain why `<T>(value: any): T => value as T` is not repaired by a more specific explicit type argument.
13. Design a cache lookup or event transformation API whose key, input, callback, and result types remain connected. Use no `any` and justify every type parameter.
14. Connect Chapters 11–13: accept `unknown`, establish a branded identifier with runtime evidence, then pass it through a generic collection transformation without losing its semantic category.

The closed-book workspace and lab specification are in `exercises/ch13.md`.

Implementation lab — Indexed Registry Transformation

Implement `transformAt` in `examples/ch13/lab/starter.ts`. The function receives a registry, a key constrained to that registry, and a callback whose input is the selected property value. Return the literal key, the original selected input, and the callback's independently inferred output. Use no `any` and no type assertion.

Static verification must retain each key/value relationship and reject unknown keys and invalid callback operations. Runtime tests must prove values and object identity. Afterward inspect `lab/evaluator/incorrect-`

`asserted-output.ts`: it ignores the callback and asserts the selected input as the caller-chosen output, so a result described as `number` contains a string at runtime.

Presentation prompt

Give a 10–15 minute explanation in your own words covering type parameters as relationships; call-site inference and explicit arguments; constraints and subtype preservation; object/key/indexed-result relationships; defaults; callback producer/consumer substitutions; method bivariance; erasure; and fake generics. Include one prediction, one causal derivation, one false model, one runtime boundary, one connection to Chapters 10–12, and one unresolved uncertainty. The complete checklist is in `presentations/prompts/ch13.md`.

Ready-when test

Proceed only when, without notes, you can identify the useful relationship in an unfamiliar generic declaration; predict inferred and explicit type arguments; explain why a constraint permits operations without supplying a subtype constructor; implement `getProperty`; distinguish static defaults from runtime values; derive safe producer and consumer substitutions; expose the method boundary; diagnose a fake generic at runtime; complete the indexed registry lab and its static fixtures; and present the chapter without attributing validation or runtime representation to a type parameter.

Chapter 14

Type Construction and Type-Level Computation

Chapter type: Integration
Prerequisites: Chapters 10–13
Capabilities introduced: Derive checked descriptions through keys, indexed access, mappings, assignability decisions, conditional-pattern inference, string construction, and bounded recursion
Earlier chapters integrated: Inference and assignability; unions and `never`; structural compatibility and semantic identity; generic relationships
Later chapters enabled: Domain-state construction, erasure audits, runtime parsing, framework type reading, and schema relationships
Estimated study time: 5–7 hours plus delayed retrieval
Runnable sources: `examples/ch14/`

Compressed thesis

Type construction derives one compile-time description from another. `keyof` derives an object's property-key union; indexed access selects property value types; mapped types transform properties; conditional types choose a result by assignability; `infer` captures a type from a matched structure; template literal types construct string literal unions; recursive aliases repeat a transformation. The TypeScript checker evaluates those relationships while checking source code. JavaScript receives no generated schema, validator, property list, object, event name, dispatcher, or recursive procedure.

This canonical chapter preserves the shared technical spine. Reader-specific explanations belong in the personal appendix rather than in the shared chapter.

The practical objective is controlled derivation, not maximum computation. A useful derived type keeps several API surfaces synchronized with one canonical source and remains readable at the point of use. An explicit interface is better when a transformation hides the domain rule, produces hostile diagnostics, slows checking, or exists only to impress the author.

1. Construction composes earlier relationships

Chapter 10 established directional assignability: a source must satisfy the target description. Chapter 11 established unions as alternatives and `never` as an impossible remainder. Chapter 12 separated structural compatibility from semantic identity. Chapter 13 connected object, key, and result positions with `Key extends keyof ObjectType` and `ObjectType[Key]`. This chapter composes those operations over whole families of types.

Consider one runtime value:

```
const deployment = {
  service: "billing",
  retries: 3,
  dryRun: false,
};

type Deployment = typeof deployment;
type DeploymentKey = keyof Deployment;
type DeploymentValue = Deployment[DeploymentKey];
```

The checker derives `Deployment` as `{ service: string; retries: number; dryRun: boolean }`, `DeploymentKey` as `"service" | "retries" | "dryRun"`, and `DeploymentValue` as `string | number | boolean`. The source object remains the only runtime object. These derived descriptions establish no provenance, so Chapter 12's distinction remains: a structurally compatible value can satisfy `Deployment` without having originated from this declaration.

2. Keys, indexed access, and the two meanings of `typeof`

For an object type `T`, `keyof T` produces the union of known property keys. Index signatures change the result: a numeric index produces `number`, while a string index generally produces `string | number` because JavaScript coerces numeric property access to strings. Do not casually read every `keyof T` as a union of visible string literals.

An indexed access `T[K]` selects the value description at keys `K`. If `K` is a union, the result is the union of each selected property's value type. Chapter 13's `getProperty` therefore retains the exact relationship:

```
function getProperty<T, K extends keyof T>(object: T, key: K): T[K] {
  return object[key];
}
```

The constraint rejects an unknown key during checking; JavaScript still performs an ordinary bracket lookup. No key list is available to the function at runtime unless JavaScript code supplies one.

Type-query `typeof` and runtime `typeof` share spelling but not authority. In `type Deployment = typeof deployment`, the checker reads the declared type of the identifier; the expression does not run. Type-query `typeof` is intentionally limited to identifiers and their properties. In `typeof deployment.retries`, JavaScript evaluates the property access and returns the runtime string `"number"`.

Predict before running: What does JavaScript print, and what disappears during emission?

```
const record = { attempts: 3 };
type RecordShape = typeof record;
type Attempts = RecordShape["attempts"];

console.log(typeof record, typeof record.attempts);
```

The checker derives `RecordShape` and `Attempts` as an object description and `number`. The emitted JavaScript contains neither alias. It prints:

```
object number
```

3. Mapped types transform a property family

A mapped type iterates over a union of property keys and constructs one property per member:

```
type BooleanFlags<T> = {
  [K in keyof T]: boolean;
};
```

The key variable `K` is a type parameter scoped to the mapping. `T[K]` can retain each source property's value type. Mapping modifiers add or remove optionality and readonly status: `+?` and `-?` control optional properties; `+readonly` and `-readonly` control checked reassignment. The `+` is implicit.

```
type MutableRequired<T> = {
  -readonly [K in keyof T]-?: T[K];
};
```

These modifiers change static assignment permissions. `readonly` does not call `Object.freeze`, and removing `readonly` does not clone or unlock a frozen object.

An `as` clause remaps each source key. A remapped result of `never` drops that key:

```
type ChangedHandlers<T> = {
  [K in keyof T & string as `${K}Changed`]: (value: T[K]) => void;
};

type WithoutId<T> = {
  [K in keyof T as K extends "id" ? never : K]: T[K];
};
```

The `& string` retains only keys template literal types can interpolate in this declaration. Neither alias creates callbacks. JavaScript code must construct the handler object and connect it to an event source.

4. Conditional types are assignability decisions

A conditional type has the form `Source extends Target ? Accepted : Rejected`. Here `extends` asks whether the source type is assignable to the target type; it does not run a JavaScript branch and it does not inspect a runtime value.

```
type KeepStrings<T> = T extends string ? T : never;
```

When the checked operand is a naked type parameter and an instantiation supplies a union, the conditional distributes over the union. `KeepStrings<"queued" | 404 | "done">` becomes:

```
KeepStrings<"queued"> | KeepStrings<404> | KeepStrings<"done">
// "queued" | never | "done"
// "queued" | "done"
```

Chapter 11's `never` disappears from the resulting union because it contributes no possible value.

Tuple wrapping suppresses distributivity by making the checked operand a one-element tuple rather than the naked parameter:

```
type WholeUnionIsString<T> = [T] extends [string] ? true : false;
```

Commit before checking: Evaluate both aliases for the same union.

```
type Members = KeepStrings<"queued" | 404>;          // "queued"
type Whole = WholeUnionIsString<"queued" | 404>;     // false
```

The first question is "which members individually satisfy `string`?" The second is "is the complete union assignable to `string`?" Distribution is not a formatting detail; it changes the question. In correlated APIs, an accidental non-distributive object can separate a discriminant union from its payload union and admit impossible pairs.

5. `infer` captures from a successful pattern

Inside a conditional target, `infer Name` introduces a type variable for the part that made the match succeed:

```
type PayloadOf<T> = T extends { payload: infer P } ? P : never;
type ParametersOf<T> = T extends (...args: infer P) => unknown ? P : never;
type ResultOf<T> = T extends (...args: never[]) => infer R ? R : never;
```

`infer` does not inspect a live object or function. The checker compares structural descriptions, captures the matching component, and substitutes that component into the accepted branch. For overloaded functions, the standard `Parameters` and `ReturnType` utilities use the last signature rather than performing overload selection from a hypothetical call.

Template literal types construct string literal descriptions by concatenating literal components. Interpolated unions produce every combination:

```
type Key = "count" | "label";
type ChangeName = `${Key}Changed`;
// "countChanged" | "labelChanged"
```

This construction is appropriate for bounded naming conventions. Large cross-products expand rapidly; ahead-of-time code generation or a simpler explicit union is clearer for large catalogs. A template literal type does not allocate either string at runtime.

6. Recursive construction needs an explicit budget

Recursive conditional aliases can descend through nested structures. They can also increase checking time, hit compiler recursion limits, and conceal where a transformation stops. A depth budget makes the termination rule visible:

```
type DeepReadonly<
  T,
  Budget extends readonly unknown[] = readonly [unknown, unknown, unknown],
> = Budget extends readonly [unknown, ...infer Rest]
  ? T extends (...args: never[]) => unknown
    ? T
    : T extends readonly (infer Element)[]
      ? readonly DeepReadonly<Element, Rest>[]
      : T extends object
        ? { readonly [K in keyof T]: DeepReadonly<T[K], Rest> }
        : T
  : T;
```

Each recursive step consumes one tuple member. The function branch stops before mapping callable members; the array branch retains an array relationship; the object branch maps properties. After three steps the alias returns the remaining type unchanged. This is a teaching example, not a complete definition of immutability: it neither freezes objects nor accounts for every built-in container. Prefer a named domain interface when callers need to understand the full result without executing the type computation mentally.

7. Standard utilities are compositions, not magic

Deriving selected utilities exposes the small operations underneath their names:

```
type MyPartial<T> = { [K in keyof T]?: T[K] };
type MyRequired<T> = { [K in keyof T]-?: T[K] };
type MyReadonly<T> = { readonly [K in keyof T]: T[K] };
type MyPick<T, K extends keyof T> = { [P in K]: T[P] };
type MyRecord<K extends PropertyKey, V> = { [P in K]: V };
```

`Partial`, `Required`, and `Readonly` preserve keys and values while changing modifiers. `Pick` selects keys; `Record` starts from a key union and assigns one value description to every key. `Omit<T, K>` can be derived as `Pick<T, Exclude<keyof T, K>>`.

Conditional distribution supplies union filters:

```
type MyExclude<T, U> = T extends U ? never : T;
type MyExtract<T, U> = T extends U ? T : never;
type MyNonNullable<T> = T extends null | undefined ? never : T;
```

`Exclude` drops assignable members, `Extract` retains them, and `NonNullable` drops `null` and `undefined`. These utilities filter descriptions; they do not remove values from an array or reject nullish input.

Conditional-pattern capture supplies function and promise projections:

```
type MyParameters<T> = T extends (...args: infer P) => unknown ? P : never;
type MyReturnType<T> = T extends (...args: never[]) => infer R ? R : never;
type MyAwaited<T> = T extends null | undefined
  ? T
  : T extends PromiseLike<infer V>
    ? MyAwaited<V>
    : T;
```

`Parameters` derives a parameter tuple; `ReturnType` derives a result; `Awaited` recursively unwraps ordinary promise-like descriptions while preserving nullish inputs. The repository alias is a pedagogical approximation for ordinary promises, not a reimplementation contract for every thenable edge case in TypeScript's library declarations. Use the standard utilities in application code unless a genuinely different rule needs a different name.

8. Derivation preserves relationships, not runtime truth

The lab derives event names and payloads from one value shape:

```
const fieldDefaults = { count: 0, label: "idle", active: false };
type Fields = typeof fieldDefaults;
type FieldKey = keyof Fields & string;

type ChangeEvent<K extends FieldKey = FieldKey> = K extends FieldKey
  ? { type: `${K}Changed`; value: Fields[K] }
  : never;
```

The naked `K` distributes, producing three correlated object members. A `switch` on `event.type` then narrows `event.value` to `number`, `string`, or `boolean`. If the declaration instead produces `{ type:${FieldKey}Changed; value: Fields[FieldKey] }`, each name can pair with every value. An assertion inside the switch can hide that loss but cannot repair it.

This API remains purely static. An external JSON object is still `unknown` until Chapter 17's runtime parser establishes its fields. A mapped handler type does not register handlers, a conditional event type does not route messages, and a template literal type does not verify an external string. Type generation is not data validation.

Consequences for program design

Choose one canonical source only when its authority is real. A runtime registry can support derived static names, but JavaScript must still enumerate that registry when runtime enumeration is required. A domain interface can drive several static views, but it cannot certify external data or establish Chapter 12's semantic brand.

Keep public signatures shallower than private helpers. Name intermediate results, constrain key domains deliberately, and test decisive outcomes with static fixtures. Treat distributivity, modifier preservation, overload selection, recursion depth, and compiler version as observable boundaries. If users must expand four aliases to understand one parameter, publish the explicit parameter type.

A canonical source also needs an ownership decision. Deriving an interface from a runtime registry is appropriate when the registry truly drives execution. Deriving several static views from a domain interface is appropriate when compile-time authoring is authoritative. Neither direction removes synchronization with databases, configuration files, network protocols, or other processes. If an external schema is authoritative, generate both runtime and static artifacts through an explicit build step or parse against a runtime schema; a type alias alone cannot make the external source conform.

Finally, separate synchronization from truth. Deriving request, patch, and handler types from one checked declaration reduces static drift. Runtime parsers, authorization checks, database constraints, and process behavior still supply their own guarantees.

False models, repaired

- **"A type derived from a value is a runtime mirror of that value."** Type-query `typeof` reads the checker's description of an identifier; it emits no reflection object and does not track later mutation.
- **"`keyof T` returns an array of keys."** `keyof T` is a compile-time union. `Object.keys(value)` is a JavaScript call with separate runtime typing and coercion boundaries.
- **"Mapped `readonly` freezes objects."** The checker rejects selected assignments. Only runtime operations such as `Object.freeze` affect runtime mutability.
- **"A conditional type runs an `if`."** The checker selects a type branch by assignability. JavaScript executes no corresponding branch.
- **"A conditional always distributes over unions."** Distribution requires a naked checked type parameter. Tuple wrapping asks one whole-union question.
- **"`infer` examines runtime contents."** `infer` captures part of a structurally matched type during checking.
- **"A generated event type validates network events."** External data remains untrusted until runtime code parses it.
- **"More derivation is always more maintainable."** Explicit types are clearer when computation hides the rule, expands excessively, or degrades diagnostics and checker performance.

Mastery questions

1. Derive `keyof` and `T[keyof T]` for an object containing string, number, and boolean properties, then predict the emitted JavaScript.
2. Compare type-query `typeof config` with runtime `typeof config`. Which component evaluates each, and what result does each produce?
3. Transform a readonly optional record into a mutable required record. Explain each mapping modifier.
4. Remap `firstName | retryCount` into getter names and preserve each getter's result type.
5. Derive `Exclude<"a" | 1 | "b", string>` member by member using distributivity and `never`.
6. Predict the difference between `T extends string ? T[] : never` and `[T] extends [string] ? T[] : never` for `T = string | number`.
7. Diagnose an event declaration whose name union and payload union are uncorrelated. Construct one statically admitted runtime failure.
8. Use `infer` to extract an array element, a function parameter tuple, and a function result. State what happens for a nonmatching structure.

9. Construct a bounded recursive readonly alias and identify exactly where its budget stops transforming properties.
10. Derive `Partial`, `Required`, `Readonly`, `Pick`, `Omit`, and `Record` from mappings and union filtering.
11. Derive `Exclude`, `Extract`, and `NonNullable`; explain why no runtime collection or value is filtered.
12. Derive `Parameters`, `ReturnType`, and an ordinary-promise approximation of `Awaited`; identify the overload and thenable boundaries.
13. Design a canonical-source API that keeps command names and payloads synchronized, then list the JavaScript code still required.
14. Replace an unreadable recursive or cross-product type with an explicit interface. Justify the lost synchronization against diagnostic and maintenance gains.

Implementation lab — Correlated Change Dispatch

Work in `examples/ch14/lab/`. `fieldDefaults` is the canonical source. Without `any` or a type assertion, implement a dispatcher for the derived `ChangeEvent` and `ChangeHandlers` descriptions. Prove statically that the three exact names select number, string, and boolean payloads; prove two invalid pairs are rejected; and prove at runtime that each valid event calls the matching operation.

Then inspect the deliberately broad event. Explain why its independent unions admit `{ type: "countChanged", value: "wrong" }`, why assertions inside its dispatcher provide no evidence, and why the runtime test throws. Stop before adding JSON parsing; Chapter 17 owns external-data validation.

Presentation prompt

Give the structured 10–15 minute presentation in `presentations/prompts/ch14.md`. Use your own words and sparse notes. Include one decisive distributivity prediction, one utility derivation, the lab fault, an earlier-chapter connection, and one uncertainty you would verify before publishing the API.

Ready-when test

You are ready when, without the chapter, you can derive and explain all twelve selected utilities; predict naked distribution and tuple suppression; implement the correlated lab without `any` or assertions; distinguish type-query and runtime `typeof`; identify one bounded recursive termination rule; and state precisely why every derived description still requires separate JavaScript construction and runtime validation.

Chapter 15

Domain Modeling and Impossible States

Chapter type: Integration and emergent **Prerequisites:** Chapters 10–14; Chapter 5's state owner, transition, invariant, and commit-point model **Capabilities introduced:** Encode alternative domain states with state-specific payloads; scope claims that invalid states are unrepresentable; design constructors, commands, events, transition results, exhaustive consumers, and evolution boundaries **Earlier chapters integrated:** State ownership and proposal/commit from Chapter 5; assignability, literal types, narrowing, `unknown`, and `never` from Chapters 10–11; structural typing and semantic brands from Chapter 12; generic relationships and type construction from Chapters 13–14 **Later chapters enabled:** Assertion policy, external-data validation, typed service boundaries, persistence, and production correctness **Estimated study time:** 5–7 hours across six sessions **Runnable sources:** `examples/ch15/src/`, `examples/ch15/test/`, and `examples/ch15/lab/`

Compressed thesis

"Make impossible states unrepresentable" is a scoped design claim. A discriminated union can require supported, checked TypeScript construction paths to supply the fields owned by the selected alternative, and it lets consumers narrow to that alternative. Fresh object literals commonly receive excess-property checks, but Chapter 12's structural width can still admit a staged source value with additional fields; exact runtime keys require a constructor or parser policy. State-specific transition functions can prevent a checked caller from proposing some command/state pairs. Constructors can validate runtime preconditions and return values whose types record the established facts. None of those facts prevents untyped JavaScript, `any`, assertions, dishonest declarations, mutation through aliases, unvalidated deserialization, stale persistence, or implementation bugs from supplying a contradictory runtime value.

A useful domain model names one stable literal discriminant, puts every field in the union member that owns it, and represents ordinary transition refusal as data. The state owner evaluates the entire command against the current state, returns the unchanged state on rejection, or publishes a complete next state and related event on acceptance. Exhaustive consumers make additions to a closed union visible to the checker, but they do not migrate stored records or validate network input.

The goal is not maximal type-level ingenuity. The goal is a small checked vocabulary whose permitted values and transitions match the domain closely enough that supported callers cannot casually express contradictions. This canonical chapter retains that shared technical spine; personal decompression belongs in source-linked sections added only after observed reader friction.

1. Derive the invalid combinations before choosing a type

Suppose a publication record has three booleans:

```
type Publication = {
  isDraft: boolean;
  isScheduled: boolean;
  isPublished: boolean;
  scheduledFor?: number;
  publishedAt?: number;
  publicUrl?: string;
};
```

The three booleans form a product: each field varies independently, so they admit eight combinations before the optional payloads are considered. If the domain permits exactly draft, scheduled, or published, five flag combinations already contradict the intended lifecycle. The optional fields add records such as a draft with a public URL, a published record without one, and a value with `publishedAt` present but every flag false.

Derive this mismatch explicitly. List the intended cases, list the fields meaningful in each case, then compare that list with the combinations admitted by the proposed type. Do not rely on a comment such as "exactly one flag is true"; the checker does not enforce prose.

A discriminated union is a sum: a value is one named alternative at a time.

```
type PublicationState =
  | { status: "draft"; body: string }
  | { status: "scheduled"; body: string; scheduledFor: number }
  | {
      status: "published";
      body: string;
      publishedAt: number;
      publicUrl: string;
    };
```

Now the `published` member owns `publishedAt` and `publicUrl`; the `scheduled` member owns `scheduledFor`. A normal object literal cannot claim `status: "published"` while omitting the publication payload. This is the central representational improvement: the presence of a field follows from the selected alternative instead of from a separate convention.

The union is not an exact-object type. A staged source containing all required `published` fields plus an extra `scheduledFor` field can remain assignable through ordinary structural width. Checked consumers viewing the value as `PublicationState` cannot rely on that extra field, but assignment did not delete it. If hybrid keys themselves must be rejected, keep raw construction behind a trusted function and make the Chapter 17 parser enforce an explicit unknown-key policy.

2. A discriminant is a checked protocol

Every member needs the same discriminant property and a distinct literal type. `status: "draft" | "scheduled" | "published"` is useful; `status: string` is not a closed protocol. A broad string admits misspellings and unknown states and prevents the checker from concluding that known cases are exhaustive. A catch-all member such as `{ status: string; ... }` overlaps every literal member and similarly weakens narrowing.

After a check on `state.status`, Chapter 11's control-flow analysis narrows the observed type:

```
function render(state: PublicationState): string {
  switch (state.status) {
    case "draft":
      return `Draft: ${state.body}`;
    case "scheduled":
      return `Scheduled for ${state.scheduledFor}`;
    case "published":
      return `Published at ${state.publicUrl}`;
    default:
      return assertNever(state);
  }
}
```

The `published` branch may read `publicUrl` because the literal comparison established that member. In the default branch, `state` is `never` only if every member of the declared union was removed. `assertNever` is therefore both a static exhaustiveness check and a runtime failure if a boundary injects a value outside the declaration.

Choose discriminants as durable protocol values. Renaming `published` affects switches, serialized records, messages, logs, and clients if those values cross boundaries. The type alias is local; the literal may be external data.

3. Optionality and nullability must carry one precise meaning

Optional properties are appropriate when absence itself is valid for that member. They are poor substitutes for lifecycle alternatives. A single record with `result?: Result`, `error?: Error`, and `finishedAt?: number` permits partial combinations unless a constructor or transition validates their relationship at runtime.

Under this repository's `exactOptionalPropertyTypes` setting, `field?: T` distinguishes an absent property from an explicitly present `field: undefined` unless `undefined` is included. The distinction matters to property-existence checks, object spread, serialization, and patch operations. `field: T | undefined` requires the key but permits an undefined value. `field: T | null` requires the key and uses `null` as an explicit runtime alternative. Select among them from domain meaning, not convenience.

Moving a field into its owning union member often removes nullable and optional cases entirely. If every successful job has an artifact identifier, make `artifactId` required on `SucceededJob`; do not make it optional on every job and then repeatedly assert its presence.

4. Constructors establish facts; result types record them

Chapter 12's phantom brands can distinguish structurally identical primitives:

```
type JobId = string & { readonly [jobIdBrand]: "JobId" };
type WorkerId = string & { readonly [workerIdBrand]: "WorkerId" };
```

A checked call cannot normally pass a `WorkerId` where a `JobId` is required. That is category separation, not proof that the job exists, the worker is authorized, the string came from storage, or either identifier has the right runtime format. Brands are erased, and an assertion or `any` can forge them.

A trusted constructor may validate a primitive before returning the branded result:

```
function positiveUnits(value: number): PositiveUnits {
  if (!Number.isSafeInteger(value) || value <= 0) {
    throw new RangeError("units must be a positive safe integer");
  }
  return value as PositiveUnits;
}
```

The conditional check establishes the runtime fact. The output type records that fact for checked consumers. Reversing the explanation—"the return type makes the number positive"—attributes causality to erased syntax. Whether invalid input throws, returns a result, or accumulates diagnostics is an API policy; the validation must still occur before the stronger type is returned.

Parsing arbitrary external structures remains Chapter 17. A local constructor does not prove that every value carrying its output type passed through it, especially across JavaScript, declaration, storage, or assertion boundaries.

5. A transition combines current state, proposal, and commit

Reuse Chapter 5's six-part contract. A state owner controls the mutation surface. A command proposes a change. The owner evaluates that command against the current state and its invariants. A rejection returns an ordinary domain result and promises no state change. An acceptance constructs a complete candidate, reaches the commit point by publishing it, and exposes observations according to the API's timing policy.

For an in-memory immutable-style transition:

```
type TransitionResult<Current, Next, Event> =
  | { accepted: true; state: Next; event: Event }
  | { accepted: false; state: Current; reason: RefusalCode };
```

The three type parameters preserve a real Chapter 13 relationship: each function names its current state, possible next state, and emitted event. They do not validate timestamps or manufacture values. A `succeedJob` implementation must still check that completion time does not precede start time and that processed units match requested units. Only after every check passes should it construct the `SucceededJob` and `JobSucceeded` event.

The lab specifies that rejection returns the exact current object identity and acceptance creates a distinct complete object. That makes the commit point observable in tests. It does not make the operation database-transactional, crash-atomic, or safe from an unrelated alias that can mutate nested writable data. The states contain readonly primitive payloads to keep that local policy narrow; Chapter 12 already established that `readonly` is not ownership or runtime freezing.

Events should also be discriminated unions with member-owned payloads. A success event owns its artifact identifier; a failure event owns its reason. An event trace then records accepted transitions without inventing optional fields shared by unrelated events.

6. Static transitions and dynamic dispatch solve different problems

A state-specific function can reject an illegal pair before execution:

```
function startJob(
  state: QueuedJob,
  command: StartCommand,
): TransitionResult<QueuedJob, RunningJob, JobStarted>;
```

A checked caller holding `DraftJob` cannot call this function. But message handlers and process managers often hold `JobState` and receive `JobCommand`; they need a total dynamic dispatcher. For every pair, `applyCommand(state, command)` either delegates to a permitted transition or returns `command-not-allowed`. Static prevention and runtime refusal are complementary, not competing designs.

Chapter 14 can derive command maps, payload selections, and result relationships from one canonical declaration. Derivation is useful when it prevents duplicated contracts. Stop when the computed type hides the transition graph, worsens diagnostics, or requires assertions in the implementation. The Chapter 15 lab uses explicit unions and a small generic result because those declarations keep the domain protocol visible.

A typestate builder can encode longer call sequences, but its value must exceed its cost. Fluent generic states do not protect untyped callers, persisted data, or concurrent owners. Prefer a plain discriminated union and explicit transitions until a repeated checked workflow demonstrates a need for more machinery.

7. Evolution makes both static and runtime work visible

Adding `PausedJob` to `JobState` should fail `never`-based renderers, serializers, metrics classifiers, and other exhaustive consumers until they decide what pause means. Those diagnostics are useful impact discovery, but only for consumers compiled against the changed union. A stale deployed client, a JavaScript consumer, or a catch-all branch will not necessarily fail.

Stored values require separate migration and validation decisions. Old records may lack a newly required field; new records may reach old binaries; renamed discriminants may no longer parse. Type construction can derive a new snapshot type, but it cannot rewrite a database. Chapter 17 establishes runtime input; later production chapters cover schema evolution, deployment ordering, compatibility, and persistence.

Before adding a union member, inventory construction paths, transitions, exhaustive consumers, serialized representations, stored data, and independently deployed readers. Static failures cover only part of that graph.

Consequences for program design

Begin with domain alternatives and invariants, not with type operators. Give each alternative one literal name and only the payload meaningful in that alternative. Expose constructors that validate stronger value-level facts. Put transitions behind the state owner, validate the entire proposal, and make acceptance and rejection distinguishable without exceptions for expected domain outcomes.

Keep guarantees attributed to their enforcers. The checker rejects incompatible supported calls and constructions. Constructor and transition code performs runtime validation. The state owner governs publication. A module boundary can hide construction functions. A database constraint, transaction, or authorization service supplies guarantees that the TypeScript union cannot.

Test static and runtime properties separately. Static fixtures prove that missing payloads, swapped brands, illegal state-specific calls, and stale exhaustive consumers are rejected. Runtime tests prove validation, identity, emitted events, total dispatch, serialization, and assertion-boundary failures.

False models, repaired

- **"The type makes every hybrid object impossible."** The union requires the selected member's payload and supports narrowing, but structural width can admit staged extra fields; trusted construction or parsing must enforce exact-key rules.
- **"Three booleans are equivalent to three union members."** Independent booleans admit eight combinations before payload fields are counted.
- **"Optional fields are a flexible state machine."** They permit absence independently unless another checked or runtime mechanism relates them.
- **"A brand proves the entity exists."** It separates static categories; database existence, authorization, and provenance require other evidence.
- **"Returning `PositiveUnits` validates a number."** Runtime checks establish positivity; the output type records the result.
- **"An exhaustive switch validates JSON."** It is exhaustive relative to the declared union after a value has already been treated as that union.
- **"An illegal transition should always throw."** Expected domain refusal can be an ordinary result; exceptions remain appropriate for broken assumptions or infrastructure failure.

- **"A typed transition removes runtime checks."** The current member can restrict command shape, but timestamps, quantities, uniqueness, and external facts still require evaluation.
- **"`readonly` makes the state immutable."** It restricts writes through that typed path; aliases and runtime mutation remain possible.
- **"Adding a union member migrates the system."** It can expose compiled consumers; stored data and deployed readers need explicit compatibility work.

Mastery questions

1. For a record with four lifecycle booleans, calculate the admitted flag combinations and identify the intended subset.
2. Convert a loading/success/failure product with optional payloads into a discriminated union. Justify ownership of every field.
3. Predict which properties are readable before and after narrowing an unfamiliar union by its discriminant.
4. Explain why adding `{ status: string }` as a catch-all weakens literal-member narrowing and exhaustiveness.
5. Distinguish `note?: string`, `note: string | undefined`, and `note: string | null` under this repository's configuration.
6. Give three facts a branded `JobId` does not establish and name a component that could establish each.
7. Design a constructor whose runtime checks justify a stronger output type. State which boundary can still forge it.
8. Derive the permitted transitions for draft, queued, running, succeeded, failed, and cancelled job members.
9. Explain why timestamp monotonicity remains a runtime check even when command and state members are statically compatible.
10. Compare a state-specific `startJob(QueuedJob, StartCommand)` with total `applyCommand(JobState, JobCommand)`. Which callers need each?
11. Specify the rejection identity and acceptance identity policy for an immutable-style transition and connect both to the commit point.
12. Add one event member to a transition design and show how its required payload follows from the accepted next state.
13. Construct an assertion-based counterexample that reaches an exhaustive renderer with a missing state-specific field.
14. Add a union member and enumerate the static consumers, external protocols, stored records, and deployed processes that require review.
15. Decide whether a proposed generic typestate builder clarifies or obscures a three-state workflow. Defend a simpler alternative.

The closed-book workspace and lab specification are in `exercises/ch15.md`.

Implementation lab — Fulfillment Job State Machine

Implement the functions in `examples/ch15/lab/starter.ts` from the states, commands, events, and generic transition result in `lab/types.ts`. The supported graph is draft → queued → running → succeeded or failed, plus queued → cancelled. State-specific functions must validate timestamps, processed units, and nonblank reasons before constructing the next state. Rejection returns the exact prior object; acceptance returns a distinct complete state and one related event. The total dispatcher returns `command-not-allowed` for every illegal dynamic pair.

Static fixtures reject incomplete success payloads, swapped branded identifiers, an illegal state-specific command, and a stale exhaustive consumer. Runtime tests cover every permitted transition, value-level refusals, identity, event trace, rendering, snapshots, constructor checks, and the deliberate loose-model assertion. Inspect the evaluator only after implementing the starter.

Presentation prompt

Give a 10–15 minute explanation in your own words covering product versus sum representations; field-owning discriminated members; stable literal discriminants; optionality and nullability; brands and constructor evidence; state owner, proposal, validation, and commit; state-specific versus dynamic transition APIs; exhaustive rendering and serialization; assertion and external-data boundaries; and union evolution. Include one combination derivation, one prediction, one false model, one runtime counterexample, one connection to Chapters 5 and 10–14, and one unresolved uncertainty. The complete checklist is in `presentations/prompts/ch15.md`.

Ready-when test

Proceed only when, without notes, you can derive invalid combinations from an independent-field model; replace it with member-owned alternatives while stating the structural-width boundary; predict narrowing and exhaustiveness; state precisely what brands, constructors, and types do and do not establish; distinguish optional, undefined, and null; implement state-specific and total dynamic transitions with ordinary refusals; connect validation and identity to the Chapter 5 commit point; expose an assertion or deserialization counterexample; explain the static and migration consequences of adding a union member; complete the fulfillment lab and its fixtures; and present the chapter without attributing runtime enforcement to erased TypeScript syntax.

Chapter 16

Erasure, Assertions, and Runtime Truth

Chapter type: Integration
Prerequisites: Chapters 1, 3, and Part III; strict TypeScript checking and ordinary JavaScript objects, classes, and control flow
Capabilities introduced: Predict representative emitted JavaScript; classify type-space and value-space names; diagnose assertions and declarations that admit false facts; select runtime operations that establish exact evidence
Earlier chapters integrated: Runtime identity, freezing, classes, private brands, inference, narrowing, structural typing, generics, constructed types, and impossible-state design
Later chapters enabled: External-data parsing, module and compiler diagnosis, and distributed production guarantees
Estimated study time: 4–6 hours across five sessions
Runnable sources: `examples/ch16/src/`, `examples/ch16/test/`, and `examples/ch16/lab/`

Compressed thesis

TypeScript checks source descriptions, then emits JavaScript according to source syntax, compiler configuration, and compiler version. Most type-only syntax has no runtime representation: annotations, interfaces, type aliases, type parameters and arguments, conditional and mapped types, type-predicate clauses, phantom brand members, `satisfies`, `as` assertions, non-null assertions, and type-only imports do not become validators or metadata in this repository's output. The JavaScript runtime receives values and executable operations, not the facts the checker accepted.

Erasure is not the deletion of every construct unavailable in a `.js` source file. A class declaration creates a runtime constructor and prototype while also supplying an instance type. Standard enums generate runtime objects. Parameter properties cause emitted property initialization. An ambient declaration supplies the checker with a promise but emits no implementation. Accurate prediction therefore requires classifying each source construct and inspecting the configured output, not repeating "types vanish."

Runtime truth comes from named runtime enforcers: JavaScript construction and private-brand initialization, executed `typeof` or property checks, `instanceof` prototype tests, `Object.freeze`, application parsing and validation, database constraints, and authorization decisions. Each operation establishes only its specified facts. An assertion, declaration file, exhaustive union, or impossible-state type can strengthen checked source while remaining false about a value received from unchecked code. This canonical chapter preserves the shared technical spine; reader-specific repair belongs in source-linked sections of the personal appendix after an observed failure.

1. Predict output by classifying source constructs

Consider this representative source:

```
import type { TrustedEnvelope } from "../lab/types.js";

interface SourceRecord { readonly kind: "record"; text: string }
type Optional<Value> = { [Key in keyof Value]?: Value[Key] };
type Text<Value> = Value extends { text: infer Result } ? Result : never;

const settings = { mode: "checked" } as const
  satisfies { mode: "checked" | "unchecked" };

function echo<Value = string>(value: Value): Value {
  return value;
}
```

Under TypeScript 6.0.3 with target `ES2024`, NodeNext modules, and `verbatimModuleSyntax`, the emitted file retains the `settings` object and `echo` function. It removes the type-only import, interface, two aliases, mapped and conditional operators, type parameter and default, return annotation, `as const`, and `satisfies` clause. The emitted function is simply `function echo(value) { return value; }`.

The same boundary applies to a predicate:

```
function isRecord(value: unknown): value is SourceRecord {
  return typeof value === "object" && value !== null
    && "kind" in value && value.kind === "record";
}
```

The parameter annotation and `value is SourceRecord` clause disappear. The boolean-returning function body remains and executes. The body can establish runtime evidence; the predicate clause tells the checker how to interpret `true` and can lie if the body is insufficient. A declaration-scoped phantom brand likewise changes static assignability without installing a symbol property on a primitive.

Type-only imports deserve an explicit rule. `import type` is guaranteed to be elided, so it cannot trigger runtime loading or provide a runtime binding. Chapter 18 develops module loading; here the relevant fact is narrower: a name imported only for types is unavailable as a JavaScript value.

2. Classes occupy type and value spaces

Some identifiers can be resolved differently by syntactic position. For this class:

```
class Receipt {
  constructor(
    public readonly id: string,
    private verificationCode: string,
  ) {}
}
```

`Receipt` in `let value: Receipt` names the instance type. `Receipt` in `new Receipt(...)` denotes the runtime constructor value. `typeof Receipt` in a type position describes the type of that constructor value, including its construct signature and static members. Runtime `typeof Receipt` instead evaluates to the string `"function"`; the two `typeof` operators occupy different syntactic contexts and answer different questions.

The class declaration remains in emitted JavaScript because JavaScript can construct and compare its values. The TypeScript access modifiers and annotations disappear, but the parameter-property syntax creates own fields and constructor assignments:

```
class Receipt {
  id;
  verificationCode;
  constructor(id, verificationCode) {
    this.id = id;
    this.verificationCode = verificationCode;
  }
}
```

`readonly` and TypeScript `private` constrain checked access; they do not freeze or hide those emitted properties. `Reflect.get(receipt, "verificationCode")` observes the soft-private field. JavaScript `#verificationCode` would instead create runtime private-brand state, as Chapter 3 established.

The class's instance type remains structurally compatible when only public members participate. A separately created object can satisfy a public class type while failing `instanceof`. The annotation does not run the constructor or alter the prototype. A JavaScript private-field brand establishes different runtime evidence from public structure or prototype membership.

3. Standard enums generate runtime objects

An enum is a selected non-erasable TypeScript construct. Under the pinned compiler:

```
enum NumericPhase { Queued, Running }
enum TextPhase { Queued = "QUEUED", Running = "RUNNING" }
```

The emitter creates JavaScript objects for both declarations. `NumericPhase.Running` is `1`, and the numeric object also receives a reverse entry so `NumericPhase[1]` is `"Running"`. `TextPhase.Running` is `"RUNNING"`, but string enums receive no reverse mapping; `Reflect.get(TextPhase, "RUNNING")` is `undefined`. These are tested properties of the emitted objects, not universal properties of union types.

A literal union such as `type Phase = "queued" | "running"` emits no runtime object. If code also defines `const phases = ["queued", "running"] as const`, the array remains because the array is a JavaScript value; `as const` disappears. Choosing an enum or a value-plus-derived-type design is an API decision, not a moral rule. `const enum`, ambient enum, target, and configuration variations have different emission boundaries; use the actual project settings before reasoning about output.

4. Runtime operators establish specific facts

Type-space descriptions cannot replace runtime operations. The runtime operations also differ from one another:

- runtime `typeof` returns one of JavaScript's specified category strings; notably `typeof null` is `"object"`, and arrays and ordinary records also produce `"object"`;
- `value instanceof Constructor` normally tests whether `Constructor.prototype` occurs in the value's prototype chain, subject to `Symbol.hasInstance`; it does not prove that the constructor ran;
- `"field" in value` tests prototype-inclusive property presence and does not check the property's value;
- `Object.hasOwn(value, "field")` tests own-property presence, not value category;
- `#field in value`, inside the declaring class, tests that class's runtime private brand;
- `Object.freeze(value)` changes selected runtime property operations on one object and is shallow; TypeScript `readonly` only rejects selected checked writes through an access path.

Predict before running: Which two tests disagree for the forged value?

```
class Envelope {
  #brand = true;
  static hasBrand(value: object) { return #brand in value; }
}

const forged = Object.create(Envelope.prototype);
console.log(forged instanceof Envelope, Envelope.hasBrand(forged));
```

Output:

```
true false
```

The prototype test succeeds because code installed the expected prototype link manually. The brand test fails because the class never initialized its private element on `forged`. Neither test validates arbitrary public fields, authorization, or storage existence. Select an operation whose established fact matches the program's requirement.

5. Assertions replace checker evidence without checking values

A type assertion `expression as T` instructs the checker to use `T` within permitted assertion rules. Angle-bracket assertions are equivalent where the file syntax permits them. A double assertion through `unknown` or `any` can force otherwise disallowed descriptions. Postfix non-null `!` removes `null` and `undefined` from the checked type. None converts, validates, throws on mismatch, or changes the runtime value.

Predict before running: What are the runtime type and failure?

```
interface Account { name: string; balance: number }

const external: unknown = { name: 17, balance: "large" };
const account = external as Account;

console.log(typeof account.name);
console.log(account.name.toUpperCase());
```

The first output is `number`; the second statement throws `TypeError`. The assertion made `toUpperCase` acceptable to the checker without changing `17`.

Non-null assertions create the same obligation:

```
const selected = ["alpha"].find(value => value === "missing")!;
selected.toUpperCase();
```

`find` returns `undefined`; `!` emits no check, so the call throws. Assertions are appropriate only when reviewable evidence exists outside the checker's reach. Use an executed assertion function, parser, or validator when the program must establish a runtime fact. Even then, the function implementation—not its TypeScript assertion signature—owns the evidence.

6. Static completeness is relative to supplied declarations

Part III improved checked source through `satisfies`, constraints and defaults, discriminated unions, exhaustive `never`, constructed types, and impossible-state encodings. Each device can make invalid checked construction harder. None examines an external runtime value.

An exhaustive switch is exhaustive relative to the union declaration supplied to the checker. If unchecked input `{ kind: "redirected" }` is asserted as a union containing only `"accepted"` and `"rejected"`, checking still proves the declared cases exhaustive. At runtime the value reaches the supposedly unreachable branch. An `assertNever` helper can throw there, providing a useful runtime alarm, but its static `never` parameter did not validate entry.

Ambient declarations and `.d.ts` files create another trust boundary. `declare const deployedService: { version : string }` lets checked source read `deployedService.version`; the ambient declaration emits no binding or implementation. If the host does not actually provide the global, JavaScript throws `ReferenceError`. If a declaration file describes a number where the library returns a string, checker success rests on a false contract. Chapter 18 and Chapter 19 cover package declarations and compiler pipelines; the present rule is that declarations describe runtime code and can be inaccurate.

7. Runtime truth requires an executing authority

At an untrusted boundary, begin with `unknown`. Execute checks for every property the trusted code will consume, then construct or return a closed value from checked observations. Recording the evidence makes the trust transition auditable: "non-null record," "kind is message," "text is string," and "priority is number" name runtime facts rather than relying on the target type's spelling.

Runtime truth can also come from construction that initializes private state, a database constraint that rejects an invalid row, an authorization decision evaluated for the current principal and resource, or a storage read that establishes existence. Name the enforcer, operation, time, and remaining failures. A type describes what checked code may assume; it does not retroactively perform any of those operations.

Emission itself has a boundary. The compiler may preserve already-supported JavaScript syntax or transform selected constructs according to target and options. This chapter verifies representative output for TypeScript 6.0.3 and the repository configuration. Chapter 19 owns target, module, source-map, and resolution depth; do not generalize one emitted file to every toolchain.

Consequences for program design

Inspect emitted JavaScript when a construct's runtime presence affects loading, reflection, bundle size, privacy, or interoperability. Keep assertions local and attach the missing evidence in a comment, test, parser, or constructor. Treat declaration files as executable-code contracts that require version alignment and boundary tests.

Separate static and runtime tests. Static fixtures prove accepted and rejected source relationships. Emitted-code tests prove representative absence or transformation without relying on whitespace. Runtime tests prove values, brands, exceptions, and enforcement. No one category substitutes for the others.

False models, repaired

- **"Everything TypeScript-specific is erased."** Many type-only forms disappear, while standard enums and parameter properties generate output and classes remain runtime values.
- **"A class name is only a type."** It names an instance type in type positions and a constructor value in value positions.
- **"`typeof ClassName` always returns a string."** Runtime `typeof` returns a string; type-query `typeof` derives the constructor value's static type.
- **"`instanceof` proves construction."** It normally establishes a prototype-chain relation and can be customized.
- **"An assertion casts or validates data."** It changes checker evidence and emits no runtime conversion or inspection.
- **"Non-null `!` throws early when the value is absent."** The operator is erased; a later JavaScript operation may fail.
- **"An exhaustive union validates incoming JSON."** Exhaustiveness covers the alternatives declared to the checker.
- **"A declaration file guarantees the implementation."** It describes an external implementation; an inaccurate description creates false checker assumptions.
- **"Readonly and phantom brands are runtime guards."** They restrict checked source; runtime enforcement requires separate operations.

Mastery questions

1. Given a file containing an interface, type alias, generic function, `satisfies`, `as const`, and a runtime object, predict which named constructs appear in its emitted JavaScript and verify semantically.
2. Classify `Receipt`, `typeof Receipt`, and runtime `typeof Receipt` by syntactic space, result, and executing component.
3. Derive the emitted fields and assignments for a public-readonly and private parameter property under the pinned configuration.
4. Predict forward and reverse lookups for numeric and string enums, then contrast a literal union plus `as const` array.
5. For one forged prototype object, predict `instanceof`, a public property check, and a private-brand check. Explain why the results differ.
6. Design the minimum runtime checks that distinguish a non-null own-property record from an array, inherited property, or wrong-valued property.
7. Diagnose an assertion that lets `{ count: "4" }` enter code expecting `{ count: number }`; identify the first concrete operation that behaves incorrectly.
8. Compare `as T`, `as unknown as T`, postfix `!`, a type predicate, and an assertion function by checker effect, executed code, and proof obligation.
9. Add an undeclared runtime variant to an asserted discriminated union. Predict the `never` switch's checker result and runtime branch.
10. Explain how an ambient `declare function` can pass checking and still fail with `ReferenceError` or return a wrongly described value.
11. Inspect an unfamiliar emitted file and classify each surviving construct as preserved JavaScript, TypeScript-generated output, or ordinary source value.
12. Transfer the chapter to an SDK whose `.d.ts` promises `Promise<User>` but whose implementation sometimes resolves to `null`. Which test and runtime check would expose the false declaration?
13. Compare TypeScript `readonly`, `Object.freeze`, a static phantom brand, and a JavaScript private brand by enforcer, phase, depth, and bypass.
14. Design a trusted boundary that accepts `unknown`, lists evidence established by executed checks, reconstructs a closed value, and rejects extra facts it did not prove.

The closed-book workspace and lab specification are in `exercises/ch16.md`.

Implementation lab — Runtime Evidence Inventory

Implement `inspectEnvelope` in `examples/ch16/lab/starter.ts`. Accept `unknown`; distinguish non-null non-array records; check a closed `kind`; validate every consumed field; reconstruct a `TrustedEnvelope`; and return an inventory naming each established fact. Do not use `any` or a type assertion.

Static fixtures must show that `unknown` is rejected before inspection and that successful results narrow to the closed union. Runtime tests must cover valid alternatives, wrong fields, null, discarded extra keys, and result identity. Then inspect `lab/evaluator/incorrect-assertion.ts`: it asserts malformed external data as trusted, after which a string and number operation throw despite successful checking.

Presentation prompt

Give a 10–15 minute explanation in your own words covering representative erasure; type-only imports; class type/value spaces; parameter-property and enum output; exact runtime checks; assertions and non-null assertions; static exhaustiveness; ambient declarations; and runtime truth. Include one source-to-emit prediction, one causal derivation, one false model, one declaration or assertion failure, one Part III connection, and one unresolved toolchain boundary. The complete checklist is in `presentations/prompts/ch16.md`.

Ready-when test

Proceed only when, without notes, you can predict representative emitted JavaScript; classify a class name and both forms of `typeof`; explain numeric and string enum output; distinguish prototype, property, brand, freeze, and validator evidence; diagnose type, double, and non-null assertions; state why exhaustive unions and ambient declarations can be wrong at runtime; implement the evidence inventory without assertions; inspect emitted semantic markers; and present the chapter without using “erasure” as an unexplained universal deletion rule.

Chapter 17

Parsing and Validating External Data

Chapter type: Integration **Prerequisites:** Chapters 11, 12, and 14–16; strict TypeScript narrowing, discriminated unions, phantom brands, constructed types, impossible-state design, and runtime evidence **Capabilities introduced:** Decode external representations; validate structure and domain values; normalize transport data; construct closed domain values; design stable, non-secret parse failures **Earlier chapters integrated:** `unknown`, control-flow narrowing, type predicates, exact optional properties, semantic brands, derived types, domain constructors, assertions, erasure, and runtime truth **Later chapters enabled:** HTTP boundaries, compiler and module pipelines, persistence, observability, resilience, security, and the production-service capstone **Estimated study time:** 5–7 hours across five sessions **Runnable sources:** `examples/ch17/src/`, `examples/ch17/test/`, and `examples/ch17/lab/`

Compressed thesis

External JSON, HTTP bodies, environment-derived configuration, storage records, and untyped-library results arrive without evidence that they satisfy the program's domain declarations. Accept such values as `unknown`. Execute checks that establish the exact structure and value invariants trusted code will consume, then construct a new closed domain value. Type annotations, assertions, predicate signatures, derived types, `satisfies`, brands, and discriminated unions validate nothing by themselves.

Parsing is a sequence of distinct operations. Decoding interprets a representation such as JSON text and may reject invalid syntax. Structural validation distinguishes records from `null` and arrays, checks own-property presence, rejects or preserves unknown keys by policy, and inspects nested containers. Value validation enforces literals, finite integers, ranges, formats, uniqueness, and cross-field rules. Normalization deliberately changes values through trimming, case conversion, defaults, or text-to-value conversion. Domain construction creates a new value whose static type records established runtime facts. Each operation has its own rejection conditions and identity effects.

A `Parser<Output>` is an ordinary JavaScript function from `unknown` to a discriminated success or failure result. Its type parameter relates a successful return value to `Output`; it does not manufacture `Output`, execute checks, or protect a dishonest implementation. Parse failures should report stable paths, codes, and expected facts without echoing secret payloads. Invalid external input is ordinary result data. Parser defects and unavailable infrastructure remain programmer or operational failures. The canonical chapter stays compressed; reader-specific decompression belongs in source-linked sections of the personal appendix tied to observed failures.

1. Decode syntax before validating meaning

`JSON.parse` interprets JSON text and returns a JavaScript value. Valid JSON may denote an object, array, string, number, boolean, or `null`. Invalid JSON syntax throws `SyntaxError`. Syntax success therefore establishes only that the text was decodable, not that the result is a request, configuration, or stored domain object.

TypeScript's library declaration gives `JSON.parse` an `any` result. Stop that implicit trust immediately:

```
function decodeJsonText(text: string): ParseResult<unknown> {
  try {
    const raw: unknown = JSON.parse(text);
    return { ok: true, value: raw };
  } catch (error: unknown) {
    if (error instanceof SyntaxError) {
      return { ok: false, issues: [
        { path: "$", code: "invalid-json", expected: "valid JSON text" },
      ] };
    }
    throw error;
  }
}
```

The catch classifies the failure owned by JSON decoding and rethrows unexpected defects. A response containing `null` has valid syntax and must fail later as the wrong structure. Conflating those cases destroys useful diagnosis.

"Parsing" is sometimes used for the entire boundary. When precision matters, name the operation: decode syntax, validate structure, validate values, normalize, then construct the domain value. HTTP routing, status selection, authentication, and body-size limits belong to Chapter 23 and the security chapters; this chapter owns the value transition after a representation reaches the parser.

2. A parser is an executed function, not a type spell

Use a discriminated result so valid external refusal remains explicit:

```
type ParseResult<Output> =
  | { readonly ok: true; readonly value: Output }
  | { readonly ok: false; readonly issues: readonly ParseIssue[] };

type Parser<Output> = (input: unknown) => ParseResult<Output>;
```

The generic parameter preserves a relationship: when a particular parser returns `ok: true`, its value has the parser's declared output type. JavaScript executes the function body. A body that returns `input as Output` is still a lie, just as Chapter 16's assertion examples were lies. A type predicate or assertion-function signature can also claim more than its boolean or throwing body established. The implementation carries the proof obligation.

`unknown` admits every incoming value but forbids property access and domain operations until control flow supplies evidence. That restriction makes the boundary inventory visible. By contrast, `any` allows a string-typed field to contain a number and postpones failure until an unrelated consumer calls a string method or performs arithmetic.

Do not return partially constructed output beside issues unless the application has explicitly designed a partial-data domain. A simple parser has one strong postcondition: success contains a complete domain value; failure contains issues and no domain value. That rule lets callers dispatch the successful closed union without defensive rechecking.

3. Establish record and property facts exactly

JavaScript requires three checks before treating an arbitrary value as a non-null, non-array object suitable for own-property inspection:

```
function isRecord(input: unknown): input is Record<string, unknown> {
  return typeof input === "object"
    && input !== null
    && !Array.isArray(input);
}
```

Runtime `typeof null` is `"object"`, and arrays are objects, so neither `typeof` alone nor a non-null check establishes this inspection boundary. The three checks still admit `Date`, `Map`, and custom-prototype objects; they do not prove a plain-object prototype. A parser that requires plain objects must also inspect and define allowed prototypes. Once the usable object boundary is established, choose property semantics deliberately. `Object.hasOwn(record, "port")` distinguishes a directly supplied field from an inherited field. The `in` operator includes the prototype chain. Neither operation checks the property's value.

For a required field, absence and a present value of `undefined` are distinct observations. Absence should usually produce `missing-field`; present `undefined` should fail the field's value rule. For an optional field under `exactOptionalPropertyTypes`, `{ note?: string }` represents either no `note` property or a present string. It does not admit `{ note: undefined }` unless `undefined` is included explicitly. Runtime parsing must implement the same policy with `Object.hasOwn`. `null` is another present value and requires its own deliberate acceptance or rejection.

Unknown keys are a compatibility and security policy, not a TypeScript inference result. A strict command parser can reject them with `unknown-key`, preventing misspellings and silently ignored authority-bearing data. A version-tolerant event reader may preserve or discard them. State the policy at every object boundary; root strictness does not automatically govern nested records.

Arrays require their own traversal. Report element locations as `$.items[3].quantity`, not a generic "items invalid." When an element is not a record, report the element container and stop descending into it. Continue inspecting independent siblings when accumulating issues remains safe.

4. Structural validity is weaker than domain validity

`typeof value === "number"` admits `NaN`, `Infinity`, fractional values, and unsafe integers. A quantity rule may require all of the following: runtime number, finite value, safe integer, and inclusive range. A port may require an integer from 1 through 65,535. Record those requirements in the issue's `expected` fact rather than saying only "invalid number."

Literal discriminants must be compared with the allowed runtime strings before dispatch. After `kind === "schedule-delivery"`, validate exactly that variant's required and allowed fields. A union declaration makes checked construction precise; it does not close an external object.

String syntax and domain truth are also separate. A request ID constructor can verify `req_<positive integer>` and then apply a contained `RequestId` assertion. The assertion is justified by the executed syntax check and should remain inside that trusted constructor. The resulting brand distinguishes checked IDs from arbitrary strings in later source code. It does not prove that the ID exists in storage, belongs to the current tenant, or is authorized for the current caller. Those facts require later database and authorization operations.

Nested and cross-value invariants often require normalized candidates. If SKUs are case-insensitive, duplicate detection must compare the canonical uppercase values, not the original spellings. A non-empty items rule applies after the array container is established. A schedule time may require the canonical `YYYY-MM-DDTHH:mm:ss.sssZ` representation, a finite timestamp, and exact round-trip through `toISOString`. Accepting every string that `Date.parse` happens to interpret would admit implementation-dependent or locale-dependent formats.

5. Normalization constructs a different value

Validation can preserve a value; normalization and transformation deliberately do not. Trimming `" Ada "` produces `"Ada"`. Lowercasing an email or uppercasing a SKU selects a canonical spelling. Converting canonical

instant text to `Date` creates an object with time operations. Applying a default converts an absent transport field into an explicit domain value.

These operations affect identity as well as content. A reliable boundary reconstructs nested records and arrays instead of returning the source object after checking it. That prevents unknown keys and aliases to mutable external objects from entering the trusted core. A returned `Date` is not the original string, and a normalized item is not the original item object. Chapter 1's alias and mutation rules still apply to the constructed graph; `readonly` does not deep-freeze it.

Coercion must be explicit. `Number(" ")` produces `0`, `Number(null)` produces `0`, and broad date parsing accepts formats the domain may not intend. A parser that accepts numeric text should first establish the exact text grammar, perform the conversion, then validate finiteness, integer safety, and range. Rejecting numeric strings is equally valid when the transport contract requires JSON numbers. Do not let JavaScript's implicit conversion choose domain policy accidentally.

Defaults also require a stated trigger. "Default when absent" differs from "default for `undefined`," "default for `null`," and "default for any falsy value." The Chapter 17 configuration example defaults `mode` only when the own property is absent; present `undefined` and `null` are invalid.

6. Design failures for callers and operators

A stable issue contains a structural path, a machine-oriented code, and an expected fact:

```
interface ParseIssue {
  readonly path: string;
  readonly code: "missing-field" | "expected-string" | "invalid-format";
  readonly expected: string;
}
```

Paths and codes support tests, client mapping, metrics, and later observability. Expected facts help humans without reproducing the received value. Avoid embedding passwords, tokens, full request bodies, or unknown field values in errors. A path such as `$.credentials.token` plus `expected: "non-empty text"` identifies the rule without leaking the token.

Fail fast when no safe descent exists: a `null` root cannot yield meaningful field issues, and a string item cannot yield `sku` and `quantity` issues. Accumulate independent issues after the necessary container exists; one request can then report a bad recipient email and two indexed item errors together. Define ordering—this repository uses deterministic traversal and sorted unknown keys—so tests and clients do not depend on incidental object enumeration.

Invalid input belongs in `ParseResult` because it is an expected boundary outcome. A violated internal parser invariant, an unexpected exception from parser code, an unavailable secret store, or a failed database operation is not another `invalid-format` issue. Catch only failures the current operation can classify. Chapter 20 develops operational error taxonomy, Chapter 22 translates failures into public API responses, and Chapter 24 owns observability. This chapter preserves their input by not flattening every failure into validation data.

7. Static descriptions still validate nothing

Writing `const body: Request = JSON.parse(text)` places a checked description on an unchecked `any` result. `as Request`, `as unknown as Request`, a non-null assertion, or a dishonest `isRequest` predicate replaces missing evidence. `satisfies Request` checks a source expression known to the compiler; it does not inspect a future response. A discriminated union enables exhaustive checked dispatch only after runtime code has established its discriminant and fields.

Derived types are equally static. `Pick`, `Omit`, mapped types, conditional types, indexed access, `typeof someValue` in a type position, and types generated from another declaration produce checker relationships, not runtime schemas. A brand records which checked paths are trusted; the brand's phantom member does not appear in

JSON. Chapter 14's single-source-of-truth technique can reduce static drift, but a runtime parser still needs an executable schema or checks.

Schema libraries can reduce repetitive checks, compose nested parsers, infer static output types, and standardize errors. They also impose bundle, performance, error-shape, coercion, unknown-key, versioning, and type-inference choices. Generated schemas can drift from handwritten domain rules; inferred types can obscure transformations. Evaluate a library against the boundary contract and keep application-specific invariants explicit. This repository adds no dependency because the lab must expose the mechanism and policies directly.

8. Construct before effect

The lab accepts a delivery command with a closed discriminant, branded request ID, nested recipient, indexed items, canonical delivery instant, optional note, uniqueness, and range invariants. It validates and normalizes into a new `DeliveryCommand`. Only the successful branch may call `commit`:

```
const result = parseDeliveryCommand(input);
if (result.ok) commit(result.value);
return result;
```

That ordering gives failure a concrete effect guarantee: invalid input does not mutate the source, dispatch a partial command, or commit state. It does not make the later commit atomic, durable, or idempotent. Chapters 20–24 allocate those production guarantees to state owners, transaction mechanisms, retry policies, and protocol design.

Chapter 11 supplies the `unknown`-to-evidence narrowing discipline. Chapter 12 supplies semantic brands after local invariant checks. Chapter 14 explains why a derived type is not a generated runtime parser. Chapter 15 supplies closed domain variants and constructor-scoped impossible-state claims. Chapter 16 names assertions and predicates as evidence obligations. Chapter 17 joins those tools at the external boundary and enables Chapter 23 to route authenticated HTTP input into a trusted application core.

False models, repaired

- **"Valid JSON is valid application data."** JSON decoding establishes syntax only; valid JSON includes primitives, arrays, and `null`.
- **"`Parser<User>` creates runtime evidence because it is generic."** The executed function body creates evidence; the type parameter records its successful output relationship.
- **"`typeof value === "object"` establishes a record."** It also accepts `null` and arrays; excluding those still does not establish a plain-object prototype.
- **"A present property satisfies a field."** Presence says nothing about the field's value category or domain invariant.
- **"Optional means present `undefined` is always accepted."** Exact optional semantics distinguish absence from a present value.
- **"A TypeScript union closes a JSON object."** Runtime code must check the discriminant, variant fields, and unknown-key policy.
- **"A brand proves the entity exists and is authorized."** A local brand can record checked syntax; storage and authorization require separate runtime authorities.
- **"Normalization merely annotates the input."** It may change strings, create `Date` objects, apply defaults, reconstruct arrays, and replace identity.
- **"Assertions and predicates validate at runtime."** Only their executed bodies can inspect or reject a value.
- **"Every parser failure should be caught and returned as invalid input."** Unexpected defects and infrastructure failures require different handling.

Mastery questions

1. For malformed JSON, valid `null`, and a valid object with the wrong fields, identify the operation that rejects each value and its issue code.
2. Explain why assigning `JSON.parse(text)` directly to a domain type admits `any`, then repair the boundary with `unknown`.
3. Derive what `Parser<Output>` guarantees statically and what its implementation must establish at runtime.
4. Design the minimum checks that distinguish an own-property record from `null`, an array, and an object inheriting its required fields.
5. Compare missing, present `undefined`, present `null`, and present valid text for `{ note?: string }` under exact optional semantics.
6. Specify strict, stripping, and preserving unknown-key policies. Choose one for a money-moving command and defend it.
7. Build a numeric rule that rejects strings, `NaN`, infinities, fractions, unsafe integers, zero, and values above 100.
8. Explain why a syntax-validated `RequestId` brand does not prove storage existence, tenancy, or authorization.
9. Predict the identities and runtime categories after trimming a name, canonicalizing a SKU, and converting ISO text to `Date`.
10. Contrast deliberate numeric-text conversion with implicit coercion for `""`, `" "`, `null`, and `"12x"`.
11. Design stable errors for three bad array elements without embedding their received values.
12. Decide where fail-fast behavior is necessary and where accumulation is safe for a nested command.
13. Construct a lying predicate that admits `{ quantity: "2" }`; identify the first runtime expression whose behavior contradicts the declared number result.
14. Compare handwritten parsing with a schema library by runtime policy, transformation, error stability, dependency cost, and static inference.
15. Prove with tests that invalid input creates no partial domain value, mutation, dispatch, or commit.

The closed-book workspace and lab specification are in `exercises/ch17.md`.

Implementation lab — Delivery Command Boundary

Implement `parseDeliveryCommand` in `examples/ch17/lab/starter.ts`. Accept `unknown`; distinguish malformed JSON from wrong runtime structure; require own fields; reject unknown keys; validate a closed command discriminant, branded ID, nested recipient, non-empty item array, quantity ranges, canonical instant, optional note, and variant-specific value rules. Normalize into a new domain value. Return stable indexed issues and no partial domain value on failure. Prove statically that `unknown` cannot dispatch and that success retains the brand and closed union. Prove at runtime that failure neither mutates nor commits.

Then inspect `lab/evaluator/incorrect-assertion.ts`. Its assertion passes TypeScript while a string quantity makes a function declared to return `number` produce the runtime string `"02"`.

Presentation prompt

Give a 12–15 minute explanation in your own words covering the five boundary operations; `unknown`; the generic parser relationship; exact record, ownership, optional, numeric, literal, array, and nested checks; normalization and identity; brands; stable secret-safe errors; accumulation versus fail-fast behavior; lying static constructs; schema-library tradeoffs; and parse-before-effect ordering. Include one prediction, one causal derivation, one deliberate failure, one Chapter 11–16 connection, and one remaining production boundary. The complete checklist is in `presentations/prompts/ch17.md`.

Ready-when test

You are ready to continue when, without notes, you can derive the five operations from external representation to closed domain value; implement a dependency-free `unknown` parser with exact object, property, scalar, array, and optional semantics; state every normalization and identity change; design stable non-secret accumulated issues;

expose a lying assertion or predicate with a concrete runtime mismatch; and prove that invalid input cannot reach a domain effect. If one derivation fails, paste the derivation, your attempt, and what remains confusing into Codex. Use the resulting personal appendix repair, then repeat the original derivation.

Chapter 18

Modules, Packages, ESM, and CommonJS

Chapter type: Acquisition and integration **Prerequisites:** Chapters 1–3 and 6–7; Chapters 12 and 16; familiarity with JavaScript files, functions, lexical bindings, promises, and erased TypeScript types **Capabilities introduced:** Predict source interpretation, specifier resolution, ESM linking and evaluation, CommonJS loading and caching, package entry selection, Node 24 interoperation, and NodeNext checking and emit **Earlier chapters integrated:** Identity and snapshots from Chapter 1; lexical scope and closures from Chapter 2; runtime values from Chapter 3; jobs and asynchronous completion from Chapters 6–7; structural declarations from Chapter 12; runtime truth and type-only erasure from Chapter 16 **Later chapters enabled:** Compiler configuration, build pipelines, deployment artifacts, runtime boundaries, package publication, and production diagnosis **Estimated study time:** 6–8 hours across seven sessions **Runnable sources:** `examples/ch18/src/`, `examples/ch18/fixtures/`, `examples/ch18/test/`, and `examples/ch18/lab/`

Compressed thesis

A source file, an ECMAScript module, a CommonJS module, and a package are not synonyms. A source file is text at a location. Node's active loader interprets that text as ESM or CommonJS from extension and package metadata. A module is the resulting scoped runtime unit and its dependency relationships. A package is a resolution and distribution boundary described by `package.json`; it may expose many modules in either format.

For ESM, ECMAScript supplies module scope, import and export bindings, graph linking, initialization, evaluation, re-export, and dynamic-import semantics; the host supplies specifier resolution, loading, and module identity. For CommonJS, Node supplies a function wrapper, synchronous `require`, `module.exports`, and a filename-keyed cache. Package metadata directs Node toward public entry points and conditions. TypeScript's NodeNext checker models those Node rules and emits JavaScript, but Node executes the emitted files and deployed metadata—not the checker's successful analysis.

Diagnose in order: source text and emitted text; format interpretation; specifier resolution; ESM linking or CommonJS loading; evaluation and cache identity; final observed value. Each stage has a different authority and failure class. The compressed model is useful only when you can name that authority rather than saying that "the module layer" worked or failed.

1. Keep four objects separate

Consider `src/counter.ts` inside a package whose nearest `package.json` contains `"type": "module"`. TypeScript reads the source file and decides how to check and emit it. If emitted as `dist/counter.js`, Node reads the emitted file, consults the nearest deployed package metadata, and interprets `.js` as ESM. An `exports` entry may make that

module reachable as `counter-package/counter`. None of those facts makes the source file, module, and package the same object.

Use a five-stage account:

1. **Emit:** the TypeScript compiler checks a source import and may produce an emitted import.
2. **Interpret:** the Node loader chooses ESM or CommonJS for the emitted file.
3. **Resolve:** that loader maps each specifier to a URL, builtin, package target, or filename.
4. **Link and evaluate:** ECMAScript module machinery links an ESM graph before executing it; the CommonJS loader synchronously executes modules as `require` reaches them.
5. **Observe:** application code reads a binding, namespace property, or `module.exports` value.

A checker diagnostic can stop stage one. `ERR_MODULE_NOT_FOUND` and `ERR_PACKAGE_PATH_NOT_EXPORTED` come from Node resolution. A missing named ESM export prevents successful linking. A temporal-dead-zone `ReferenceError` occurs during evaluation. A CommonJS file that reassigns `exports` can finish with status zero yet expose the wrong value. The failure's phase determines which authority can repair it.

2. ESM is a linked graph of bindings

Static `import` and `export` declarations describe a dependency graph. During linking, ECMAScript resolves requested exports and creates module environments before module bodies execute. An imported name is an immutable indirect binding: the importer cannot assign through it, but reading it obtains the exporter's current binding value. This is why `import { count }` is neither a copied primitive nor a writable alias. If the exporter later executes `count += 1`, the importer observes the new value.

Re-export does not turn that binding into a snapshot. `export { count } from "./counter.js"` forwards export resolution to the original binding. Module namespace objects also expose the module's exports, subject to their specified exotic-object behavior; they are not ordinary mutable records that let consumers replace exported bindings.

Loading and linking precede evaluation. In a normal synchronous acyclic graph, dependencies evaluate before the importing module body. Source order within the importing file does not mean "run this import here": static imports are graph declarations, not calls. Top-level side effects therefore run according to graph evaluation, once for a given module record, not once per textual import statement.

The "once" claim needs an identity boundary. ECMAScript reuses evaluation state for a linked module record. Node's ESM loader resolves and caches modules as URLs. Two imports resolving to the same URL share one instance; Node treats distinct query strings or fragments as distinct URLs and can load the same pathname more than once. Separate package copies or processes also create separate identities. A singleton is therefore singleton-per-resolved-loader-identity, not universal state.

`import(specifier)` is an expression. ECMAScript returns a promise and asks the host to load the requested module; fulfillment exposes its namespace after successful loading and evaluation. Node supports dynamic import in both ESM and CommonJS. It changes when the edge is requested and makes failure asynchronous; it does not exempt the specifier from Node resolution or package exports.

3. Cycles are initialization problems, not automatic failures

ECMAScript explicitly models cyclic module records and strongly connected components. Linking can create the bindings for a cycle without executing either body. Evaluation then traverses the graph while each lexical declaration still obeys initialization rules.

A cycle succeeds when early evaluation does not read an uninitialized binding. Two modules may export functions that refer to one another, complete their top-level initialization, and call those functions only afterward. The functions' closures then read initialized live bindings. The safe cycle fixture produces `A:B:A`.

A cycle fails when evaluation demands a lexical value too early. If `b.mjs` initializes `bValue` from imported `aValue` while `a.mjs` is still waiting on `b.mjs`, the binding exists but `aValue` has not been initialized. ECMAScript's temporal-dead-zone rule produces `ReferenceError`; the problem is not that the import was absent or that all cycles are forbidden.

Draw the graph, locate top-level reads, and trace initialization. Source order in the entry file is insufficient. Top-level `await` adds asynchronous dependency ordering and can delay parent evaluation; it does not remove the need to reason about the entire strongly connected component.

4. Node 24 chooses a file format before executing it

Node 24 always treats `.mjs` as ESM and `.cjs` as CommonJS. For `.js`, the nearest containing package scope with `"type": "module"` selects ESM and `"type": "commonjs"` selects CommonJS. With no explicit marker, Node 24 can inspect ambiguous input for module syntax; package authors should still declare `type` so format is not an accidental inference. A `.js` file written with `module.exports` will fail if package metadata makes Node interpret it as ESM, because the ESM scope does not receive CommonJS's `module` parameter.

Node's ESM resolver distinguishes relative specifiers such as `./counter.js`, bare package specifiers such as `counter-package`, and absolute URL specifiers. Relative and absolute ESM specifiers require explicit file extensions; Node does not append `.js` or search a directory index. Bare specifiers use package resolution and `node_modules` lookup. ESM files are URL-addressed, so `?variant=1` and `?variant=2` can identify distinct instances even when their pathname is equal.

These are Node host rules, not universal ECMAScript rules. A browser resolves URLs relative to a document or module URL and may use an import map. A bundler can implement aliases, extension inference, virtual modules, and graph rewriting. Success under a bundler does not prove that unbundled Node will resolve the same specifier; Node success does not prove that a browser server will provide the same URL or MIME behavior.

5. CommonJS loads synchronously and exports one value

Before evaluating a CommonJS file, Node wraps it approximately as:

```
(function (exports, require, module, __filename, __dirname) {
  // file body
});
```

The wrapper gives the file module scope and its apparent module globals. `require()` resolves and loads synchronously. The value returned to the caller is `module.exports`.

Initially, the wrapper parameter `exports` refers to the same object as `module.exports`. Property mutation through that alias works: `exports.parse = parse`. Assignment does not: `exports = { parse }` only rebinds the local parameter, leaving `module.exports` unchanged. To replace the exported value, assign `module.exports = { parse }`. This is ordinary Chapter 1 reference rebinding applied inside Node's wrapper.

Node caches CommonJS modules after first load by resolved filename. Requiring the same resolved filename normally returns the same `module.exports` object without re-executing the body. Different resolved filenames, including separate installed copies, are separate cache entries. Code that needs repeated work should export a function rather than depending on repeated module evaluation.

During a CommonJS cycle, synchronous loading cannot wait for every module to finish. Node can return an unfinished exports object to break the recursion. Consumers may observe properties published before the recursive `require` and miss later initialization. This differs from ESM's pre-linked live lexical bindings, but both models require graph reasoning rather than a blanket "cycles work" or "cycles fail" rule.

6. Node interoperation is asymmetric

An ESM `import` can target CommonJS. Node supplies the CommonJS `module.exports` value as the default export. It may also synthesize named exports by statically analyzing recognizable CommonJS source patterns before evaluation. That detection is heuristic. After CommonJS evaluation, Node copies detected property values onto the namespace; later changes to `module.exports` are not reflected in those named exports. The fixture's imported `count` remains `0` while the default export object's `count` becomes `1`. Use the default import when a CommonJS package does not explicitly guarantee compatible named exports.

In current Node 24, CommonJS `require()` can load a synchronous ESM graph and returns its module namespace object. It cannot load an ESM graph containing top-level `await`; Node throws `ERR_REQUIRE_ASYNC_MODULE`. Dynamic `import()` is the asynchronous bridge from CommonJS when the target may be asynchronous. This boundary is version-sensitive: older Node versions rejected `require(esm)` more broadly.

TypeScript NodeNext models current Node interoperation but cannot prove every runtime outcome. TypeScript optimistically permits some named imports from CommonJS declarations even though Node's static detection may not find them. It also does not reject a `require` merely because the target graph contains top-level `await`. The checker can approve the program while the Node 24 loader later rejects or exposes a different shape.

7. Package metadata controls public resolution

`main` names one legacy package entry. `exports` is the modern public entry map: it can define the root, subpaths, and conditions, and it takes precedence over `main` in Node versions that support it. Once `exports` exists, package-name imports may reach only listed or matched subpaths. Adding it to an established package can therefore break consumers that deep-imported internal files; the resulting Node error is `ERR_PACKAGE_PATH_NOT_EXPORTED`. This encapsulation defines supported package-name entry points, not a security boundary against direct filesystem access.

Conditional exports select targets for conditions such as `import`, `require`, `node`, and `default`. Node examines object keys in declaration order, so arrange conditions from more specific to less specific and retain `default` as a broad fallback when appropriate. The active resolver and request kind determine which conditions participate: an ESM import and CommonJS require can legitimately receive different files and therefore different module identities.

The `imports` field defines private `#` mappings usable from within that package; it can target local files or external packages. A package with both `name` and `exports` may also refer to its own exported entries by name. Self-reference obeys the same `exports` restrictions, making it useful for testing the actual public boundary without installing the package.

Runtime and declarations must align. A package may expose `.mjs` to import and `.cjs` to require while directing TypeScript to `.d.mts` and `.d.cts` declarations. The TypeScript resolver can successfully find a declaration that claims a shape the selected runtime file does not provide. Chapter 16's rule remains: declarations describe external implementation; Node executes that implementation. Test each published condition as a consumer would load it.

8. NodeNext models the pipeline; it does not run it

With `module: "NodeNext"`, TypeScript determines each file's prospective Node format and applies the corresponding resolution and interoperation rules. `.mts` and `.cts` are explicit source formats and emit `.mjs` and `.cjs`; ordinary `.ts` follows the nearest package `type` and emits `.js`. NodeNext implies NodeNext module resolution.

An ESM TypeScript source imports the emitted target spelling. If `counter.ts` emits `counter.js`, write `import "./counter.js"` in the source. TypeScript resolves that specifier to the source for checking while preserving the `.js` spelling for Node. Writing `./counter` fails the Node ESM extension rule; writing `./counter.ts` requires a separate extension-rewrite or no-emit policy and is not this repository's contract.

`import type` and `export type` are guaranteed to be elided. They let the checker relate declarations but create no emitted runtime binding, no Node resolution request, and no module-evaluation side effect. If initialization is required, use a value import or an explicit side-effect import and test it. Conversely, importing a value merely for its type can accidentally retain an evaluation edge unless the syntax and compiler options make the intent explicit.

A clean `tsc` result establishes compatibility with TypeScript's selected declarations and modeled resolver. It does not execute emitted JavaScript, verify that build artifacts were copied, confirm deployed `package.json`, validate a CommonJS named-export heuristic, or exercise every condition. Inspect emitted files and run them with the supported Node version. Chapter 19 develops the complete configuration and build contract.

9. Initialization is part of API design

Top-level module code is initialization with cache-scoped lifetime. It may register handlers, read configuration, open resources, or mutate shared state before the importing application reaches its own body. A type-only import does none of that; a second resolved identity can do it twice; a cycle can expose it halfway through.

Prefer explicit constructors and startup functions when work needs parameters, ordering, retry, cleanup, or failure policy. Keep top-level work small, deterministic, and safe to perform once per loader identity. If a side-effect import is intentional, name it in the entry point and test the observation. If exact singleton identity matters across package copies, processes, workers, or URL variants, a module cache is insufficient authority; choose an application-owned registry or external coordination mechanism whose boundary matches the guarantee.

False models, repaired

- **"A file extension is cosmetic."** Node uses `.mjs`, `.cjs`, `.js`, and nearest package `type` to select grammar and scope.
- **"Imports copy exported values."** ECMAScript imports are read-only indirect bindings to an exporter's current binding.
- **"An import runs where it appears."** Static imports declare graph edges; linking precedes module-body evaluation.
- **"Every circular import fails."** Cycles fail only when resolution, linking, initialization, or evaluation demands something unavailable; delayed reads can succeed.
- **"A module runs exactly once."** It evaluates once per resolved loader identity; URL queries, separate files, package copies, processes, and loaders can create more identities.
- **"`exports = value` changes a CommonJS export."** It rebinds the wrapper parameter; replacement requires `module.exports = value`.
- **"ESM named imports from CommonJS are live."** Node's detected names are heuristic copied values; the default export is the reliable `module.exports` value.
- **"Node 24 can require every ESM file."** `require` accepts only synchronous ESM graphs and rejects top-level-await graphs.
- **"`exports` falls back to `main` for omitted paths."** In supported Node resolution, `exports` takes precedence and blocks unlisted package subpaths.
- **"If TypeScript resolves it, Node will."** The checker models selected rules and declarations; the deployed loader resolves and executes emitted artifacts and metadata.
- **"A type-only import initializes the module."** TypeScript erases that edge, so Node never receives the request.
- **"Node resolution is ECMAScript resolution."** ECMAScript defines module semantics and host hooks; Node, browsers, and bundlers supply different resolution policies.

Mastery questions

1. Given one `.js` file in two nested package scopes, predict its format and name the metadata Node consults.
2. Distinguish a source file, emitted file, ESM record, CommonJS module, and package using one concrete library.
3. Explain why an imported `let` is read-only to the importer yet still live.
4. Trace load, link, initialization, and evaluation for a three-module ESM graph with one re-export.
5. Modify the safe cycle so it fails. Name the exact uninitialized read rather than blaming circularity.
6. Predict whether three specifiers differing only by query share Node's ESM instance.
7. Explain `exports.foo = 1`, `exports = { foo: 1 }`, and `module.exports = { foo: 1 }` from wrapper parameter identity.
8. Trace which partial CommonJS properties two modules observe during a synchronous cycle.
9. Compare default and named ESM imports from a CommonJS counter after it mutates an exported property.
10. Predict `require()` behavior for an ESM graph with and without top-level `await` under Node 24.
11. Design an `exports` map with root, public subpath, and import/require targets; state how key order affects conditions.

12. Explain why adding `exports` can be a breaking change even when every file remains on disk.
13. Show how a `.mts` source, `.d.mts` declaration, `.mjs` output, and package condition must align.
14. Explain why TypeScript source spells a relative ESM dependency with the emitted `.js` extension.
15. Prove from emitted text that `import type` creates no runtime edge.
16. Classify a failure at each phase: checking, interpretation, resolution, linking, evaluation, and observed value.
17. Decide whether a top-level database connection should be module initialization or an explicit startup function. Name the lifetime and retry consequences.

The closed-book workspace is in `exercises/ch18.md`.

Implementation lab — Package Graph Diagnosis

Diagnose the five entries in `examples/ch18/lab/fixture/` without modifying them. Begin with the investigation plan in `lab/starter.ts`: interpretation, resolution, linking, evaluation, and value. The package self-reference proves that `exports` overrides `main`, selects the `import` condition, and blocks an unexported deep path. Other entries omit a relative ESM extension, place CommonJS text in package-typed ESM, and rebind the CommonJS `exports` parameter.

Run each entry in an isolated child process. Record exit status, stdout, error code, and error class. Your diagnosis must name the first failed phase and the authority responsible. The evaluator asserts three named loader failures and one zero-exit wrong-value fault; it does not depend on network access, timers, installed fixture packages, or shared globals.

Presentation prompt

Give a 12–15 minute explanation that follows one source import through TypeScript checking and emit, Node format selection, specifier resolution, linking or CommonJS loading, evaluation, cache identity, and final value. Include one live-binding prediction, one safe and one failing cycle, one `exports`-map decision, one interoperation boundary, one emitted `import type` inspection, and one lab failure localized to its first incorrect phase. The complete checklist is in `presentations/prompts/ch18.md`.

Ready-when test

Proceed only when, without notes, you can distinguish files, modules, and packages; attribute format and resolution to Node rather than ECMAScript; predict live bindings, re-exports, graph evaluation, dynamic import, and URL identity; explain safe and failing ESM cycles through initialization; derive the CommonJS wrapper, export alias, cache, and partial cycle behavior; state Node 24's exact ESM/CommonJS interoperation boundary; design and trace `type`, `main`, `exports`, conditions, `imports`, and self-reference; predict NodeNext extension and emit behavior; prove that a type-only import creates no runtime edge; complete the lab; and localize each failure to an authority and phase before proposing a repair.

Chapter 19

Compilation, tsconfig, and Module Resolution

Chapter type: Integration **Prerequisites:** Chapters 16–18; strict TypeScript, ESM and CommonJS interpretation, package metadata, and emitted-JavaScript inspection **Capabilities introduced:** Identify the effective TypeScript project; trace source files and declarations through checking and emission; map compiler options to their responsible phase; predict which file and format Node executes **Earlier chapters integrated:** Type descriptions and strict checking; erasure and runtime truth; external-data boundaries; module scope, live bindings, package metadata, ESM, CommonJS, and loading **Later chapters enabled:** Systematic source-to-runtime diagnosis, production build design, test-runner analysis, and deployment verification **Estimated study time:** 5–7 hours across six sessions **Runnable sources:** `examples/ch19/src/`, `examples/ch19/test/`, `examples/ch19/fixtures/`, and `examples/ch19/lab/`

Compressed thesis

A `tsconfig.json` defines a TypeScript project: root-file discovery and compiler options for one invocation. The command selects which configuration is effective. TypeScript then constructs a program from root files, imported files, ambient declarations, and resolved package declarations; parses and binds source; checks expressions, assignments, calls, and declarations; and may emit JavaScript, declaration files, and source maps. Node or a test runner later chooses, resolves, loads, and executes a runtime file. No single configuration option performs all of those operations.

Options have bounded authority. `target` affects selected syntax transformations; `lib` supplies declaration sets; `module` helps determine emitted module format; `moduleResolution` finds source and declaration files according to a host-shaped lookup policy; strictness options expose specific uncertainty; `outDir` places new output. None installs a runtime API, changes Node's loader, validates external data, removes stale output, or proves that the file deployed is the file checked.

Diagnosis therefore starts with evidence: exact compiler and runtime versions, invocation, effective configuration, input graph, resolved import, emitted path and text, package metadata, and loader command. The canonical chapter preserves that shared technical spine. Reader-specific explanations belong in the personal appendix rather than in the shared chapter.

1. Trace responsibilities, not one imaginary compiler-runtime layer

Use the following phase names as a diagnostic decomposition. TypeScript may interleave or cache internal work, but each responsibility produces different evidence:

1. **Configuration discovery** selects a `tsconfig` and applies command-line overrides.

2. **File inclusion** chooses initial roots from `files` or `include`, then adds imports, references, libraries, and ambient declarations.
3. **Parsing** recognizes source syntax. **Binding** associates declarations and names with scopes.
4. **Module resolution** maps each specifier to a source file, JavaScript file, or declaration file for checking.
5. **Checking** evaluates inferred and declared constraints and reports diagnostics.
6. **Emission** erases type-only syntax and preserves or transforms selected source syntax. Optional declaration and source-map production creates additional artifacts.
7. **Runtime loading** belongs to Node, a test runner, or another loader, which interprets package metadata and runtime specifiers.
8. **Execution** evaluates the loaded JavaScript and encounters actual globals, values, I/O, and failures.

An editor can display diagnostics from one inferred project while a shell builds another. TypeScript can resolve `"./worker.js"` to `worker.ts` for checking while Node later resolves the retained `"./worker.js"` to emitted JavaScript. A source map can direct a debugger to `worker.ts` while Node still executes `worker.js`. "The config makes it work" hides all three consumers.

2. The invocation selects the project

Running `tsc` without input filenames searches for the nearest `tsconfig.json` from the current directory upward. `tsc -p path/to/tsconfig.json` selects a project explicitly. `tsc --showConfig -p ...` prints the effective configuration after inheritance, defaults, and command-line overrides; `--listFilesOnly` or `--explainFiles` exposes program membership, and `--traceResolution` explains lookup decisions. Record the local compiler's `--version` with that evidence.

TypeScript 6.0 makes an old ambiguity explicit. In a directory containing `tsconfig.json`, `tsc main.ts` reports `TS5112` because a filename invocation would ignore the project; `tsc --ignoreConfig main.ts` opts into that isolated compilation. Combining `-p` with a source filename reports `TS5042`. These commands are not spelling variants:

```
tsc -p tsconfig.json             project configuration and project roots
tsc --ignoreConfig main.ts       explicit file with command-line/default options
tsc -p tsconfig.json main.ts     invalid mixture
```

Predict before running: If the project enables `strict` and `noEmit`, what does `tsc --ignoreConfig main.ts` prove about those options? Nothing. The command intentionally excludes the project configuration. Inspecting source alone cannot recover the invocation that checked it.

3. The program graph is larger than an include glob

`files` is an explicit root-file allowlist. `include` discovers roots through patterns relative to its configuration file. `exclude` removes matches only from `include` discovery. If an included file imports `./internal.js` and TypeScript resolves that specifier to `internal.ts`, the imported file enters the program even when `exclude` names it. A `types` selection, triple-slash reference, or `files` entry can also add a file that `exclude` would have filtered from an include search.

Ambient `.d.ts` files participate in binding and checking but normally produce no JavaScript implementation, as Chapter 16 established. An included test file can pull production source into the program; a referenced declaration can alter global names without creating a runtime value. Use `--listFilesOnly` and resolution traces rather than guessing from directory layout.

`rootDir` does not select input files. It defines the source directory used to preserve relative structure under `outDir` and prevents emission that would require writing outside the output directory. `outDir` selects where new artifacts go; it is not an ownership or cleanup declaration. The Chapter 19 fixture places an unrelated `stale.js` under its temporary output, rebuilds, and observes that `tsc -p` leaves the stale file present. A build must name which tool owns cleanup and which exact output directory it may remove. Do not infer current output solely from a successful incremental write.

4. `target`, `lib`, and `types` answer different questions

`target` tells TypeScript which JavaScript syntax level the consumer is expected to support and therefore which selected constructs require transformation. In the fixture, ES2024 output preserves optional chaining and nullish coalescing; ES2019 output replaces them with older operations. Both outputs print `ready` under Node 24. The exact emitted text is pinned to TypeScript 6.0.3, not promised across compiler versions.

`lib` supplies declaration files for built-in language and host APIs. It does not install an implementation or polyfill. Adding `"DOM"` makes `document.title` acceptable to the checker, but Node 24 still throws `ReferenceError` because Node supplies no `document` global. The reverse mismatch is also possible: a runtime may implement an API whose declaration is absent from the selected libraries, so TypeScript rejects otherwise executable source until accurate declarations are supplied.

`types` chooses which visible `@types` packages contribute globals and automatic type-package visibility. It does not install a runtime dependency or force Node to load a package. TypeScript 6.0 defaults `types` to an empty array; this repository explicitly selects `["node"]`. Imported packages can still bring their declarations through module resolution even when their `@types` package is not globally selected.

Strictness options describe named uncertainty. This repository's `strict` umbrella enables a family of checks. `noUncheckedIndexedAccess` adds `undefined` to values obtained through undeclared index-signature keys. `exactOptionalPropertyTypes` distinguishes absence from an explicitly present `undefined` unless the property type includes it. `noUncheckedSideEffectImports` reports unresolved side-effect-only imports. `verbatimModuleSyntax` preserves imports and exports without a `type` modifier, erases those marked `type`, and reports incompatible ESM syntax instead of silently rewriting it to CommonJS. `skipLibCheck` skips checking declaration-file internals; application source is still checked against the declaration information it uses. None of these options parses network input or audits a declaration against its runtime implementation.

5. `module` and `moduleResolution` must agree with the consumer

`module` controls module-format decisions and transformations. `moduleResolution` controls how TypeScript maps specifiers to files and package conditions for checking. They interact, but changing either option does not reconfigure Node.

Under `module: "NodeNext"` and `moduleResolution: "NodeNext"`, TypeScript models Node's per-file ESM/CommonJS interpretation. An `.mts` input emits `.mjs`; an `.cts` input emits `.cjs`. A plain `.ts` file follows the nearest `package.json` `"type"` field and emits `.js` in the corresponding format. For an ESM source import:

```
import { load } from "./worker.js";
```

TypeScript can use extension substitution to resolve `./worker.js` to `worker.ts` for checking. The emitted specifier remains `./worker.js`, which is the runtime file Node must find. Chapter 18's package `exports`, `imports`, conditions, extensions, live bindings, and initialization rules still govern loading. A TypeScript `paths` mapping likewise does not rewrite emitted imports; a runtime or bundler needs matching resolution support.

The format of the importing file also determines whether NodeNext resolution uses an `import` or `require` path and which package condition it selects. Treat `moduleResolution` as a prediction of the intended consumer's lookup, not as the consumer itself.

6. Checking, emission, maps, and execution remain separate outcomes

`noEmit` suppresses JavaScript, declaration, and source-map output while allowing checking. `noEmitOnError` suppresses new output when diagnostics occur; its ordinary `tsc` default is `false`. The Chapter 19 fixture therefore demonstrates a diagnostic and a newly emitted JavaScript file in the same compilation. `tsc -b` project-reference builds effectively require no-emit-on-error semantics.

The repository's `npm run check` supplies `--noEmit` on the command line. `npm run build` invokes `tsc` separately and can emit because the root configuration does not set `noEmit` or `noEmitOnError`. Running only the build command

is not equivalent to the verification script's check-then-build policy. Even `noEmitOnError: true` prevents only new output; it does not prove an older file is absent.

`declaration: true` produces `.d.ts` API descriptions in addition to ordinary output unless another emit option changes the combination. `sourceMap: true` writes a `.js.map` beside JavaScript and adds a map-location comment to the JavaScript. A source map lets debuggers and stack tooling relate emitted locations to source locations. `sourceRoot` changes where those tools look for sources by writing a path or URL into the map. Node executes JavaScript, not the map, and a map does not repair a wrong artifact or missing runtime API.

7. Direct TypeScript and multi-project configuration are explicit alternatives

Node 24.18 can directly execute `.ts` containing erasable TypeScript syntax by stripping it without typechecking. Node ignores `tsconfig.json`, performs no downlevel transformation, and does not apply TypeScript `paths`. Syntax requiring JavaScript generation, including enums and parameter properties, needs Node's `--experimental-transform-types` flag. Node also preserves the selected module system rather than converting ESM to CommonJS or the reverse.

The direct-execution fixture contains `target: "ES2019"`, `module: "CommonJS"`, and `noEmit: true`, yet plain `node erasable.ts` runs according to Node's package metadata and built-in type stripping. The configuration is irrelevant to that command. This repository's canonical path remains `tsc` checking and emission followed by Node 24 executing emitted ESM.

Configuration inheritance and project references introduce more selection points. `extends` loads a base first, then overrides it; relative paths remain relative to the configuration file where each path originated. Child `files`, `include`, and `exclude` replace rather than concatenate the base fields, and `references` is not inherited. A referenced project must enable `composite`, which also requires declaration output and complete root-file coverage. `tsc -b` can order and build referenced projects; ordinary `tsc -p` does not promise to build every referenced dependency. These facts are enough to read a multi-project repository without turning this chapter into monorepo architecture.

8. Build the evidence packet before changing options

For a pipeline failure, record the actual input filename; invoking directory and command; effective config from the exact project; compiler version; included files; resolved source or declaration for each disputed import; emitted path and semantic text; package metadata governing that emitted file; test runner or Node command; and runtime version. Then classify the first disagreement by phase.

Do not toggle `module`, add `DOM`, weaken `strict`, or copy output until the evidence identifies which consumer disagrees. Chapter 20 turns this evidence packet into a full debugging procedure.

Consequences for program design

Make one command authoritative for each deliverable: checking, application emission, declarations, tests, or direct execution. Pin versions and keep output ownership narrow. Test semantic markers in emitted code and execute the artifact that deployment will execute. In a multi-project repository, record the exact project file for every command because inherited options and roots can differ.

Prefer option comments that name phase and consumer. "`lib: ["ES2024"]` because Node has no browser globals" is recoverable; "modern settings" is not. Treat compiler configuration as executable policy: verify it with fixtures whenever a compiler, runtime, package mode, or runner changes.

False models, repaired

- **"The editor and build use the same project."** They agree only when they select the same configuration, roots, compiler version, and declarations.

- **"`exclude` bans a file."** It filters `include` discovery; an import, reference, `types` selection, or `files` entry can still add the file.
- **"`rootDir` selects source."** It shapes and constrains output paths; `files`, `include`, and graph edges select program files.
- **"A newer `lib` polyfills Node."** `lib` supplies declarations, not runtime values.
- **"`target` controls which globals exist."** It controls selected syntax transformations and default declaration selection; the runtime supplies globals.
- **"`moduleResolution` changes Node's loader."** It predicts lookup for checking; Node independently resolves emitted specifiers.
- **"Successful checking means JavaScript was produced."** `noEmit` permits a successful check with no output.
- **"Diagnostics always prevent output."** Ordinary `tsc` may emit unless `noEmitOnError` or build policy prevents it.
- **"A source map is executable TypeScript."** It maps locations for tooling; Node executes JavaScript.
- **"Node running `.ts` applies `tsconfig`."** Node's built-in stripping ignores the file and performs no TypeScript checking.

Mastery questions

1. Given an editor command and two nested `tsconfig` files, determine which project each consumer selects and specify the evidence needed to prove it.
2. Predict `TS5112` and `TS5042` for two incorrect TypeScript 6.0 invocation forms, then repair each without weakening project options.
3. Starting from `include: ["src/main.ts"]` and `exclude: ["src/internal.ts"]`, derive why an import of `internal.js` can add `internal.ts` to the program.
4. Given `rootDir: "src"` and `outDir: "build"`, derive emitted paths for two nested inputs and diagnose one input outside `rootDir`.
5. Compare ES2019 and ES2024 emission of optional chaining. Which runtime assumption does `target` record, and which fact does it not prove?
6. Construct a `lib` configuration that accepts `document.title`, then predict Node 24 execution and identify the missing authority.
7. Map `strict`, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`, `noUncheckedSideEffectImports`, `verbatimModuleSyntax`, and `skipLibCheck` to their exact checked uncertainty.
8. For `module/moduleResolution: NodeNext`, derive the output filenames and formats of `entry.mts`, `entry.cts`, and package-classified `entry.ts`.
9. Trace `import "./worker.js"` from TypeScript resolution of `worker.ts` through emitted text to Node lookup.
10. Compare `noEmit`, `noEmitOnError`, declaration output, and source-map output for a project with one diagnostic and an existing stale JavaScript file.
11. Diagnose a test stack trace that names TypeScript while Node executed JavaScript. What can the source map establish, and what can it not establish?
12. Predict direct Node 24 execution for a type annotation, an enum, and a parameter property, with and without `--experimental-transform-types`.
13. Merge a base and child configuration containing relative paths and overlapping `include` arrays; compute the effective values rather than assuming array concatenation.
14. Transfer the evidence packet to an unfamiliar runner that transpiles TypeScript in memory. Which emitted-text, loader, and version evidence changes?

The closed-book workspace and lab specification are in `exercises/ch19.md`.

Implementation lab — Configuration Evidence Report

Implement `inspectPipeline` in `examples/ch19/lab/starter.ts`. Materialize only the named fixture in a fresh temporary directory. Invoke the local TypeScript 6.0.3 compiler with `--showConfig`; record effective target, module, resolution, libraries, emit controls, output directory, and files; compile; inspect the expected JavaScript and map; then run Node only when checking passed and output exists.

Use the `lib-runtime` fixture to explain why `tsconfig.dom.json` checks and emits but Node throws `ReferenceError`, and why `tsconfig.node.json` reports `TS2584` yet still emits under `noEmitOnError: false`. Use the `no-emit` fixture to prove that checker success can produce no JavaScript. Then inspect `incorrect-success-assumption.ts`, which treats a zero checker status as proof of both output and runtime support.

Presentation prompt

Give the structured 10–15 minute presentation in `presentations/prompts/ch19.md` using your own words and sparse notes. Include one option-to-phase map, one effective-config prediction, one file-graph derivation, one emit inspection, one `lib`/runtime counterexample, one Chapter 18 module connection, and one unresolved runner or version boundary.

Ready-when test

Proceed only when, without notes, you can identify the effective project for an unfamiliar command; derive the complete file graph and output paths; map each repository option to its checking, resolution, emission, tooling, or runtime boundary; predict NodeNext format and `.js` specifier handling; distinguish `target`, `lib`, and runtime support; explain checking with no output and output with diagnostics; inspect a source map without treating it as executable; contrast `tsc` with Node 24 type stripping; complete the evidence-report lab; and diagnose a new pipeline disagreement from named artifacts rather than option folklore.

Chapter 20

Diagnosing the Source-to-Runtime Pipeline

Chapter type: Emergent **Prerequisites:** Chapters 16–19; static/runtime boundaries, external-data parsing, Node module loading, and effective TypeScript configuration **Capabilities introduced:** Capture a reproducible pipeline incident; locate its first divergent phase; assign the repair to the responsible authority; prove the repair through the real consumer **Earlier chapters integrated:** Declaration honesty and erasure; runtime validation; ESM/CommonJS loading and package metadata; configuration, resolution, emission, and source maps **Later chapters enabled:** Production build design, observability, incident response, and capstone debugging **Estimated study time:** 5–7 hours across six sessions **Runnable sources:** `examples/ch20/src/`, `examples/ch20/test/`, `examples/ch20/fixtures/`, and `examples/ch20/lab/`

Compressed thesis

A source-to-runtime failure is a disagreement between artifacts or consumers, not evidence that "TypeScript," "Node," or "the build" is generally broken. For one exact command, predict each phase's result, compare the prediction with an observed artifact, and stop at the earliest disagreement. Repair the authority that owns that disagreement, then rerun the remaining phases in order.

This method requires identities, not impressions: source and specifier; invoking directory and environment facts; compiler version and effective project; included and resolved files; emitted or in-memory JavaScript; copied or deployed entry; package metadata and loader conditions; Node version and flags; runtime input; and source-map pairing. An editor, test runner, development command, production process, and deployment may select different projects, transforms, files, conditions, or runtimes. Agreement in one path supplies no guarantee about another.

The chapter is compressed by design. Preserve the canonical procedure and place reader-specific explanations in the personal appendix after the reader reports a concrete point of confusion. Do not replace evidence collection with option changes, broad cleanup, assertions, or cache folklore.

1. Freeze the incident before explaining it

Write one reproducible incident statement:

```
cwd: /service/current
command: node --enable-source-maps build/main.js
Node: 24.18.0
environment: SERVICE_CONFIG present; secret values redacted
expected: prints port=8080
actual: status 1, TypeError, mapped frame src/main.ts
```

Record the executable path when multiple installations are plausible. For a child process, retain numeric status, signal if relevant, stdout, stderr, stable error category, and `error.code` when Node supplies one. Exact diagnostic prose and stack formatting are version-sensitive; status, semantic markers, paths, export names, and stable codes usually make better assertions.

Reproduce the command before changing source, configuration, package metadata, input, or caches. A changed symptom after an unrecorded cleanup destroys evidence. Redact secrets but retain structural facts such as presence, encoding, key names, and observed runtime types. "Works in the editor" is not a command; record the editor's TypeScript version and selected or inferred project only when that result is part of the incident.

2. Inspect one ordered pipeline

Use this diagnostic order. It is a responsibility model, not a claim that every tool implements identical internal stages.

Phase	Prediction and decisive artifact	Authority that can disagree
Source and config discovery	exact source, cwd, command, selected project	editor, CLI, runner project selection
Parser and checker	diagnostic status and codes for that project	TypeScript syntax and type constraints
Type/declaration resolution	selected source or `.d.ts`, package condition, trace	TypeScript resolver and declaration publisher
Emission or runner transform	emitted or in-memory JavaScript, semantic marker	compiler, bundler, or runner transform
Artifact selection and copy	output path, hash, contents, deployment manifest	build and deployment process
Runtime format interpretation	ESM or CommonJS classification	extension and nearest package metadata
Runtime specifier/package resolution	resolved runtime file and export condition	Node or the actual loader
ESM linking or CJS loading	available exports and graph construction	loader plus package implementation
Evaluation and initialization	import order, bindings, top-level effects	module implementation
Runtime data, host, and resources	validated input, globals, files, network, lifetime	boundary parser, host, producer, resource owner
Observation and source-map mapping	executed JavaScript paired with its map	build metadata and diagnostic tooling

For every inspected phase, write: artifact, command and version that produced or consumed it, expected value, actual value, whether they match, and the next hypothesis if they do not. "Module problem" omits at least format interpretation, runtime resolution, linking/loading, and evaluation. "Compilation succeeded" omits project discovery, graph construction, checking, emission, artifact freshness, and execution.

Source maps come last because they alter observation. With `--enable-source-maps`, Node can display a TypeScript location derived from a map while executing JavaScript. That frame helps locate generated behavior; it does not prove that the right JavaScript was emitted, copied, or selected, or that runtime input had the declared shape.

3. Stop at the first divergence

The first-divergence procedure is mechanical:

1. Predict the source, project, artifact, format, resolution target, export contract, evaluation result, runtime boundary, and observation for the exact failing command.
2. Compare each prediction with captured evidence in phase order.
3. Stop at the earliest mismatch, even when a later exception is louder.
4. Name the authority that produces or consumes the mismatched artifact.
5. Choose one next command that can confirm or reject the resulting hypothesis.
6. Repair that authority and rerun forward through the actual deployment or runtime consumer.

Suppose TypeScript resolves `runtime-kit` to `index.d.ts`, which declares `renderToken`, and emits a retained named import. The package's runtime `import` condition selects `index.js`, which exports only `render`. Node finally reports a missing named export during ESM linking. The final symptom belongs to the linker, but the earliest contract disagreement is the declaration/implementation pair. Switching the consumer to CommonJS changes a downstream phase without repairing the package.

Now suppose source contains `SOURCE_MARKER:fresh-v2`, effective config has `noEmit: true`, and `build/main.js` retains `OUTPUT_MARKER:stale-v1` with the same hash before and after a successful `tsc` command. Node prints stale-v1 successfully. Runtime execution matches its selected artifact. The first divergence is emission policy: the supposed build produced no new JavaScript. Deleting caches before recording the hash would hide that fact.

Finally, suppose checking and emission succeed, the selected output is fresh, and `SERVICE_CONFIG` decodes to `{ "port": "8080" }`. A type assertion calls the object `Config`; execution fails on `port.toFixed`. The pipeline remains coherent through evaluation. The first mismatch is external data versus the application's runtime boundary, as Chapter 17 predicts. A mapped TypeScript frame does not move ownership to the checker.

4. Build the evidence packet deliberately

Begin with inexpensive discriminators, then deepen only the disputed phase:

```
node --version
node -p 'JSON.stringify({version:process.version,execArgv:process.execArgv})'
./node_modules/.bin/tsc --version
./node_modules/.bin/tsc -p tsconfig.json --showConfig
./node_modules/.bin/tsc -p tsconfig.json --listFilesOnly
./node_modules/.bin/tsc -p tsconfig.json --traceResolution --noEmit
```

Use `--explainFiles` when the question is why a file entered the program; use `--traceResolution` when the question is which declaration or source satisfied a specifier. Capture the selected lines rather than treating a large trace as self-explanatory. For editor-only disagreement, record whether the file belongs to a configured project or an inferred project and which TypeScript installation the editor uses.

Inspect emitted text. Type-only imports should be absent; retained runtime imports and unresolved aliases should be visible. A TypeScript `paths` entry can guide checking without rewriting the emitted specifier, so a successful trace does not make that alias meaningful to Node. If a runner transforms in memory, capture its transform configuration and representative output or a documented semantic marker. Never assume the test runner uses `tsc` merely because it accepts `.ts`.

Then identify the selected artifact: expected path, existence, modification context, content marker, and cryptographic hash before and after the build. Record copy/package steps and inspect the deployed `package.json`, entry file, `exports` targets, declarations, maps, and dependency files. The repository's source metadata is irrelevant when deployment contains different metadata.

At runtime, record executable, version, `process.execArgv`, loader flags, conditions, entry path, and sanitized environment facts. A development runner may transform TypeScript and resolve aliases; a test runner may isolate each file in a child process and inject conditions; production may execute emitted JavaScript under another Node release. Compare those facts explicitly.

5. Distinguish recurring disagreement classes

- **Editor accepts, build rejects.** Compare project, TypeScript version, roots, and effective options. Repair selection or source under the authoritative build; do not weaken strictness merely to imitate an inferred editor project.
- **Checker sees an export, runtime does not.** Compare the selected declaration with the runtime condition target. Package declarations describe an implementation; they do not create it.
- **`paths` checks, emitted alias fails.** Inspect emitted text. Repair the runtime/bundler mapping or use runtime-valid specifiers; changing unrelated Node format cannot rewrite the alias.
- **A type-only import disappears.** This is correct erasure. If execution required its side effect, repair the source edge rather than forcing a declaration to become a runtime value.
- **Wrong or stale `outDir` runs.** Compare semantic markers and hashes across build, copy, image, and process entry. Repair the producing or selection step, then rebuild a narrowly owned output.
- **Tests pass, Node fails.** Compare transform, aliases, package conditions, loader hooks, isolation, flags, environment, and Node version. Reproduce the production artifact and loader before modifying modules.
- **Link/load failure versus evaluation failure.** Missing exports and unresolved specifiers occur before ordinary module-body evaluation; `ReferenceError` or initialization-order faults can occur during evaluation. Preserve the distinction when assigning repair ownership.
- **TypeScript-looking stack, JavaScript execution.** Verify `.js` and `.js.map` pairing. Repair maps only when frames are wrong; repair the earlier phase when the behavior is wrong.
- **Clean pipeline, bad host fact.** Parse external values, confirm runtime globals and resources, and inspect process lifetime. Static declarations cannot make input, storage, network, or shutdown behavior correct.

6. Reproduce, minimize, repair, regress

First reproduce the exact artifact under the exact loader. Then copy the smallest necessary fixture into a fresh temporary directory and remove one phase at a time while preserving the failure. A useful minimization retains the disputed declaration and implementation pair, package metadata, emitted import, and Node command. A useless minimization replaces the package import with a local function and accidentally removes the phase under investigation.

Do not begin by changing `module`, adding DOM declarations, disabling strictness, copying files by hand, deleting caches, or adding assertions. Each action changes evidence or moves ownership. Cleanup is justified only after artifact identity is captured, and it must target a directory owned exclusively by the current build—not a repository root, home directory, or shared cache.

Match regression evidence to the repair class. A declaration/package repair compiles and executes a packaged consumer for supported conditions. An emit repair proves a fresh semantic marker and changed hash, then executes that output. A deployment repair validates the copied entry and metadata. A loader repair runs the supported Node version against the packaged graph. A runtime-input repair exercises valid, missing, and wrong-shaped input through an actual parser. A source-map repair compares mapped and unmapped frames while holding JavaScript constant.

Production builds should record compiler/runtime versions, effective build configuration, and artifact identity. Startup may report a build identifier, runtime version, entry path, and enabled loader mode, but must not reveal secrets. Broader observability policy belongs to Chapter 24; this chapter supplies only the identities needed to reproduce a pipeline incident.

False models, repaired

- **"The last exception names the broken layer."** Later consumers often expose an earlier mismatch; compare phases in order.
- **"If the editor accepts it, TypeScript accepts it."** Name the editor project and version, then compare the exact build.
- **"If TypeScript resolves it, Node resolves it."** TypeScript can select declarations or source substitutions that do not change emitted runtime lookup.

- **"A green build means the deployed output is current."** Prove output production, copy, selection, content, and hash.
- **"A `.ts` frame means TypeScript executed."** A source map can project an executed JavaScript location onto TypeScript source.
- **"Clearing caches is harmless diagnosis."** It can destroy the only stale-artifact evidence and may affect unrelated owners.
- **"Changing module mode is a general import fix."** Format is one phase; declaration drift, aliases, stale copies, and runtime data have different owners.
- **"An assertion repairs a runtime shape."** It changes checking only; validate unknown external data at execution.

Mastery questions

1. Construct an evidence packet for an editor/build disagreement and choose the single next command with the highest discriminating value.
2. Given successful checking, emitted named import, package `types`/`import` targets, and a Node linking error, identify the first divergence and package-owner regression.
3. Prove that a `paths` alias affected TypeScript lookup but not emitted text or Node resolution.
4. Distinguish an erased type-only edge from an accidentally missing side-effect import.
5. Diagnose a successful check followed by execution of unchanged stale output; name the required before/after evidence.
6. Compare a transforming test runner with production Node across transform, aliases, conditions, isolation, flags, and versions.
7. Separate runtime format interpretation, specifier resolution, ESM linking, and module evaluation for one failure in each phase.
8. Explain why a mapped TypeScript frame cannot prove artifact freshness or runtime-input validity.
9. Minimize a declaration/runtime mismatch while preserving the package boundary and failing phase.
10. Reject three tempting option changes for a runtime-input fault and design a parser regression instead.
11. Transfer the method to an unfamiliar deployment that copies only selected package files. Which identities must be captured before repair?
12. Given multiple mismatches, choose the first, state one next hypothesis, and describe how to rerun forward after repair.

The closed-book workspace and lab specification are in `exercises/ch20.md`.

Implementation lab — First-Divergence Incident Report

Implement `diagnoseIncident` in `examples/ch20/lab/starter.ts`. It receives a captured `EvidencePacket`; do not rerun tools, mutate input, or change compiler and runtime settings. Return the first divergent phase, authority, artifact, expected and actual facts, decisive evidence, one next hypothesis, repair class, repair, regression command, and rejected folklore fixes.

The evaluator captures three temporary incidents. The primary incident pairs a declaration that promises `renderToken` with runtime JavaScript exporting only `render`; an unrelated legacy artifact and map are distractions. A stale-output incident combines checker success, effective `noEmit`, unchanged hashes, and an old output marker. A runtime-input incident combines a clean build with a string-valued port and a TypeScript-looking mapped frame. Inspect `last-symptom-fault.ts`, which deliberately changes ownership from the final exception instead of finding the first mismatch.

Presentation prompt

Give the structured 10–15 minute presentation in `presentations/prompts/ch20.md`. Start with one evidence packet, walk phase-by-phase to the first divergence, select one next command, reject two folklore changes, name the responsible authority, and close with regression evidence under the actual runner or deployment.

Ready-when test

Proceed only when, without notes, you can freeze an incident; enumerate the ordered phases; name one decisive artifact and authority per phase; use TypeScript and Node evidence commands deliberately; distinguish checking, transform, copy, format, resolution, linking/loading, evaluation, runtime input, and mapping; stop at the first mismatch; reproduce and minimize without removing that phase; reject shotgun option and cleanup changes; assign a bounded repair; and prove it through the exact artifact, runner, loader, runtime version, and deployment path that originally failed.

Chapter 21

Types, Tests, Assertions, and Constraints

Chapter type: Integration **Prerequisites:** Chapters 1, 5, 11, 15–17, and 20; familiarity with strict TypeScript, `unknown`, discriminated unions, runtime parsing, branded values, effect boundaries, and evidence-first diagnosis **Capabilities introduced:** Allocate correctness propositions among static checking, runtime validation, assertions, executable tests, authorization, database constraints, transactions, and production observation **Earlier chapters integrated:** State ownership and commit from Chapter 5; narrowing and trusted predicates from Chapter 11; brands from Chapter 15; erased types and runtime truth from Chapter 16; unknown-input parsing from Chapter 17; evidence packets and first divergence from Chapter 20 **Later chapters enabled:** Boundary-heavy system design, authorization and transactional depth, production observability, and capstone correctness arguments **Estimated study time:** 5–7 hours across six sessions **Runnable sources:** `examples/ch21/src/`, `examples/ch21/test/`, and `examples/ch21/lab/`

Compressed thesis

Correctness is a set of propositions, not a property conferred by one tool. "Known callers provide an `OrderCommand`," "this received payload contains a bounded positive amount," "this principal may perform this action now," "no participating database write duplicates this key," and "selected production outcomes were recorded" are different propositions. They require different evidence and enforcement.

TypeScript checks source relationships against declarations and compiler configuration. A runtime parser inspects a particular value. A runtime assertion executes and fails if its condition is false. A test observes selected code, inputs, environments, and schedules. Authorization decides whether a current principal may perform a current action on a current resource. A database constraint rejects writes violating its declared predicate. Monitoring observes selected events during or after execution. None inherits the authority of the others.

For every correctness statement, write a **guarantee entry**: the exact proposition; the evidence produced; the component that checks, enforces, or observes it; when that component acts; the scope covered; and the remaining bypass. This chapter's central skill is allocating each proposition to an authority strong enough to support it, then retaining independent defenses where distinct trust boundaries justify them.

1. Build a guarantee ledger before selecting mechanisms

Suppose an API accepts an order request. "The order is valid" hides several independent claims:

Proposition	Evidence class	Responsible component	Time and scope	Remaining bypass
Known source callers supply branded fields	Early detection	TypeScript checker	Included source under active declarations/config	assertions, external values, runtime mutation
This payload contains supported, bounded values	Prevention	Runtime parser	Request boundary; inspected value	unparsed paths, later mutation, facts outside payload
This internal candidate still satisfies an invariant	Early detection	Executing assertion	Immediately before dependent code	dishonest assertion body, caught failure, later mutation
Selected cases behave as expected	Sampled evidence	Test runner and assertions	Tested code, cases, runtime, dependencies, configuration	untested inputs, schedules, clients, deployments
Current principal may create this tenant's order	Prevention	Authorization policy/decision	Protected operation and current context	missing placement, stale context, alternate effect path
Tenant and order key remain unique	Durable enforcement	Named database `UNIQUE` constraint	Participating writes to constrained table	data elsewhere, absent schema, downstream effects
Selected outcomes were recorded safely	After-the-fact observation	Instrumentation and telemetry pipeline	Instrumented branches reaching configured sink	missing/wrong events, sampling, sink failure, secret leakage

The row format prevents "we have types" or "tests pass" from ending the argument. It also distinguishes **prevention**, **early detection**, **sampled evidence**, **durable enforcement**, and **after-the-fact observation**. These labels describe what the evidence can support; they are not rankings. A compiler diagnostic can be earlier than a database rejection, while the database can govern clients the compiler never saw.

2. TypeScript checks declared source relationships

Under this repository's strict configuration, TypeScript checks whether expressions, assignments, calls, and declarations satisfy inferred or declared type constraints. A branded `PositiveCents` prevents ordinary checked source from supplying a raw `number`. A closed currency union rejects a known literal outside its members. A static fixture with `@ts-expect-error` can verify that these rejections remain part of the compiler contract.

That evidence is scoped to the source files, declarations, library definitions, and compiler options in the checked program. The checker does not inspect an HTTP body arriving later, prove that a declaration describes its implementation, prevent `as`, validate a database row, authorize a user, freeze an aliased object, or establish which artifact is deployed. Type annotations and inferred types generally do not exist in emitted JavaScript. Some TypeScript constructs emit runtime code, but a type assertion does not.

Use types to make invalid states hard to express for cooperating checked code. Construct refined or branded values only after the responsible runtime check. Do not claim that the brand proves a record still exists, belongs to a tenant, or remains authorized. Those propositions depend on other authorities and times.

Static tests are useful because they execute the compiler against deliberate relationships: "raw `number` must not assign to `PositiveCents`." They remain tests of a checker configuration, not runtime tests. A suppression comment, assertion, excluded file, changed declaration, or different build configuration can change the evidence.

3. Runtime validation establishes checked facts about one value

External input begins as `unknown`. A boundary parser inspects runtime structure and values, refuses malformed input, and constructs a new closed domain value containing only accepted fields. In the lab, the parser checks

a tenant identifier pattern, an order-key pattern, an integer amount from 1 through 1,000,000 cents, and one of three supported currencies. Its success establishes exactly those facts for the constructed value at that time.

The parser does not establish tenant existence, current permission, cross-row uniqueness, account balance, or eventual delivery. It also cannot protect a different endpoint that skips it. If the accepted object remains mutable or an alias reaches mutable nested data, later writes can invalidate earlier facts. Construction plus readonly types reduce ordinary mutation; they do not retroactively enlarge what was checked.

Validation is normally part of a public invalid-input protocol: return stable issue codes and bounded paths, not raw payloads or stack traces. This differs from an internal assertion whose failure usually means an implementation assumption was violated. A parser's rejection is expected user-facing control flow; an invariant assertion is an alarm.

4. "Assertion" names three different mechanisms

A **TypeScript type assertion** such as `input as OrderCommand` changes how the checker treats an expression. It is removed during compilation and performs no conversion, restructuring, validation, or runtime failure. The deliberate lab fault asserts a negative amount into `OrderCommand`; the program checks successfully and stores the negative value in the repository simulator.

A **JavaScript runtime assertion** executes. With Node's strict assertion API, `assert.ok(condition)` returns normally when the condition is truthy and throws `AssertionError` when it is falsy. Whether a caller catches that error is a separate control-flow decision. The throw detects a violated assumption; it does not automatically map itself into a stable public response.

A **TypeScript assertion-function signature** connects executing code to control-flow analysis:

```
function assertInternalOrder(value: unknown): asserts value is OrderCommand {
  const parsed = parseOrderCommand(value);
  assert.ok(parsed.ok, "internal order invariant violated");
}
```

After normal return, TypeScript narrows `value` to `OrderCommand`. The signature is a declaration of trust in the body. If the body forgets the amount check but retains `asserts value is OrderCommand`, the checker accepts a false statement. Treat the body as a proof obligation and test its rejection behavior.

Use assertions for states that should be impossible if internal code is correct: unreachable variants, corrupted internal candidates, missing initialization, or violated preconditions. Use explicit parsing and result types for ordinary untrusted input. The same predicate may appear at both boundaries only when the failure protocols and trust boundaries are intentionally different.

5. Tests produce selected, executable evidence

Node's test runner executes test functions. A synchronous test fails when it throws; an asynchronous test fails when its promise rejects. Assertions inside the test compare the observed result with an expected result. A passing test therefore supports a bounded statement: this selected case passed with this code, test data, configuration, runtime version, dependency set, and schedule.

Different tests answer different questions:

Test kind	Typical proposition supported	Common missing evidence
Static fixture	A compiler relationship is accepted or rejected	Runtime value and emitted/deployed behavior
Unit test	One component maps selected inputs to outputs	Real collaborators and configuration
Integration test	Selected components agree through a real boundary	Other dependencies, data volumes, schedules

Test kind	Typical proposition supported	Common missing evidence
Contract test	An implementation matches selected interface examples	Universal provider behavior and deployment identity
Regression test	A known counterexample remains repaired	Unknown failure classes

Property-style loops can examine many generated or table-driven cases without a property-testing library. The Chapter 21 example checks ten boundary values deterministically. That is broader than one happy example and still finite. It does not prove the predicate for every JavaScript number, concurrent interleaving, database client, dependency release, or deployment.

A failing test is concrete counterevidence: at least one stated expectation does not hold in that tested context. Diagnose the first divergence before weakening the expectation. A passing suite is never proof of bug absence. Coverage measures executed structure under a run; it does not supply assertions, realistic inputs, authorization, or a production environment.

6. Database constraints enforce predicates at the storage boundary

PostgreSQL constraints apply to writes reaching the constrained table, including writes from other application instances, scripts, and services. That makes them valuable independent enforcement at a shared durable boundary.

```
CREATE TABLE orders (
  order_id bigint PRIMARY KEY,
  tenant_id text NOT NULL,
  order_key text NOT NULL,
  amount_cents integer NOT NULL,
  currency text NOT NULL,
  CONSTRAINT orders_positive_amount CHECK (amount_cents BETWEEN 1 AND 1000000),
  CONSTRAINT orders_supported_currency CHECK (currency IN ('USD', 'CAD', 'EUR')),
  CONSTRAINT orders_tenant_order_key_key UNIQUE (tenant_id, order_key)
);
```

`NOT NULL` rejects null in its column. `CHECK` requires its expression not to be false; in PostgreSQL, true **or null** satisfies a check, so pair nullable expressions with `NOT NULL` when null must be forbidden. `UNIQUE` governs equality across the named columns and has explicit null semantics; making both columns non-null avoids relying on the default treatment of nulls. A primary key combines uniqueness and non-null row identity. A foreign key requires non-null referencing values to match a referenced key and maintains referential integrity under its configured actions.

Constraints cannot express every business rule, current authorization decision, remote side effect, or fact stored outside their database. `CHECK` is intended for the new or updated row, not arbitrary cross-row or cross-table conditions; use the constraint forms designed for those relationships. Map a named database rejection into a stable domain result such as `duplicate-order-key`. Do not perform a uniqueness precheck and assume it reserves the future: another writer can win before insert. Let the database arbitrate the participating write.

A transaction bundles participating database steps into an all-or-nothing operation and hides intermediate states from concurrent transactions under PostgreSQL's documented model. It does not automatically include an already-sent email or remote HTTP call. Chapter 23 develops transaction boundaries, isolation, retries, and authorization timing in depth.

The lab uses an in-memory **repository-contract simulator** to deterministically exercise named uniqueness rejection. It is evidence about the application mapping, not evidence that PostgreSQL was configured or behaved as claimed. The database claims come from the official PostgreSQL documentation and require schema inspection and an integration test against the supported database.

7. Authorization protects an operation, not a shape

Authorization evaluates a current principal, action, resource, and context at the protected operation. A structurally valid order can still target another tenant. A row's existence and a branded `TenantId` do not make the caller its owner. A passing parser, test suite, or uniqueness constraint does not grant permission.

Place the decision so no unauthorized path reaches the effect. The lab injects authorization and proves that denial causes no repository insertion. That test supports the selected control path; system confidence also requires auditing alternate handlers, jobs, admin paths, and stale decision caching. Chapter 23 owns policy depth and transaction-aware timing.

8. Observation detects outcomes; it usually does not prevent them

Logs, metrics, and traces observe selected events during or after execution. An alert can trigger human or automated response after a threshold or event. Unless instrumentation is itself an enforcement gate, recording "negative amount observed" does not prevent the write. The deliberate fault's accepted event is evidence that the wrong path ran; it does not repair the stored value.

Instrumentation can be missing, misconfigured, sampled, delayed, mislabeled, or unavailable with the failing service. High-cardinality labels can make metrics expensive; payload logging can leak credentials or personal data. Emit bounded outcome and reason codes, keep secrets out, and test the event contract. Chapter 24 expands production observability.

9. Duplicate defenses by boundary, not by accident

Redundancy is valuable when independent components enforce the same important proposition for different populations or times. A request parser rejects a non-positive or over-limit amount early and returns useful feedback; a database `CHECK` constraint rejects the same amount predicate for every participating table write, including other clients. Their overlap is intentional because the parser supplies a request protocol while the constraint supplies storage-boundary enforcement. An application or repository duplicate lookup is different: it can provide early feedback from the state it observes, but it is race-prone and cannot replace the `UNIQUE` constraint that arbitrates the write.

Duplication is harmful when teams hand-copy currency lists, identifier regular expressions, status transitions, or constraint names without a canonical authority or synchronization test. Divergent copies create contradictory accepted sets. Identify the canonical authority for each proposition. Derive representations where possible; otherwise add contract or integration tests that compare the translation and assign an owner for changes.

When a failure appears, apply Chapter 20's evidence-first sequence. Inspect the actual compiler configuration, parser code, assertion body, executed test command, deployed schema, authorization placement, runtime artifact, and telemetry configuration. Locate the first divergence from the guarantee entry. Do not weaken a type, remove a test, catch an assertion, or discard a constraint merely because doing so makes the symptom disappear.

10. Lab: route one command through every authority

The lab submits an order with a branded tenant identifier, branded order key, positive bounded cents, and supported currency. The correct path accepts `unknown`, parses and constructs the domain value, executes an internal invariant assertion, asks an injected authorization decision at the protected operation, calls a repository interface, maps the named uniqueness rejection to `duplicate-order-key`, and records a bounded secret-safe outcome.

Tests establish separate propositions: static fixtures reject raw branded values; malformed payloads are refused; the runtime assertion throws; ten representative amount cases pass; authorization denial produces no repository effect; a named repository conflict maps correctly; and observation excludes an extra secret-bearing input field. The deliberate implementation replaces parsing with `as OrderCommand`. It passes the checker and its authorization branch, yet the evaluator demonstrates a negative storage effect and a misleading accepted event.

Use `GUARANTEE_LEDGER` as an architectural review object, not decoration. For each new row, ask whether its component can actually inspect or govern the proposition, at the stated time, for the stated population. If not, either narrow the proposition or move enforcement.

False models, repaired

- **"The value has a TypeScript type, so runtime input is valid."** The checker analyzed source relationships; a parser must inspect the received value.
- **"`as OrderCommand` validates or converts the object."** A type assertion is erased and performs no runtime work.
- **"An assertion-function signature proves its predicate."** The checker trusts the signature; the executing body must honestly test or throw.
- **"All assertions are input validation."** Public invalid input normally needs a stable result protocol; internal assertions signal violated programming assumptions.
- **"The tests pass, so the property is proven."** Passing tests support selected cases and environments; failing tests provide concrete counterevidence.
- **"Checking for a duplicate before insert guarantees uniqueness."** A concurrent writer can invalidate the precheck; the storage authority must arbitrate the write.
- **"A database constraint makes the whole workflow correct."** It governs its declared database predicate, not authorization, remote effects, or data elsewhere.
- **"A valid row means the caller may use it."** Authorization is a current principal/action/resource/context decision.
- **"Monitoring catches the problem, so it prevents the problem."** Observation normally occurs during or after the effect and can itself fail.
- **"Repeated validation is always defense in depth."** Independent boundary enforcement can be valuable; copied unowned definitions can diverge.

Mastery questions

1. Rewrite "the order is valid" as at least six distinct propositions. Give component, time, scope, evidence, and bypass for each.
2. Why can a brand reject ordinary source code while accepting a runtime lie introduced by `as`?
3. What exactly does successful parsing establish, and which three facts remain outside the payload?
4. Compare a type assertion, Node runtime assertion, and TypeScript assertion-function signature by emit, execution, failure, and narrowing.
5. Design a dishonest assertion function that compiles. What evaluator case exposes the false signature?
6. Allocate one bug among a static fixture, unit test, integration test, contract test, and regression test. What evidence remains missing from each?
7. Why does a deterministic boundary loop improve evidence without proving a universal numeric property?
8. Explain the difference among `NOT NULL`, `CHECK`, `UNIQUE`, primary key, and foreign key, including PostgreSQL's `CHECK`/null boundary.
9. Construct a two-writer schedule that defeats a uniqueness precheck. Which component settles the conflict?
10. Why is the in-memory repository simulator valid for mapping tests but invalid as evidence about PostgreSQL?
11. Give one valid order that must still be rejected by authorization. Where must the decision occur?
12. Design a secret-safe event for rejection. Which operational failures can make it absent or misleading?
13. Give one useful duplicate defense and one harmful duplicated definition. Name the canonical authority in each design.
14. A production conflict rises after a deployment. Build an evidence packet before changing code or schema.
15. Transfer the guarantee-ledger method to a job queue, payment command, or file upload with at least seven propositions.

Presentation prompt

Present a 12–15 minute correctness argument for the order-command lab. Begin with the guarantee ledger, not the code. Demonstrate the erased assertion fault and the executing assertion failure. Classify each test's evidence, trace an unauthorized request to a no-effect result, and explain why the repository simulator cannot substantiate PostgreSQL behavior. End with one useful redundant defense, one divergence risk, and one unfamiliar system transferred into the same ledger format. Use `presentations/prompts/ch21.md` for the evaluator rubric.

Ready-when test

You are ready to continue when, without notes, you can turn a vague correctness claim into a guarantee ledger; distinguish checker, parser, type assertion, runtime assertion, assertion signature, test, authorization decision, database constraint, transaction, and observation; predict the deliberate fault's invalid effect; explain PostgreSQL constraint and simulator boundaries; design both useful redundancy and synchronization against divergence; and diagnose a new failure from actual evidence before weakening any guarantee.

Chapter 22

Errors, Public APIs, and Domain Boundaries

Chapter type: Integration **Prerequisites:** Chapters 8, 13, 15, 17, and 21; familiarity with promises, typed APIs, discriminated unions, runtime parsing, and correctness authorities **Capabilities introduced:** Classify failures by producer and recovery obligation; choose explicit results or exceptions by contract; translate internal failures into stable, secret-safe public semantics; preserve uncertain effects and compatibility **Earlier chapters integrated:** Promise rejection from Chapter 8; generic relationships from Chapter 13; stable named domain categories from Chapter 15; `unknown` input and construction from Chapter 17; proposition, evidence, enforcement, and boundary ownership from Chapter 21 **Later chapters enabled:** HTTP service boundaries, authorization and transactional failures, production diagnostics, and capstone API design **Estimated study time:** 5–7 hours across six sessions **Runnable sources:** `examples/ch22/src/`, `examples/ch22/test/`, and `examples/ch22/lab/`

Compressed thesis

A useful failure contract answers concrete questions: who produced the failure, how it is represented, which path carries it, what the consumer can do, what remains stable across releases, and which facts must remain private. "Something went wrong" answers none of them. "Every failure throws" and "never throw" replace design with slogans.

Invalid external input, a domain refusal, an authorization refusal, a storage conflict, temporary dependency unavailability, an effect whose outcome is unknown, cancellation, a deadline, and an invariant defect have different producers and recovery obligations. They should not collapse into one `Error`, one boolean, or one public status. Expected alternatives often belong in explicit discriminated results. Exceptions abruptly leave the normal path and are useful when the current operation cannot continue or does not own the recovery. A promise can still reject even when its fulfillment value is `Result<T, E>`; `Result` syntax does not abolish defects, cancellation, or dependency failure.

Translate a failure only where a component owns a vocabulary change. Preserve the original cause for internal diagnosis. Expose a stable public code, bounded safe message, correlation identifier, selected field issues, and justified caller action. Withhold stacks, dependency names, SQL, filesystem paths, credentials, arbitrary input, and nested causes. A timeout or rejected promise proves that an observation ended; it does not prove that a remote effect did not occur.

1. Classify by producer, path, and consumer action

Start with the event, not the class hierarchy. The same English word—"invalid," "conflict," or "timeout"—can hide incompatible semantics.

Failure category	Immediate producer	Useful representation and path	Consumer action	Fact not established
Invalid external input	Runtime parser at an ingress boundary	Explicit issues in an invalid result	Correct named fields	Permission, existence, or later success
Domain refusal	Domain transition or policy owned by the domain	Named result variant	Change the requested business action	Infrastructure failure
Authorization refusal	Current authorization decision	Named refusal translated at the service boundary	Authenticate, request access, or stop as policy allows	Resource nonexistence or malformed input
Conflict or constraint rejection	Version check or named storage authority	Conflict result with bounded metadata	Refresh state, choose another key, or stop	That a precheck reserved the write
Transient infrastructure unavailability	Dependency adapter	Classified exception internally; bounded unavailable result at its owner	Retry later only when policy permits	That retry is safe or will succeed
Uncertain partial effect	Effect adapter after dispatch	Distinct unknown-outcome failure	Query status or reconcile before retry	Whether the effect committed
Cancellation	Caller, shutdown signal, or lifetime owner	Distinct control/lifetime outcome	Stop or unwind owned work	That already-dispatched effects were reversed
Timeout or deadline	Deadline controller or dependency adapter	Distinct deadline outcome	Verify, compensate, or retry under an idempotency policy	That the timed operation had no effect
Programmer or invariant defect	Executing implementation	Exception or process-level failure path	Repair code; keep the defect observable	Ordinary client invalidity

For each category, record a **failure entry**: producer; representation; propagation path; consumer action; stable public promise, if any; and withheld facts. This is Chapter 21's guarantee discipline applied to unsuccessful execution. A parser has authority to report facts about the value it inspected. A repository can report its contract outcome. Neither has authority to invent an authorization decision or claim that a remote commit did not occur.

Authorization appears here only as a distinct category. Chapter 23 owns policy placement, concealment choices, transactions, and the timing of authorization relative to effects.

2. Choose results or exceptions by contract, not ideology

Use an explicit result when the alternative is part of the operation's ordinary vocabulary and the immediate caller is expected to branch on it. `assignSeat` can return `assigned | sold-out`; the caller can render a waitlist without leaving normal control flow. A domain debit can return `debited | insufficient-funds`. The variants make the expected decision visible, support exhaustive handling in checked source, and can carry different data without sentinel values.

Use an exception when normal execution cannot continue and the current function neither owns nor usefully expresses recovery. An impossible negative seat count is an invariant defect, not "sold out." A low-level adapter may throw a classified unavailable error because several intervening functions cannot act on it; the application boundary that owns the public vocabulary can catch and translate it. Depth alone does not decide: expectedness, ownership, frequency, fan-out, language conventions, and the caller's recovery contract do.

Avoid duplicating the same failure across both channels without a rule. A function that sometimes returns `{ kind: "unavailable" }` and sometimes throws the same unavailable condition forces every caller to inspect two

paths. State which operational failures are result variants, which classified exceptions may cross the boundary, and which unexpected reasons must propagate unchanged.

A generic surface such as

```
type Result<T, E> =
  | { readonly ok: true; readonly value: T }
  | { readonly ok: false; readonly error: E };
```

relates success and failure types. It does not choose the failure vocabulary, validate runtime data, serialize an `Error`, catch promise rejections, or determine retry safety. `Promise<Result<T, E>>` has at least two channels: fulfillment with a result and rejection with an arbitrary reason. Whether rejection is reserved for defects and control outcomes is an API contract that code must implement and tests must exercise.

3. Thrown and rejected reasons begin as `unknown`

JavaScript permits `throw` to carry the value produced by any expression. Promise rejection reasons are likewise not restricted to `Error`. Dependencies, legacy code, and tests can throw strings, numbers, plain objects, `null`, or hostile proxy objects. TypeScript's strict catch-variable behavior therefore gives a caught reason the type `unknown`.

```
try {
  await dependencyCall();
} catch (reason: unknown) {
  if (reason instanceof DependencyUnavailableError) {
    // Classified handling is now justified.
  } else {
    throw reason;
  }
}
```

Do not read `reason.message`, spread `reason`, or convert it with an unchecked assertion before narrowing. `reason instanceof Error` is useful for standard error behavior, but it does not prove a domain category. Conversely, a non-`Error` reason may still need to propagate unchanged. Normalizing every reason with `new Error(String(reason))` can lose identity, structured details, and the original causal chain; evaluating `String` can itself invoke user-defined behavior.

Prediction: a `catch` around `throw "offline"` runs, but `reason instanceof Error` is false. A rejection with `{ vendorCode: "CAPACITY" }` also reaches a rejection handler as an object, not an `Error`. The runnable example pins both cases.

4. Structure internal errors without confusing them with public data

A custom internal error can provide a stable class or code for the component that owns it, selected operational metadata, and an original `cause`:

```
class DependencyUnavailableError extends Error {
  readonly internalCode = "DEPENDENCY_UNAVAILABLE" as const;

  constructor(cause: unknown, readonly retryAfterMs?: number) {
    super("account repository unavailable", { cause });
  }
}
```

The code enables deliberate matching; the bounded delay can support a retry policy; `cause` preserves identity for diagnosis. None should be copied automatically into a public response. An internal message may contain implementation vocabulary, and a cause may contain an entire vendor object. Validate metadata before trusting or publishing it. Custom classes also cross process, realm, and duplicate-package boundaries poorly, so use them deliberately inside the environment whose identity rules you control; use stable serialized codes at external boundaries.

Standard `Error` properties are not an automatic serialization protocol. The language installs an own `message` and `cause` as non-enumerable properties when supplied. In the tested Node baseline, `stack` is also an own non-enumerable diagnostic property, but stack format is non-standard and engine-dependent. `JSON.stringify` visits enumerable own string-keyed properties of ordinary objects. Consequently, `JSON.stringify(new Error("private "))` produces `{}` in the verified baseline, while an enumerable custom `code` appears and the standard message and cause do not.

That behavior is a prediction to verify, not a secrecy system. A custom `toJSON`, an enumerable field, a logger that specially expands errors, or manual copying can expose diagnostics. Messages and stacks are diagnostic, secret-sensitive, unstable text. Do not ask clients to parse them, snapshot exact stack text as a public contract, or infer retryability from a substring.

5. Translate only at an ownership boundary

An adapter owns the translation from vendor vocabulary to application vocabulary. If a storage vendor reports `CAPACITY`, the adapter may throw `DependencyUnavailableError(vendorError, 250)`. If it reports that a commit acknowledgement was lost after dispatch, the adapter should produce an unknown-outcome category, not ordinary unavailability. For an unrecognized vendor value, the safe default is usually to rethrow the original reason. Inventing “invalid input” destroys the first useful distinction.

The application boundary then owns translation from classified internal outcomes to public API codes. Intermediate functions that cannot change caller action should not catch merely to add prose. Catch only the classes or predicates the boundary is designed to handle, preserve the original cause internally, and rethrow everything else unchanged. A catch-all that labels every problem `INVALID_INPUT` hides invariant defects, converts outages into caller blame, may trigger harmful retries or corrections, and makes the original failure harder to observe.

Cause preservation and public redaction are compatible. The lab’s internal diagnostic retains the exact cause under a non-enumerable property while its public mapper creates a fresh closed object. The diagnostic can record a bounded request identifier, operation, and category. Chapter 24 owns where that diagnostic is recorded, how access is controlled, and how logs, metrics, and traces correlate it.

6. Design the public failure as a stable action contract

A public error is a new value, not a sanitized `Error` spread. Its useful fields are deliberately small:

- a stable machine-readable code;
- a bounded human-readable message that is safe but not intended for parsing;
- a correlation or request identifier safe to return;
- field or path issues for correctable input, using stable issue codes;
- a justified caller action or retry instruction;
- selected bounded metadata, such as a validated retry delay, only when its semantics are defined.

Withhold exception causes and stacks, SQL and query text, filesystem paths, dependency and vendor identifiers, hostnames, credentials, tokens, raw payloads, and arbitrary rejected values. Also withhold internal state that merely happens to be convenient. A client should receive the facts needed to choose its next action, not a map of the implementation.

Keep public and diagnostic structures separate in the type system. In the lab, `InternalDiagnostic` owns `cause`; `PublicError` cannot. Static verification rejects reads of `publicResponse.error.cause` and `.stack`. Runtime tests serialize both paths and search for the vendor statement, credential, path, code, and stack marker. Neither check alone proves universal secrecy: the types cover checked source relationships, while the tests cover selected runtime mappings and serializers.

7. Retryability depends on effects and policy

"Transient" does not mean "retry now." A failure is safely retryable only relative to an operation, effect history, idempotency mechanism, caller budget, deadline, backoff policy, and dependency semantics. An unavailable read before dispatch may be retryable. A lost commit acknowledgement after dispatch is uncertain: the debit may have happened. Repeating it without an idempotency key or status check may duplicate the effect.

Treat at least three states separately:

1. **Known not performed:** retry may be permitted by policy.
2. **Known performed:** return or recover the outcome; do not repeat the effect.
3. **Outcome unknown:** query by request or idempotency key, reconcile, or escalate before retry.

A timeout establishes that the observer's deadline elapsed, not that the callee stopped. Cancellation requests that owned work should stop; they do not rewind already-completed or independently-owned effects. Both are control and lifetime outcomes, not evidence that input was invalid. Chapter 9's idempotency and uncertain-effect model remains the authority: preserve the original request identity, bound attempts, and never convert uncertainty into retry folklore.

The lab therefore maps ordinary dependency unavailability to `retry-later`, unknown commit outcome to `check-status`, caller cancellation to `stop`, and deadline expiry to `verify-before-retry`. Those actions are application choices supported by the modeled effect boundary; they are not universal meanings of the class names.

8. Version both checked and deployed consumers

Inside one strict TypeScript build, a discriminated union makes change visible. Adding a new `ApplicationResult` member breaks an exhaustive switch whose default requires `never`; every compiled consumer must decide what the member means. Removing a member or changing its fields can likewise break checked code. This is useful local evidence that a vocabulary change reached included consumers.

Deployed and untyped clients are different. An older client can receive a new server error code without being recompiled. Its wire type should admit `code: string` and its dispatcher should retain an unknown-code fallback such as a safe generic message. A generated closed union is valuable for current clients, but runtime parsing must still account for server/client version skew.

Treat removing a public code, repurposing its meaning, or changing its caller action as breaking. Adding a code can still break consumers that assumed closure. Adding an optional field is often compatible for tolerant readers, but strict schema parsers or exact-object decoders may reject it. Version decisions therefore require both static-consumer analysis and deployed-wire behavior; neither substitutes for the other.

9. Lab: debit through parse, domain, repository, and public boundaries

The lab accepts an unknown debit command, rejects additional fields, constructs a closed value, loads an account, performs a domain transition, and asks an injected repository to save it. The application result distinguishes success, validation failure, insufficient funds, revision conflict, dependency unavailability, uncertain commit, cancellation, and deadline expiry. Unexpected `Error` and non-`Error` reasons are rethrown with identity preserved.

The public mapper supplies a stable code, safe message, request identifier, action, and selected issues or retry delay. It supplies no internal diagnostic or cause. A separate client dispatcher handles known codes and a future code. The deliberate fault catches everything, labels it invalid input, and copies the message, stack, vendor statement, credential, code, and raw reason into the public object. Its test passes by proving that this apparently convenient implementation leaks secrets and directs the caller to correct a request during an infrastructure failure.

Read each evaluator test as a bounded failure entry. It identifies the producer, expected representation, propagation path, caller action, stability promise, and excluded fields. Add a new category only after supplying all six.

False models, repaired

- **"Exceptions are bad; return `Result` everywhere."** Representation follows recovery ownership and control flow. A result can coexist with exceptional defects and rejected promises.
- **"Only `Error` objects can be thrown or rejected."** JavaScript reasons can be any value; catch and rejection handlers begin with `unknown`.
- **"`JSON.stringify(error)` is safe because it returns `{}`."** Enumerability and serializer behavior are implementation facts, not a redaction guarantee.
- **"The message is the error code."** Messages are diagnostic, unstable, localized, and secret-sensitive. Publish a stable code separately.
- **"Wrapping every error improves context."** Unclassified wrapping can destroy identity and causal evidence. Translate only where the vocabulary owner can justify the mapping.
- **"A timeout means nothing happened."** The observer stopped waiting; an effect may be complete, incomplete, or still running.
- **"A transient error is retryable."** Retry safety depends on effect state, idempotency, budgets, and policy.
- **"An exhaustive TypeScript switch protects every client."** It protects included checked consumers. Deployed older and untyped clients need runtime fallback.
- **"Adding an optional response field cannot break anyone."** Strict parsers may reject unknown fields; compatibility belongs to the actual wire contract.
- **"Catching defects as invalid input is user-friendly."** It blames the caller, hides defects, leaks internals easily, and prescribes the wrong action.

Mastery questions

Answer from memory, then verify with the chapter and runnable artifacts:

1. Write a failure entry for malformed JSON and name one fact its parser cannot establish.
2. Why is insufficient balance an explicit domain refusal while a mismatched repository identity is an invariant defect?
3. When can a classified dependency exception be preferable to adding a result variant to every intermediate function?
4. Name the two control-flow channels of `Promise<Result<T, E>>`.
5. What value type can JavaScript throw, and why should a catch variable begin as `unknown`?
6. Predict `Object.keys` and `JSON.stringify` for an `Error` with an enumerable custom code and a standard cause.
7. Which component may translate a vendor capacity code, and what should happen to an unrecognized vendor reason?
8. Why should an internal diagnostic preserve cause identity while the public response omits the cause?
9. Which public fields help a caller act without exposing the implementation?
10. Distinguish known-not-performed, known-performed, and outcome-unknown effects.
11. Why are cancellation and timeout not synonyms for invalid input or no effect?
12. How does adding a union member affect an exhaustive compiled consumer versus an already-deployed client?
13. Why can an additive optional field still be incompatible with some parsers?

14. Diagnose the deliberate catch-all fault by producer, wrong representation, wrong action, and leaked facts.
15. Design one unfamiliar public failure by naming its producer, representation, path, action, stable promise, and withheld facts.

Presentation prompt

Present a 12–15 minute boundary review for the debit lab. Begin with a failure entry for each application result, then justify its result or exception channel. Predict the arbitrary-reason and `Error`-serialization examples before revealing their output. Trace vendor capacity and uncertain commit failures through adapter, internal diagnostic, public response, and client action. Demonstrate that an unexpected reason preserves identity and that a future public code reaches a safe fallback. End by diagnosing the catch-all fault and transferring the design to one unfamiliar effectful operation. Use `presentations/prompts/ch22.md` for the evaluator prompts.

Ready-when test

You are ready to continue when, without notes, you can classify an unfamiliar failure by producer and recovery action; choose its result or exception path; narrow arbitrary reasons; predict standard `Error` serialization without treating it as redaction; preserve cause identity through an owned translation; build a fresh secret-safe public response; distinguish unavailability, uncertainty, cancellation, and deadline; and evolve both an exhaustive internal union and a deployed public code without abandoning runtime fallback.

When retrieval fails, log only the smallest missing term, relationship, prediction, procedure, or boundary in the personal appendix. Keep this canonical chapter compressed; repair the demonstrated gap, then return to the original question.

Chapter 23

HTTP, Persistence, Transactions, and Authorization

Chapter type: Integration **Prerequisites:** Chapters 8, 9, 15, 17, 21, and 22; asynchronous cleanup, idempotency, domain transitions, runtime parsing, correctness authorities, and failure translation **Capabilities introduced:** Carry an untrusted HTTP request through authentication, current-state authorization, a domain transition, transactional persistence, and a deliberate public response while naming which component establishes each fact **Earlier chapters integrated:** Promise and resource lifetime from Chapter 8; retry, idempotency, and uncertain effects from Chapter 9; closed domain states from Chapter 15; `unknown` ingress parsing from Chapter 17; guarantee allocation from Chapter 21; failure ownership and public translation from Chapter 22 **Later chapters enabled:** Production diagnostics and the capstone HTTP service **Estimated study time:** 6–8 hours across six sessions **Runnable sources:** `examples/ch23/src/`, `examples/ch23/test/`, and `examples/ch23/lab/`

Compressed thesis

An HTTP write is not one operation governed by one authority. The runtime host receives bytes. Transport admission checks a route, method, media type, body bound, decoding, and JSON syntax. An application parser inspects the decoded `unknown` value and constructs a closed command. Authentication establishes a principal. Authorization decides whether that current principal may perform this action on this resource in its current state. Domain logic proposes a permitted state transition. A transaction-scoped repository performs tenant-scoped reads and writes. PostgreSQL applies configured constraints, locking, isolation, commit, and durability behavior. The application translates the resulting internal outcome into a stable, secret-small HTTP response.

Each stage establishes only its named facts. A successful JSON decode proves neither command shape nor permission. Authentication proves identity, not authority. A domain transition computed in memory proves no row changed. A transaction cannot roll back an email sent outside it. A `200`, `409`, or `500` records the service's public decision; the number alone proves neither database rollback nor absence of an external effect.

This chapter is compressed by design. It preserves the shared technical spine as a connected lifecycle model. Make the lifecycle usable by predicting schedules, reconstructing it from memory, implementing the lab, and presenting it aloud. When your work exposes confusion, paste the relevant passage, prediction, code, and observed result into Codex. Codex will determine the smallest useful personal appendix repair without expanding the canonical chapter for every reader.

1. Carry guarantees through the request lifecycle

The lifecycle is ordered because later claims depend on earlier evidence. For the cancellation lab, use this chain:

Stage	Fact established here	Failure still possible afterward
Node/HTTP reception	Request bytes, headers, method, and target were observed by the host	Truncation, excess size, unsupported media, malformed payload, disconnect
Transport admission	Intended route/method/media were accepted; a bounded byte sequence decoded as JSON	Wrong object shape, extra fields, invalid identifiers, hostile values
Application parsing	A closed `CancelCommand` was constructed from inspected values	Bad credentials, forbidden action, stale state, storage failure
Authentication	Credentials established a current principal under the authenticator's contract	Principal lacks permission; resource or policy state changes
Authorization	This principal may attempt this action on the loaded current state	Domain refusal, concurrent change, storage rejection
Domain transition	The current domain state permits a proposed next state	Conditional write loses a race; constraint or dependency failure
Transactional persistence	Participating work is classified as committed, rolled back/not committed, rollback-unknown, or commit-unknown from completed operations and adapter evidence	Durability depends on configuration; remote effects remain separate
Public translation	The caller received a deliberate status, headers, and bounded body	Delivery can fail; a retry can be unsafe; telemetry can still be incomplete

Do not summarize the table as "one component validates for the next." Name the proposition. Transport proves that these bytes fit this admission policy. The parser proves that this inspected value satisfied these predicates at this time. Authorization proves a policy decision over named inputs. PostgreSQL proves only the constraints and transaction behavior actually configured for participating statements.

2. Admit HTTP before interpreting the domain

In Node.js, an `http.IncomingMessage` is a readable request stream with method, headers, target, and payload bytes. Before JSON parsing, select the route and method, reject an unsupported `Content-Type`, and enforce a byte limit while reading. Decode UTF-8 under a declared policy, then run `JSON.parse`. The result is `unknown`. JSON syntax success says only that the bytes represent a JSON value; `null`, an array, a string, or an object with dangerous extra fields can all be syntactically valid.

HTTP method semantics matter independently of implementation convenience. `GET` is not a suitable disguise for a state-changing cancellation. A `POST` is not automatically retry-safe. Chapter 9's request key, scope, retention, and effect rules remain necessary if a client may repeat it. A request's method and an idempotency mechanism answer different questions.

Keep transport failures distinct: an unknown route can be `404`; a known route with the wrong method can be `405` with `Allow: POST`; an oversized body can be `413`; unsupported request content can be `415`; malformed JSON can be `400`. A syntactically correct JSON value that fails the application command contract can be `422`. A generic `Accept` response header does not explain a `415`; omit it unless the application has defined response-representation negotiation. These mappings are a service contract built on HTTP semantics, not meanings supplied by TypeScript.

Node's `message.complete` can help detect whether a complete HTTP message was received. It does not prove that the application parsed, authorized, stored, or committed anything. Likewise, returning a status does not prove that the client received it.

3. Convert `unknown` into a closed application command

The transport representation, command, domain state, and row representation have different owners:

```
bytes -> decoded unknown -> CancelCommand -> Order transition -> persisted row
```

The parser owns the second arrow. It checks that the value is a non-null, non-array object; rejects unknown keys; requires own fields; validates route and body identifiers; and constructs branded tenant, order, and request identifiers. Its success does not prove that an order exists, that the authenticated user belongs to it, or that the expected version remains current. Those propositions require different authorities.

Keep repository operations parameterized and tenant-scoped. A useful contract requires both `tenantId` and `orderId` for every protected read and write. A production adapter might execute a statement shaped like:

```
UPDATE orders
SET status = 'cancelled', version = version + 1
WHERE tenant_id = $1
  AND order_id = $2
  AND version = $3
  AND status = 'pending'
RETURNING order_id, version;
```

Place values in parameters, not interpolated SQL. The statement establishes a conditional storage outcome; it does not by itself implement an ownership policy unless the policy predicates and their authoritative state are included in the same protected operation.

4. Authenticate identity; authorize the current effect

Authentication and authorization are not synonyms. Authentication answers "which principal does this credential establish?" Authorization answers "may this principal perform this action on this resource in this context now?" A valid bearer token can authenticate a user who is forbidden to cancel a particular order.

Use all relevant dimensions: current principal, action, tenant, resource, resource state, and policy context. Tenant scope belongs in every protected read and write, not only in an early route check. Otherwise an order identifier that is unique only within a tenant can cross a boundary accidentally, and a later query can discard the earlier evidence.

Authorization must be close enough to the protected effect that the state it used cannot become stale under the selected concurrency protocol. Consider this schedule:

```
A loads order 41 owned by user 7 outside a transaction
A authorizes user 7
B transfers order 41 to user 9
A begins a transaction, loads the row, and cancels it without reauthorizing
```

A's first decision was true about earlier state. It supplies no guarantee for the later write. The lab's deliberately faulty evaluator executes exactly this schedule. The repair loads and authorizes current tenant-scoped state inside the transaction before the conditional write.

That repair is bounded. If authorization depends on another table, membership service, policy revision, or clock, locking only the order row may be insufficient. Put database-owned policy state under an appropriate transaction/lock, include relevant policy facts in a conditional write, or use another named protocol. "Checked inside a transaction" is not a universal proof.

An authorization refusal can map to `403`. A service may instead conceal a protected resource as `404`, but this is a deliberate disclosure policy applied consistently to forbidden and absent outcomes. It is not evidence that the row was absent. An unauthenticated request maps separately to `401` and includes an applicable `WWW-Authenticate` challenge.

5. Make asynchronous control flow own the transaction lifetime

The resource lifetime is procedural, not decorative:

```
acquire -> begin -> await read -> authorize -> transition -> await write
        -> await commit -> form success
failure after begin -> attempt rollback
all acquired paths -> attempt release
```

The transaction must remain alive across every `await` that participates in it. Do not return success before the commit promise fulfills. Do not release in a callback that can run before later awaits. Acquire the lease, begin, run all participating work, await commit, attempt rollback after a pre-commit failure, and release in `finally`.

Preserve failure evidence. If the read throws and rollback and release also throw, the read remains the primary failure; rollback and release are separate cleanup failures. A rejected rollback cannot be reported as `rolled-back`; its outcome is unknown. A rejected commit remains `commit-unknown` even if a later rollback call returns, because rollback acknowledgement does not prove that the earlier commit did not occur. Conversely, if commit fulfills and release fails, the database outcome remains `committed`; a pool cleanup problem must not be relabeled as commit uncertainty.

A participating save rejection is provisional until cleanup completes. If rollback then fulfills, the transaction is known uncommitted and the public category follows the save failure's stable transient or internal classification. If rollback rejects, the database outcome is `rollback-unknown` and the request key is required for reconciliation before retry. This differs from a rejected commit: after commit starts, a later rollback acknowledgement cannot establish whether the earlier commit took effect.

PostgreSQL transactions group participating database statements so they complete as one unit and hide intermediate changes from other transactions according to isolation behavior. Those claims stop at the database boundary. They do not include an email, HTTP call, queue publish, filesystem write, or another database connection unless a named protocol includes it.

Commit durability is configuration-sensitive. With PostgreSQL's ordinary synchronous commit behavior, success waits for the configured durable acknowledgement. With asynchronous commit, the server can report success before WAL is flushed, so a crash can lose a recently reported transaction. "Commit succeeded" must therefore be read with the adapter and database configuration, not as an unqualified physical guarantee.

6. Name the concurrency mechanism

"It is in a transaction" does not say which race is prevented. PostgreSQL's default Read Committed isolation gives each statement a new snapshot; two `SELECT` statements in one transaction can observe different committed state. A writer can wait for another writer and then re-evaluate its `WHERE` condition. Locks, conditional updates, unique constraints, and stronger isolation solve different problems.

For optimistic versioning, predict this exact schedule:

```
A reads order version 7
B reads order version 7
A updates WHERE tenant_id = T AND order_id = O AND version = 7; one row changes
A commits version 8
B updates with the same predicate; zero rows change
B maps zero rows to stale-order
```

The version predicate prevents both writers from silently claiming the same prior version. It does not protect authorization-relevant state that can change without participating in that version. A row lock such as `SELECT ... FOR UPDATE` can serialize modifiers of the locked row until transaction end, but it can block and contribute to deadlocks. A database can abort one deadlocked transaction. Serializable isolation can reject a schedule that cannot be represented serially. Neither mechanism means "races are impossible"; each supplies named behavior under its documented scope.

A composite unique constraint such as `(tenant_id, request_key)` can enforce one stored cancellation request key per tenant. Map PostgreSQL failures by stable SQLSTATE and constraint identity—such as `23505` plus `order_cancellations_tenant_request_key_uq`—rather than parsing English error text. The application adapter

translates that authority-owned result to `duplicate-request`. The in-memory lab simulator only proves that this mapping and call order work; it is not evidence that a production schema contains the constraint.

Retry only after classifying the failure and effect state. A serialization failure (`40001`) or deadlock (`40P01`) can justify retrying the whole transaction under a bounded policy: same logical request identity, limited attempts, backoff, deadline/cancellation checks, and a fresh transaction. A named uniqueness conflict or domain refusal normally requires a different caller action. A lost commit acknowledgement is not ordinary transient unavailability; query or reconcile by the request key before repeating the effect.

7. Keep remote effects outside the transaction claim

Suppose cancellation stores a row, sends an email, then rolls back. The database can remove its staged row, but the delivered email remains. Moving the email before or after commit changes failure order; it does not make database and mail delivery atomic.

A transactional outbox is one bounded protocol: store the domain change and an outbox record in the same database transaction, then let a worker publish later. It closes the gap between the database change and durable publication intent. It does not promise exactly-once delivery, instant publication, or automatic remote rollback. The publisher still needs retry, observation, and usually an idempotent consumer. Compensation is another later effect—such as a corrective message—not time reversal.

Avoid remote calls while holding database locks unless a named protocol requires them and accounts for latency, cancellation, ambiguity, and lock duration. Keep the transaction's participating set explicit.

8. Translate internal outcomes into a public action contract

Retain Chapter 22's producer-based distinctions inside the application: invalid command; unauthenticated; forbidden; not found; domain refusal; storage conflict; known pre-effect unavailability; outcome unknown; unexpected defect. Translate them where the HTTP boundary owns the vocabulary.

The lab deliberately maps syntactically valid but invalid commands to `422`; unauthenticated to `401`; forbidden and absent to one concealed `404`; domain and storage conflicts to stable `409` codes; known pre-effect unavailability to `503` with a bounded `Retry-After`; unexpected failure to a generic `500`; and commit uncertainty to a separate `500` body instructing `check-status` with the safe request key. Other services can choose different stable contracts, but they must state the disclosure and caller-action policy.

Never copy SQL, constraint messages, stack traces, dependency names, credentials, policy reasoning, or raw rejected values into the response. A status is not a causal proof. `409` does not say whether an earlier external call happened. `404` does not distinguish absence from concealment. `500` does not authorize a blind retry. The internal outcome and execution report retain facts that the public value intentionally withholds.

9. Lab: cancel an order across every boundary

Implement `handleCancelRequest` in `examples/ch23/lab/starter.ts`. The transport stage has already produced `decodedBody: unknown` and an authentication result. Your handler must parse a closed command; reject unauthenticated input before transaction acquisition; acquire and begin; load by tenant and order under the transaction; authorize current state; derive the domain transition; perform a named conditional save; await commit before success; and retain primary and cleanup failures separately.

`TransactionContractSimulator` is intentionally not PostgreSQL. It establishes deterministic application evidence: tenant-scoped calls, lifecycle ordering, staged writes, named repository outcomes, and response timing. It supplies no claim about SQL execution, locks, isolation, WAL, durability, or cross-process races. Those claims require the actual PostgreSQL 18 adapter, schema/configuration inspection, and integration/concurrency tests.

The evaluator covers malformed values, unauthenticated and forbidden requests, domain refusal, both storage conflicts, commit failure, rollback and release failure, post-commit release failure, safe pre-effect unavailability, no write after denial, and response order. The faulty evaluator checks authorization outside the transaction; its passing test demonstrates a real unauthorized cancellation after concurrent ownership transfer.

Presentation prompt

Give a 10–15 minute explanation in your own words using a sparse outline or one lifecycle diagram. Predict one transport case and the version-7 writer schedule; derive the stale-authorization failure and the success/cleanup transaction traces; compare one named concurrency mechanism; explain the remote-effect boundary; and defend a stable, secret-small public response mapping. Include one earlier-chapter connection, one repaired false model, and one unresolved question that would require adapter, schema, configuration, or production evidence. The complete submission and evaluation instructions are in `presentations/prompts/ch23.md`.

False models, repaired

- **"Valid JSON is a valid command."** JSON decoding returns a runtime value; the application parser must inspect it and construct the command.
- **"Authenticated means authorized."** Identity is one input to a current principal/action/resource/context decision.
- **"A tenant check in the route is enough."** Every protected read and write must retain tenant scope.
- **"Authorization passed, so the later write is allowed."** The decision can go stale unless authorization-relevant state participates in the write's concurrency protocol.
- **"Transactions prevent races."** Isolation, locks, conditional predicates, and constraints provide different named behavior.
- **"Rollback succeeded after commit failed, so nothing committed."** A rejected commit acknowledgement can leave the effect outcome unknown.
- **"Rollback failed, but we can report rolled back."** A rejected cleanup operation supplies no completion evidence.
- **"A transaction rolls back email."** Only participating database operations are inside the database transaction.
- **"A transient error is safe to retry."** Retry safety depends on effect history, idempotency, attempt budget, deadline, and dependency semantics.
- **"The HTTP status explains what happened internally."** It is a deliberate public mapping, often concealing distinctions and never proving unseen effects.

Mastery questions

1. Reconstruct the eight lifecycle stages and state one fact established and one remaining failure after each.
2. Predict the transport result for wrong method, unsupported media, excess bytes, malformed JSON, and a valid JSON array. Which cases reach the application parser?
3. Explain why a branded `CancelCommand` proves neither resource existence nor permission, even under strict TypeScript.
4. Given the stale-authorization schedule in Section 4, identify the first proposition that becomes false and design two repairs with different locking or conditional-write costs.
5. Write the exact transaction event order for success, read failure plus successful rollback, commit rejection plus successful rollback, and successful commit plus release failure.
6. Why must `rollback-unknown`, `commit-unknown`, and `committed with cleanup failure` remain distinct?
7. Predict both writes in the version-7 concurrency schedule. What changes if authorization-relevant ownership changes without incrementing the version?
8. Compare a version predicate, `SELECT ... FOR UPDATE`, a unique constraint, and Serializable isolation by the schedule each can reject or serialize.
9. Classify `23505`, `40001`, `40P01`, a zero-row conditional update, and a lost commit acknowledgement. Which, if any, may enter a bounded automatic retry?
10. Derive why an email sent inside an application `try` block is still outside a PostgreSQL transaction. What guarantee does an outbox add, and what remains possible?
11. Design a stable public mapping for invalid input, authentication failure, authorization concealment, domain refusal, storage conflict, pre-effect unavailability, outcome unknown, and unexpected failure. Name withheld facts.

12. Transfer the model to "reserve inventory and charge a card." Identify the authoritative state, current authorization decision, participating transaction, remote effect, idempotency identity, and reconciliation path.

Ready-when test

You are ready to proceed when, without the chapter, you can draw the full request lifecycle; distinguish transport, command, domain, and row representations; place authentication and current-state authorization; predict the two-writer schedule; write the success and four failure cleanup traces; explain why commit and rollback rejection create different uncertainty; classify a retry with effect history and request identity; explain the outbox's bounded guarantee; map each internal outcome to a stable public response; and pass every Chapter 23 lab test, including the deliberate stale-authorization fault.

Chapter 24

Process Lifecycle, Observability, and Partial Failure

Chapter type: Integration and emergent **Prerequisites:** Chapters 8, 9, 20, 21, 22, and 23; cancellation, resource ownership, reliable asynchronous workflows, source-to-runtime diagnosis, evidence allocation, failure translation, and transactional service boundaries **Capabilities introduced:** Own a service process as a state machine; publish readiness only after startup invariants; admit and track work by capability; drain within a bound; supervise background tasks; design secret-safe operational evidence; and decide explicitly under partial failure **Earlier chapters integrated:** Cancellation and cleanup from Chapter 8; admission, retry, and idempotency from Chapter 9; artifact/configuration/runtime identity from Chapter 20; observation boundaries from Chapter 21; failure categories from Chapter 22; HTTP, transaction, and authorization ownership from Chapter 23 **Later chapters enabled:** The capstone production service and incident-grade operational reasoning **Estimated study time:** 6–8 hours across six sessions **Runnable sources:** `examples/ch24/src/`, `examples/ch24/test/`, and `examples/ch24/lab/`

Compressed thesis

A service process is not ready because JavaScript began executing, a port bound, or a log line says "ready." One lifecycle owner must govern legal transitions, external readiness, admitted work, in-flight work, acquired resources, background-task settlement, and terminal outcome. For the lab, the legal path is `starting -> ready -> draining -> stopped`, with `starting -> failed -> stopped` and `starting -> draining -> stopped` as explicit alternatives. Degraded operation is a ready mode with named unavailable capabilities, not an adjective that permits arbitrary success.

Startup must establish exact artifact, runtime, and parsed configuration identity; install shutdown ownership before publication; acquire required dependencies in declared order; decide optional capabilities; start and track background work; and only then make readiness true. Shutdown reverses the publication: make readiness false, stop admission, signal cooperative cancellation, await owned work within policy, close resources once in reverse order, and record primary, background, and cleanup failures without collapsing them. A deadline can require forced termination, but it cannot prove that pending effects stopped or cleanup completed.

Operational evidence must answer a query. Logs are bounded event records, metrics are numeric aggregates with bounded dimensions, and traces connect sampled operation segments. Any of them can be absent, delayed, duplicated, misclassified, sampled, or produced by the wrong build. Observation detects selected events after they occur; it does not replace admission, parsing, authorization, transactions, idempotency, or cleanup ownership.

1. Give the lifecycle one owner

Represent lifecycle state as a closed union whose member owns the facts valid in that state. Do not maintain unrelated `isReady`, `isShuttingDown`, and `failed` booleans that can contradict each other.

State or mode	Readiness	Admission	Owned facts	Legal next state
`starting`	false	reject	successful acquisitions so far; startup cancellation signal	`ready`, `draining`, or `failed`
`ready/full`	true	admit supported work	all required resources; all declared capabilities; tracked tasks	`draining`
`ready/degraded`	true for core	reject named unavailable capability	required resources; unavailable-capability set; tracked tasks	`draining`
`draining`	false	reject	in-flight set; background set; resources; force decision	`stopped`
`failed`	false	reject	startup stage; primary failure; rollback failures	`stopped`
`stopped`	false	reject	terminal cleanup outcome and retained failure facts	none

The state owner performs every transition. A listener or request handler may ask it to shut down; it may not set readiness independently. The owner publishes a snapshot, not mutable internal collections.

Liveness asks whether this process is alive enough for the platform's restart policy. **Readiness** asks whether the process should receive a named class of new work. A starting or draining process can be live and not ready. A core endpoint can be ready while notifications are unavailable. Binding a socket establishes only that the host accepted the bind. Writing a ready event establishes only that the sink received or attempted to receive a record. Neither mutates the external readiness probe.

2. Treat startup as ordered resource acquisition

Record the exact build identifier, `process.version`, entry artifact, effective configuration identity, and startup policy. Chapter 20's first-divergence rule applies immediately: if two instances differ, compare the actual artifact, runtime, invocation, and parsed configuration before changing application logic.

Configuration begins as unknown strings. Parse it into a closed value before dependency effects. Reject missing, malformed, out-of-range, and unknown fields under the declared policy. Diagnostic output may name stable field and issue codes; it must not copy database URLs, tokens, raw environment values, or whole configuration objects.

Use a declared acquisition sequence such as:

```
parse config
-> install shutdown adapter
-> acquire required database
-> acquire optional notifier
-> start and register background reconciler
-> publish full or capability-scoped readiness
```

Installing the shutdown adapter first gives termination requests an owner even during startup. A successful acquisition transfers a close obligation to the supervisor. If acquisition `N` fails, close only successful acquisitions `1..N-1`, once each, in reverse order. Preserve the acquisition failure as primary and every close failure as separate cleanup evidence. The service never becomes ready. A rejected acquisition remains responsible for any invisible partial resources it created before rejecting; the supervisor cannot close a handle it never received.

Required and optional are policy terms. Required database failure refuses startup. Optional notifier failure can publish core readiness in degraded mode while notification admission returns a named capability-unavailable result. Each retry needs an owner, classification, attempt bound, injected delay policy, cancellation check, and effect/idempotency analysis. This startup sequence resembles a resource transaction, but it is not Chapter 23's database transaction and provides no atomic rollback across external systems.

3. Couple readiness to admission

Readiness is useful only when it governs new work. The admission path reads the current state and capability, rejects unless that capability is ready, then registers accepted work before returning control to asynchronous execution. The supervisor increments a bounded in-flight count and attaches fulfillment and rejection handling immediately. Settlement removes the work from the registry in `finally`-equivalent logic.

A partial failure needs a named decision. "The service is degraded" does not say whether to admit a core read, reject a notification request, queue it, or return an unavailable result. Specify capability, request class, refusal, bound, and evidence. Readiness should change before a drain begins so an external load balancer stops sending new work; local admission must also reject because probe propagation is delayed and clients can bypass it.

Correlation identifiers are untrusted input. Accept only a bounded grammar and length, or replace the value with a generated identifier. A valid correlation ID connects evidence; it does not authenticate the caller, authorize access, guarantee uniqueness, or belong in metric labels.

4. Supervise background work as owned work

Every background task needs an ownership entry:

Question	Required answer
Owner	Which supervisor observes settlement and decides failure policy?
Start condition	Which resources and state must exist first?
Cancellation	Which signal is passed, and where are pre-abort checks made?
Settlement	Which tracked promise records fulfillment or rejection?
Failure policy	Restart within a bound, degrade a capability, or drain the process?
Cleanup	Which resource or listener is released, by whom, and once?
Effect boundary	Which attempts can duplicate effects, and what idempotency key/scope/retention applies?

Attach rejection handling when the promise is created, not later after an `unhandledRejection` warning. Remove settled promises from the registry. An unexpectedly fulfilled required loop can be as important as a rejection:

the loop stopped while the service claimed readiness. A floating promise has no reliable owner, drain barrier, or failure classification.

Calling `unref()` on a Node timer only means that timer does not keep the event loop alive. The process may exit before its callback. It does not cancel the task, observe its settlement, close its resources, or make its effects safe. Event-loop retention is not lifecycle correctness.

5. Drain cooperatively, then enforce a bound

The first shutdown request atomically changes `ready -> draining` or diverts `starting -> draining`. Readiness becomes false and admission stops before any await. The supervisor aborts its owned controller, awaits registered in-flight and background settlements, then closes acquired resources in reverse order. Repeated shutdown requests return the same drain promise and do not start another close pass.

Abort is notification. Code must check pre-abort, subscribe without a lost-wakeup gap, and stop at cooperative boundaries. Abort does not preempt JavaScript, retract a dispatched database statement, roll back a committed transaction, or cancel remote work whose adapter ignores the signal. A request deadline can expire while the remote effect continues. Preserve that as an uncertain outcome and apply Chapter 9's same logical key, operation scope, retention window, attempt bound, and reconciliation rules.

A production adapter races graceful completion against an explicit deadline. In this chapter's policy, a second signal is idempotent; only deadline expiry requests forced termination. Other policies are possible, but they must say who owns the decision and prevent duplicate cleanup. The deterministic lab injects the deadline promise and records `force-required`; it never kills the test process. A real adapter may then call `process.exit()`, after which pending asynchronous output and cleanup can be truncated. Nothing after that call is guaranteed to settle.

Terminal `outcome: "clean"` has deliberately narrow meaning: the owned cleanup pass completed without a cleanup rejection. It does **not** mean startup succeeded, requests succeeded, remote effects are known, or background work was healthy. Primary, background, and cleanup facts remain separate.

6. Use Node process events for their documented scope

In the verified Node.js 24.18.0 baseline:

- `unhandledRejection` is emitted when a promise is rejected without a handler within a turn of the event loop. The default `--unhandled-rejections=throw` mode raises it as an uncaught exception when no hook handles it. A hook is diagnostic policy, not a substitute for attaching ownership at creation.
- `uncaughtException` is the last boundary for an exception that reached the event loop. Node's default prints a stack and exits with code 1. Continuing normal service after installing a handler is unsafe because application state may be inconsistent. Use a monitor listener for last-chance evidence without changing default crash behavior, or perform only carefully bounded termination coordination.
- `SIGINT` and `SIGTERM` can be observed on the main process. Installing listeners changes default behavior on supported platforms, so the listener must initiate the owned shutdown policy. Signal availability and details are platform-sensitive; keep the Node adapter thin and test the supervisor with injected requests.
- `beforeExit` occurs when the event loop has no additional scheduled work and may schedule more work. It is not emitted for `process.exit()` or uncaught exceptions, so it is not a universal shutdown hook.
- `exit` listeners can perform synchronous operations only. Queued asynchronous work is abandoned after the listeners return.
- `process.exit()` terminates as quickly as possible even with pending asynchronous work, including stdout or stderr writes. Prefer natural completion plus `process.exitCode` when graceful completion is still possible.

Fatal handlers cannot repair an unknown state. Capture bounded evidence if possible, coordinate termination, and rely on an external process supervisor for restart.

7. Design evidence from operational queries

Start with a question: "Which build admitted notification work while the notifier was unavailable?" "How many requests remained in flight when drain exceeded its bound?" "Which startup acquisition failed first, and which

closes then failed?" Choose evidence that can answer it.

Form	Useful content	Boundary
Structured log event	event name, outcome, reason code, build/runtime identity, validated correlation ID, safe duration, bounded count	Can be dropped, duplicated, delayed, reordered, malformed, or secret-bearing
Metric	numeric aggregate such as readiness, admitted, rejected, in-flight, latency bucket, cleanup failure	Labels must be bounded; request, user, tenant, URL, and arbitrary error text cause cardinality and privacy failures
Trace	connected spans across admitted work and dependencies	Context can be lost or forged; sampling omits work; collector failure removes evidence

The lab uses one canonical event shape: `eventName`, `outcome`, `reason`, `buildId`, `runtimeVersion`, `correlationId`, `durationMs`, and `count`. Tests compare those fields so log, metric derivation, and trace attributes do not silently assign different meanings. The same operational event may produce all three forms, but duplication is not agreement: different sampling, clocks, buffering, retries, and label mapping can make them disagree.

Keep raw payloads, environment values, credentials, database URLs, stack traces, arbitrary rejection reasons, and vendor objects out of routine events. Store diagnostic causes only in a protected internal path with explicit access and retention policy.

8. Bound every service-level claim

A service-level indicator is a calculation over selected evidence, not the behavior itself. Useful lab indicators include core readiness, notification readiness, admission/refusal counts, in-flight work, bounded outcome counts, duration distributions, background failures, and cleanup failures. State numerator, denominator, population, time window, missing-data rule, build/runtime scope, and sampling rule.

Observation can itself throw, block, allocate without bound, drop, sample, mislabel, or become unavailable. The lab catches a synchronous sink failure and increments a local observation-failure count; even that count can disappear if the process dies or the next sink fails. Correctness must not depend on a best-effort sink. If evidence must gate behavior—an audit record required before a protected action, for example—model it as a required effect with explicit failure semantics, not ordinary telemetry.

9. Make partial-failure decisions explicit

For each case, record lifecycle, admission, evidence, retry owner, classification, bounds, idempotency, and residual uncertainty:

Failure	Lifecycle and admission	Required decision
Required database unavailable at startup	fail; admit none	bounded external restart; reverse rollback; retain primary and cleanup failures
Optional notifier unavailable	ready/degraded; core only	reject notifications explicitly; bounded reconnect owned by notifier supervisor
Telemetry sink unavailable	lifecycle unchanged	drop or buffer within a bound; never block correctness accidentally
Required background loop fails	drain; stop admission	preserve failure; bounded restart or process restart; analyze duplicated effects

Failure	Lifecycle and admission	Required decision
Request deadline while remote work continues	usually lifecycle unchanged	return uncertainty; reconcile before retry with the same idempotency identity
Shutdown close rejects	stop with cleanup-failed evidence	do not relabel as clean; external owner handles residual resource uncertainty

Chapter 23 remains authoritative for database transaction participation, isolation, authorization timing, and commit uncertainty. Process shutdown can await a transaction promise and signal cancellation; it cannot broaden the transaction or prove rollback. Chapter 21 remains authoritative for observation as scoped evidence rather than enforcement. Chapter 20 supplies deployed identity and first-divergence diagnosis. Chapters 8 and 9 supply cancellation, cleanup, retries, attempts, and idempotency.

10. Lab: deterministic service supervisor

The evaluator parses unknown configuration; acquires a shutdown adapter, database, and optional notifier in order; starts a tracked reconciler; publishes full or degraded readiness; admits only supported capabilities; validates or replaces correlation identifiers; counts in-flight work; and drains exactly once. Deferred promises let the test control startup, request, background, cleanup, and deadline settlement without real time or external systems.

Tests prove selected traces: no readiness before required acquisition; immediate background failure before publication; reverse rollback with primary and cleanup evidence; degraded capability refusal; shutdown during startup; pre-reserved lifecycle ownership under reentrant callbacks; readiness false before drain; cooperative cancellation without preemption; in-flight/background settlement before close; reverse close; duplicate shutdown identity; injected force decision; background rejection policy; observation-sink failure containment; bounded canonical event fields; and the absence of request identifiers from metric dimensions.

The deliberate fault publishes readiness before database acquisition settles. TypeScript accepts it. A deterministic test admits work while the database promise is still pending. The repair is one ordering rule: publish readiness only after every required invariant and tracked-task registration succeeds.

False models, repaired

- **"The process is alive, therefore it is ready."** Liveness and capability readiness answer different operational questions.
- **"The server bound its port, so startup succeeded."** Bind success proves no database, configuration, background, or admission invariant.
- **"We logged ready."** A log records an attempted observation; the lifecycle owner controls readiness.
- **"Optional means ignore failure."** It means apply a named degraded-admission, retry, and evidence policy.
- **"Abort stops the operation."** Abort requests cooperative stopping; dispatched effects and ignoring code may continue.
- **"`unref()` supervises background work."** It changes event-loop retention only.
- **"A second signal should run cleanup again."** Repeated shutdown requests share one owned drain and one close pass.
- **"`process.exit()` finishes cleanup."** It can abandon asynchronous work and truncate output.
- **"An uncaught-exception handler makes recovery safe."** Node explicitly warns that normal continuation is unsafe.
- **"Metrics cannot leak identifiers."** Unbounded labels can expose data and destabilize the metrics system.
- **"Telemetry failure should fail the request."** Best-effort observation is not a correctness gate unless the contract makes it a required effect.
- **"No alert means no failure."** Missing, sampled, delayed, misclassified, or wrong-build evidence remains possible.

Mastery questions

1. Reconstruct every legal lifecycle transition and name the state owner, readiness value, admission rule, owned collections, and terminal evidence at each state.
2. Given a service that binds before database acquisition and emits `service.ready` immediately, identify each unsupported proposition and design the smallest ordering repair.
3. Predict acquisition and close calls when the third acquisition rejects and the second close also rejects. Which failure is primary, which are cleanup failures, and why?
4. Design readiness and local admission for a healthy database and failed notifier. Which probes and request classes return success or refusal?
5. Trace a shutdown request that arrives while database acquisition is pending. Show how late acquisition, cancellation, readiness, rollback, and terminal state must settle.
6. An admitted request ignores abort and its remote write completes after the caller's deadline. Derive the valid outcome, retry policy, idempotency requirements, and remaining uncertainty.
7. Compare a tracked background rejection, a floating rejection, and an unexpectedly fulfilled required loop. What can the supervisor know and do in each case?
8. Explain why `unref()`, `beforeExit`, `exit`, `uncaughtException`, and `process.exit()` cannot replace the owned drain protocol.
9. Design a two-signal and deadline policy different from the lab's policy. Prove that it cannot admit new work or close a resource twice.
10. Form three concrete operational queries and choose log, metric, trace, or a combination. State fields, dimensions, sampling, time window, and missing-data interpretation.
11. Audit an event containing raw request ID, tenant ID, URL, exception message, build ID, outcome, and duration. Decide what belongs in logs, metric labels, traces, or nowhere, with reasons.
12. Two dashboards disagree about admitted failures after a deployment. Use artifact/runtime identity, canonical event fields, sampling, aggregation, and first divergence to order the investigation.
13. Build the full decision record for telemetry sink failure and for shutdown cleanup failure. Explain why neither decision can be inferred from the word "failure."
14. Transfer the supervisor to an inventory reservation service with a transactional write and remote notification. Mark process-owned work, transaction-owned effects, uncertain outcomes, idempotency scope, and evidence boundaries.

Presentation prompt

Present the lifecycle as one state machine, then execute three traces: startup success with optional degradation, startup failure with rollback failure, and bounded shutdown with an abort-ignoring request. Show the external probes and local admission decision at every transition. Explain one Node 24 process-event boundary, one secret-safe log, one bounded metric, one trace limitation, and the deliberate readiness fault. End by transferring the partial-failure decision record to a service you have not implemented.

Ready-when test

Without notes, you can reconstruct the legal transitions; distinguish liveness, core readiness, and capability readiness; predict ordered acquisition and reverse rollback; preserve primary, background, and cleanup failures; register admitted and background work before it can settle; explain cooperative abort and uncertain effects; design one exactly-once drain with an injected forced-exit decision; state Node 24 rejection, exception, signal, `beforeExit`, `exit`, and `process.exit()` limits; design bounded logs, metrics, traces, and indicators; diagnose disagreement by build/runtime identity and first divergence; complete all six partial-failure decision records; and pass every Chapter 24 test, including the deliberate premature-readiness fault.

Chapter 25

Java and TypeScript as Contrasting Systems

Chapter type: Integration **Prerequisites:** Chapters 1–24 **Capabilities introduced:** Transfer an operation between Java and TypeScript by identifying compile-time authority, runtime representation, dispatch, equality, failure, and boundary evidence **Earlier chapters integrated:** Identity and mutation from Chapters 1–3; asynchronous execution and resource lifetime from Chapters 6–10; TypeScript relationships from Chapters 11–16; runtime construction and source-to-execution behavior from Chapters 17–20; correctness, public boundaries, production evidence, and architecture from Chapters 21–24 **Later chapters enabled:** Architecture defense, unfamiliar-codebase explanation, and the capstone **Estimated study time:** 6–8 hours across seven sessions **Runnable sources:** `examples/ch25/src/`, `examples/ch25/test/`, and `examples/ch25/lab/`

Compressed thesis

Similar syntax is weak evidence of shared semantics. To transfer one operation between Java and TypeScript, ask which compiler checks it, what runtime value represents it, how the runtime selects code, which equality relation governs its values, how failure travels, and which external component can still invalidate the result. The useful comparison is `Map` lookup by a particular key or dispatch of a particular command—not "Java is nominal" versus "TypeScript is structural."

Java retains class, interface, array-component, and selected declaration metadata in class files and runtime objects while erasing most generic type arguments. TypeScript usually erases type-only declarations and annotations, yet JavaScript classes, functions, objects, symbols, and emitted enum objects remain ordinary runtime values. Neither language is "fully runtime typed," "purely nominal," or "purely structural." Each concrete operation crosses a different set of compile-time, runtime, loader, and application authorities.

This canonical chapter preserves the shared technical spine. When a comparison produces friction, paste the relevant passage and your description of the confusion into Codex. Codex will repair the smallest demonstrated gap in your personal appendix. Do not expand the canonical chapter into two introductory language courses.

1. Transfer operations, not slogans

Create a transfer card before translating a design:

Question	Java example: `hashMap.get(key)`	TypeScript/JavaScript example: `map.get(key)`
Compile-time authority	`javac` checks the declared receiver, key argument, and generic relationships	TypeScript checks the declared receiver, key argument, and generic relationships
Runtime representation	A JVM `HashMap` object; generic arguments are not reified	A JavaScript `Map` object; type arguments and interfaces are absent
Selection	Interface invocation resolves an implementation method	Ordinary property lookup obtains `get`, then a JavaScript call executes it
Key relation	`HashMap` uses a key's hash and `equals` contract; another `Map` implementation can define a different relation	`Map` uses SameValueZero; object keys therefore require the same object identity
Failure and escape	User `hashCode`/`equals`, implementation behavior, or concurrent misuse can fail outside generic checking	Getters, proxies, wrong runtime values, mutation, and external input can fail outside TypeScript checking

Repeat the card for construction, assignment, overload resolution, narrowing, serialization, dependency lookup, exception handling, and commit. The same surface word can name different operations: Java overload selection is a compile-time choice among declarations, whereas overriding selects an implementation at runtime. TypeScript overload declarations describe permitted calls to one emitted JavaScript implementation.

2. Structural and nominal are qualifiers, not identities

Java class and interface relationships are primarily declared. A class explicitly extends a superclass or implements interfaces; assignment and method invocation use those nominal declarations plus specified primitive conversions, array rules, wildcards, and inference. This does not mean that every Java relation is a class-name comparison: primitives, arrays, lambdas targeted to functional interfaces, and type inference have their own rules.

TypeScript compatibility is primarily structural. A value with the required readable and callable members can satisfy an interface without declaring that interface. A class type containing only public instance members is also structural:

```
class RuntimeTask {
  constructor(readonly id: string) {}
  run() { return `class:${this.id}`; }
}

const plain = { id: "task_25", run: () => "plain" };
const accepted: RuntimeTask = plain; // accepted by the checker
accepted instanceof RuntimeTask;     // false at runtime
```

The assignment supplies no prototype, constructor call, or class identity. TypeScript private or protected members restrict compatibility according to declaration origin. Excess-property checks apply specially to fresh object literals in contextual positions; moving the same value through a variable can change that diagnostic without changing the runtime object. A brand can deliberately introduce a nominal-like static distinction, but an erased brand supplies no runtime proof. TypeScript is therefore not "pure structural typing," and Java is not explained by the opposite slogan.

3. Say exactly what survives compilation

TypeScript interfaces, type aliases, type parameters, annotations, and most accessibility modifiers do not exist in emitted JavaScript. Inferred types also do not become runtime descriptors. JavaScript classes and functions

do survive because they are value-producing JavaScript constructs. Some TypeScript constructs, including non-`const` enums under ordinary emission, produce JavaScript values; a blanket "TypeScript disappears" statement is false.

Java compilation preserves binary class and interface identities, member descriptors, array component types, and other class-file structures used by the JVM. Java generic type arguments are mostly implemented by erasure: a parameterized type erases to its raw type, a type variable erases to a bound, and compilation may insert casts or bridge methods. A `Signature` attribute can retain generic declaration metadata for libraries and reflection even though the JVM does not use that metadata as a universal runtime generic checker. Java arrays are different: an array object retains its component type and checks reference stores at runtime.

Ask two separate questions: "Can a reflection API inspect declaration metadata?" and "Does the executing operation enforce that generic argument?" A reflective `List<String>` return signature does not make every list element store dynamically reject non-strings. Conversely, a `String[]` store can fail at runtime even after an assignment through an `Object[]` reference.

4. Values, equality, and keys do not transfer by spelling

Java distinguishes primitive values from reference values. `==` compares numeric or boolean primitive values after applicable conversions, but compares reference operands by identity. Boxing introduces reference objects and unboxing can fail when a wrapper reference is `null`. `String.equals` compares character content; reference `==` does not. Record classes synthesize component-based `equals` and `hashCode` methods for instances of the same record class. Hash-based maps depend on the `equals`/`hashCode` contract of their keys.

JavaScript strings and numbers are primitive values. Strict equality compares equal string contents, while distinct ordinary objects are unequal even when their enumerable fields match. JavaScript `Map` uses SameValueZero: it treats `NaN` as the same key as `NaN` and `0` as the same key as `-0`, but two equal-looking objects remain different keys.

Prediction:

```
const first = { accountId: "acct_25" };
const second = { accountId: "acct_25" };
const byObject = new Map([[first, "first"], [second, "second"]]);

console.log(byObject.size, byObject.get({ accountId: "acct_25" }));
```

Exact output:

```
2 undefined
```

The two insertions use distinct object identities, and the lookup creates a third. A domain identifier such as the primitive string `"acct_25"` supplies the intended key relation here. Do not import Java record equality or a Java key class's `equals` method into plain JavaScript objects without implementing and consistently invoking a domain equality operation.

5. Arrays and variance move risk between checker and runtime

Java reference arrays are covariant: `Dog[]` can be viewed as `Animal[]`. The array object still knows that its component is `Dog`, so storing a `Cat` through the broader view throws `ArrayStoreException`. Java generic classes are invariant unless wildcards express a use-site relationship such as producer or consumer variance. Erasure does not turn generic invariance into an array store check.

TypeScript permits useful but unsound relationships around mutable arrays. A `Dog[]` can be used where `Animal[]` is expected, and a write through the broader access path can place a non-dog into the same JavaScript array. No runtime component descriptor rejects the write. A `readonly Animal[]` prevents writes through that checked reference, but `readonly` is erased and does not freeze the array or prevent mutation through another alias.

Function parameter checking has separate variance rules. With `strictFunctionTypes`, ordinary function parameters receive stricter contravariant checking, while method syntax retains a compatibility exception. Do not derive function variance from array behavior or from a one-word characterization of the language.

The runnable array case stores a cat through an `Animal[]`, then a later `dogs[1].bark()` throws `TypeError`. Java's corresponding risk is detected at the store. TypeScript's accepted source allows the invalid value to remain until a later operation exposes it.

6. Alternative cases and code organization have distinct runtime shapes

A TypeScript discriminated union describes a checked relationship among ordinary JavaScript values. The union itself is not a runtime container. A parser must inspect unknown input and construct a member with a validated discriminant; a `switch` can then narrow and check exhaustiveness in included source. Assertions, unchecked dependencies, and later mutation can bypass that evidence.

Java SE 26 can express closed alternatives with sealed classes or interfaces, records, enums, and exhaustive pattern `switch` constructs. Permitted classes retain runtime class identities; records retain class identity and synthesized members; an enum constant is an instance of its enum class. A sealed hierarchy and a TypeScript discriminated union can support analogous case analysis, but their representations and escape routes differ.

Organization also resists caricature. TypeScript can organize work with functions, closures, plain objects, classes, and ECMAScript modules. Java can use classes, interfaces, records, enums, static members, instance members, lambdas, method references, and modules. "TypeScript is functional" and "Java requires everything to be an object" conceal the actual owner, lifetime, captured state, and dispatch operation. Translate those four facts, not a preferred syntax.

7. Null, exceptions, and completion use different contracts

Java `null` inhabits reference types but not primitive types. Dereferencing it fails; unboxing a null wrapper also fails. Standard Java type checking does not provide a universal non-null guarantee for ordinary reference declarations. Library annotations and external analyzers can add evidence, but their authority must be named.

With `strictNullChecks`, TypeScript requires `null` and `undefined` to appear in checked relationships before ordinary use. With `exactOptionalPropertyTypes`, an optional property can mean absence without automatically permitting an explicit `undefined` write. Those checks do not parse JSON, change a JavaScript object's properties, or stop an assertion from misdescribing `{ label: null }`. The runnable case asserts external data as `{ label: string }`; calling `toUpperCase` still throws `TypeError`.

Java divides exceptions into checked and unchecked categories. The compiler's exception analysis can require a `catch` or `throws` declaration for checked exceptions. It does not prove recovery, and unchecked exceptions remain possible. TypeScript has no checked `throws` relation: JavaScript can throw any value, so a caught reason begins as `unknown` and must be narrowed. A synchronous throw before a promise is returned, a returned promise that rejects, and an explicit result variant are three different paths.

Java `CompletionStage` and JavaScript `Promise` both describe selected completion relationships. They do not make their bodies execute on the same scheduler. A Java stage action may run in a completing thread or an executor according to the selected method; a JavaScript `async` function runs synchronously until suspension and resumes through queued promise jobs. Neither abstraction by itself establishes a transaction, a lock, or rollback.

8. Concurrency requires a scheduler and memory account

Java threads can execute concurrently. Virtual threads are still `Thread` instances scheduled by the Java runtime; they improve the scalability of thread-per-task code rather than making work intrinsically faster. Shared-memory reasoning uses Java Memory Model rules. A monitor unlock before a later lock on the same monitor, a volatile write before a later matching read, and thread start/join relationships can establish happens-before ordering and visibility. Without a required happens-before edge, a data race cannot be repaired by source-order intuition.

Ordinary Node.js JavaScript in one agent runs one job to completion. An `await` creates a temporal boundary at which other queued work can run before continuation. This prevents two JavaScript callbacks from executing simultaneously in that agent, but it does not make a read–await–write sequence atomic:

```
let balance = 0;
async function increment() {
  const observed = balance;
  await 0;
  balance = observed + 1;
}
await Promise.all([increment(), increment()]);
console.log(balance);
```

Exact output in the pinned baseline is `1`: both invocations read `0` before either continuation writes. Worker threads, child processes, native code, remote services, and databases introduce separate agents or authorities. `Promise.all` joins completion; it supplies no mutual exclusion and no database isolation.

9. Dependency injection and reflection need runtime values

Constructor parameters and factories support dependency injection in both languages without a container. In TypeScript, an interface can check that an injected repository has `load` and `save`, but the interface supplies no runtime lookup key. A container needs an actual value such as a unique symbol, string under a controlled convention, or class constructor. Two `Symbol("Repository")` calls produce distinct tokens even though their descriptions match.

Decorators and emitted metadata are optional compiler and framework choices, not inherent consequences of an interface declaration. A class token also couples lookup to that emitted constructor identity and can split across duplicated packages or realms.

Java reflection can inspect loaded classes, fields, methods, constructors, and runtime-retained annotations subject to platform and access restrictions. Source-only annotations disappear; class-retained annotations need not be visible to runtime reflection. Generic signature metadata is incomplete as runtime enforcement. Java dependency-injection frameworks may use reflection and annotations, but direct constructor composition, factories, and generated code are also valid. “Java DI is reflection; TypeScript DI is structural” misstates both operations.

10. Modules, overloads, and service boundaries expose different authorities

Java package names organize declarations; JAR files package resources and class files; the Java Platform Module System can declare dependencies, exports, services, and reflective opening. The compiler, launcher, class path or module path, class loaders, module descriptors, and archive contents jointly determine what loads.

TypeScript/JavaScript execution depends on ECMAScript import/export syntax, emitted specifiers, Node or browser resolution, package metadata, installed package topology, compiler options, and runtime loaders. TypeScript resolving an import for checking does not prove that Node will load the emitted specifier. Similar `import` syntax supplies almost no evidence that Java and Node locate code the same way.

Java overloads are separate method declarations with distinct JVM descriptors when their erased parameter types differ. The compiler selects an applicable declaration; runtime invocation can then dispatch an override. Erasure can require bridge methods and forbids overload sets whose signatures erase to the same form. TypeScript overload signatures are erased declarations in front of one JavaScript implementation:

```
function describe(value: string): string;
function describe(value: number): string;
function describe(value: string | number) { return String(value); }

console.log(describe.length); // 1
```

Across a service boundary, neither a Java DTO declaration nor a TypeScript interface validates bytes. The receiving application must decode, validate, authorize, and construct its own trusted value. Version skew and hostile input survive language choice.

11. Lab: migrate a command boundary without importing class identity

The lab receives an unknown `open-account | credit-account` transport command. Implement three operations: parse and reconstruct a closed discriminated union; dispatch exhaustively; compose the service with an injected repository. Use the primitive account ID as domain identity. Return explicit invalid-input and domain-refusal outcomes. Let unexpected repository reasons reject the returned promise unchanged.

The deliberately faulty evaluator defines `CreditAccountWireCommand` and checks decoded JSON with `instanceof`. That idea is plausible when a Java deserializer has actually constructed a declared command class. `JSON.parse` instead creates an ordinary object with `Object.prototype`; equal fields do not install `CreditAccountWireCommand.prototype`. The faulty evaluator therefore rejects valid wire structure, while a manually constructed class instance passes. The repair is not a broader assertion. Parse fields, validate values, and construct the application's union member.

The lab proves only application behavior against an injected repository. It does not establish transaction isolation, durable storage, authorization, distributed concurrency, or a production serializer.

False models, repaired

- **"Java is nominal; TypeScript is structural."** Those are useful defaults, not complete operational accounts. Name the assignment, private member, brand, array, lambda target, or declared hierarchy involved.
- **"Java types exist at runtime; TypeScript types do not."** Java erases most generic arguments and retains selected metadata; TypeScript erases type-only constructs while JavaScript classes and other value constructs remain.
- **"Records make Java objects value types."** A record remains an object with class identity; its generated equality is component-based for the same record class.
- **"Readonly TypeScript arrays are immutable."** `readonly` restricts a checked access path and is erased; another alias can mutate the same array.
- **"A promise is JavaScript's thread."** A promise records eventual settlement and schedules reactions; it does not identify a thread or protect shared state.
- **"`instanceof` validates a JSON DTO."** It checks a runtime prototype relationship. JSON decoding does not ordinarily construct application class instances.
- **"An interface can be a dependency-injection token."** An erased interface has no value identity. Runtime lookup needs a runtime value.
- **"The same import or overload syntax means the same mechanism."** Loaders, package metadata, descriptors, erasure, and emitted implementations decide the actual operation.
- **"Static declarations validate a service boundary."** External bytes remain `unknown` until the receiving runtime decodes and validates them.

Mastery questions

1. Build a six-field transfer card for a repository `save` call in Java and TypeScript. Which facts does each compiler establish, and which database facts remain unproved?
2. Predict the static acceptance and runtime `instanceof` result when a plain object satisfies a TypeScript class with only public instance members. How does adding a private member change only the checked relationship?
3. Explain separately what Java erasure removes, what a class-file `Signature` can retain, and what a Java array enforces at a store.
4. A Java `HashMap<AccountKey, V>` uses a record key, while a JavaScript `Map<object, V>` uses plain object keys. Derive the different lookup results for equal-looking fresh keys and repair the JavaScript design.
5. Compare a `Cat` store through a broadened Java array reference with the Chapter 25 TypeScript array case. At which operation does each failure become observable?
6. Design corresponding closed command alternatives using a TypeScript discriminated union and a Java SE 26 sealed hierarchy. Name their distinct runtime representations and boundary escape routes.
7. Diagnose why `strictNullChecks` and `exactOptionalPropertyTypes` do not justify accepting an asserted JSON object. What runtime evidence must be added?
8. Classify these paths independently: a Java checked exception, a Java unchecked exception, a JavaScript synchronous throw, a rejected promise, and an explicit result variant.
9. Derive the final value of the two-increment `await` schedule. Why does single-agent run-to-completion fail to make the whole operation atomic?
10. Specify a runtime DI key for a TypeScript repository. What collision or identity risks follow from a string, symbol, and class constructor respectively?
11. Explain how Java runtime-retained annotations, generic signature metadata, and direct constructor composition supply three different kinds of evidence to a DI framework.
12. Compare Java overload selection plus override dispatch with TypeScript overload checking plus its single emitted implementation.
13. Diagnose the lab's `instanceof` fault without using "Java is nominal" as the explanation. Name decoder output, prototype identity, checked structure, and the correct reconstruction operation.
14. Transfer the chapter method to C# or Kotlin: choose one operation, identify every uncertain field, and state which claims require new primary-source verification.

Independent implementation and diagnosis lab

Without opening `examples/ch25/lab/evaluator/`, implement `parseTransportCommand`, `dispatchCommand`, and `createCommandService` in `examples/ch25/lab/starter.ts`. Preserve the public types. Test invalid records, unknown fields, both variants, domain refusals, overflow, call counts, domain key identity, and propagation of an arbitrary repository rejection reason.

Then diagnose `incorrect-java-instanceof.ts` from its public behavior alone. State why the fault may feel natural in a Java migration, which exact JavaScript relation fails, and why adding a TypeScript assertion would preserve the defect.

Presentation prompt

Using sparse notes and your own words, present the transfer card, the structure-versus-runtime-identity prediction, one erasure distinction, the array failure comparison, the logical race schedule, and the lab fault. Connect at least one result to Chapters 1, 17, 19, and 21. End with one boundary that neither compiler owns and one uncertainty that requires inspecting a concrete loader, framework, or deployment.

Ready-when test

You are ready when, without the chapter open, you can take an unfamiliar Java or TypeScript operation and name its compile-time authority, runtime representation, dispatch rule, equality relation, failure path, and external

escape; predict all focused Chapter 25 outputs; implement the lab without classifying JSON by application-class identity; and identify which comparison requires a source check instead of a remembered slogan.

Chapter 26

Capstone: Build, Explain, and Diagnose a Production-Shaped Service

Chapter type: Synthesis **Prerequisites:** Chapters 1–25 **Capabilities introduced:** Assemble, defend, test, and diagnose one strict TypeScript/Node service; identify which component supplies or enforces every guarantee and which failures remain possible elsewhere **Earlier chapters integrated:** Runtime identity and state; asynchronous completion and ownership; TypeScript's static constraints and erasure; parsing and source-to-execution identity; HTTP, transactions, authorization, lifecycle, and operational evidence **Later chapters enabled:** Independent transfer to unfamiliar production TypeScript systems **Estimated study time:** 8–12 hours across implementation, diagnosis, and architecture defense **Runnable sources:** `examples/ch26/src/`, `examples/ch26/test/`, `examples/ch26/faults/`, and `examples/ch26/lab/`

Compressed thesis

JavaScript determines what executes. TypeScript analyzes the program before execution and reports whether its expressions, assignments, calls, and declarations satisfy a static set of type constraints. Type annotations and inferred types generally do not survive into the emitted JavaScript. Production behavior also depends on the runtime host, module loader, package metadata, compiler configuration, external data, storage systems, and process lifetime. Mastery requires identifying which component supplies each guarantee, which component can enforce it, and which failures remain possible elsewhere.

This book is compressed by design. It presents a connected technical model rather than a sequence of gentle tutorials. Make that model usable by predicting program behavior, reconstructing explanations from memory, implementing mechanisms independently, presenting them aloud, and repairing only gaps demonstrated by your own work. When a passage produces friction, paste the relevant passage, code, prediction, and observed result into Codex and describe the confusion in ordinary language. Codex will determine the smallest useful repair and put it in your personal appendix instead of expanding the canonical chapter. Use that repair, then return here.

The capstone is a tenant-scoped reservation-release service. A request crosses a pure HTTP transport boundary, an `unknown` application parser, current-state authorization, a database transaction, a conditional version write, a named request-key constraint, commit settlement, public translation, and operational observation. The same transaction stores an outbox intent; a separately owned publisher performs the remote notification. One lifecycle coordinator parses configuration, records runtime identity, acquires resources, registers background work, publishes readiness, admits requests, drains owned work, and closes resources exactly once. The service is production-shaped because these relationships are explicit. Its adapters remain deterministic fakes, so it makes no claim that a real network, PostgreSQL server, process supervisor, or notification provider behaved.

1. Start with executable identity

The repository is an ESM package. TypeScript checks with `module` and `moduleResolution` set to `NodeNext`; source imports name `.js` targets because emitted ESM is what Node loads. A successful check establishes no emitted artifact. A successful build establishes no deployed artifact. A correct source file establishes no entry command. The diagnostic packet therefore records the build identifier, artifact marker, Node version, entry identity, package mode, compiler mode, and a safe configuration identity.

Configuration enters as `unknown`. `parseServiceConfig` requires a non-null, non-array record; rejects unknown and missing fields; parses bounded numeric strings; checks the database URL shape, token length, notifier policy, and body limit; and constructs a new closed `ServiceConfig`. Invalid configuration acquires no resource. Issue records contain stable field and code names, never the configured database URL or token value. `safeConfigIdentity` reports only propositions such as `database=required` and `auth=configured`.

This separation is exact:

Proposition	Supplying or enforcing component	Remaining failure
Checked source imports are compatible with NodeNext rules	TypeScript 6.0.3 checker and effective config	Node may receive a different artifact, package mode, topology, or entry command
The loaded file is the intended capstone artifact	Node loader plus manifest marker and invocation evidence	Deployment can launch an old file or record false metadata
Configuration has the closed application shape	Runtime parser	Secret validity, database reachability, and operator intent remain external
Diagnostics omit configured secret values	Safe formatter plus tests over sentinel values	Another sink or adapter can publish secret-bearing data

When behavior disagrees with source, find the first divergent edge: build command, selected configuration, emitted file, copied artifact, runtime entry, import resolution, parsed configuration, then application path. Do not edit domain logic until evidence reaches it.

2. Keep transport, parsing, and authentication separate

`admitTransport` is a pure function over method, path, content type, byte chunks, and a configured size limit. It matches one route, requires `POST`, checks JSON media type, counts bytes before decoding, uses fatal UTF-8 decoding, and calls `JSON.parse`. Its successful result contains `decodedBody: unknown`. JSON decoding proves only that the bytes form a JSON value.

`parseReleaseCommand` owns the application schema. It checks the route tenant and reservation identifiers, requires exactly `requestKey`, `expectedVersion`, and `reason`, validates every value, then creates branded application identities. The brands help the checker reject accidental interchange inside checked code; they are not present at runtime. A type assertion would create no validation and no brand evidence. The deliberate assertion fault returns `"71"` from the expression that the checker believes produces a number.

Authentication supplies a principal or an explicit missing/invalid-credentials result. It does not authorize the release. The public mapping keeps transport failure, invalid application input, unauthenticated access, concealed forbidden/not-found access, domain conflict, storage conflict, pre-effect unavailability, uncertain outcome, and unexpected failure distinct. A `401` includes `WWW-Authenticate`. This service intentionally maps forbidden and absent reservations to the same fresh `404` representation so callers cannot distinguish current existence. Denial reaches no save.

Every public response is newly constructed from an allowed outcome. It contains no internal `Error`, arbitrary rejection reason, database URL, token, SQL message, stack, user ID, or tenant ID. A fresh generic response is a security boundary; serializing the caught object and deleting a few properties is not.

3. Let the domain state own valid fields

`Reservation` is a discriminated union of `held`, `fulfilled`, and `released`. Shared identity, owner, version, and units exist in every state. `expiresAt` belongs only to `held`; `fulfilledAt` belongs only to `fulfilled`; release reason, releaser, and request key belong only to `released`. Narrowing `state` gives the checker evidence for those fields. The runtime value is still an ordinary JavaScript object, so external values must be parsed and repository adapters must honor their contracts.

`releaseReservation` accepts only a currently held reservation. It constructs a released successor with version incremented and an unpublished outbox intent. It does not read storage, authorize a principal, publish a notification, or commit anything. This purity makes the state transition independently predictable.

Authorization reads the current row inside the transaction and immediately precedes the effect decision. `mayRelease` requires the principal's tenant to match the current reservation tenant and then requires owner or support role. A token carrying yesterday's owner cannot authorize today's row. A route tenant cannot broaden a principal's tenant. Moving authorization before the current read creates a stale-authorization race even if the checker accepts every type.

The checker can reject access to `releasedBy` while a value is narrowed to `held`. It cannot prove that a database decoder returned a genuine union member, that another process respected the version, that the policy table is current, or that a production adapter issued the expected SQL. Those require parser, transaction, constraint, and integration evidence.

4. Make transaction lifetime and outcome explicit

The application transaction follows this order:

```
acquire lease
-> begin
-> load current tenant-scoped row for update
-> authorize current state
-> derive domain transition
-> conditional version save plus outbox insert
-> await commit
-> construct success response
-> release lease
```

If a noncommitted path began, the coordinator attempts rollback. It always attempts release after acquisition. The primary failure remains separate from rollback and release failures. Observation of the report is best effort: a throwing evidence sink cannot change committed success or replace an original arbitrary rejection reason.

The report names database knowledge rather than using one `failed` bucket:

Database outcome	Meaning in this contract
`not-started`	No transaction began; selected acquire/begin/load unavailability is safe for the service's retry-later response
`rolled-back`	A begun, uncommitted transaction reports successful rollback
`rollback-unknown`	Rollback rejected; the original operation and cleanup failure remain separate
`commit-unknown`	Commit started and rejected; absence or durability is not known from that rejection
`committed`	Commit fulfilled and lease release reported success
`committed-with-cleanup`	Commit fulfilled, but release rejected; success is not relabeled as rollback

Success is constructed only after awaited commit fulfillment. The deliberate premature-success fault calls commit without awaiting it and returns `200`; its test holds commit pending, observes the response, then rejects commit.

The first divergence is response construction, not a type error.

A commit rejection returns `outcome-unknown` with `action: check-status` and the caller's request key. The client reconciles that key before deciding whether another attempt is legal. Timeout, abort notification, commit rejection, rollback rejection, and confirmed conflict are separate facts. None implies the others.

A save rejection occurs before commit starts. If the participating transaction then reports successful rollback, this service knows that transaction did not commit: stable storage-unavailable classification becomes retry-later, while an unexpected storage category becomes internal failure. If rollback itself rejects, absence is no longer established and the response is outcome-unknown. This distinction prevents a contradictory report such as public uncertainty paired with `databaseOutcome: rolled-back`.

5. Put concurrency policy in storage-visible identities

The command carries `expectedVersion`. The repository save is conditional on tenant, reservation, and current version. If zero rows match, the adapter returns `version-conflict`; the application does not blindly retry a domain decision derived from stale state. A retry must reread, reauthorize, and rederive under an explicit attempt policy.

The request key has tenant scope and a declared operation scope: reservation-release notification. PostgreSQL, not the TypeScript brand, must enforce the named production constraint. The schema names it `reservation_release_tenant_request_key_uq`. The adapter maps SQLSTATE `23505` only when the reported constraint name matches. Message text is not a stable classification API. The simulator's request-key set proves only its controlled contract; it does not prove the migration exists in a deployed database.

Retention is part of identity. The simulator retains keys for its lifetime. The notification consumer contract requires at least 24 hours after publication. A real system must align API retry horizon, database retention, outbox retention, consumer deduplication, and disaster-recovery replay. Reusing a key outside its declared scope or after premature deletion reopens duplicate effects.

6. Commit intent; publish outside the transaction

The release row and `reservation.released` outbox intent are staged by the same simulated transaction and become visible together on commit. No remote notifier call occurs inside that transaction. A separate publisher asks for a pending intent, calls the notifier with the request key as idempotency key, then marks the intent published. If notification succeeds and marking fails, the next attempt can publish again; the consumer's idempotency scope and retention bound the duplicate.

The publisher has an owner, start condition, abort signal, tracked settlement promise, failure policy, and drain obligation. It checks abort around controlled effects. Unexpected fulfillment is a failure because a required loop stopped. Rejection is equally a failure even when the rejection reason is `undefined`; JavaScript permits arbitrary rejection values. "Exactly once" would be false here. The service claims an atomic database state-plus-intent transition and at-least-once publication attempts under the consumer idempotency contract.

7. Publish readiness only after ownership exists

`ProductionShapedService` owns `starting`, `ready/full`, `ready/degraded`, `draining`, `failed`, and `stopped`. Liveness is true while starting, ready, or draining under this policy. Readiness is true only in a ready mode. Local admission rejects every other state even if an external probe update is delayed.

Startup parses configuration before effects, installs a shutdown adapter, acquires the database, acquires the required or optional notifier, starts the publisher, registers its settlement, performs a microtask checkpoint for already-settled work, and only then publishes readiness. Each returned handle transfers one close obligation. Failure closes acquired handles once in reverse order and preserves the acquisition or background reason as primary beside cleanup failures.

Startup and shutdown promises are installed before any observer or adapter callback can reenter their methods. Otherwise a synchronous sink callback can start a duplicate acquisition or cleanup pass before assignment occurs.

Immediate publisher fulfillment, `Promise.reject(new Error(...))`, and `Promise.reject(undefined)` all fail startup before a ready event. These are ordering guarantees supplied by the lifecycle coordinator and deterministic microtask tests, not by TypeScript's promise types.

Admission creates an in-flight completion gate and adds it to the registry before invoking user work. A callback can synchronously request shutdown; the drain still sees and awaits that work. Shutdown makes readiness false, stops admission, aborts the owned controller, awaits startup and registered request/background settlement, then closes resources in reverse order. Repeated shutdown calls receive the same promise. An injected deadline can return `force-required`; the test never calls `process.exit`, and that decision does not prove pending effects stopped.

Thin signal adapters may call `requestShutdown`. Tests inject requests directly. Process signal delivery, platform behavior, socket draining, and forced exit remain outside the deterministic supervisor evidence.

8. Observe bounded facts without changing correctness

Correlation input is accepted only under a bounded grammar and otherwise replaced with a deterministic local identifier. It connects selected events; it does not authenticate, authorize, or belong in metric dimensions. Events contain a closed event name, bounded outcome and reason code, build/runtime identity, correlation ID or null, and numeric count. Metrics are fixed numeric aggregates. They have no request, user, tenant, request-key, raw URL, arbitrary error, or secret dimension.

`SafeObserver` catches synchronous sink failures and increments an in-process observation-failure counter. Request and transaction evidence record bounded categories. Unexpected `Error` and non-`Error` reasons keep exact identity until the top operational boundary records a category and returns a fresh generic response. The event sink never receives the arbitrary reason. A required audit record would be a correctness effect with its own failure contract; it must not masquerade as best-effort telemetry.

Observation can be absent, dropped, duplicated, delayed, or emitted by a stale artifact. Start diagnosis with a proposition and compare the earliest evidence-producing boundary. One terminal request outcome is emitted per admitted request; an internal failure cannot first emit `failed` and then be counted again as generic `succeeded` merely because a `500` object was returned.

9. Read the guarantee ledger adversarially

`guarantee-ledger.ts` names twelve propositions across checker, configuration parser, deployment identity, application parser, domain transition, authorization, repository contract, PostgreSQL, test simulator, lifecycle coordinator, observation boundary, and outbox protocol. For each row ask:

1. What exact proposition is claimed?
2. Which component supplies or enforces it?
3. At what time and over what population?
4. What evidence was actually observed?
5. Which bypass or failure remains possible elsewhere?

"The type system guarantees this," "the database handles concurrency," and "the service is observable" fail this test. Name the expression relationship checked, conditional statement or constraint enforced, event recorded, and residual escape. A checker-clean program may assert malformed input. A correct repository interface may be backed by incorrect SQL. A passing simulator may disagree with a missing migration. A present event may come from an old build.

10. Diagnose faults and unfamiliar code from first divergence

The chapter contains three isolated checker-clean faults:

- asserted input bypasses the runtime parser;
- success becomes visible before commit settlement;
- readiness becomes true before required acquisition.

For each, the test records first divergence, enforcing authority, decisive evidence, minimal repair, and regression. Do not begin with the final symptom. A `200` followed by missing state is not first evidence that "the database lost data" when source constructs the response before awaiting commit. Admission failure is not first evidence that "Node is racing" when readiness is assigned before an injected promise settles.

The unfamiliar-codebase fixture is intentionally shuffled. Reconstruct its runtime entry, effective configuration, erased static evidence, runtime validators, dependency construction, request path, transaction lifetime, state owner, background owner, and public translator. Its checker passes while the selected `tsconfig.check.json` has `noEmit: true`; `node dist/entry.js` runs `stale-v2`. The first divergence is the build command and effective configuration, not the source edit or Node's execution of the named old file. Repair by selecting the build configuration, verifying the emitted marker, and then launching that artifact.

The service is ready for defense when you can implement it without evaluator access, predict its controlled schedules, explain every ledger row, diagnose every injected fault at the first divergent boundary, and transfer the reconstruction procedure to the unfamiliar fixture. Passing tests are necessary evidence, not the whole argument.

False models, repaired

"The checked command type validates JSON." The assertion fixture type-checks while a string version reaches numeric-looking code and produces concatenation. Repair: transport exposes `unknown`; the parser validates each field and reconstructs the command.

"A repository save rejection is always outcome-unknown." The transaction's later cleanup supplies more evidence. Fulfilled rollback establishes no commit for the participating save; rejected rollback does not. Repair: classify after cleanup settlement rather than freezing the public outcome inside the first catch.

"Calling commit means the response can say success." The premature-success fixture returns `200` while commit is pending. Repair: await commit fulfillment before constructing success; reconcile after rejection.

"One JavaScript agent makes the transaction atomic." Other processes and the database interleave across awaits. Repair: make the version predicate and tenant/request-key constraint storage-visible, then inspect the exact production isolation and SQL.

"Starting a background promise establishes readiness." An already-fulfilled or arbitrarily rejected promise has no running loop. Repair: attach settlement ownership, perform the microtask checkpoint, classify either settlement as startup failure, then publish readiness.

"A ready log and a readiness probe are the same guarantee." The event sink can throw, drop, delay, duplicate, or receive a stale build's event. Repair: the lifecycle state controls local admission and probe response; the event is bounded observation.

"A passing simulator proves production correctness." It proves selected application traces under its programmed semantics. Repair: allocate database constraints, deployment identity, platform routing, signal delivery, notifier idempotency, and telemetry delivery to their actual components and evidence.

Mastery questions

1. Given a successful TypeScript check and stale runtime output, derive the shortest evidence sequence that can locate the first divergence without assuming the source executed.
2. Transform the configuration parser so one field becomes optional. State the construction rule, safe diagnostic change, acquisition policy change, and new residual failure.
3. Predict transport and application results for invalid UTF-8, valid JSON scalar input, an unknown body key, and an invalid branded identifier. Name the component deciding each result.
4. Design a test that distinguishes stale token claims from current-state authorization without using time, a network, or a real database.
5. Derive the full trace for a domain conflict followed by rollback rejection and lease-release rejection. Which reason is primary, and what public response is allowed?

6. A conditional save succeeds and commit rejects. Explain why neither automatic retry nor a generic `503` is justified. Specify the reconciliation query and next decision.
7. Change the request-key constraint from tenant scope to global scope. Predict one false conflict and identify the migration, adapter, and documentation changes required.
8. Schedule two release attempts from the same version. Explain what the simulator establishes, what PostgreSQL must enforce, and why TypeScript brands establish neither fact.
9. The notifier succeeds and outbox marking rejects. Derive the next publisher attempt and state the consumer key, scope, and retention needed to bound duplicates.
10. Predict startup when the required background task fulfills immediately, rejects with `undefined`, or rejects after readiness. State the lifecycle and cleanup path for each.
11. Construct a synchronous reentrancy schedule that duplicates startup or shutdown if promise ownership is assigned after an observer callback. Explain the reservation repair.
12. An admitted callback requests shutdown before its first `await`. Derive the in-flight registry and close order with and without pre-registration.
13. Design an operational query for commit uncertainty. Allocate bounded event fields and numeric metrics without using tenant, user, URL, request key, or arbitrary reason as a metric dimension.
14. Defend or refute: "The complete deterministic chapter test suite proves this service is production safe." Enumerate the strongest proved propositions and at least five unproved external facts.
15. Reconstruct the unfamiliar fixture without opening its evaluator, identify its first divergence, and present the architecture in 10–15 minutes using only one boundary diagram and the guarantee ledger.

Implementation lab — reconstruct an unfamiliar service

Do not open `examples/ch26/lab/evaluator/` first. `lab/starter.ts` contains a genuine implementation point: return the complete `ArchitectureExplanation` for the supplied `UnfamiliarServiceFixture`. Your answer must identify runtime entry, selected configuration, erased static evidence, runtime validation, dependency construction, request flow, transaction lifetime, database state ownership, background ownership, public translation, first divergence, authority, decisive evidence, and repair. A field list without causal connections does not pass.

Test your implementation against `lab/fixture.ts`, then compare with `lab/evaluator/solution.ts`. Finally open `incorrect-stale-artifact-diagnosis.ts` and prove why blaming the current handler begins after the demonstrated no-emit divergence. Complete the three service fault diagnoses separately: each requires first divergence, enforcing component, evidence, repair, and regression. Finish by defending which fixture facts are real runtime evidence and which are only a static description of the unfamiliar architecture.

Presentation prompt

Use `presentations/prompts/ch26.md` for a 10–15 minute architecture defense. Present the source-to-runtime and request-to-effect paths, one transaction uncertainty schedule, one lifecycle reentrancy schedule, the outbox idempotency boundary, one checker-clean fault, and the unfamiliar stale-artifact diagnosis. The required guarantee table must name a proposition, supplying or enforcing component, decisive evidence, and residual failure. The defense ends with unproved production facts rather than the test count.

Ready-when test

You are ready when you can, without notes: reconstruct the source-to-runtime and request-to-effect paths; implement strict parsing, state transition, current-state authorization, transaction cleanup, outbox publication, lifecycle ownership, and public translation; predict commit and drain schedules before execution; identify every tested simulator boundary; diagnose all three faults by first divergence; explain the unfamiliar fixture; and defend which component supplies or enforces every guarantee and which failures remain possible elsewhere. Paste any failed item, your attempt, and the observed evidence into Codex. Use the resulting personal appendix repair, then repeat the original task.

Personal Appendix

This is one cumulative, reader-specific appendix. Each level-two heading groups repairs by canonical source section, and each level-three numbered repair is keyed to the passage that triggered it. Follow a triggering-passage link to return to the top of the paragraph that contains the linked words; those words link directly back to that repair.

The appendix supplies only the decompression needed to make the original passage usable. It does not add a standard question set or turn every repair into a separate appendix chapter.

Appendix Introduction

Introduction.1 - Runtime terms, production components, and the five-region map

Triggered by: Introduction, production-components paragraph and Introduction, five-region paragraph

"Runtime" names execution rather than source analysis

Start with a small program:

```
type Phase = "idle" | "done";

let phase: Phase = "idle";

async function finish(): Promise<number> {
  const before = phase;
  await Promise.resolve();
  phase = "done";
  return before === "idle" ? 1 : 0;
}

const pending = finish();
```

In this repository's checked build path, TypeScript analyzes the source before Node executes the emitted JavaScript. TypeScript checks that `"idle"` and `"done"` are the permitted values for locations described by `Phase`. It checks that `finish` returns a promise whose fulfillment value is a number. That work is **static analysis**: TypeScript reasons from source declarations and expressions without running this particular execution. A later chapter treats direct TypeScript execution paths that do not run the checker first.

When the emitted JavaScript executes, the type alias `Phase` is absent. The executing program has a binding named `phase` containing the string value `"idle"`. It has a function object created for `finish`. Calling that function creates a promise object, runs the function synchronously until `await`, and arranges for the suspended continuation to resume later. These are runtime events.

The word *runtime* is used in several related ways:

- **A runtime value** is one actual JavaScript value present during execution. In the example, `"idle"`, the `finish` function object, the promise stored in `pending`, and the eventual number `1` are runtime values. `Phase` is not a runtime value; it is a TypeScript type alias used during checking.

- **Runtime state** is the current arrangement of facts that can affect what happens next. At one moment, the `phase` binding contains `"idle"`, the promise is pending, the local `before` contains `"idle"`, and a continuation is waiting to resume. Later, `phase` contains `"done"` and the promise is fulfilled with `1`. Runtime state is not necessarily one object named `state`. It includes relevant bindings, object properties, promise states, queued work, open resources, and application records.
- **Runtime behavior** is the sequence of observable events produced by execution: which code runs first, which values are returned, which promises settle, which errors occur, which state changes, and which external effects happen.
- **A runtime host** is the environment that executes JavaScript and supplies facilities that the ECMAScript language does not supply by itself. For this book the main host is Node.js.
- **Runtime truth** is a narrower phrase used later in the book. It means a fact established by an operation that actually executed, with the operation's time and scope kept explicit. A parser that just checked one request established facts about that request. It did not prove that every future request will be valid.

A useful distinction is: a runtime value is one piece of data; runtime state is the current collection and arrangement of relevant pieces. A promise is a runtime value. Whether that promise is pending, fulfilled, or rejected is part of runtime state. A function is a runtime value. The current contents of the bindings that its closure can reach are part of runtime state.

Why production behavior depends on more than JavaScript and TypeScript

Imagine a Node service that imports a request handler, reads configuration, accepts JSON over HTTP, and writes an order to PostgreSQL. The TypeScript source is only one participant in that service's behavior.

Runtime host

Node.js is the runtime host. It creates the process, evaluates JavaScript, supplies APIs such as timers, files, sockets, environment variables, and process signals, and provides scheduling behavior around JavaScript Jobs and host callbacks. A browser is a different host with different APIs and event-loop rules.

TypeScript can describe `process.env` or a timer callback, but TypeScript does not create the environment variable, open the socket, deliver the signal, or decide when a timer callback becomes eligible. Node performs those operations while the program runs.

Module loader

The module loader turns an import request into a module that can be loaded, linked, and evaluated. Given an import such as:

```
import { startServer } from "./server.js";
```

Node's loader decides what file or package that specifier denotes under its rules. It also participates in deciding whether the file is ECMAScript module or CommonJS code, when the module is evaluated, and whether a previously loaded module instance is reused.

TypeScript also performs module resolution while checking source, but TypeScript resolution and Node runtime resolution are distinct operations. The editor can find a declaration while Node later fails to find the executable file. Conversely, Node can load JavaScript that TypeScript never checked.

Package metadata

Package metadata is data such as `package.json`. Fields including `type`, `exports`, `imports`, and package entry declarations influence how tools and loaders interpret files and public package paths.

Metadata is not merely documentation. Changing `"type": "module"` can change how Node interprets a `.js` file. An `exports` map can permit one import path and reject another. Correct source code can therefore fail because the deployed package metadata differs from the metadata used during development.

Compiler configuration

Compiler configuration includes the selected `tsconfig.json`, inherited configuration, defaults, and command-line flags. It controls facts such as which files are checked, how strictly they are checked, how module specifiers are interpreted for checking, which JavaScript language target is emitted, and whether JavaScript is emitted at all.

A successful checker invocation proves only that the selected files satisfied the selected constraints under that compiler version and configuration. It does not prove that the file was emitted, copied into the deployment, selected by the loader, or executed by Node. Another command can select another configuration and therefore produce different evidence.

External data

External data is any value arriving from outside the checked construction path. Examples include HTTP bodies and headers, environment variables, command-line arguments, file contents, database rows, queue messages, and responses from other services.

The source may declare an interface describing the hoped-for shape, but the sender does not become governed by that declaration. Runtime parsing and validation must establish the facts the application needs. Even after validation, later external data can differ; evidence about one value does not validate the next value.

Storage systems

Storage systems include databases, caches, files, queues, and object stores. They own facts that TypeScript cannot manufacture: whether a record exists now, whether two writers conflict, whether a transaction committed, whether a uniqueness constraint is installed, and whether data survives a process restart.

A TypeScript repository interface describes permitted calls and result shapes for checked callers. The actual database and adapter must still implement that contract. If the production migration omitted a unique constraint, a perfectly typed interface does not restore it.

Process lifetime

Process lifetime is the path from process creation to termination. A production service must load and validate configuration, acquire resources, decide when it is ready for traffic, track background and request work, react to failures and shutdown signals, stop admitting work, wait or cancel according to policy, and release owned resources.

A request handler can be locally correct while the service remains operationally wrong. The process might announce readiness before the database is available, lose a background rejection, accept new work during shutdown, or exit before a buffered effect is flushed. TypeScript can describe lifecycle states and callback signatures; the running lifecycle controller must enforce their ordering.

What “component” means

In this book, a **component** is a named participant responsible for a particular operation or fact. The word does not necessarily mean a class, an npm package, a process, or a separately deployed microservice.

The TypeScript checker is a component because it performs assignability and control-flow checks. An application parser is a component because it inspects an unknown runtime value and either constructs trusted data or reports issues. The Node module loader is a component because it resolves and loads runtime modules. PostgreSQL is a component because it can enforce an installed constraint during participating writes. A lifecycle supervisor is a component because it controls readiness, admission, draining, and cleanup.

Naming the component prevents a vague statement such as “the system guarantees uniqueness.” A sharper statement is: “The PostgreSQL unique constraint rejects two participating rows with the same retained tenant and request key.” Now the enforcer, operation, scope, and remaining bypasses can be discussed.

What "failures remain possible elsewhere" means

Every check or enforcement mechanism has a boundary. "Elsewhere" means outside the named component's authority, time, data, or execution path.

Consider several forms of elsewhere:

- **Another phase:** TypeScript accepted the source, but the Node loader cannot find the emitted file.
- **Another entry path:** The HTTP handler validates input, but a maintenance script writes directly to storage without using that parser.
- **Another value:** One request passed validation; the next request can still be malformed.
- **A later time:** Authorization was true when data was read, but ownership changed before the protected write.
- **Another process or deployment:** One process has an in-memory idempotency set; a second process does not share it.
- **Another authority:** Application code checks for a duplicate before insertion, but only the database can arbitrate two concurrent participating inserts reliably.
- **A stronger failure than the protocol covers:** Graceful shutdown closes resources after a signal, but a machine loss or forced kill can prevent that cleanup code from running.

This is why the introduction asks both "who supplies the guarantee?" and "what can still fail elsewhere?" The purpose is not pessimism. It is to avoid accidentally stretching one piece of evidence beyond the operations it governs.

How asynchronous execution is relevant to TypeScript

Asynchronous execution is JavaScript runtime behavior, but TypeScript analyzes the source that creates and consumes asynchronous operations.

For `finish`, TypeScript can check that callers receive a `Promise<number>`. It can reject a normal return of `"wrong"` because that string does not satisfy the declared fulfillment type. It can infer the type of the expression produced by `await`. It can describe an `AbortSignal`, a result union, a callback, or a lifecycle state machine.

`Promise<number>` does **not** say when the promise will settle. It does not say that settlement will succeed, because JavaScript promises can reject with arbitrary reasons and TypeScript does not provide checked exceptions. It does not say that the operation observes cancellation, owns cleanup, avoids duplicated effects, respects a concurrency limit, or settles before process shutdown.

The `await` boundary also matters because time passes between two pieces of one function. Other callbacks can run before the continuation resumes. A local value captured before `await` remains that value, but shared application state, a database record, authorization facts, or resource availability may have changed. TypeScript can keep a variable's type consistent while the real-world fact represented by the variable becomes stale.

The book therefore teaches asynchronous runtime mechanics before the TypeScript static model. You first learn what promises, `await`, scheduling, cancellation, and cleanup actually do. Then the TypeScript chapters show how to describe values and relationships without mistaking those descriptions for scheduling or lifetime enforcement. Later integration chapters combine the two.

The five regions and their chapter ranges

Yes: the five regions are presented in order, and each region is a contiguous chapter range. The current PDF table of contents lists chapters without displaying the five part headings, which makes the architecture harder to see than it should be.

Region 1 — Runtime values and program state: Chapters 1–5

- Chapter 1 establishes values, identity, aliases, mutation, and copying.
- Chapter 2 adds functions, lexical scope, closures, and `this`.
- Chapter 3 adds prototypes, classes, and object organization.
- Chapter 4 adds collections, iterators, generators, and transformations.
- Chapter 5 integrates those mechanisms into explicit state transitions and local-reasoning rules.

Region 2 — Asynchronous execution and resource lifetime: Chapters 6–9

- Chapter 6 establishes execution order, Jobs, callbacks, microtasks, timers, and the event loop.
- Chapter 7 develops promises, `async`, `await`, and dependent control flow.
- Chapter 8 connects errors, cancellation, cleanup, and resource ownership.
- Chapter 9 integrates scheduling and state into retries, bounded concurrency, idempotency, and reliable workflows.

Region 3 — TypeScript's static analysis: Chapters 10–15

- Chapter 10 introduces inference, annotations, assignability, and types as sets of possible values.
- Chapter 11 connects executed runtime checks to narrowing, `unknown`, and exhaustiveness.
- Chapter 12 separates structural compatibility from semantic identity.
- Chapter 13 uses generics to express relationships among checked positions.
- Chapter 14 derives new types with mapped, conditional, indexed, and related type operations.
- Chapter 15 integrates these tools into domain models and state machines that exclude selected invalid combinations for checked callers.

Region 4 — Runtime truth and the source-to-execution pipeline: Chapters 16–20

- Chapter 16 returns explicitly to erasure, assertions, runtime operators, and runtime truth.
- Chapter 17 validates external data and constructs trusted domain values.
- Chapter 18 explains module scope, ESM, CommonJS, package metadata, and loaders.
- Chapter 19 connects compiler configuration, checking, emission, and module resolution.
- Chapter 20 provides a phase-ordered method for diagnosing disagreements from source through execution.

Region 5 — Production correctness: Chapters 21–26

- Chapter 21 allocates guarantees among types, tests, runtime assertions, constraints, and observations.
- Chapter 22 designs stable public errors and domain boundaries.
- Chapter 23 connects HTTP, validation, transactions, persistence, authentication, and authorization.
- Chapter 24 handles process lifecycle, observability, background work, and partial failure.
- Chapter 25 contrasts Java and TypeScript/JavaScript across the accumulated mechanisms.
- Chapter 26 combines all five regions in a production-shaped service and diagnosis capstone.

The order is cumulative, not a set of sealed boxes. Region 2 uses the mutable state from Region 1. Region 3 describes programs that still execute according to Regions 1 and 2. Region 4 shows where the descriptions stop and connects source to the artifact Node actually runs. Region 5 assigns each production guarantee to the component that can really supply or enforce it.

A type assertion can preserve a false runtime belief

Suppose this source passes TypeScript checking:

```
interface Config {
  port: number;
}

function startServer(port: number): void {
  console.log(port.toFixed(0));
}

const config = JSON.parse(process.env.APP_CONFIG ?? "{}") as Config;
startServer(config.port);
```

The checker accepts `config.port` as a number because the assertion instructs it to use `Config`. At runtime, however, `APP_CONFIG` might contain `{"port":"8080"}`. `JSON.parse` then produces an object whose `port` property is a string. The assertion emits no validator and changes no runtime value, so `port.toFixed(0)` throws because a string has no `toFixed` method.

Several components are now relevant. Node supplies `process.env` and executes the code. The loader must locate the module containing `startServer`. Package metadata influences module interpretation. Compiler configuration determines what was checked and emitted. The environment variable is external data and requires runtime validation. The server's socket is a host resource whose acquisition and release belong to process lifetime.

One "TypeScript passed" observation cannot replace all of those responsibilities.

Static analysis still contributes real evidence

It would also be wrong to conclude that TypeScript contributes nothing to runtime correctness. Static analysis can reject inconsistent calls, impossible checked combinations, missing branches, and broken relationships before a program is deployed. Those rejections remove many candidate defects.

The boundary is that TypeScript's evidence applies to the source positions, inputs, declarations, configuration, and escape hatches participating in that checker invocation. It does not itself execute the program, validate arbitrary external values, install a database constraint, settle a promise, or run cleanup. Static evidence is valuable precisely when its scope is stated accurately.

Reading the original passage with the repaired model

The triggering passage can now be read as follows:

JavaScript determines the operations that execute and the runtime values they produce. TypeScript checks source-level constraints before that execution, and most type-only information is absent from the emitted JavaScript. The deployed result still depends on Node supplying host facilities; the loader and package metadata selecting and interpreting modules; compiler configuration selecting what was checked and emitted; runtime parsers dealing with external data; storage authorities enforcing persistence facts; and lifecycle code controlling startup through shutdown. Each participant supplies evidence or enforcement only for its own operation, time, and scope. Failures remain possible through other phases, values, paths, processes, authorities, or later state changes.

The five regions then form an ordered path: Chapters 1–5 establish runtime state, Chapters 6–9 add time and lifetime, Chapters 10–15 develop TypeScript's static descriptions, Chapters 16–20 reconnect those descriptions to executed artifacts and data, and Chapters 21–26 allocate and integrate production guarantees.

Back to the production-components paragraph | Back to the five-region paragraph

Appendix 1.0 - Compressed thesis

1.0.1 - Argument values, parameters, and shallow spread

Triggered by: Chapter 1, compressed thesis, paragraph 2

An argument value is the result of evaluating a call-site expression

In this call:

```
showTotal(price + tax);
```

`price + tax` is the **argument expression**: the source expression written between the parentheses. JavaScript evaluates that expression for this particular call. If `price` is `10` and `tax` is `2`, evaluation produces the number `12`. That resulting `12` is the **argument value**.

The function declaration gives the receiving position a name:

```
function showTotal(total: number): void {
  console.log(total);
}
```

`total` is the **parameter**. A parameter is a new binding belonging to the function call. For this call, JavaScript initializes the `total` binding with the argument value `12`.

So the short sequence is:

```
argument expression at the call site
-> evaluation produces an argument value
-> the value initializes the function's parameter
```

JavaScript does not pass the source text `price + tax`, and it does not give the function control over the caller's variables `price` or `tax`. It passes the value produced by evaluating the expression.

An object argument still passes a value

Now consider an object:

```
const original = { points: 1 };

function update(account: { points: number }): void {
  account.points = 2;
  account = { points: 99 };
}

update(original);
console.log(original.points); // 2
```

At `update(original)`, the argument expression is `original`. Evaluating it produces a value that gives access to the object whose `points` property is initially `1`. The book calls this an object-access or **reference value**. JavaScript copies that value into the new parameter binding `account`.

Immediately after the call begins, `original` and `account` are two bindings whose values reach the same object. They are aliases. Therefore `account.points = 2` changes that shared object, and a later read through `original` observes `2`.

The next statement behaves differently:

```
account = { points: 99 };
```

It replaces the value in the call-local parameter binding. It does not replace the value in the caller's `original` binding. After that assignment, `account` reaches a new object while `original` still reaches the first object, whose `points` property is now `2`.

This is why the chapter avoids the loose statement "objects are passed by reference." JavaScript passes a value. That value may preserve access to an object, so copying it creates an alias. The copied access explains the visible mutation; the separate parameter binding explains why rebinding the parameter does not rebind the caller's variable.

Object spread creates a new outer object and copies property values

Object spread does not recursively duplicate everything reachable from an object:

```
const original = {
  name: "Ada",
  preferences: { theme: "dark" },
};

const copy = { ...original };

console.log(copy === original);                      // false
console.log(copy.preferences === original.preferences); // true
```

Evaluating `{ ...original }` creates a new outer object. It then copies the value of each included property into that object. The string value stored in `name` is copied. The value stored in `preferences` gives access to a nested object, so copying that value leaves both outer objects pointing to the same nested object.

The result is therefore **shallow**: the outer identity is new, but a nested identity can remain shared. Replacing `copy.name` does not change `original.name`. Mutating `copy.preferences.theme` does change the shared preferences object, so the change is visible through `original.preferences` as well.

At a call such as:

```
update({ ...original });
```

the entire spread expression is the argument expression. Evaluating it creates the new outer object and produces a value that reaches that new object. JavaScript passes that resulting value into the parameter. The parameter does not alias `original`'s outer object, but nested properties may still reach identities also reachable through `original`.

Array spread creates a new outer array and copies element values

Array spread follows the same identity pattern:

```
const first = { id: 1, selected: false };
const original = [first];
const copy = [...original];

console.log(copy === original);       // false
console.log(copy[0] === original[0]); // true
```

Evaluating `[...original]` creates a new array and copies each iterated element value into it. The two arrays are different outer objects. Their first elements nevertheless contain values that reach the same `first` object.

Changing the array structure is separate from changing a shared element. `copy.push(...)` changes only `copy`, and `copy[0] = anotherObject` replaces only the first slot of `copy`. By contrast, `copy[0].selected = true` mutates the one object reached from both arrays, so `original[0].selected` becomes `true` too.

Spread therefore protects only the newly created outer container. Whether deeper state is isolated depends on which copied values are primitives and which preserve access to objects.

Returning to the compressed thesis

Read "JavaScript passes argument values" as:

> For each function call, JavaScript evaluates every argument expression and uses the resulting value to initialize the corresponding parameter binding.

If the result is a number such as `12`, the parameter receives that number. If the result gives access to an object, the parameter receives a copied access value and initially reaches the same object. If the argument expression itself creates a spread copy, the parameter reaches that new outer container while any copied nested access values may still reach older objects. In every case JavaScript passes the value produced by evaluating the argument expression.

Back to Chapter 1

1.0.2 - `const`, `readonly`, and `Object.freeze`

Triggered by: Chapter 1, compressed thesis, paragraph 2

`const` restricts one binding

A binding is the association between a name and the value currently held under that name. In this declaration, `session` names a binding and the binding initially contains a value that reaches an object:

```
const session = { attempts: 1 };
```

`const` prevents a later assignment from replacing the value in that binding:

```
session = { attempts: 0 }; // rejected: this would rebind session
```

It does not make the reached object immutable:

```
session.attempts += 1; // allowed: this changes a property on the object
```

Those statements have different assignment targets. The first targets the binding named `session`. The second targets the `attempts` property of an object reached through `session`. `const` restricts the first target, not the second.

The restriction belongs to the particular binding. Another binding can still hold the same object, and the object remains mutable unless some separate mechanism restricts it:

```
const first = { attempts: 1 };
const alias = first;

alias.attempts = 2;
console.log(first.attempts); // 2
```

Both bindings are `const`; neither is rebound. The shared object is mutated.

TypeScript `readonly` restricts checked writes through a typed path

`readonly` is a TypeScript checking rule. It tells the checker that selected property assignments should not be accepted through a particular type:

```
const mutable = { attempts: 1 };
const view: Readonly<{ attempts: number }> = mutable;

view.attempts = 2;    // TypeScript error
mutable.attempts = 2; // accepted
```

`view` and `mutable` still reach the same object. The difference is in the types through which the source code accesses it. The checker rejects the write through `view`; the writable alias `mutable` can still perform the write, and a later read through `view` observes the new value.

Most TypeScript type information is erased from the emitted JavaScript. `readonly` therefore does not install a runtime guard on the object. Untyped JavaScript, a writable alias, or code that bypasses the checker can attempt the same write.

The restriction is also only as deep as the declared type. `Readonly<T>` makes the listed properties of `T` readonly; it does not recursively transform every nested object:

```
type Settings = Readonly<{
  profile: { theme: string };
}>;

declare const settings: Settings;

settings.profile = { theme: "light" }; // TypeScript error
settings.profile.theme = "light";      // accepted: theme was not declared readonly
```

That is why the thesis says `readonly` rejects **selected writes**. The selected write depends on the declared type and the access path the checker is examining.

`Object.freeze` restricts one object during execution

`Object.freeze` is a JavaScript runtime operation. It acts on the actual object passed to it:

```
const frozen = Object.freeze({
  attempts: 1,
  options: { trace: false },
});
```

For this ordinary object, freezing prevents changes to its own properties. Code cannot successfully replace `attempts`, add another own property, delete an own property, or reconfigure those properties. This book runs ECMAScript modules, which are strict code, so an ordinary invalid assignment such as the following throws a `TypeError` at runtime:

```
frozen.attempts = 2;
```

Freeze is shallow. The `options` property cannot be replaced on `frozen`, but its value still reaches a separate nested object. That nested object was not passed to `Object.freeze`:

```
frozen.options.trace = true; // succeeds: options itself was not frozen
```

Every alias to `frozen` reaches the same frozen outer object, so changing the variable name used to access it does not bypass the runtime restriction. An alias to the nested `options` object can still mutate that nested object because it is a different, unfrozen identity.

The three mechanisms answer different questions

Mechanism	What it targets	When it acts	What it does not establish
`const`	One named binding	During checked source analysis and JavaScript execution	That the reached object or its nested objects cannot mutate
TypeScript `readonly`	Selected writes through a checked type or access path	During TypeScript checking	That an actual runtime object rejects writes, or that writable aliases do not exist
`Object.freeze`	One actual object's own properties	While JavaScript executes	That separately reachable nested objects are frozen

The compact sentence can therefore be expanded into three questions:

- **Can this name be assigned a different value?** `const` says no for that binding.
- **Will TypeScript accept this property write through this typed access path?** `readonly` may say no during checking.

- **Will this actual object accept a change to one of its own properties while the program runs?** `Object.freeze` may say no at runtime.

Using more than one mechanism can be useful because they communicate and enforce different facts. A `const` binding can prevent accidental rebinding, a readonly type can reject writes in checked callers, and a frozen object can reject selected writes from executing code. Combining them still does not automatically freeze an entire reachable object graph or prove that no mutable alias to a nested object exists.

Back to Chapter 1

1.0.3 - Ownership, aliases, and mutation authority

Triggered by: Chapter 1, compressed thesis, paragraph 2

Ownership answers who controls a value or resource

In this chapter, **ownership** means an enforced or deliberately designed relationship that answers questions such as:

- Which part of the program is allowed to mutate this object?
- May several parts of the program retain access to it?
- Can one part assume that no writable alias exists elsewhere?
- Who is responsible for preserving the object's invariant?

Merely having a variable that reaches an object does not make that variable the object's unique owner. JavaScript permits the access value to be copied freely:

```
const profile = { name: "Ada", settings: { theme: "dark" } };
const firstAlias = profile;
const secondAlias = profile;
```

All three bindings reach the same outer object. The language does not designate one binding as the owner and the others as temporary borrowers. It also does not automatically prevent any alias from mutating the shared object.

An ownership system stronger than ordinary JavaScript aliasing could enforce rules such as "after transferring this value, the previous holder may no longer use it" or "while one mutable access path exists, no competing access path may mutate the same state." JavaScript and TypeScript do not generally enforce those rules for ordinary objects.

`const`, `readonly`, and freeze do not select an owner

The three mechanisms from the previous sentence restrict operations, but none answers "which part of the program uniquely controls this object?"

`const` restricts one binding:

```
const firstAlias = profile;
```

It prevents rebinding `firstAlias`. It does not stop another alias from mutating `profile`, and it does not prove that `firstAlias` is the only access path.

TypeScript `readonly` restricts selected writes through a checked type:

```
const writable = { name: "Ada" };
const view: Readonly<{ name: string }> = writable;

writable.name = "Grace";
console.log(view.name); // "Grace"
```

The readonly view did not become an owner. In fact, it cannot assume that another writable alias does not exist.

`Object.freeze` restricts changes to one actual object's own properties. It applies the restriction to every alias that reaches that object; it does not grant one alias special authority. It is also shallow, so a nested object can remain mutable:

```
const profile = Object.freeze({
  settings: { theme: "dark" },
});

const settingsAlias = profile.settings;
settingsAlias.theme = "light"; // nested object was not frozen
```

Freeze restricted the outer `profile` object. It did not establish ownership over the entire graph reachable through `profile`.

Reachable state includes nested objects

An object is **reachable** when the program has an access path to it. Starting from `profile`, the path `profile.settings` reaches the nested settings object. If another binding stores that nested value, the nested object remains reachable through both paths:

```
const settingsAlias = profile.settings;
```

The phrase "prevents every alias from mutating reachable state" would be a very strong claim. It would require controlling not only assignments through `profile`, but also every copied alias to `profile`, every alias to `profile.settings`, and every other mutable object farther down the graph.

A shallow readonly type or shallow freeze does not provide that control. Even a recursively readonly type remains a TypeScript checking rule rather than proof that unchecked runtime code or an existing writable alias cannot mutate an object. A recursively frozen graph can reject many runtime writes, but that is still a runtime immutability policy, not a rule that identifies one unique owner allowed to use the value.

JavaScript programs can design an owner without receiving an ownership proof

A program can choose one component to act as the owner of mutable state. For example, a closure can retain an account record and expose operations rather than the record itself:

```
function createAccount(initialBalance: number) {
  let balance = initialBalance;

  return {
    deposit(amount: number) {
      if (amount <= 0) throw new Error("amount must be positive");
      balance += amount;
    },
    snapshot() {
      return { balance };
    },
  };
}
```

The closure is designed as the state owner: it retains the mutable binding, checks the transition, and returns a new snapshot instead of exposing that binding. Callers cannot directly assign `balance` because no access path to the binding is returned.

This is an API and encapsulation design, not a general proof supplied by the JavaScript or TypeScript object model. If the implementation leaks a mutable internal object, stores an uncontrolled alias, or accepts a callback that can mutate the same state unexpectedly, the intended ownership boundary can fail.

Resource ownership later in the book

Later chapters also use *ownership* for resources such as files, sockets, transactions, timers, or background tasks. There the central question is responsibility: **which component acquired the resource and therefore must ensure that it is closed, committed, rolled back, cancelled, or awaited?**

The two uses are related. State ownership identifies who controls mutation and invariants. Resource ownership identifies who carries the lifetime and cleanup obligation. In both cases, saying "someone owns it" is incomplete unless the program names the owner, the operations under its control, the duration of the responsibility, and how that responsibility ends or transfers.

Returning to the compressed thesis

Read "None of them introduces ownership" as:

> `const`, `readonly`, and `Object.freeze` each restrict particular operations, but none makes JavaScript track one unique controller of an object, prohibit every competing alias, or assign responsibility for all reachable mutable state.

The sentence is warning against a false inference. A restricted access path is not the same as exclusive control of the object graph.

Back to Chapter 1

1.0.4 - The ECMAScript specification and Reference Records

Triggered by: Chapter 1, compressed thesis, paragraph 3

The ECMAScript specification is the rulebook for JavaScript language behavior

ECMAScript is the standardized language commonly called JavaScript. The **ECMAScript specification** is the formal document that defines the language's syntax and semantics: which source text is valid, what language operations mean, how expressions are evaluated, how objects and functions behave, and which observable results a conforming implementation must produce.

Node.js, Chrome, Firefox, Safari, and other hosts contain JavaScript engines that implement these rules. The specification is not one of those engines. It is also not a tutorial, a library, a compiler configuration, or a file that your program imports.

A useful mental model is:

> The specification is a precise rulebook and a shared pseudocode model that lets different engine implementations agree about JavaScript behavior.

The specification often explains a rule with named algorithms, abstract operations, and internal records. These are tools in the standard's model. An engine must preserve the required behavior, but it does not have to copy the specification's pseudocode line for line or allocate a physical object for every record the specification describes. An engine may optimize aggressively as long as the program cannot observe a forbidden difference.

When reading the specification, ask three questions:

1. What source construct or operation is this algorithm defining?
2. What conceptual information does the algorithm need to carry into its next step?
3. Which final effects or values can ordinary JavaScript observe?

Do not assume that every capitalized specification term names a JavaScript value available to your program. Many such terms describe the standard's internal reasoning machinery.

A location is a place that can be read from or written to

Consider:

```
let score = 10;
score = 11;
```

The name `score` does two related jobs. In an expression such as `console.log(score)`, the program needs the value currently stored in the `score` binding. On the left side of `score = 11`, the program needs to identify the binding that should receive a new value.

The current number `10` is not enough to perform the assignment. Many bindings could contain the number `10`; changing the number itself would not identify which binding should be updated. The evaluation machinery therefore needs a conceptual description of the **place** involved.

In this discussion, a *location* means an assignable place such as:

- a binding selected by an identifier like `score`; or
- a property selected by a base object and key, as in `profile.name`.

This does not require you to imagine a raw hardware memory address. It means the language has identified enough information to perform the appropriate read or write.

Evaluating an identifier can first identify its binding

The phrase "evaluating an identifier" means applying the language rules to an identifier expression such as `score` during program execution.

At a high level, evaluating `score` first resolves which binding that name denotes in the current lexical environment. The specification represents this intermediate result with a **Reference Record**. The record conceptually says something like:

```
the resolved base or environment
the referenced name: "score"
information needed to apply the correct language rules
```

If the surrounding expression needs the stored value, the specification's `GetValue` operation reads through that Reference Record. If an assignment needs to update the binding, `PutValue` writes through it.

For this statement:

```
score += 1;
```

the simplified specification story is:

1. Evaluate `score` and obtain an internal Reference Record identifying its binding.
2. Read the current value through that reference: `10`.
3. Add `1`, producing `11`.
4. Write `11` through the same reference into the `score` binding.

The Reference Record is the **internal result** mentioned by the chapter. It is internal because ordinary JavaScript cannot store it in a variable, print it, pass it to a function, or inspect its fields. It is a conceptual result used by later specification steps.

Evaluating a property can identify an object and property key

Now consider:

```
profile.name = "Ada";
```

To perform this assignment, JavaScript must identify both the object and the property to change. At a simplified level:

1. Evaluate `profile` and read the object value stored in that binding.
2. Evaluate `profile.name` as a property location.
3. Produce an internal Reference Record whose base is the object and whose referenced name is `"name"`.
4. Write the string value `"Ada"` through that reference.

Again, the Reference Record is not the object and is not the string `"Ada"`. It is the specification's temporary description of the property location needed for the assignment.

The same property expression can be read:

```
console.log(profile.name);
```

The location-identifying step is similar, but the surrounding expression asks for the property value. The specification therefore applies a read operation rather than using the location as an assignment target.

"An internal result of evaluating locations" in plain language

The original phrase can now be expanded as follows:

> When JavaScript evaluates an expression such as the identifier `score` or the property access `profile.name`, the specification may first produce a private conceptual record identifying which binding or property the expression refers to. Later specification steps use that record to read the current value or perform a write.

The phrase does not mean that `score` or `profile.name` evaluates to a Reference Record that your JavaScript program can capture. It describes intermediate machinery inside the specification's account of evaluation.

A Reference Record is not the book's reference value

This distinction becomes visible in a familiar example:

```
let first = { points: 1 };
const second = first;

first = { points: 99 };
console.log(second.points); // 1
```

When the initializer `first` is evaluated in `const second = first`, the specification can use a Reference Record to identify the `first` binding, then `GetValue` reads the object value stored there. That object value initializes the new `second` binding.

The Reference Record identifying the `first` binding is not copied into `second`. If it were, rebinding `first` could make `second` follow the caller's binding to the new object. That does not happen. The two bindings separately contain values that initially reach the same object; later rebinding `first` changes only `first`.

The book uses **reference value** as an operational teaching phrase for the ordinary copied value that preserves access to one object identity. The specification's **Reference Record** is different: it is internal location machinery used while evaluating identifiers and properties. The similar names are unfortunate, but the jobs are distinct.

Keep this compact comparison available:

Term	Mental model	Available to JavaScript code?	Main job
Object value, described by the book as an object-access or reference value	A value that preserves access to one object identity when copied	Yes	Explains aliasing, identity, and shared mutation
Specification Reference Record	A private evaluation record identifying a binding or property location	No	Lets later specification steps read from or write to the correct place

Back to Chapter 1

Appendix 1.1 - Bindings and values

1.1.1 - Live bindings and ordinary object properties

Triggered by: Chapter 1, destructuring paragraph

A binding and a property are different storage relationships

Start with two expressions that happen to use the word `count`:

```
count
state.count
```

The identifier `count` is resolved through a **binding** in the current scope. A declaration such as `const count = 1` creates that binding and initializes it with a value.

The expression `state.count` performs **property access**. JavaScript first evaluates `state` to obtain an object, then asks that object for the property whose key is `"count"`. The property belongs to the object; it is not the lexical binding named `count`.

Those two locations can contain equal values without being connected:

```
const state = { count: 1 };
const count = 1;

state.count = 2;
console.log(count); // 1
```

Changing the property does not change the separate binding. Giving both locations the same visible name does not make them one storage location.

A live binding reads through to another module's binding

In JavaScript module terminology, a **live binding** is a binding whose reads continue to obtain the current value of an exported binding rather than a value copied once during import.

Suppose one module exports a variable and a function that reassigns it:

```
// counter.ts
export let count = 0;

export function increment(): void {
  count += 1;
}
```

Another module imports that name:

```
// report.ts
import { count, increment } from "./counter.js";

console.log(count); // 0
increment();
console.log(count); // 1
```

The import did not copy `0` permanently into an unrelated local variable. The imported name is connected indirectly to the exporter's `count` binding. When `report.ts` evaluates `count` again after `increment()`, it obtains the exporter's current value.

"Live" does not mean that an event is pushed to the importing module, that a callback runs, or that the importer polls on a timer. It describes the binding relationship: later reads observe later values of the exporting binding. The importer also cannot assign to the imported name itself.

This becomes especially visible when the exporter rebinds a variable to a different object:

```
// current-user.ts
export let currentUser = { name: "Ada" };

export function replaceUser(): void {
  currentUser = { name: "Grace" };
}
```

Code that imports `currentUser` reads the new object after `replaceUser()` runs. That is different from merely sharing one object and observing mutations to it.

Object destructuring performs a property read and initializes a new binding

For the relevant part of this chapter, this destructuring declaration:

```
const { count } = state;
```

can be reasoned about in three steps:

1. Evaluate `state` to obtain the source object.
2. Read that object's `"count"` property and obtain its current value.
3. Create the new local binding `count` and initialize it with that value.

The result of step 2 is used once in step 3. JavaScript does not install a permanent read-through connection from the new binding back to `state.count`.

```
const state = { count: 1 };
const { count } = state;

state.count = 2;
console.log(count); // 1
```

This is why the chapter says destructuring did not create a live binding. The later property write changes `state.count`; it does not redirect or reinitialize the local `count` binding.

Even an accessor property does not change the destructuring rule. Destructuring calls the getter while reading the property, then stores the getter's returned value in the new binding. It does not arrange for every later use of the binding to call the getter again.

An ordinary object property is a normal property on a regular object

In the sentence that triggered this repair, **ordinary object properties** means properties such as `count` and `options` on the object created by this literal:

```
const state = {
  count: 1,
  options: { trace: false },
};
```

For the properties in this example, each property key has an associated data-property value. Evaluating `state.count` reads the value currently stored under `"count"`; assigning to `state.count` replaces that property value, subject to the property's descriptor and the usual JavaScript rules.

The word **ordinary** distinguishes this common object-literal behavior from specialized language mechanisms. A module namespace object, for example, is an exotic object whose exported properties reflect module bindings. A `Proxy` can also customize what happens during a property read. Neither mechanism is present in the chapter's `state` object.

You do not need to imagine a property as a lexical variable hidden inside the object. The useful distinction is operational:

- evaluating `count` performs identifier binding lookup;
- evaluating `state.count` performs object property access;
- assigning to `count` targets the binding, if that binding is assignable;
- assigning to `state.count` targets the object's property.

Shared object identity can look live without being a live binding

The original example destructures both a primitive and an object value:

```
const state = {
  count: 1,
  options: { trace: false },
};

const { count, options } = state;
```

The local `count` binding and the `state.count` property receive the same primitive number value. A later replacement of the property does not affect the binding.

The local `options` binding and the `state.options` property instead contain object values that reach the same object identity. A mutation through either access path is visible through the other:

```
options.trace = true;
console.log(state.options.trace); // true
```

That observation comes from aliasing, not from a live binding. Replacing the property makes the difference obvious:

```
const state = { options: { trace: false } };
const { options } = state;

state.options = { trace: true };

console.log(options.trace);       // false
console.log(state.options.trace); // true
```

`options` still reaches the original nested object. `state.options` now reaches a new object. If destructuring had created a live binding back to the property, reading `options` would have followed that replacement; it does not.

The compressed sentence therefore means:

> Object destructuring reads each requested property at that moment and initializes a new local binding with the resulting value. It does not make the local name read through to the property on every later access. Imported module names use a different language mechanism: they remain connected to the current values of their exported bindings.

Back to the destructuring paragraph

Appendix 1.2 - Object identity and reference values

1.2.1 - Domain IDs and new runtime object identities

Triggered by: Chapter 1, object-identity explanation and Chapter 1, domain-ID comparison

A domain ID identifies something in the application's subject matter

Here, **domain** means the subject matter the program models. It does not mean an internet domain name. In an application that manages users, users are part of the domain. In an ordering system, orders, products, and customers may be domain entities.

A **domain ID** is a value the application uses to say which domain entity some data refers to. Examples include:

- a user ID such as `"17"`;
- an order ID such as `"order-842"`;
- a generated UUID;
- or a composite identity such as a tenant ID paired with an invoice number.

The ID is ordinary runtime data: often a string or number stored in an object property. Its special meaning comes from the application's rules. JavaScript does not inspect `userId: "17"` and automatically understand that the string identifies a user.

A **database identifier** is an ID used to distinguish or retrieve a stored record. A database primary key is a common example. A domain ID and database identifier are often the same value, but they need not be. An application might use a public order UUID as its domain ID while the database also maintains a private numeric row key. The chapter's immediate point does not depend on that distinction: neither kind of application-level identifier is JavaScript object identity.

Runtime object identity identifies one particular object

Every separately created object has its own runtime identity. Consider two objects that carry the same domain ID:

```
const cachedUser = { userId: "17", name: "Ada" };
const fetchedUser = { userId: "17", name: "Ada" };

console.log(cachedUser === fetchedUser); // false
console.log(cachedUser.userId === fetchedUser.userId); // true
```

The two object literals created two objects, so `cachedUser === fetchedUser` is false. The application may interpret both objects as representations of the same user because both contain the domain ID `"17"`. Those are two different questions:

- **Same runtime object?** Compare the object values by identity. Here the answer is no.
- **Same domain entity?** Compare according to the application's domain rule. If `userId` is the authoritative identity for a user, the answer is yes.

Runtime object identity normally lasts only as long as that particular object exists in that particular execution. A domain ID can survive serialization, database storage, network transfer, process restarts, and the creation of many different in-memory representations.

The converse also matters. An object does not need a domain ID to have runtime identity:

```
const temporaryOptions = { trace: false };
const alias = temporaryOptions;

console.log(alias === temporaryOptions); // true
```

This options object has no application property identifying a durable domain entity. It still has runtime identity, and both bindings reach that same identity.

Assignment stores a value; the right-hand side may create an identity

It is common to say that an assignment "created a new object," but the precise sequence is useful. An assignment has a target on the left and an expression on the right. JavaScript first evaluates the right-hand expression, then stores its resulting value in the target location.

If the right-hand side produces an already existing object value, assignment creates no new object:

```
const first = { userId: "17" };
const second = first;

console.log(first === second); // true
```

The object literal in the first line created one object. The second line evaluated `first`, obtained the existing object value, and initialized `second` with that value. The two bindings became aliases.

Now put an object literal on the right-hand side of a later assignment:

```
let current = { userId: "17", version: 1 };
const previous = current;

current = { userId: "17", version: 2 };

console.log(current === previous); // false
console.log(current.userId === previous.userId); // true
```

The final assignment proceeds conceptually like this:

1. Evaluate `{ userId: "17", version: 2 }`. The object literal creates a new object identity.
2. Produce the new object value.
3. Store that value in the `current` binding, replacing the value that binding previously held.

The assignment rebounded `current`; it did not mutate the old object. `previous` still reaches the older identity. Both objects nevertheless contain the same domain ID, so the application may treat them as two representations or versions of the same user.

This is why the most precise wording is: **evaluating the object-producing right-hand side creates the identity; the assignment stores the resulting value.** Object literals, array literals, function expressions, class expressions, and many constructor calls can produce new identities. Merely assigning or returning an existing object value does not.

Property assignment can combine mutation of one object with creation of another:

```
const profile = { settings: { trace: false } };
const originalSettings = profile.settings;

profile.settings = { trace: true };

console.log(profile.settings === originalSettings); // false
```

Evaluating `{ trace: true }` creates a new settings object. Assigning it to `profile.settings` mutates the existing `profile` object by replacing one property value. The outer `profile` identity remains the same; the nested settings identity changes.

Reading the original comparison precisely

The chapter's claim can now be read as follows:

> JavaScript object identity answers whether two evaluated object values reach the same particular runtime object. A domain or database ID is application data used to identify an entity or stored record. Several runtime objects may carry the same ID, and an object may have runtime identity without carrying any such ID.

When assignment appears near object creation, separate two operations: determine what evaluating the right-hand side creates or retrieves, then determine which binding or property receives the resulting value.

Back to the assignment and identity paragraph | Back to the domain-ID paragraph

1.2.2 - Class instances through base-shaped and `readonly` views

Triggered by: Chapter 1, class-instance identity paragraph

One instance can be reached through variables with different static types

Consider a class instance:

```
class AdminUser {
  constructor(
    public readonly id: string,
    public name: string,
    public permissions: string[],
  ) {}
  rename(name: string): void { this.name = name; }
}

const admin = new AdminUser("user-17", "Ada", ["deploy"]);
```

Evaluating `new AdminUser(...)` constructs one runtime object. That object has an `AdminUser` prototype, its own fields, and one runtime identity. The `admin` binding receives the value that reaches that instance.

TypeScript can let another binding receive the same value while describing it with a smaller type:

```
type UserSummary = {
  id: string;
  name: string;
};

const summary: UserSummary = admin;

console.log(summary === admin); // true
```

The assignment did not construct a `UserSummary` object. `UserSummary` is only a TypeScript type, so it has no runtime constructor and produces no emitted wrapper. TypeScript checked that the public members available on `admin` include the `id` and `name` required by `UserSummary`. JavaScript then copied the existing object value into `summary`. Both bindings reach the same instance.

The static types permit different checked operations. Through `admin`, TypeScript knows about `permissions` and `rename`. Through `summary`, checked source can use only the members promised by `UserSummary`:

```
console.log(admin.permissions);   // accepted
console.log(summary.name);        // accepted
// console.log(summary.permissions); // TypeScript error
```

That restriction is about what TypeScript can justify through the `summary` expression. The `permissions` property did not disappear from the runtime object.

What “base-shaped” means here

A **base-shaped type** describes the smaller, more general part of a value. The phrase can refer to an actual base class:

```
class Entity { constructor(public readonly id: string) {} }

class Customer extends Entity {
  constructor(id: string, public name: string) { super(id); }
}

const customer = new Customer("customer-4", "Lin");
const entity: Entity = customer;

console.log(entity === customer); // true
```

It can also refer more loosely to a structural type such as `UserSummary` that lists only the common members a caller needs. TypeScript usually checks public object types structurally: if the source value supplies the required members with compatible types, the assignment can be accepted even when no runtime conversion occurs.

In both cases, "base-shaped" describes the static view available through one expression. It does not mean JavaScript sliced the object down to its base fields.

A `readonly` view does not clone or freeze the instance

Now give another binding a readonly description:

```
type ReadonlyUserSummary = {
  readonly id: string;
  readonly name: string;
};

const view: ReadonlyUserSummary = admin;

console.log(view === admin); // true
// view.name = "Grace";       // TypeScript error through this path

admin.name = "Grace";       // accepted through the writable path
console.log(view.name);      // "Grace"
```

The `readonly` members tell TypeScript to reject selected property writes made through `view`. They do not create a readonly runtime object. They do not call `Object.freeze`, clone the instance, remove its methods, or prevent another writable alias from changing it.

The final log prints `"Grace"` because `view` and `admin` still reach the same runtime object. Mutation through `admin` changes that object, and a later read through `view` observes the changed state.

The word **variable** in the original sentence means a binding whose uses TypeScript checks with the annotated type. It does not mean the object acquired a different identity or permanent runtime category when placed in that binding.

The annotation changes checked access, not runtime identity

Keep the sequence explicit:

1. `new AdminUser(...)` creates one instance with one runtime identity.
2. Assigning `admin` to `summary`, `entity`, or `view` copies the existing object value into another binding.
3. Each binding's TypeScript type determines which operations checked source may perform through that expression.
4. At runtime, all accepted expressions still reach the original instance.

This is the sense in which a class instance "retains its identity." A narrower, base-shaped, or readonly type can restrict the checker's view without performing a JavaScript conversion.

Back to the class-instance identity paragraph

1.2.3 - Heap fragments as alias graphs

Triggered by: Chapter 1, heap-fragment paragraph

The graph is a picture of relationships during one execution

A **graph** is a collection of nodes connected by edges. In this chapter's reasoning model:

- an object is drawn as a **node**;
- a binding, object property, array element, map entry, or similar access path can carry an **edge** to an object node;
- and a primitive value such as a number or string can be written directly beside the binding or property that contains it.

This is not initially a claim about how an engine arranges bytes. It is a compact picture of relationships that affect observable JavaScript behavior: which paths reach which objects, which paths are aliases, and which mutations those paths can observe.

Consider:

```
const user = {
  name: "Ada",
  settings: { theme: "dark" },
};

const alias = user;
const settings = user.settings;
```

A useful graph is:

```
binding user  ----+
                  +--> Object A
binding alias ----+      name: "Ada"
                         settings -----> Object B
                            theme: "dark"

binding settings ----------------------> Object B
```

`user` and `alias` are aliases because both binding edges reach Object A. The `settings` property of Object A and the separate `settings` binding both reach Object B, so those are aliases for the nested object.

The strings `"Ada"` and `"dark"` are primitive values. For the purpose of predicting shared mutation, they can be written directly as property contents. The model does not need to draw mutable object nodes for them.

"Heap" means the runtime storage in which objects live

Programmers commonly use **heap** for the area of runtime storage where dynamically created objects live and are managed. A JavaScript engine normally allocates and garbage-collects object storage, but the ECMAScript language rules do not require your program to observe a particular address or physical arrangement.

For this chapter, the useful mental model is simply:

> Objects created during execution have identities and remain available while the running program can still reach them through relevant paths.

The word **running** emphasizes that the graph describes one execution at one moment. The graph changes as the program creates objects, rebinds variables, replaces properties, or loses the last path to an object.

A fragment is only the relevant part of the running state

A real process can contain enormous numbers of objects, bindings, module records, internal host objects, and other runtime structures. Drawing the whole thing would be useless.

A **heap fragment** is the small portion relevant to the prediction being made. In the example, the fragment includes three bindings and two objects because those are sufficient to explain the aliases and mutations under discussion. It omits unrelated arrays, functions, modules, and engine machinery.

This is like drawing only the streets needed to explain one route rather than reproducing an entire country's road network.

Mutation follows an existing edge path

Now execute:

```
alias.settings.theme = "light";

console.log(user.settings.theme); // "light"
console.log(settings.theme);      // "light"
```

Starting from `alias`, follow its edge to Object A, then follow the `settings` property edge to Object B. The assignment changes Object B's `theme` property. Reads starting from either `user.settings` or the separate `settings` binding reach that same node, so both observe `"light"`.

The graph makes the result visible without using the vague rule "objects are shared somehow." It identifies the exact shared identity and the paths that reach it.

Rebinding moves one edge; it does not rename the old object

Suppose `alias` were declared with `let` and later rebound:

```
let alias = user;
alias = { name: "Grace", settings: { theme: "light" } };
```

Evaluating the second object literal creates new object nodes. The assignment then changes the edge from the `alias` binding so that it reaches the new outer object. The `user` binding still reaches Object A. Rebinding did not change Object A's identity or make `user` follow `alias`.

That transition can be pictured as:

```
before:  user --> Object A <-- alias

after:   user --> Object A     alias --> Object C
```

Shallow spread creates one node while preserving selected edges

The same model explains a shallow copy:

```
const copy = { ...user };
```

The object literal with spread creates a new outer object node. It copies `user.name`'s primitive string value and copies the object value stored in `user.settings`. Consequently, `copy` reaches a new outer identity while `copy.settings` still reaches Object B:

```
user --> Object A -- settings --> Object B

copy --> Object C -- settings --> Object B
```

This graph shows both truths at once: `copy !== user`, yet `copy.settings === user.settings`.

The graph is semantic, not a literal engine diagram

An engine may use addresses, handles, compacting garbage collection, hidden classes, pointer compression, escape analysis, or other implementation techniques. It may optimize some allocations in ways that are not directly visible to the program. The chapter's graph commits to none of those details.

Its nodes mean "distinct object identities as JavaScript can observe them." Its edges mean "a binding or contained value currently provides access to that identity." Use the model to predict equality, aliasing, rebinding, mutation, and shallow copying. Do not treat the drawing as a map of exact memory addresses.

The original sentence therefore means:

> Draw only the currently relevant bindings and objects. Give every distinct object its own node, and draw every access path to the node it reaches. Multiple paths ending at one node are aliases.

Back to the heap-fragment paragraph

1.2.4 - Object equality, operands, and reachable subgraphs

Triggered by: Chapter 1, heap-fragment and object-equality paragraph

The operands are the values produced on the two sides of `===`

In an expression such as:

```
first === second
```

`first` and `second` are the two **operand expressions** of `===`. JavaScript evaluates each expression before applying the equality operation. If both bindings contain object values, evaluation produces two object-valued operands.

The operand is not the spelling `first` or `second`. It is the value obtained by evaluating that expression during this execution. More complicated expressions work the same way:

```
getCurrentUser() === cache.get("user-17")
```

JavaScript calls `getCurrentUser()`, evaluates the method call on the right, and then compares the two resulting values. The comparison does not care which source expressions produced them once those values are available.

"Designate the same node" means reach the same object identity

In the chapter's graph, every distinct object identity receives its own node. An object value **designates** a node when it provides access to that particular object.

```
const first = { name: "Ada" };
const alias = first;

console.log(first === alias); // true
```

Evaluating `first` and `alias` produces two copied object values, but both values reach the same object identity:

```
first --+
        +--> Object A { name: "Ada" }
alias --+
```

For object operands, `===` answers whether the two resulting values designate that same node. Here they both designate Object A, so the result is `true`.

This wording applies to object equality. Primitive operands follow their primitive equality rules instead; the number `3` does not need a mutable object node in this model.

A reachable subgraph includes nested objects

Start at one object node and follow its property edges to other objects. Continue following relevant object-valued properties. The starting node together with the nodes and edges reachable from it forms a **reachable subgraph**.

For example:

```
const user = {
  name: "Ada",
  settings: {
    theme: "dark",
  },
};
```

The graph reachable from `user` contains the outer user object, the nested settings object, and the edge between them:

```
user --> Object A
           name: "Ada"
           settings --> Object B
                          theme: "dark"
```

The word **subgraph** does not imply that JavaScript constructed a special subgraph object. It names the portion of the reasoning diagram reachable from one starting node.

Equal-looking nested data does not make the outer objects identical

Now create the same-looking data twice:

```
const left = {
  name: "Ada",
  settings: { theme: "dark" },
};

const right = {
  name: "Ada",
  settings: { theme: "dark" },
};

console.log(left === right);                    // false
console.log(left.settings === right.settings); // false
```

Four object literals were evaluated: two outer literals and two nested settings literals. They created four object identities. The two reachable subgraphs have the same visible property names and equal-looking primitive contents, but their corresponding object nodes are distinct.

`left === right` compares only the two outer operand identities. It does not recursively visit `name`, `settings`, or `theme`. Because the outer operands designate different nodes, the result is immediately `false`.

Distinct outer objects can share part of their subgraphs

Reachable subgraphs need not be completely separate:

```
const sharedSettings = { theme: "dark" };

const left = { name: "Ada", settings: sharedSettings };
const right = { name: "Ada", settings: sharedSettings };

console.log(left === right);                    // false
console.log(left.settings === right.settings); // true
```

The outer literals create distinct Objects A and C, while both `settings` properties contain values that reach Object B:

```
left  --> Object A -- settings --+
                                 +--> Object B { theme: "dark" }
right --> Object C -- settings --+
```

The reachable subgraphs overlap at Object B. Nevertheless, `left === right` is false because the two evaluated outer operands designate A and C. Comparing the nested operands produces true because both designate B.

"Look alike" hides a policy that the application must define

JavaScript's object `===` does not attempt built-in deep equality. A general deep comparison would need rules that the language cannot infer from appearance alone:

- Should prototypes have to match?
- Should non-enumerable and symbol-keyed properties count?
- Are two dates equal when their timestamps match?
- How should maps, sets, typed arrays, accessors, and functions be compared?
- Should two graphs with duplicated nested objects equal one graph with a shared nested object?
- How should cycles be detected and compared?

Different applications need different answers. A test helper, domain-specific comparator, or library can implement a chosen structural-equality policy, but `===` deliberately does not invent one.

The compressed sentence therefore expands to:

> Evaluate the expressions on both sides of the object comparison. If their resulting object values reach one particular runtime identity, object `===` is true. If they reach different identities, it is false without recursively comparing the properties and nested objects reachable from those identities.

Back to the object-equality paragraph

Appendix 1.5 - Copying and nested sharing

1.5.1 - Preserved identities and avoided graph traversal

Triggered by: Chapter 1, preserved-identity paragraph

Suppose `profile.identity` is an object that the update does not change. Selective copying may place the same object value into the new profile:

```
updated !== profile;                        // true
updated.preferences !== profile.preferences; // true
updated.identity === profile.identity;       // true
```

The `identity` object's **identity is preserved** because no new replacement was created for it. This can be useful when the program intentionally treats that object as stable, or when caches and equality checks use identity to recognize unchanged data. "Useful" is conditional: sharing is unsafe if later code mutates that object when the two versions were supposed to diverge.

An indiscriminate deep copy would recursively visit every reachable branch and recreate everything it can. Selective copying instead descends only through the branch being updated. Each object spread still reads the own enumerable properties of the container it copies; it simply does not recursively traverse the off-path objects stored in those properties.

Back to the preserved-identity paragraph

1.5.2 - Selective copying along an update path

Triggered by: Chapter 1, selective-copying paragraph

For `profile.preferences.theme`, the **path** is:

```
profile -> preferences -> theme
```

Selective copying creates a new object for each object on that path before the changed field: a new outer profile and a new preferences object. It can reuse objects reached through other branches:

```
const updated = {
  ...profile,
  preferences: {
    ...profile.preferences,
    theme: "light",
  },
};

updated !== profile;                            // true
updated.preferences !== profile.preferences;    // true
updated.identity === profile.identity;          // possibly true
```

The last comparison can be true because `identity` is not on the update path. Reuse is safe only if the program will not mutate that shared object in a way that should remain private to one version. “Its contract prevents unsafe mutation” means the API, types, encapsulation, or program rules make that sharing acceptable; selective copying does not enforce deep immutability by itself.

Copying too few nodes leaves a path into old state: copying only `profile` would leave `preferences` shared. Copying every reachable node would also recreate unrelated branches, discard identities that callers may rely on, and do unnecessary work. The practical rule is: copy the root and every containing object down to the changed field; reuse off-path branches only deliberately.

Back to the selective-copying paragraph

1.5.3 - JSON serialization is a representation change, not general cloning

Triggered by: Chapter 1, JSON-serialization paragraph

The commonly suggested “JSON deep copy” is:

```
const copy = JSON.parse(JSON.stringify(value));
```

`JSON.stringify` does not copy a JavaScript object graph directly. It converts supported information into JSON text. `JSON.parse` then constructs new arrays, ordinary objects, and primitive values from that text. The new containers make the result look like a deep copy, but the round trip preserves only what JSON can represent.

For example, dates normally become strings; object properties containing `undefined`, functions, or symbols are omitted; `Map` and `Set` do not preserve their entries by default; `BigInt` causes an error; prototypes, methods, property descriptors, symbol keys, and non-enumerable properties are not represented; and cycles cause an error. Getters and `toJSON` methods may run and choose the serialized result.

The round trip also loses alias structure:

```
const shared = { enabled: true };
const value = { first: shared, second: shared };
const copy = JSON.parse(JSON.stringify(value));

value.first === value.second; // true
copy.first === copy.second;   // false
```

The same source object was written into the JSON text twice, so parsing created two separate objects. Use this pipeline when JSON is the desired interchange or storage representation. If the goal is cloning, choose a cloning mechanism only after deciding which types, cycles, prototypes, and shared identities must survive.

Back to the JSON-serialization paragraph

Appendix 1.7 - Consequences for program design

1.7.1 - Identity, encapsulation, and controlled mutation

Triggered by: Chapter 1, controlled-mutation paragraph

The paragraph gives two reasons mutation can have a manageable reasoning boundary.

When **identity is the abstraction**, the object represents one continuing thing whose state changes over time. A connection object, for example, may remain the same connection while moving from `connecting` to `open` to `closed`. Code may have registered listeners or retained that particular handle, so replacing it with a new object could be less faithful than changing the state of the existing one.

When change is **tightly encapsulated**, mutable state exists but only a small, named owner can reach or modify it. The examples use that idea in different ways:

- A private cache mutates entries, but callers see the operation's result rather than the cache object.
- A parser cursor advances while one parser consumes one input; unrelated code does not receive the cursor.
- A connection state machine changes one connection through controlled transitions such as `connecting -> open -> closed`.
- A builder may fill a new object while no other code can reach it. **Publication** occurs when the object is returned, stored somewhere shared, or otherwise made reachable by another part of the program. It must be valid before that happens.

The final questions locate the boundary. **Who can observe it?** asks which code can read the changing state. **Through which aliases?** asks which bindings, properties, collections, or closures reach the same object. **During what lifetime?** asks how long those paths remain usable, including across asynchronous work. **What invariant controls it?** asks which rule every permitted change must preserve, such as a cursor staying within its input or a connection taking only allowed state transitions.

Mutation is easier to justify when the owner is clear, aliases are limited and known, the mutable lifetime is bounded, and all operations preserve a stated invariant. The same property write becomes harder to reason about when arbitrary code can retain and mutate the object for an unknown duration.

Back to the controlled-mutation paragraph

Glossary

This glossary fixes the sense of load-bearing terms used across chapters. Later chapters may extend a definition for a new context, but they must state the extension rather than silently replace the earlier meaning.

Abort signal — A Web-platform and Node runtime object that records whether cancellation was requested, retains a reason, and notifies observers. The signal does not stop an operation unless that operation observes it and cooperates.

`any` — A TypeScript type that suppresses many ordinary checking constraints for the affected expression and allows unchecked assumptions to propagate through property access, calls, assignments, and results. It is not a runtime value category.

Alias — A binding, property, or collection element whose value gives access to the same object identity as another access path.

Annotation — Written TypeScript syntax that declares a target type for a binding, parameter, property, return position, or other checked location. It asks the checker to verify relevant source values; it does not convert or validate runtime values.

Application-level race — An ordering-dependent interaction between logical operations whose steps interleave across Jobs, callbacks, or suspension boundaries. It does not require two JavaScript statements to execute simultaneously.

Artifact identity — Evidence that distinguishes one build or deployment artifact from another, commonly including path, contents or a semantic marker, cryptographic hash, producer command and version, and copy or deployment context. A timestamp alone does not establish identity or freshness.

Ambient declaration — A TypeScript description of a value or implementation expected to exist elsewhere. It affects checking but emits no binding or implementation, so its accuracy remains an external contract.

Assignability — TypeScript's relation for deciding whether a source type may be used where a target type is expected. For ordinary structural object comparison, the source must supply compatible members required by the target, subject to contextual and special rules. Assignability is not runtime conversion or validation.

Assertion — TypeScript syntax that instructs the checker to use a stated type for an expression within the language's permitted assertion rules. A type assertion supplies no runtime evidence and emits no runtime check.

Assertion function — A function whose TypeScript signature states that normal return establishes a condition or refined type. The implementation must perform the required runtime checks or otherwise justify the assertion; TypeScript trusts the signature rather than proving its logic.

Authentication — A credential authority's operation for establishing a principal identity under its stated mechanism and current evidence. Authentication does not grant permission for an action or prove ownership of a resource.

Authorization — A policy decision over a current principal, action, resource, state, and context at a protected operation. Authorization can become stale when participating facts change outside its concurrency protocol.

Authority — The named component or owner able to inspect, enforce, produce, or repair a particular fact within a stated boundary. An authority supplies no guarantee about operations or values outside that boundary.

Background task — Work started outside one request's direct return path that still requires a named owner, start condition, cancellation protocol, tracked settlement, failure policy, cleanup path, and effect/idempotency

boundary.

Binding — A named association maintained by an environment, such as one introduced by `const`, `let`, a parameter, or an import. A binding contains a value; the binding is not the value.

Bivariance — A TypeScript compatibility allowance in which a parameterized position can be accepted in either assignability direction. Method parameter checking is an important boundary where this can admit a target-authorized call that the implementation cannot safely handle.

Bounded concurrency — An application admission contract that allows no more than a stated number of logical operations to remain active according to a named active-count definition. Promise aggregation alone does not supply this contract.

Call-atomic — A qualified in-memory guarantee: a synchronous transition publishes no partial domain change before returning, and another queued job on the same JavaScript agent does not preempt that call. It does not imply rollback after arbitrary exceptions, durability, database isolation, or cross-agent exclusion.

Cancellation — A cooperative protocol in which one component requests that work stop and the operation observes that request, stops supported future work, and performs its cleanup. Cancellation does not automatically reverse completed effects.

Cleanup failure — A throw or promise rejection produced while releasing a resource, removing a listener, or performing another required lifetime-closing operation. Preserve it separately when primary work also failed.

CommonJS module — A Node module evaluated inside a wrapper with `exports`, `require`, and `module`, loaded synchronously through `require`, and represented to callers by `module.exports`. These are Node host semantics, not ECMAScript module semantics.

Closure — A function object together with retained access to the lexical environment associated with its creation. A closure reads the current values of captured bindings; creation does not freeze a value snapshot.

Commit point — The operation after which a prepared next state becomes visible to the state owner's readers. An in-memory assignment used as a commit point is not a database `COMMIT` and supplies no durability by itself.

Commit-unknown — A database effect classification used when a commit attempt rejected without evidence that the database did not commit. A later rollback acknowledgement cannot by itself prove that the earlier commit was absent.

Component — A separately named participant responsible for an operation or fact, such as the TypeScript checker, Node module loader, application parser, database, or lifecycle supervisor. A component need not be a class, package, process, or independently deployed service; the term identifies responsibility and authority in context.

Conditional type — A TypeScript type expression `Source extends Target ? Accepted : Rejected` that selects a type result using assignability. It does not emit or execute a JavaScript branch.

Conditional-pattern inference — A TypeScript `infer` declaration inside a conditional target pattern that captures a matched component for use in the accepted branch. It is compile-time structural matching, not runtime inspection.

Constructor — A function object with a `[[Construct]]` operation that can be invoked by `new`. The ordinary `.prototype` property used for candidate instances is distinct from the constructor object's own `[[Prototype]]` link.

Constraint — A rule whose enforcer is named, such as the TypeScript checker, an input parser, a database, or an authorization policy. In a generic declaration, an `extends` constraint restricts admissible type arguments and permits operations known on the constraint; it does not construct an arbitrary selected subtype.

Contravariance — A relationship in which assignability between generic instantiations reverses the assignability direction between their type arguments, as with strictly checked consumer function parameters.

Contextual typing — TypeScript inference in which a known use site supplies type information to an expression, such as a callback position supplying the parameter types of an arrow function.

Covariance — A relationship in which assignability between generic instantiations follows the assignability direction between their type arguments, as with producer return values.

Decoding — Runtime interpretation of a representation such as JSON text into a JavaScript value. Successful decoding establishes representation syntax, not application structure or domain validity.

Degraded mode — A ready process mode in which named capabilities are unavailable under an explicit admission, retry, evidence, and recovery policy. The label does not permit arbitrary partial success.

Direct TypeScript execution — A runtime or loader path that accepts `.ts` input without first using this repository's `tsc` emit. Node 24's built-in path strips or transforms a limited syntax set, performs no TypeScript checking, and does not apply `tsconfig` transformations.

Domain failure — An expected refusal defined by application rules, commonly represented as an ordinary result that callers must inspect. It is distinct from an infrastructure failure merely preventing evaluation from completing.

Domain construction — Creation of a complete domain value after required runtime facts have been established. The returned static type records those facts but does not cause them.

Domain identity — The application-defined relation by which values denote the same entity or semantic category. It is distinct from JavaScript object identity and is not established by structural compatibility alone.

Discriminant — A property whose distinct literal value identifies one member of a union and supplies evidence for control-flow narrowing.

Discriminated union — A closed TypeScript union whose members share a discriminant property with distinct literal values and carry the fields required by their respective alternatives.

Distributive conditional type — A conditional type whose checked operand is a naked type parameter and which is applied separately to members of a supplied union. Wrapping the operand, commonly in a one-element tuple, asks a whole-union question instead.

Drain — A lifecycle transition that first withdraws readiness and stops new admission, then requests cooperative cancellation, awaits tracked owned work within policy, and closes acquired resources once in reverse dependency order.

Erasure — The absence of most type-only TypeScript syntax and information from emitted JavaScript. The term is not universal deletion: classes remain runtime values, and constructs such as standard enums and parameter properties can generate JavaScript under the configured compiler.

ECMAScript module (ESM) — A module governed by ECMAScript import/export binding, linking, initialization, and evaluation semantics after a host resolves and loads its requested dependencies.

Effective configuration — The final compiler configuration for one invocation after config selection, inheritance, defaults, and command-line overrides. It must be inspected for the actual command rather than inferred from the nearest JSON file.

Emission — Production of new JavaScript, declaration files, or source maps by a compiler under one syntax, configuration, and version. Emission does not imply cleanup, packaging, loader acceptance, or execution.

Enforcement — A named component's ability to reject or prevent an operation that violates a stated predicate within that component's authority. Evidence that enforcement occurred in one fixture does not prove that the production authority is present or correctly configured.

Excess-property check — TypeScript's additional contextual check on selected fresh object literals for properties not recognized by the target type. It is not a general exact-object guarantee and removes no runtime properties.

Field owner — The discriminated-union member in which a payload field is meaningful and therefore required. It is distinct from the state owner that controls publication of state.

First divergence — The earliest observation in a deliberately ordered diagnostic pipeline where predicted and actual facts disagree. It can precede and belong to a different owner from the final exception or displayed source location.

Failure entry — A boundary-design record naming a failure's producer, representation, propagation path, consumer action, stable public promise if any, and facts deliberately withheld. The record does not itself classify or catch a runtime value.

Error — A failure representation or event. Qualify the term when a TypeScript diagnostic, thrown value, rejected promise, domain failure, database rejection, or operational incident could be confused with another.

Event loop — A scheduling facility supplied by a host such as HTML or Node.js. ECMAScript defines Jobs and host hooks but does not define one universal event-loop algorithm.

Evidence — An observation that supports a particular proposition for a stated command, value, configuration, time, and scope. Evidence from one checker, test, artifact, authority, or execution path does not automatically transfer to another.

Execution context — An ECMAScript specification structure containing state needed to evaluate code. An agent maintains an execution-context stack; the term does not require an engine to use one particular machine-stack representation.

Generator — An object created by calling a generator function. It is an iterable iterator whose execution starts or resumes when a consumer requests the next result.

Generic parameter default — A TypeScript type argument selected when neither explicit syntax nor inference supplies one for an optional type parameter. It creates no runtime default value.

Guarantee ledger — A review record that states one proposition together with its evidence class, responsible component, checking or enforcement time, governed scope, and remaining bypasses. The record does not strengthen the underlying evidence.

Identity — The property by which an object is the same object across observations. Separate object creation operations produce separate identities even when their properties are equal.

Idempotency — A qualified property under which repeated processing of the same logical request produces no additional committed effect for a named key, scope, and retention boundary.

Inference — TypeScript's derivation of a type from available source evidence, including initializers, return expressions, call relationships, and contextual use sites. It does not inspect a future execution.

Incident identity — The executable, exact command, working directory, selected entry, relevant versions and flags, sanitized environment or input facts, expected result, and actual result that distinguish one reproducible failure from nearby executions.

Impossible state — A domain combination excluded by a declared set of alternatives, always scoped to supported checked construction paths. Structural width, assertions, unchecked code, mutation, and external data remain relevant escape boundaries.

Indexed access type — A TypeScript type expression `ObjectType[KeyType]` that describes property value types selected from an object type by a key type. JavaScript bracket lookup remains the runtime operation.

Intersection type — A TypeScript type that requires a value to satisfy every member constraint. The type operator does not merge runtime objects.

Internal diagnostic — A bounded application value retained for authorized diagnosis, potentially including causal identity and implementation-specific categories. It is distinct from a public error and must not cross an external boundary through automatic serialization or field copying.

Iterable — A runtime value with a callable `[Symbol.iterator]()` method that supplies an iterator. TypeScript's `Iterable<T>` describes this operation during checking but does not install or validate it at runtime.

Iterator — A runtime object whose `next()` operation advances traversal state and returns an iterator result. An iterator may also be iterable by returning itself, in which case it is commonly single-use.

Job — An ECMAScript abstract closure that begins a computation when no other ECMAScript computation is active on that agent. HTML tasks and Node event-loop callbacks are not automatically ECMAScript Jobs.

`keyof` — A TypeScript type operator that derives the permitted property-key types of an object type. It does not enumerate keys or inspect an object at runtime.

Key remapping — A mapped-type operation using an `as` clause to derive a new property key or drop a property by remapping its key to `never`. It creates no runtime property.

Lexical environment — An ECMAScript specification structure that associates identifier names with bindings and links to an outer environment for lexical name resolution. The term does not require a particular engine memory layout.

Literal type — A TypeScript type describing one specific primitive value, such as `"queued"` or `3`, rather than the wider primitive category.

Linking — The ECMAScript-module operation that resolves requested imports and exports and initializes graph environments before module evaluation. Specifier resolution and file loading remain host responsibilities.

Live binding — An imported ECMAScript-module binding whose reads indirectly obtain the exporter's current binding value. The importer cannot assign through it, and it does not imply deep immutability.

Liveness — A probe policy describing whether a process is alive enough for an external restart decision. Liveness is distinct from readiness and does not prove dependency health, request correctness, or absence of partial failure.

Mapped type — A TypeScript type expression that iterates over a property-key union and constructs one checked property description per member. It does not enumerate or create a runtime object.

Mapping modifier — Mapped-type syntax that adds or removes optional or readonly status from constructed property descriptions. It changes checker permissions, not runtime property descriptors.

Microtask — Host terminology for work processed through a microtask queue. Promise reaction Jobs and `queueMicrotask` callbacks use host microtask scheduling in HTML and Node, but the exact authority must still be named.

Microtask checkpoint — A host operation that drains queued microtasks, including microtasks enqueued by earlier microtasks, before the host proceeds to later eligible work.

Module — A source or runtime unit with its own scope and explicit import/export relationships. Its exact interpretation depends on the language, compiler, loader, file extension, and package metadata.

Module identity — The resolved identity under which a particular loader caches or reuses a module instance. URL variants, different resolved filenames, package copies, workers, processes, or loaders can create separate identities.

Module resolution — Mapping a module specifier to source, declaration, package, URL, or runtime file information according to a named resolver. TypeScript resolution for checking and Node resolution for execution are separate operations.

Mutation — A change to the state of an existing object. Mutation differs from rebinding a name to a different value.

`never` — A TypeScript type describing no possible values. It can result when control flow eliminates every union member or describe a function that cannot return normally; JavaScript creates no `never` runtime value.

Non-null assertion — TypeScript's postfix `!`, which removes `null` and `undefined` from the checked description without executing a test or changing the runtime value.

Narrowing — TypeScript's refinement of the possible values represented by a type using control-flow evidence.

Normalization — A deliberate runtime transformation into a canonical representation, such as trimming text, changing case, applying a stated default, or converting text into a `Date`. It may change content and identity.

Object — A JavaScript language value with identity and properties, including functions and arrays. Object here does not mean only a plain object literal.

Observation — Selected runtime or tool evidence recorded during or after an operation. Unless the observing component is deliberately placed as a gate, observation does not prevent the observed effect and can itself be absent, sampled, delayed, mislabeled, or unavailable.

Own property — A property stored directly on an object rather than found by following the object's `[[Prototype]]` chain.

Outcome unknown — An effect state in which the caller lacks evidence that a dispatched operation either committed or did not commit. Safe next action may require status lookup, reconciliation, or an idempotency mechanism rather than blind retry.

Outbox — A protocol that stores a domain change and a publication-intent record in one participating database transaction, then publishes separately. It does not supply instant or exactly-once remote delivery and still requires retry, observation, and consumer idempotency decisions.

Partial success — A workflow outcome in which some logical items commit or fulfill while others fail, are refused, or remain unstarted. Reporting per-item outcomes does not make the batch transactional.

Package — A distribution and resolution boundary described by package metadata; it is not synonymous with a module.

Package entry point — A public package-name target selected through metadata such as `exports` or legacy `main`, potentially by subpath and condition. Entry-point encapsulation controls package-name resolution, not direct filesystem security.

Parsing — Converting an input representation into a structured result while establishing the syntax and structural conditions that the parser is designed to recognize.

Parse issue — A stable input-failure record containing a structural path, machine-oriented code, and expected fact without requiring the received value to be echoed.

Parse result — A discriminated success-or-failure value in which success contains one complete parsed output and failure contains issues without a partial domain output, unless a different partial-data contract is explicitly designed.

Phantom brand — A static distinction created by intersecting a runtime representation with a declaration-scoped property, commonly keyed by an unexported `unique symbol`, without installing that property on the runtime value. Assertions and unchecked code can forge the distinction.

Private brand — Runtime evidence that a JavaScript object was initialized with a class's private elements. A matching prototype chain does not by itself supply the brand.

Product type — A domain representation whose fields vary together as combinations of their possible values. Independent booleans and optional fields can therefore admit combinations that the domain does not intend.

Process — An operating-system-level running program. A process may host one or more language execution agents and resources.

Promise — A JavaScript object in one of three states: pending, fulfilled with a value, or rejected with a reason. Constructing a promise does not by itself make its executor asynchronous.

Promise reaction Job — An ECMAScript Job that applies a fulfillment or rejection handler and uses the handler's completion to resolve or reject a derived promise. A host schedules it subject to `HostEnqueuePromiseJob` constraints.

Promise resolution — The procedure that fixes a promise's eventual outcome. Resolution with an ordinary value normally fulfills; resolution with a promise or thenable can leave the resolving promise pending while it adopts the supplied value's eventual outcome.

Promise settlement — A promise's transition from pending to fulfilled or rejected. A settled promise does not change state or result.

Prototype — The object or `null` stored in an object's internal `[[Prototype]]` slot and consulted during inherited property lookup. A constructor's ordinary `.prototype` property is related but not the same slot.

Public error — A deliberately constructed external contract containing only stable, caller-relevant, safe fields. It is not an `Error` object spread or a sanitized internal diagnostic, and it should exclude causes, stacks, credentials, dependency details, and arbitrary input unless an explicit contract proves otherwise.

Rebinding — Changing the value associated with a binding. Rebinding does not by itself mutate either the old or new value.

Recursive conditional type — A TypeScript alias that refers to itself through a conditional derivation. Its authored termination rule, compiler recursion limits, and checking cost must be distinguished; no runtime recursion is emitted for a type-only alias.

Readonly — A TypeScript restriction that causes the checker to reject selected writes through a typed access path. Because the modifier is absent from emitted JavaScript, runtime immutability requires separate JavaScript enforcement or encapsulation.

Readiness — A lifecycle and capability decision stating whether a process should receive a named class of new work. A socket bind, live process, or logged ready event does not by itself establish readiness.

Rollback-unknown — A database effect classification used when an attempted rollback rejected and therefore cannot be reported as completed. Preserve the rollback failure separately from the primary operation failure.

Receiver — The runtime value supplied as `this` to an ordinary function invocation. Storing a function in an object property does not permanently attach that receiver to the function object.

Reference value — This book's operational term for the value copied between bindings when they give access to the same object identity. It must not be confused with the ECMAScript specification's internal *Reference Record*, which represents an evaluated access location.

Resource owner — The component responsible for exactly one required release path after successful acquisition, unless an API explicitly transfers that obligation.

Repository-contract simulator — A deterministic in-memory collaborator that supplies evidence about an application's repository calls, ordering, and outcome mapping. It does not supply evidence about a real database's schema, constraints, isolation, locking, rollback, or durability.

Retry policy — An application rule that decides whether and when to make another attempt using a failure classification, total-attempt limit, delay or backoff rule, cancellation state, and the possibility of already committed effects.

Run-to-completion — The same-agent guarantee that one ECMAScript Job finishes before another begins. A host callback likewise completes its synchronous JavaScript work before another event-loop callback begins. This does not make a multi-callback workflow transactional.

Runtime — The phase in which JavaScript instructions are loaded and executed, or, when explicitly qualified, the execution environment that performs that work. Prefer the more specific phrase `runtime value`, `runtime state`, `runtime host`, or `runtime behavior` when the distinction matters.

Runtime behavior — The ordering, returned values, thrown or rejected reasons, state changes, external effects, and other observations produced while a program executes. TypeScript diagnostics occur before this behavior and do not determine every runtime outcome.

Runtime host — An execution environment such as Node.js that supplies facilities outside the ECMAScript language core, including module loading integration, timers, I/O, process APIs, and scheduling rules. Host facilities have their own versions, configuration, and failure boundaries.

Runtime state — The current collection of execution-relevant facts, such as binding contents, object properties, promise settlement, queued work, open resources, and application records. Runtime state can change between observations; it is not one universal object named `state`.

Runtime truth — A fact established by a named executed operation or runtime authority, qualified by exactly what was checked, when it was checked, and what remains unproved.

Runtime value — An actual JavaScript value present during execution, such as a number, string, object, function, promise, class constructor, or emitted enum object. A TypeScript annotation or type alias may describe possible runtime values but is not itself a runtime value.

SameValueZero — The equality operation used by `Map` keys and `Set` membership. It treats `NaN` as equal to itself, treats positive and negative zero as equal, and compares objects by identity.

`satisfies` — A TypeScript operator that checks an expression against a target type while retaining useful inferred information about the expression. It emits no runtime validator.

Snapshot — An observation of state whose alias policy must be qualified. A live snapshot preserves access to owned state; an alias-preserving copy creates a new outer container; a detached snapshot recreates every mutable identity the caller must not share; a frozen-detached snapshot also rejects selected runtime writes.

Source map — A tooling artifact relating generated-code locations to source locations. It can improve debugging presentation but is not executable TypeScript and does not prove that the correct artifact or runtime value was used.

State owner — The component authorized to mutate or replace a particular invariant-bearing state root and responsible for preventing uncontracted aliases from escaping.

State transition — An operation that evaluates a proposal against current state and produces either a rejected result with no promised change or a committed next state according to its stated boundary.

State-specific transition — A function whose checked parameter types admit a particular current-state and command pair and whose implementation enforces the remaining runtime invariants before returning a next state or refusal.

Stable code — A machine-readable category whose meaning and caller action are maintained across compatible public releases. Human-readable messages, exception class names, and vendor prose are not stable codes unless an explicit external contract makes them so.

Standard enum — A non-`const` TypeScript enum that generates a runtime JavaScript object under the configured compiler. Numeric members receive reverse mappings in the representative output; string members do not.

Structural typing — TypeScript's practice of relating types primarily through their members and call signatures rather than requiring shared declaration names or constructor provenance. Additional contextual and special compatibility rules still apply.

Structural validation — Executed checks for container category, own-property presence, field value categories, unknown-key policy, and safe nested traversal. It is distinct from TypeScript structural assignability.

Sum type — A domain representation in which one member is selected from a closed union of alternatives, commonly identified by a literal discriminant. TypeScript emits no algebraic-data-type runtime object for the union itself.

Task — HTML host terminology for work selected from a task queue by an HTML event loop. A Node timer, immediate, or I/O callback should be named by its Node scheduling context rather than silently relabeled as an HTML task.

Template literal type — A TypeScript type expression that constructs string literal types from literal components and their unions. It allocates no runtime strings.

Thenable — An object whose `then` property is callable. Promise resolution assimilates a thenable by arranging for that method to receive resolving functions.

Thread — An execution concept supplied by a host or operating system, not a synonym for an ECMAScript Job or event-loop task.

Transaction-scoped repository — A repository whose participating operations use one acquired transaction lease for a stated asynchronous lifetime. The name does not include remote effects, another connection, or another database unless a specific coordination protocol does so.

Transport admission — Checks over an incoming request's route, method, media type, byte bound, decoding, and representation syntax. Success supplies a decoded runtime value, not application shape, domain validity, identity, permission, or persistence evidence.

Timeout — A deadline policy selecting what a caller does after time expires. A timeout may request cancellation, but settling a raced timeout promise does not itself stop the losing operation.

Total dynamic dispatch — An application dispatcher that returns an accepted or rejected result for every pair in its declared state-and-command Cartesian product. The guarantee does not imply validation of arbitrary external input, liveness, or absence of unexpected exceptions.

Type — Unless qualified otherwise, a TypeScript compile-time description of possible values and permitted operations; it is not a runtime class or validator.

Type construction — Checker-evaluated derivation of one compile-time type description from another using operations such as indexed access, mapping, conditionals, inference, or template literals. It is not JavaScript value construction.

Type argument — The explicit or inferred type selected for a generic type parameter at a use site. It is not a JavaScript argument unless the program separately passes a runtime value.

Type parameter — A checked variable over types declared by a generic function, type, class, or interface. Its useful role is to express a relationship among positions; it has no runtime value in ordinary emitted JavaScript.

Type predicate — A TypeScript return signature of the form `parameter is Type` that lets a boolean-returning function refine callers' types. The checker trusts the declared refinement, so the implementation owns the runtime proof obligation.

TypeScript project — The root-file graph and effective compiler options selected for one TypeScript invocation. It is not synonymous with a directory, repository, or editor workspace.

Type-query `typeof` — TypeScript's `typeof` operator in a type position, which reads the checked type of an identifier or property. It is distinct from JavaScript's evaluated runtime `typeof` expression.

Type space — A TypeScript syntactic context in which a name denotes a compile-time type description. Some declarations, including classes and enums, also supply a separate value-space name.

Union type — A TypeScript type describing values admitted by at least one member. Before narrowing, checked operations must be valid for every possible member; no union container exists at runtime.

Unavailability — A classified dependency outcome stating that the modeled operation cannot currently be served. It does not establish that a dispatched effect did not occur, that retry is safe, or that another dependency or operation has the same state.

Unknown-key policy — An explicit parser decision to reject, discard, or preserve extra own properties at a runtime object boundary. TypeScript excess-property checks and structural typing do not choose this policy.

`unknown` — A TypeScript type that can receive any value while requiring evidence before value-specific operations or assignment to a narrower target. It records uncertainty during checking and has no separate runtime representation.

Validation — Runtime examination that establishes identified properties of external or otherwise untrusted data.

Value validation — Executed checks for domain-specific literals, formats, finiteness, integer safety, ranges, uniqueness, or cross-field relationships after enough structure exists to evaluate them.

Variance — The relationship between assignability of type arguments and assignability of generic instantiations. Concrete producer, consumer, and method positions determine the applicable direction and TypeScript compatibility boundary.

Value space — A syntactic context requiring a runtime JavaScript value or binding. A name that exists only in TypeScript's type space cannot satisfy a value-space use after emission.

Value — A JavaScript datum that can be stored, passed, returned, or compared. Primitive values and object values differ in identity and mutation behavior.

Widening — TypeScript's loss of literal precision when a location must describe a broader range of permitted values, such as a mutable `let` binding or object property intended to accept later writes.

Worker pool — An application scheduling structure in which a fixed or bounded set of worker loops claim logical items while preserving a stated admission and active-count invariant. It does not by itself guarantee completion order or universal fairness.